

微网与外部电网电力调控策略

摘 要

本文研究含光伏与储能的并网微网购电调度问题。针对确定性电价、负荷与光伏功率变化、多时点光伏预报以及实时波动电价四种运行条件，建立由预测、风险修正、优化调度和时序执行组成的统一模型。模型以购电费用最小为目标，并满足功率平衡、储能状态递推、容量与功率边界及充放电互斥等约束。

问题一将一天划分为 144 个 10 分钟时段，构建单日混合整数线性规划模型。HiGHS 求得的最优方案中，全天计划购电量为 59482.70 kWh，购电费为 35126.95 元，平均购电价格为 0.5905 元/kWh，全天弃光量为零。两种线性规划松弛均得到 35126.95 元的最优费用，且均不存在同时充放电；标准松弛解与混合整数解一致，由此验证了所得方案的全局最优性。与无储能方案和无光伏、无储能方案相比，购电费用分别降低 26.90% 和 60.23%。

问题二根据历史负荷与光伏数据建立日前预测模型，并以历史预测误差的经验分位数修正净负荷预测。由紧急购电价格为常规电价 5 倍的非对称成本关系，得到 80% 分位数的基准风险水平；随后通过连续时序仿真标定风险分位数、储备系数和日末储备惩罚。以 2025 年 1 月为预热期，在 2 月 1 日至 12 月 31 日的 334 天内，计划购电量为 21463158.37 kWh，紧急购电量为 140301.24 kWh，总购电费为 13939720.46 元，其中紧急购电费占 5.42%。该方案较预测均值策略和无储能策略分别降低费用 6.47% 和 24.73%。

问题三利用每日 0:00、6:00、12:00 和 18:00 发布的光伏预报，建立分时滚动调整模型。模型区分初始计划购电量、调整后购电量和紧急购电量，并将全天划分为四个六小时调整区间。对附件 3 的时间口径进行对齐检验后，采用以发布时刻实际功率为左端点的分段线性插值。退款计费口径下，购电量的调整范围由 $Q_{0.7}$ 和 $Q_{0.9}$ 两个分位数确定。334 天的时序仿真结果表明，总购电费为 13,418,626.35 元，其中计划费用、调整净额和紧急购电费用分别为 12,583,676.62、562,013.56 和 272,936.18 元；相较于仅使用 0:00 预报且不进行日内调整的策略，费用降低 10.41%。预报组合分析显示，18:00 预报的边际贡献最大，而在已使用 6:00 与 18:00 预报时，加入 12:00 预报可进一步降低费用 11.44 万元。

问题四进一步将电价视为随日期和时段变化的随机量。价格预测参数在 1 月确定，并在 2 月至 12 月保持不变；所采用的同星期加权模型权重为 0.5/0.3/0.2，三个日内修正系数依次为 1.0、0.75 和 0.5，评价期平均绝对误差为 0.0446 元/kWh。计划固定的 4-2 模型总费用为 14740813.21 元，允许日内调整的 4-3 模型总费用为 14153285.53 元，后者降低 3.99%。在当天价格完全已知的参考条件下，4-3 费用为 14075672.00 元。结果表明，价格预测与日内调整能够共同降低波动电价环境下的购电成本。

关键字：混合整数线性规划 微网调度 储能优化 峰谷套利 风险分位数 滚动决策 灵敏度分析

一、 问题重述

1.1 问题背景

微网是为解决光伏发电等分布式能源本地消纳、减少并网问题而快速发展起来的小型发用电系统。出于经济方面的考虑，非孤岛式微网可在外部电网（以下简称外网）电价较低或分布式能源有剩余电量的时间段为储能设备充电，并在外网电价较高的时间段释放电能。

微网与外网调控的目标是：利用小区负载、光伏发电量和储能设备当前储电量等数据，尽可能精准地给出微网的购电量和储能设备的充放电量，使微网提供的电力既满足小区负载，又尽可能节省购电费用。

1.2 问题提出

问题一：若每天的电价和小区负载相同，同时给出光伏发电功率一天的预测数据，要求微网提供的电能不低于小区负载，且储能设备在 0:00 和 24:00 的储电量相同。需要在每天 0:00 制定微网当天计划购电策略的数学模型；根据附件 1 和附录 1 的数据给出具体策略，并以规定格式给出指定时段的购电量、全天购电量与购电费，以及储能设备的充放电量与 0:00、24:00 储电量，完整策略保存至 `result1.xlsx`。

问题二：若每天的电价相同，而小区负载和光伏发电功率随时间变化，且微网提供的电能不可低于小区负载，如果低于负载需向外网紧急购电，其电价是交易时刻电价的 5 倍；除紧急购电费用外，其他时间段的购电费用均按计划购电量计算。需要在每天 0:00 制定当天的计划购电策略。根据附件 1 的电价和附件 2 的数据，给出表 3 中指定日期（2025.3.20、2025.6.21、2025.9.23、2025.12.21）的结果，并以规定格式给出紧急购电结果，完整策略保存至 `result2.xlsx`。

问题三：每天 0:00、6:00、12:00 和 18:00 可获得未来 24 小时整点的光伏发电功率预报。微网先依据 0:00 预报制定当天计划购电策略，并可依据其余时刻的新预报调整购电量。计划购电量高于调整购电量的部分按交易时刻电价的 50% 收取违约金；调整购电量高于计划购电量的部分按交易时刻电价的 1.5 倍计费。总购电费用包括计划购电费、紧急购电费与调整相关费用。需要建立相应的购电策略模型，给出四个指定日期的结果，将完整策略保存至 `result3.xlsx`，并分析其他时刻预报的引入价值。

问题四：外网电价改为实时波动。基于附件 2 的负载与光伏实际值、附件 3 的光伏预报和附件 4 的分时电价，在波动电价条件下重新计算问题二和问题三，给出相应的购电策略与计算结果，并将完整策略分别保存至 `result4-2.xlsx` 和 `result4-3.xlsx`。

二、 问题分析

四个问题采用相同的储能状态、能量平衡与效率约束，主要区别在于决策时可获得的信息以及购电费用的计算规则。下面分别分析各问题的建模重点。

2.1 问题一分析

问题一中，各日采用相同的电价、负荷和光伏预测曲线，因此可用一个完整日的调度方案表示重复运行策略。按照 10 分钟间隔离散后，模型包含 144 个决策时段。

储能通过低价时段充电、高价时段放电实现跨时段能量转移，同时吸收光伏富余电量。任一时段的充放电决策都会改变后续储能状态，因而需要对全天统一优化。目标函数、能量平衡、状态递推和运行边界均为线性关系；加入充放电模式变量 $z_t \in \{0, 1\}$ 后，模型构成混合整数线性规划。

模型设置弃光变量，将供需关系写为等式；充电量计入用电侧，购电、光伏和放电量

计入供电侧。日循环约束 $E_{144} = E_0$ 保证方案可以逐日重复，并给出 $\sum_t d_t = \eta_c \eta_d \sum_t c_t$ 的状态一致性关系。二元变量用于约束同一时段的充电和放电状态。

2.2 问题二分析

问题二需要同时处理日前预测、非对称购电成本和跨日储能状态。

制定第 n 天计划时，当天的实际负荷和光伏功率尚未获得。因此，点预测和误差分位数均由第 n 天以前的数据计算，形成符合信息时序的日前预测结果。

计划购电每增加 1 kWh 产生费用 p ，而计划不足的边际期望费用为 $5p \Pr(N > g)$ 。令两者相等可得 $F_N(g) = 0.8$ ，因此以净负荷预测误差的 80% 分位数作为风险余量的理论基准。考虑储能的跨时段耦合后，再通过连续运行费用对分位数水平进行标定。

跨日运行时，第 $n+1$ 天的初始储电量取第 n 天执行后的末值。相应的状态校验采用 $\Delta E = \eta_c \sum_t c_t - \eta_d^{-1} \sum_t d_t$ 。为控制单日优化对期末储能的消耗，目标函数中加入日末储备软约束 $\lambda \xi$ ；该项用于策略制定，不计入实际购电费用。

据此，问题二采用“预测—风险修正—日前优化—实时执行—跨日更新”的建模流程，并以包含紧急购电费用的实际总费用评价策略。

2.3 问题三分析

问题三允许根据日内新增预报调整购电计划，需要协调预报发布时间、调整范围和费用结算方式。

每天共有四个预报版本，各自覆盖发布时刻以后 24 个整点。数据处理时同时记录发布时间和目标时间，并对缺少的发布时刻功率设置基于已观测数据的左端点。不同预报版本的效果最终由连续运行费用评价。

问题三的调整价格改变了风险分位数。按照退款计费口径，增加 1 kWh 购电量的成本为 $1.5p$ ，减少 1 kWh 购电量的违约成本为 $0.5p$ 。比较调整成本与 $5p \Pr(N > x)$ 的期望紧急购电成本，可得

$$x^* = \text{clip}(g^0, Q_{0.7}(N), Q_{0.9}(N)),$$

其中 $[Q_{0.7}, Q_{0.9}]$ 定义为计划保持区间：初始计划位于该区间时保持原值，低于下界时增加至 $Q_{0.7}$ ，高于上界时减少至 $Q_{0.9}$ 。该关系与库存管理中的临界比规则一致 [7]。不退款计费口径下，调减不产生费用优势，模型仅保留 $Q_{0.7}$ 的增加购电阈值。考虑储能耦合后，实际分位数参数通过连续时序仿真标定。

为明确多次调整的结算对象，本文将全天划分为四个六小时调整区间。每个区间仅在起始时刻提交一次调整量，每个十分钟时段均相对于其初始计划量结算；后续区间的优化结果用于储能状态规划，达到相应发布时刻后再正式提交。

在此基础上建立多时点预报驱动的滚动购电模型，并对三个新增预报时刻形成的八种组合分别进行全年时序仿真。另设置仅更新系统状态而沿用 0:00 预报的参考策略，用于分析新增预报信息与滚动优化对费用变化的贡献。

三、模型假设

1. **数据准确假设**：附件 1 的电价、小区负载与光伏功率准确反映实际，不计预测误差（其影响留待问题三）。
2. **分段恒定假设**：每个 10 分钟时段内各功率恒定，时段电量等于功率乘以 $\Delta t = 1/6 \text{ h}$ ；供电保障对应这一离散模型。
3. **母线侧计量假设**： c_t 为微网送入储能的电量、 d_t 为储能向微网交付的电量，均按微网母线侧口径计量，额定功率同侧解释。
4. **储能理想假设**：单程效率恒为 $\eta_c = \eta_d = 0.90$ （往返 0.81），不随功率、储电量、温

度或设备老化变化，无自放电与漏电损失。

5. **单向购电假设**：题目未给出售电价格， $g_t \geq 0$ ，不计余电上网收益。
6. **弃光可行假设**：光伏出力超过负载且储能无法完全吸纳时允许弃光， $0 \leq s_t \leq v_t$ ，富余光伏总有确定去向。
7. **不外加上网限值假设**：附录中 5000 kW 限制的是**储能设备**的充放电功率，本文不另设微网购电功率上限。
8. **无紧急购电假设**：问题一中计划购电得到完全执行，不触发因计划不足而生的紧急购电（该情形在问题二中讨论）。
9. **日循环假设**：储能每日回到当日 0:00 的水平，即 $E_{144} = E_0$ ，该计划可逐日重复执行。
10. **忽略衰减假设**：不计容量衰减与寿命损耗成本。

问题二改用以下假设（与问题一冲突处以本节为准）：

11. **电价逐日重复假设**：附件 1 的日内电价在 2025 年每天重复生效，不使用附件 4 的波动电价（留待问题四）。
12. **实际值可观测假设**：当天运行时，执行层对负载与光伏作**段内实时测量**，本时段的充放电与紧急购电由本时段的实际值决定（理想即时响应、无控制滞后）；10 分钟仅作为汇总与记账步长，时段末读数用于该时段的电量与费用结算。问题二、三、四共用同一执行层，本假设一致适用。
13. **预测信息受限假设**：0:00 制定计划时只能使用当天之前的负载与光伏实际值、已知的日内电价、日期与星期，以及当前实际储电量；当天实际曲线与附件 3 的外部光伏预报均不可使用。
14. **计划购电量冻结假设**： $g_{n,t}$ 一经 0:00 确定便不再修改，不足部分只能按 5 倍电价紧急补购，无计划调整交易（留待问题三）。
15. **紧急购电不退款假设**：已按计划购入的电量无论用完与否均全额收费，用不完的部分可转为充电，其余按无收益弃用处理，即**允许弃用已购电**。
16. **储备惩罚不计入账单假设**：软惩罚项 $\lambda \xi_n$ 只用于制定策略，**不计入实际上报的购电费**。

四、 符号说明

符号	含义	单位
t	时段编号， $t = 1, 2, \dots, T$	—
T	一天划分的时段总数， $T = 144$	个
Δt	单个时段时长， $\Delta t = 10/60 = 1/6$	h
p_t	第 t 时段的外网电价（附件 1）	元/kWh
L_t	第 t 时段的小区负载功率（附件 1）	kW
V_t	第 t 时段的光伏发电预测功率（附件 1）	kW
ℓ_t	第 t 时段的负载电量， $\ell_t = L_t \Delta t$	kWh
v_t	第 t 时段的光伏发电量， $v_t = V_t \Delta t$	kWh
g_t	第 t 时段从外网购入的电量（决策变量）	kWh
c_t	第 t 时段向储能设备输入的充电量（决策变量）	kWh
d_t	第 t 时段储能设备向微网输出的放电量（决策变量）	kWh
s_t	第 t 时段弃用的光伏电量（决策变量）	kWh
E_t	第 t 时段末的储电量（决策变量）， $t = 0, \dots, T$	kWh
z_t	充放电模式变量， $z_t \in \{0, 1\}$ （决策变量）	—
E^{init}	0:00 时刻的初始储电量，取 6000	kWh
M	单时段最大充电量或放电量， $M = 5000 \Delta t = 2500/3$	kWh
η_c, η_d	储能充电效率、放电效率，均为 0.90	—
$P_{\max}$	储能最大充放电功率， $P_{\max} = 5000$	kW
$E_{\min}, E_{\max}$	储电量运行下界与上界，1200 与 10800	kWh
J	全天购电费用（目标函数）	元

问题二新增或改变含义的符号如下（其余沿用上表；带横线的变量表示日前参考计划，不带横线的为实际执行值）：

符号	含义	单位
n	日期编号, $n = 0, 1, \dots, 364$ ($n = 0$ 为 2025-01-01)	日
$L_{n,t}, V_{n,t}$	第 n 天第 t 时段的实际负载、光伏功率 (附件 2)	kW
$\ell_{n,t}, v_{n,t}$	第 n 天第 t 时段的实际负载电量、光伏发电量	kWh
$\hat{\ell}_{n,t}, \hat{v}_{n,t}$	由 n 之前历史得到的负载、光伏预测电量	kWh
$\hat{N}_{n,t}$	净负荷预测, $\hat{N}_{n,t} = \hat{\ell}_{n,t} - \hat{v}_{n,t}$	kWh
$\varepsilon_{n,t}$	净负荷预测误差, $\varepsilon_{n,t} = (\ell_{n,t} - v_{n,t}) - \hat{N}_{n,t}$	kWh
$Q_{\alpha,n,t}$	此前误差在对应时段的经验 α 分位数	kWh
$\tilde{N}_{n,t}$	风险修正净负荷, $\tilde{N}_{n,t} = \hat{N}_{n,t} + Q_{\alpha,n,t}$	kWh
α	风险分位数 (待标定)	—
$\bar{E}_{n,t}$	日前参考储电量	kWh
$\bar{c}_{n,t}, \bar{d}_{n,t}, \bar{w}_{n,t}$	日前参考充电量、放电量、剩余电量	kWh
E^{tar}	希望为次日保留的参考储电量	kWh
λ	日末储备不足的规划代价 (待标定)	元/kWh
ξ_n	日末储备不足量, $\xi_n \geq E^{\text{tar}} - \bar{E}_{n,144}$	kWh
$b_{n,t}$	使用储能前的电量盈余, $b_{n,t} = g_{n,t} + v_{n,t} - \ell_{n,t}$	kWh
$R_{n,t}$	储备线, $R_{n,t} = E_{\min} + \rho(\bar{E}_{n,t} - E_{\min})$	kWh
ρ	储备线系数, $0 \leq \rho \leq 1$ (待标定)	—
$c_{n,t}, d_{n,t}$	实际充电量、实际放电量	kWh
$r_{n,t}$	紧急购电量 (按 $5p_t$ 计价)	kWh
$w_{n,t}$	实际弃用电量 (不退款)	kWh
$E_{n,t}$	实际储电量, $t = 0, \dots, 144$	kWh
J_n^{plan}	全天计划购电费, $\sum_t p_t g_{n,t}$	元
J_n^{emg}	全天紧急购电费, $\sum_t 5p_t r_{n,t}$	元
$\mathcal{H}_n$	第 n 天预测所用的历史日集合, $\mathcal{H}_n \subseteq \{i : i < n\}$	—

五、 问题一：单日计划购电优化模型

5.1 数据预处理与时段划分

附件 1 共给出 144 组数据，涵盖一天 24 小时，故将一天等分为 $T = 144$ 个长度为 $\Delta t = 10 \text{ min} = 1/6 \text{ h}$ 的时段。

附件 1 的时间列按区间结束时刻处理：（首行 0:10、末行 0:00+1），其中 0:00+1 按题目附录 2 的规定表示当天 24:00。因此附件 1 的第 t 行对应时段 $[(t-1)\Delta t, t\Delta t)$ ，即第 t 时段为 $(t-1) \times 10$ 分钟至 $t \times 10$ 分钟，该行的电价与功率作为这一区间的代表值。

附件 5 中 **result1.xlsx** 的“计划购电量”表首段标签为 0:10–0:20、末段为 0:00+1–0:10+1，其末段已越过当天，与正文要求的 0:00–24:00 口径整体错位一格。本文统一采用正文口径，在输出副本中将购电时间标签改写为 0:00–0:10 至 23:50–0:00+1，行序与模板一一对应。

单位换算：附件 1 中电价为元/kWh，负载与光伏功率为 kW。为便于与题目结果表逐项对应，本文不直接以功率作决策变量，而是先把负载与光伏换算为时段电量，再以电量为决策变量单位：

$$\ell_t = L_t \Delta t, \quad v_t = V_t \Delta t. \quad (1)$$

图 1 给出附件 1 三类输入数据的全天变化特征：电价呈明显的双峰结构，早高峰位于 7:50–9:10（峰值 1.1683 元/kWh）、晚高峰位于 17:40–20:50，最高电价 1.3952 元/kWh（出现在 20:30–20:40），而夜间低谷约 0.42 元/kWh、全天最低 0.3713 元/kWh（出现在 5:30–5:40），峰谷价差达 3.76 倍，为峰谷套利提供了充足空间；小区负载在 8:00–18:00 维持在 4208 kW 以上、峰值 5958.97 kW（出现在 9:00–9:10），夜间回落至约 3400 kW；光伏出力呈标准钟形分布，4:30 前后开始发电，正午达到峰值 7612.32 kW，19:20 后归零，全天有 55 个时段出力为零。三者共同决定了优化问题的基本格局。

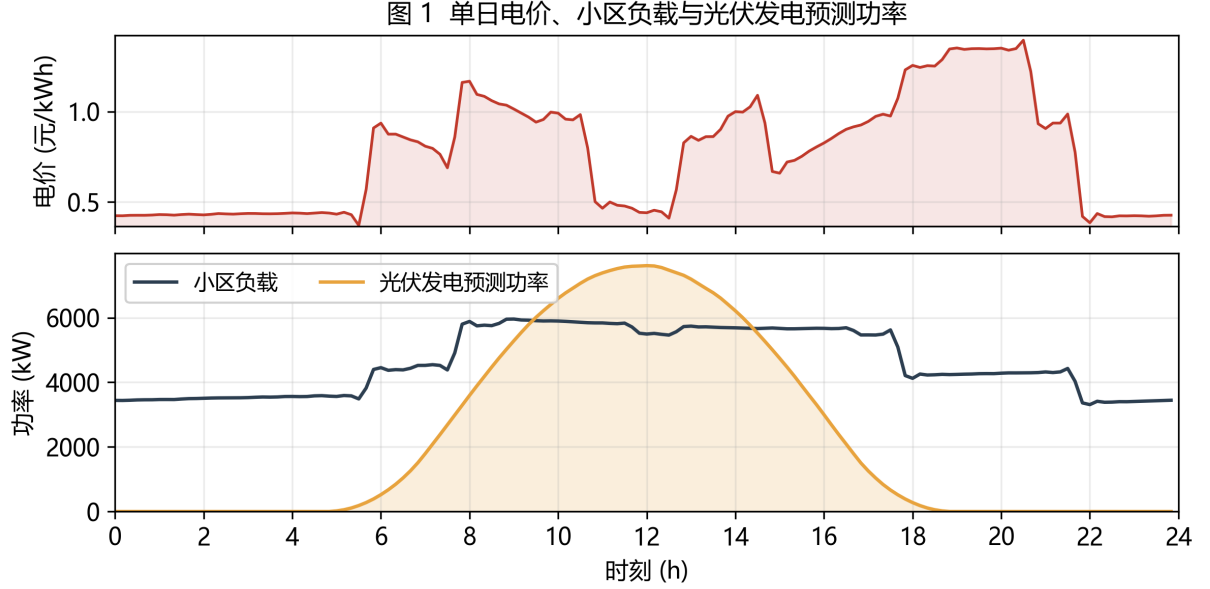


图 1 附件 1 三类输入数据的全天变化特征

题目所述“每天的电价相同”表示每日重复使用同一条分时电价曲线。模型保留附件 1 中的十分钟电价分辨率。

5.2 模型建立

5.2.1 决策变量

对每个时段 $t = 1, 2, \dots, 144$ ，定义如下决策变量（前四个以电量 kWh 计）：

- $g_t \geq 0$ ：从外网购入的电量 (kWh)；
- $c_t \geq 0$ ：向储能设备输入的充电量 (kWh)；
- $d_t \geq 0$ ：储能设备向微网输出的放电量 (kWh)；
- $s_t \geq 0$ ：弃用的光伏电量 (kWh)；
- E_t ：第 t 时段末的储电量 (kWh)， $t = 0, 1, \dots, 144$ ；
- $z_t \in \{0, 1\}$ ：充放电模式变量， $z_t = 1$ 表示该时段允许充电、 $z_t = 0$ 表示允许放电。

注意 E_0 是 0:00 电量、 E_{144} 是 24:00 电量，因此储电量序列共有 **145** 个状态点。

5.2.2 目标函数：全天购电费最小

微网的购电费用仅来自从外网购入的电量，故目标为最小化全天购电费：

$$\min J = \sum_{t=1}^T p_t g_t. \quad (2)$$

由于 g_t 的单位为 kWh，目标函数中无需再乘 Δt 。储能能量损耗可能增加购电总量，但低价充电、高价放电仍可降低购电费用。

5.2.3 约束条件

(1) 供需平衡约束（等式加显式弃光）

微网在第 t 时段可用电量由外网购电、扣除弃光后的光伏出力与储能放电三部分构成，须恰好等于小区负载耗电与储能充电之和：

$$g_t + (v_t - s_t) + d_t = \ell_t + c_t, \quad t = 1, 2, \dots, T \quad (3)$$

(2) 光伏与购电的非负及弃光上界

$$g_t \geq 0, \quad 0 \leq s_t \leq v_t, \quad t = 1, 2, \dots, T \quad (4)$$

式 (3)–(4) 以显式弃光变量处理光伏富余，并可直接统计其电量；约束 $s_t \leq v_t$ 保证弃光量不超过光伏发电量。

(3) 储能电量递推约束

第 t 时段末的储电量由上一时段末的储电量、本时段的充电量与放电电量决定：

$$E_t = E_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad t = 1, 2, \dots, T \quad (5)$$

其中 $\eta_c = \eta_d = 0.90$ 。按照母线侧计量口径，充入 1 kWh 电量使储能增加 0.9 kWh，向微网输出 1 kWh 电量使储能减少 1/0.9 kWh。该设定对应储能往返效率 $\eta_c \eta_d = 0.81$ 。当 $t = 1$ 时 $E_{t-1} = E_0$ ，即 0:00 的初始储电量。

(4) 储电量运行区间约束

为避免过度充电或放电，储电量须全程保持在给定区间内，145 个状态点逐一受限：

$$E_{\min} \leq E_t \leq E_{\max}, \quad t = 0, 1, \dots, T \quad (6)$$

其中 $E_{\min} = 1200$ kWh, $E_{\max} = 10800$ kWh。

(5) 功率上限与充放电互斥约束

受逆变器与电池倍率限制，单个 10 分钟内最大充电量与最大放电量均为

$$M = P_{\max} \Delta t = 5000 \times \frac{1}{6} = \frac{2500}{3} \text{ kWh}, \quad (7)$$

并利用二元变量约束同一时段不能同时充放电：

$$0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad z_t \in \{0, 1\}, \quad t = 1, 2, \dots, T \quad (8)$$

$z_t = 1$ 时允许充电、放电电量被压为零； $z_t = 0$ 时允许放电、充电量被压为零；两种模式下都可以令充放电电量同时取零，表示待机。

(6) 日循环约束

题目要求储能设备在 0:00 与 24:00 的储电量相同，结合附录 1 给出的 2025 年 1 月 1 日 0:00 电量为 6000 kWh，有：

$$E_0 = E^{\text{init}} = 6000, \quad E_T = E_0 \quad (9)$$

该约束使日末储电量恢复至日初水平，从而形成可逐日重复的运行方案。

对状态递推式 (5) 求和并结合日循环条件 (9)，得到

$$\eta_c \sum_{t=1}^T c_t = \frac{1}{\eta_d} \sum_{t=1}^T d_t \implies \sum_{t=1}^T d_t = \eta_c \eta_d \sum_{t=1}^T c_t = 0.81 \sum_{t=1}^T c_t. \quad (10)$$

因此，全天放电量与充电量之比应为往返效率 0.81。

5.2.4 完整模型

综合以上各式，问题一的数学模型为：

$$\begin{aligned}
\min_{g, c, d, s, E, z} \quad & J = \sum_{t=1}^{144} p_t g_t \\
\text{s.t.} \quad & g_t + (v_t - s_t) + d_t = \ell_t + c_t, \quad t = 1, \dots, 144 \\
& E_t = E_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad t = 1, \dots, 144 \\
& E_{\min} \leq E_t \leq E_{\max}, \quad t = 0, \dots, 144 \\
& 0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad t = 1, \dots, 144 \\
& 0 \leq s_t \leq v_t, \quad g_t \geq 0, \quad t = 1, \dots, 144 \\
& z_t \in \{0, 1\}, \quad t = 1, \dots, 144 \\
& E_0 = E_{144} = 6000
\end{aligned} \tag{11}$$

5.2.5 模型类别：MILP 及其 LP 松弛

模型 (11) 中目标函数与全部约束关于决策变量均为线性，唯一的非线性来源是 $z_t \in \{0, 1\}$ 这一整数要求，因此它是一个**混合整数线性规划 (MILP)**。按变量排列 $x = [g_{1:T}, c_{1:T}, d_{1:T}, s_{1:T}, E_{1:T}]^T \in \mathbb{R}^{5T}$ 、 $z = [z_1, \dots, z_T]^T$ ，可写成标准形式 $\min_{x, z} \bar{c}^T x$ s.t. $A_{\text{eq}}[x; z] = b_{\text{eq}}$, $A_{\text{ub}}[x; z] \leq b_{\text{ub}}$, $l \leq x \leq u$, $z \in \{0, 1\}^T$ ：共 **864** 个决策变量 ($5 \times 144 + 144$)、289 条等式约束（供需平衡、状态递推与日循环）与 288 条不等式约束（互斥约束）。连续变量约束中的系数主要由 $0, \pm 1, \pm \eta_c, \pm 1/\eta_d$ 构成，充放电模式约束另含常数 M 。储能状态变量连接相邻时段，使模型具有时序网络流结构。

程序设置两种线性规划下界：LP1 直接删除模式变量及互斥约束，包含 720 个变量；LP2 保留互斥约束但放宽为 $0 \leq z_t \leq 1$ ，包含 864 个变量。完整 MILP 也包含 864 个变量。理论上有 $J_{\text{LP1}} \leq J_{\text{LP2}} \leq J_{\text{MILP}}$ 。程序分别求解三者，并检查两种松弛解的充放电互斥性。其中 $M = 5000 \times (1/6) = 833.33 \text{ kWh}$ ，为单个十分钟时段的最大充放电量。

5.3 模型求解

5.3.1 求解方法与算法

本文使用 Python 科学计算栈中的 `scipy.optimize.milp` 接口调用 **HiGHS** 求解器（当前主流的开源 LP/MILP 求解器），以稀疏矩阵存储约束系数，对两种 LP 松弛与完整 MILP 分别求解。求解参数为默认的 10^{-7} 量级可行性容差。

求解流程为：

1. 读取附件 1 的电价、负载与光伏三类序列，校验时段标签严格递增、等间隔且恰好覆盖当天 144 个时段，确保时段口径成立；
2. 按式 (1) 将功率换算为时段电量，整理容量、功率、效率与日初电量等参数；
3. 按上一小节的矩阵形式装配目标向量、约束矩阵与变量边界，分别构造 LP1 结构松弛、LP2 标准松弛与完整 MILP；
4. 调用 HiGHS 求解，记录求解状态、目标值与最优性间隙；
5. 对解进行独立复核（供需平衡、状态递推、物理边界、日循环、充放电互斥、费用复算），并判定 LP 松弛解是否已是 MILP 最优解；
6. 由通过校验的解汇总论文表格、绘制策略图，并按统一时间标签写出 `result1.xlsx`。

5.3.2 求解结果与 LP/MILP 一致性

两种 LP 松弛与完整 MILP 的求解结果如下：

- **LP1 结构松弛**：最优费用 35126.9486 元；同时充放电时段数为 0；
- **LP2 标准松弛**：最优费用 35126.9486 元；同时充放电时段数为 0；
- **完整 MILP**：最优费用 35126.9486 元，最优性间隙 0（已证明全局最优）；最优解同样不含同时充放电。

三者的最优费用在数值精度内一致。LP2 与 MILP 的逐段解一致；LP1 因退化最优解，有 2 个时段的决策量与 MILP 不同，但同样满足互斥约束。由两侧一致的目标值可知 MILP 已达到线性规划下界。后续结果采用完整 MILP 的解。

5.3.3 求解结果的校验

对最优解进行时段完整性、逐段供需平衡、状态递推、物理边界、日循环和费用汇总六项检验，结果均满足要求。各项最大残差均在 10^{-6} kWh 的检查容差以内（日循环校验实测 $16799.94/20740.67 = 0.810000$ ）。各项检验结果见附录第 1.14 节。

5.4 求解结果

5.4.1 表 1：微网在指定时间段的购电量及全天的购电量和购电费

表 1 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量/kWh				59482.70	
全天购电费/元				35126.95	

5.4.2 表 2：储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

表 2 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	4500.00	0.00	4:00-8:00	833.33	6365.84
8:00-12:00	4787.96	1703.00	12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13	20:00-24:00	5333.33	2859.87
0:00 储电量/kWh				6000.00	
24:00 储电量/kWh				6000.00	

表 2 中每段的充电量与放电量**分别求和**，未用正负相抵后的净电量代替。若需另报电池内部的能量变化，对应为 $\eta_c C_k$ 与 D_k/η_d ，应与本文的母线侧报表口径明确区分。

完整的计划购电策略（144 个 10 分钟时段的逐时段购电量，以及各时段充放电量与储电量）已按要求写入文件 `result1.xlsx`。

5.5 结果分析

购电主要集中在低电价时段：0:00–6:00 的夜间低谷维持在每 10 分钟 1400 kWh 上下（全天最大 1409.8 kWh/10 min），5:40 储能充满后连续 31 个时段购电量降为零；早高峰 7:30–10:00 购电量为零，由储能放电支撑；晚高峰 17:00–21:00 回落至 0–826.08 kWh/10 min。正午 11:00–12:30 共 8 个时段，在光伏已经富余的情况下仍保

留 179.28–575.17 kWh/10 min 的购电。此时模型利用正午电价的局部低谷 (0.4113–0.5007 元/kWh) 后提前购入、存入储能留待傍晚 1.3952 元/kWh 的高价时段使用。

这一策略正是”谷充峰放”的定量体现，其经济性条件可直接由边际分析给出：在低价时段购入 1 kWh 用于充电，未来只能向负载交付 $\eta_c \eta_d$ kWh，故跨时段套利值得进行的条件是 $p_{\text{低}}/\eta_c \eta_d < p_{\text{高}}$ 。代入 $\eta_c \eta_d = 0.81$ ，只要峰谷价差超过约 1.23 倍即值得套利，而附件 1 的实际峰谷比达 3.76，空间充裕。逐段购电量与电价的对照图、以及各时段的完整分解见附录 A。

储电量轨迹呈典型的**两充两放**格局 (0:00–5:40 谷电充电至 10800 kWh, 5:50–9:30 早高峰放电至 1834.62 kWh; 9:30–16:00 吸收光伏盈余与正午低谷电再次充满, 18:00–20:40 晚高峰放电至下限 1200 kWh; 21:50–24:00 送回 6000 kWh 满足日循环)。储电量在充电过程中达到上界 10800 kWh，在放电过程中达到下界 1200 kWh，表明容量约束在最优解中起作用。全天充电量 20740.67 kWh 与放电量 16799.94 kWh 之比恰为 0.810000，与式 (10) 的理论值 $\eta_c \eta_d = 0.81$ 完全一致，满足能量守恒关系。轨迹图与逐段说明见附录 A。

全天电量两侧严格相等：供给侧为光伏 55482.84 kWh、购电 59482.70 kWh 与储能放电 16799.94 kWh，需求侧为负载 111024.81 kWh 与储能充电 20740.67 kWh，合计均为 131765.48 kWh。本模型的弃光电量为零，储能吸收了光伏超过负载的全部盈余 (6247.96 kWh)。不同决策粒度下的弃光变化见第 5.6 节。平衡构成图见附录 A。

5.6 基准对照与灵敏度分析

为量化储能与光伏的经济价值，本文构造”光伏供电、缺口直接外购、电池保持不动”的基准方案（它是原问题的一个可行解，故给出同一口径下的费用上界）；针对题目只给出一个”充放电效率 90%”、其单程/往返分配方式存在解释空间的情况，又设计了效率分配、日循环水平与决策粒度共六组对照口径。两组对照的完整数据与逐项推理见附录 L，正文只保留结论。

1. **储能的综合作用**。配置储能后全天购电费由 48052.05 元降至 35126.95 元，**节省 12925.10 元 (降幅 26.90%)**；较”无光伏无储能”方案节省 53192.41 元 (60.23%)。含储能方案的购电量反而更少 (59482.70 对 61789.94 kWh)，因为它把无储能时被迫弃掉的 6247.96 kWh 光伏盈余转移到了无光照时段，**提升了光伏本地消纳率**。
2. **效率参数的影响**。单程与往返两种解释构成购电费区间 [33416.53, 35126.95] 元，跨度不足 5%；两种口径下”谷充峰放”的策略结构与零弃光结论均不变。
3. **决策粒度的影响**。决策由 10 分钟粗化为 1 小时时，购电费上升 4.68% 并产生 768.44 kWh 弃光；粗化为 4 小时时上升 24.26% (43648.06 元)，弃光增至 2293.67 kWh。因此采用 10 分钟粒度能够更充分地利用电价和光伏出力的时段差异。
4. **给定的初始储电量已接近最优循环水平**。解除 $E_0 = 6000$ kWh 的固定约束后模型收敛到 $E_0 = 8550$ kWh，购电费仅下降 8.35 元 (0.024%)，无需为此偏离题目给定值。

六、 问题二：预测驱动的滚动日前计划与实时执行模型

6.1 问题二的建模特征

问题二放宽了问题一的确定性设定：**电价仍逐日相同**（重复使用附件 1 的日内曲线），但**小区负载与光伏出力改为附件 2 给出的全年实际值**，且**计划购电不足时须按交易时刻电价的 5 倍紧急补购**。据此，模型需要处理以下三项特征：

1. **信息约束**：每天 0:00 制定计划时，当天的负载与光伏实际值尚未发生，计划只能依据此前历史预测；
2. **代价不对称**：少买需要按 5 倍价格紧急补购，多买仍按计划量全额收费且不予退

款。两侧代价不对称，因此需要按照费用结构配置风险余量；

3. **跨日状态连续**：问题二没有再要求每天 0:00 与 24:00 的储电量相同，因此电池状态按实际运行结果跨日连续传递，不再每日重置为 6000 kWh，也不再直接采用问题一“全天放电量为充电量的 0.81 倍”这一关系（该关系以首尾电量相等为前提）。

相应地，模型由问题一的**单日确定性 MILP**扩展为**三层滚动结构**：预测层（逐日预测负载与光伏）→ 计划层（在风险修正净负荷下求解日 MILP，冻结当天 144 段计划购电量）→ 执行层（逐段观测实际值，因果地决定充放电、紧急购电与弃电，并把实际日末储电量传给次日）。

6.2 数据准备、时间口径与信息时点

附件 2 提供 2025 年 365 天、每天 144 个 10 分钟时段的负载功率与光伏实际功率，经校验无空值、无负值、日期严格递增且恰好覆盖 2025:01:01–2025:12:31。全天合计负载 40524.1 MWh、合计光伏 20251.2 MWh，功率范围分别为 1995.72–7978.88 kW 与 0–10216.20 kW。

时段口径与问题一完全一致：附件 2 的列标签同样采用**区间结束时刻**，故第 j 列对应区间 $[(j-1)\Delta t, j\Delta t)$ 。附件 5 中 **result2.xlsx** 的“计划购电量”表首段标签为 0:10–0:20、末段为 0:00–0:10+1，与问题一模板一样整体错位一格。本文在输出副本中按正文口径统一改写为 0:00–0:10 至 23:50–0:00+1，行序与模板一一对应，原模板文件保持不变。

信息时点纪律：2025 年 1 月被指定为训练与状态预热期，不进入正式结果；正式输出与评价区间为 2 月 1 日至 12 月 31 日，共 334 天。2 月 1 日的初始储电量由 1 月逐日实际运行自然递推得到，而非人为设定。每天的预测、误差分位数与标定参数一律只使用当天之前的已完成数据，计划电量 $g_{n,t}$ 在 0:00 一经确定便不再修改。

6.3 预测模型与数据结构

本文首先分析附件 2 的 365 天数据，并据此选择预测模型：

(1) 负载存在极强的星期结构。把全年按星期分组求日总电量均值，得到周一至周四及周日约为 123900–124144 kWh/日，而周五、周六仅约 78332–78407 kWh/日，两者之比达 1.58 倍。这一结构是**整体水平**上的：两族的日内归一化形状高度相似（相关系数 0.888，同样呈现凌晨低谷、上午高峰、午间回落、午后次高、入夜走低的形态），但并不完全一致——以周五、周六一族为基准，族间逐时段相对差的中位数为 8.9%，在傍晚 17:30 前后最大可达 38.5%（该族在傍晚的相对水平明显更低），而到日末 23:50 处又反向相差 30.9%（周五、周六的相对水平反而更高）。因此若把不同星期的历史日混在一起做加权平均，既会把两族水平“平均掉”、又无法刻画这一形状差异，产生系统性偏差。

(2) 光伏不存在星期结构，而以季节性为主。按星期分组的光伏日电量均值极差仅 1.80%，且与星期无明显对应关系，故光伏预测应侧重**最近**的历史曲线以跟踪季节变化。据此采用“近期加权历史值”的预测形式：

$$\hat{\ell}_{n,t} = \sum_{i \in \mathcal{H}_n} a_{n,i} \ell_{i,t}, \quad \hat{v}_{n,t} = \sum_{i \in \mathcal{H}_n} b_{n,i} v_{i,t}, \quad a_{n,i}, b_{n,i} \geq 0, \quad \sum_i a_{n,i} = \sum_i b_{n,i} = 1 \quad (12)$$

其中权重按“周”衰减， $w_{n,i} = d^{(n-1-i)/7}$ 。负载预测取回看窗口 28 天、周衰减因子 $d = 0.30$ ，并只保留与待预测日同一星期的历史日；光伏预测取回看窗口 7 天、 $d = 0.30$ ，使用全部历史日。两组超参数的取值沿用在 1 月数据上选定，并在正式区间开始前固定，此后不再调整，从而不引入未来信息。1 月预览也表明按星期分组是必要的：负荷曲线的星期内规律远强于其逐日平滑性，仅用全部历史日外推会把休息日与工作日混在一起，净负荷日绝对误差要大出一个量级。

正式区间（2月1日–12月31日）上的预测精度见表15：负载平均绝对误差1915.8 kWh/日（相对1.73%）、光伏2114.2 kWh/日（3.70%）、净负荷3158.8 kWh/日。

上面三个指标都是**逐日全天电量**口径的绝对误差，取绝对值后抹去了方向，而本题的代价恰恰是**不对称的**——低估触发5倍紧急购电，高估只造成弃电。因此另按**逐时段**口径统计两个方向，见表15：实际净负荷高于点预测的时段占48.9%、合计3158.3 kWh/日（低估时段平均低估44.8 kWh/段）；低于点预测的时段占51.1%、合计3167.2 kWh/日。两个方向在频次与电量上接近（48.9%对51.1%，3158.3对3167.2），说明点预测没有明显的系统性偏差。下一节据此利用经验分位数确定风险余量。

令净负荷预测为 $\hat{N}_{n,t} = \hat{\ell}_{n,t} - \hat{v}_{n,t}$ 。负的净负荷表示光伏存在盈余，可用于储能充电，因此建模时保留其原始数值。

6.4 风险余量与分位数选择

设某时段随机净负荷为 N ，计划购电 $g \geq 0$ 。忽略储能时，期望费用为

$$\min_{g \geq 0} pg + 5p \mathbb{E}[(N - g)_+], \quad (x)_+ = \max(x, 0). \quad (13)$$

若 N 的分布连续且最优解位于 $g > 0$ ，一阶条件给出

$$\frac{\partial}{\partial g} pg + 5p \Pr(N > g) = 0 \implies F_N(g) = \frac{5 - 1}{5} = 0.8. \quad (14)$$

即简化模型中的最优计划量对应80%分位数：多买1 kWh的确定成本是 p ，少买1 kWh的期望成本是 $5p \cdot \Pr(N > g)$ ，二者在 $F_N(g) = 0.8$ 处平衡。这一结果给出风险参数一个清楚的经济解释，但它**只在忽略储能与跨时段耦合时严格成立**，因此本文把它作为参数搜索的起点而非结论，最终由实际连续运行的总费用校准。

实践中无法直接得到 N 的分布，转而对**净负荷预测误差**建模。对每个已经结束的历史日期 i ，保存其当时利用更早数据作出的预测，计算误差

$$\varepsilon_{i,t} = (\ell_{i,t} - v_{i,t}) - \hat{N}_{i,t|\text{日期} < i}. \quad (15)$$

这里采用逐日滚动预测产生的样本外误差，以反映实际运行中的预测偏差。取此前28天内对应时段误差的经验 α 分位数 $Q_{\alpha,n,t}$ ，构造风险修正净负荷：

$$\tilde{N}_{n,t} = \hat{N}_{n,t} + Q_{\alpha,n,t}. \quad (16)$$

第 n 天的分位数只取 $\varepsilon_{i,t}$ ($i < n$) 中已结束日期的样本；历史样本不足10个时改取该时段 ± 2 个相邻时段合并后的分位数，冷启动阶段仍有可用估计。直接对净负荷误差建模，还自动保留了负载误差与光伏误差之间的相关性。

6.5 日前计划模型：风险净负荷下的 MILP

每天0:00求解一次。决策变量以**电量 (kWh)** 为单位：计划购电量 $g_{n,t}$ 、参考充电量 $\bar{c}_{n,t}$ 、参考放电量 $\bar{d}_{n,t}$ 、参考剩余电量 $\bar{w}_{n,t}$ 、参考储电量 $\bar{E}_{n,t}$ ，以及模式变量 $z_{n,t} \in \{0, 1\}$ 。带横线的变量表示**日前参考计划**，不是实际执行结果。

目标函数除购电费外，加入日末储备不足的软惩罚：

$$\min \sum_{t=1}^{144} p_t g_{n,t} + \lambda \xi_n, \quad \xi_n \geq E^{\text{tar}} - \bar{E}_{n,144}, \quad \xi_n \geq 0. \quad (17)$$

$E^{\text{tar}} = 6000$ kWh 是希望为次日保留的参考电量， λ (元/kWh) 是储备不足的规划代价。该项用于表征日末储电量对后续运行的影响，仅参与计划优化，不计入实际购电

费。本文并不预设 λ 的取值，而是把它与 α 、 ρ 一并交给数据标定（见 6.7 节）。

约束条件包括四组：

$$\begin{aligned}
g_{n,t} + \bar{d}_{n,t} &= \tilde{N}_{n,t} + \bar{c}_{n,t} + \bar{w}_{n,t}, & g_{n,t} &\geq 0, \bar{w}_{n,t} \geq 0 \\
\bar{E}_{n,t} &= \bar{E}_{n,t-1} + \eta_c \bar{c}_{n,t} - \frac{\bar{d}_{n,t}}{\eta_d}, & \bar{E}_{n,0} &= E_{n,0}^{\text{actual}} \\
E_{\min} &\leq \bar{E}_{n,t} \leq E_{\max}, & E_{\min} &= 1200, E_{\max} = 10800 \\
0 &\leq \bar{c}_{n,t} \leq M z_{n,t}, \quad 0 \leq \bar{d}_{n,t} \leq M(1 - z_{n,t}), & z_{n,t} &\in \{0, 1\}
\end{aligned} \tag{18}$$

其中 $M = 5000\Delta t = 2500/3$ kWh, $\eta_c = \eta_d = 0.90$ ，与问题一保持同一套母线侧充放电口径。注意平衡式中使用的是**风险修正后**的净负荷，因此 $\bar{w}$ 表示规划阶段的剩余量，实际弃电量在执行层根据观测值重新计算。 $\bar{c}$ 与 $\bar{w}$ 均不收费；允许 $\bar{w}$ 包含未使用的计划购电量，以保证预测偏高时模型仍保持可行。

按变量排列，模型 (18) 共 $6 \times 144 + 1 = 865$ 个决策变量，其中 144 个为二元变量 $z_{n,t}$ ，其余为连续变量，故是**混合整数线性规划**。结构矩阵与问题一一样只依赖时段划分，可**预先装配一次**，此后每天只更新等式右端项，单次求解约 46 ms。

6.6 实时执行模型：因果反馈规则

执行层使用当天 0:00 已确定的计划、当前已观测的功率和当前储电量。设使用储能前的电量盈余为

$$b_{n,t} = g_{n,t} + v_{n,t}^{\text{actual}} - \ell_{n,t}^{\text{actual}}, \tag{19}$$

$b > 0$ 表示计划购电与光伏有剩余， $b < 0$ 表示出现缺口。

(1) 有剩余时充电。充电量由本段剩余电量、充电功率和储能容量共同确定：

$$c_{n,t}^{\text{actual}} = \min\left\{(b_{n,t})_+, M, \frac{E_{\max} - E_{n,t-1}^{\text{actual}}}{\eta_c}\right\}. \tag{20}$$

(2) 有缺口时按储备线放电。在 0:00 即可算出当天的储备线

$$R_{n,t} = E_{\min} + \rho(\bar{E}_{n,t} - E_{\min}), \quad 0 \leq \rho \leq 1, \tag{21}$$

实际放电量为

$$d_{n,t}^{\text{actual}} = \min\left\{(-b_{n,t})_+, M, \eta_d(E_{n,t-1}^{\text{actual}} - R_{n,t})_+\right\}. \tag{22}$$

$\rho = 0$ 表示尽可能用现有储能补足当前缺口； $\rho = 1$ 表示将当段参考电量作为放电下限，以保留更多储能。参数 ρ 由历史运行费用标定。

(3) 紧急购电、弃电与真实电池状态。

$$r_{n,t} = (-b_{n,t} - d_{n,t}^{\text{actual}})_+, \quad w_{n,t}^{\text{actual}} = (b_{n,t} - c_{n,t}^{\text{actual}})_+, \tag{23}$$

$$E_{n,t}^{\text{actual}} = E_{n,t-1}^{\text{actual}} + \eta_c c_{n,t}^{\text{actual}} - \frac{d_{n,t}^{\text{actual}}}{\eta_d}. \tag{24}$$

该规则自动避免同时充放电，并保证实际逐段能量守恒：

$$g_{n,t} + r_{n,t} + v_{n,t}^{\text{actual}} + d_{n,t}^{\text{actual}} = \ell_{n,t}^{\text{actual}} + c_{n,t}^{\text{actual}} + w_{n,t}^{\text{actual}}. \tag{25}$$

w^{actual} 允许来自光伏，也允许来自已经付费的计划购电，因此不设置“弃电量只能小于光伏电量”的限制。

(4) **跨日状态传递**。次日初始电量取当天实际执行后的日末电量：

$$E_{n+1,0}^{\text{actual}} = E_{n,144}^{\text{actual}}. \quad (26)$$

由式 (24) 对全天求和可得状态累计恒等式

$$E_{n,144}^{\text{actual}} - E_{n,0}^{\text{actual}} = \eta_c \sum_{t=1}^{144} c_{n,t}^{\text{actual}} - \frac{1}{\eta_d} \sum_{t=1}^{144} d_{n,t}^{\text{actual}}, \quad (27)$$

它是本文逐日校验的核心关系之一。只有当某日首尾电量恰好相同时， $\sum_t d_{n,t} = 0.81 \sum_t c_{n,t}$ 才成立；问题二中绝大多数日子并不满足，因此该比例关系不适用于一般的跨日运行情形。

6.7 参数标定：用过去连续区间的实际总费用选参

模型含三个需要选择的参数：风险分位数 α 、储备线系数 ρ 与日末储备代价 λ 。标定分为两步。**第一步**在预热期 1 月对 (α, ρ, λ) 的全部 180 个组合做一次联合标定，用窗口内的实际总费用选出 λ ，并在正式区间内保持不变。该参数用于刻画日末储电量的后续经济价值。**第二步**从正式区间第一天起每隔 42 天重选一次 (α, ρ) ——对每个组合，从该日之前的 35 天窗口起点出发，用窗口起点日 0:00 的实际储电量作初值时序仿真整个窗口，比较窗口内**包含 5 倍紧急购电费的实际总费用**，取最小者。首次标定（2 月 1 日）的窗口受数据起点 1 月 1 日限制，实际只有 31 天；其余 7 次均为完整的 35 天。参数网格为

$$\begin{aligned} \alpha &\in \{0.70, 0.75, 0.80, 0.85, 0.90\}, \\ \rho &\in \{0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}, \\ \lambda &\in \{0, 0.1, 0.2, 0.3, 0.5, 0.8\}, \end{aligned} \quad (28)$$

第一步遍历全部 180 个组合，第二步遍历其中 $5 \times 6 = 30$ 个 (α, ρ) 。标定所用的历史和时序仿真窗口均截止于标定日前，满足信息时序约束。标定结果见表 17。

6.8 求解结果与校验

6.8.1 六项一致性检验

按前述要求，对 365 天的完整运行逐日执行六项检查，表 16 给出结果，六项检验均通过：信息时点（历史日下标上界小于当天）、物理可行性（逐段供需平衡最大残差 2.3×10^{-13} kWh，越界量与同时充放电时段均为 0）、跨日连续性、状态累计（最大偏差 1.3×10^{-11} kWh）、费用一致性（最大偏差 0 元）与汇总一致性（390 个紧急购电事件合并前后电量相等）。逐条明细见附录第 1.13 节。

6.8.2 总体结果

表 15 给出正式区间的总体结果。334 天中计划购电量 21463158.37 kWh、计划购电费 13184221.29 元；紧急购电量 140301.24 kWh、紧急购电费 755499.17 元；**合计购电费 13939720.46 元**，平均购电单价约 0.645 元/kWh。紧急购电费占总费用的 5.42%，334 天中有 143 天发生紧急购电。全年弃用电量 2463544.98 kWh，期末储电量 1559.72 kWh，储电量全程位于 [1200, 10800] kWh 区间内。

表 3 微网在指定日期的计划购电量、全天购电量与购电费（问题二）

日期	10:00	12:00	14:00	16:00	18:00	20:00
2025.3.20	0.00	569.14	0.00	488.39	709.72	0.00
2025.6.21	0.00	0.00	0.00	321.79	472.16	0.00
2025.9.23	0.00	469.77	0.00	529.04	749.13	0.00
2025.12.21	0.00	991.98	0.00	845.88	735.33	0.00
全天汇总（单位：kWh，费用单位：元）						
日期	计划购电量		紧急购电量	合计购电费		
2025.3.20	67,400.98		14.56	41,084.75		
2025.6.21	40,911.64		0.00	24,901.80		
2025.9.23	66,102.01		1,253.46	48,677.65		
2025.12.21	99,337.46		0.00	64,146.24		

表 4 储能设备在指定日期的充放电量及 0:00 和 24:00 的储电量（问题二）

时段	2025.3.20		2025.6.21		2025.9.23	
	充电	放电	充电	放电	充电	放电
0:00-4:00	8,166.69	149.45	1,616.53	0.00	8,876.21	66.21
4:00-8:00	1,975.10	5,511.27	1,480.09	1,719.95	1,858.70	6,490.18
8:00-12:00	6,181.55	2,777.38	5,966.02	0.00	4,923.36	1,896.62
12:00-16:00	4,205.72	429.17	0.00	0.00	5,549.91	641.70
16:00-20:00	242.48	5,746.30	0.00	4,755.36	84.06	5,441.21
20:00-24:00	547.83	2,977.22	2,388.03	2,485.90	323.24	2,795.73
0:00 储电量	1,982.53		4,554.68		1,236.54	
24:00 储电量	1,624.64		4,903.38		1,433.10	

表 5 2025 年 12 月 21 日储能设备的充放电量及 0:00 和 24:00 的储电量

时段	充电量/kWh	放电量/kWh
0:00-4:00	8,789.00	77.56
4:00-8:00	2,130.34	1,430.76
8:00-12:00	5,865.62	6,819.25
12:00-16:00	6,282.00	2,220.11
16:00-20:00	0.00	5,616.56
20:00-24:00	545.96	2,846.36
0:00 储电量/kWh	1,759.38	
24:00 储电量/kWh	1,888.12	

表 6 微网在指定日期的紧急购电量（问题二，按连续时段合并）

日期	紧急购电时间段	紧急购电量/kWh	该日紧急费/元
2025.3.20	20:30-20:40	14.56	101.56
2025.6.21	18:50-20:40	无	0.00
2025.9.23	20:50-22:00	236.10	7,777.51
	22:20-22:30	32.16	
2025.12.21	无	0.00	

注：紧急购电量按框架要求逐段计算后，将同日连续发生紧急购电的时段合并为一个事件；合并后电量与逐段之和严格相等。

6.8.3 指定日期的结果

四个指定日期（3月20日周四、6月21日周六、9月23日周二、12月21日周日）覆盖了不同星期类型与季节条件，具体结果见表3、表4、表5与表6。在紧急购电方面，四个日期中有两个**全天无紧急购电**（6月21日与12月21日）：9月23日发生3个事件（18:50–22:30，合计1253.46 kWh、7777.51元），3月20日仅1个事件（14.56 kWh、101.56元）。事件全部出现在晚间18:50以后——这两个日期的储能都在20:40触及下限1200 kWh，之后预测未覆盖的缺口只能按5倍电价现场补足。逐日的结构分析见附录第1.15节。

6.9 结果分析

6.9.1 预测误差如何转化为紧急购电成本

逐时段核对表明，预测偏低会在储能可放电量不足时转化为紧急购电成本。其触发量由实际净负荷相对风险修正值的超出量与储能剩余可放电量共同决定。3月20日与9月23日的储能都在20:40达到下限1200 kWh，其后分别出现1段与19段紧急购电；而6月21日与12月21日虽分别有1段与18段超出风险净负荷，储电量最低仍分别停在2754 kWh与1401 kWh、未触及下限，因而全天未发生紧急购电。这一结果说明分位数计划需要与储能可用容量结合分析。逐段分解及“ Q_α 有8.5%的时段为负”这一细节，见附录A。

6.9.2 计划购电与紧急购电的费用贡献

图2给出四个指定日期的计划购电量、紧急购电量与电价的对照。计划购电量（蓝色柱）呈现与电价高度相关的形态：在0:00–6:00与22:00–24:00的低价时段最大，在7:50–9:10的早高峰被压到零。而紧急购电量（红色斜纹）则以很细的柱体叠加在计划量之上，仅在个别时段出现，说明在风险余量的保护下绝大多数时段的计划量已经足够。

计划购电量并不总是等于实际净负荷：在中午光伏富余的时段，12:00附近仍保留数百kWh的计划购电。这与问题一中出现的现象同源。该时段电价处于局部低谷，提前购入并留待傍晚高价时段使用可以降低费用；问题二中的该项决策依据风险修正后的净负荷作出。

图2 指定日期的计划购电量、紧急购电量与电价

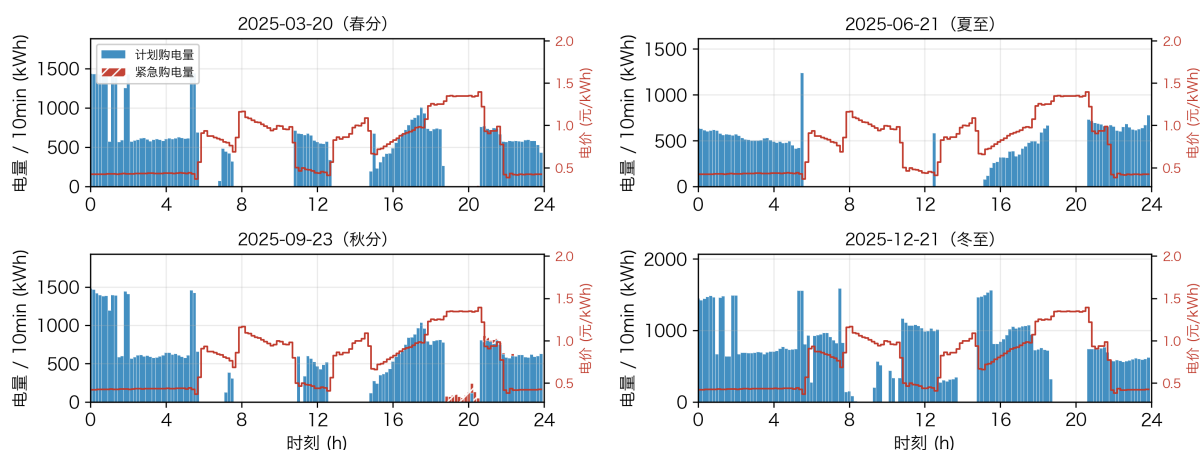


图2 指定日期的计划购电量、紧急购电量与电价

6.9.3 预测误差对储能轨迹的扰动

四个指定日期的日前参考储电量 $\bar{E}$ 与实际执行后的储电量 E 全程未越出 [1200, 10800] kWh，且日初与日末储电量各不相同（3月20日 1982.53 → 1624.64 kWh，6月21日 4554.68 → 4903.38 kWh），与问题二采用跨日连续状态且不设置日循环约束

的设定一致。轨迹图见附录 A。

6.9.4 全年费用分解与对照策略

为量化风险余量的价值，构造三组对照策略（**预测均值策略**令 $\alpha = \text{None}$ 完全不加风险余量；**无储能策略**仍按 5 倍价格补缺但不启用储能；**事后理想策略**用当天实际净负荷替换预测值求解同一 MILP），全部使用**同一预测模型、同一实时执行规则、同一评价区间（2–12 月共 334 天）**，并各自单独标定参数。表 13 给出四者对比：本文策略 13939720 元，较预测均值策略 14903679 元省 963959 元（6.47%）、较无储能策略 18519391 元省 4579670 元（24.73%）。事后理想策略的费用为 12110855 元，比本模型低 13.12%；其 334 天的日末储电量均为下限 1200 kWh，且沿用主策略的日初储电量，因此该结果作为理想信息条件下的费用参照。

预测均值策略的紧急购电费为 2708867 元，是本文策略 755499 元的 3.6 倍，而其计划购电费较低（12194812 元对 13184221 元）。风险余量通过增加计划购电支出来降低高倍价紧急购电支出。逐月分解、月费用与紧急购电费排序不一致的成因、以及四个策略期末储电量并不相同（本文策略 1559.7 kWh，预测均值策略 5960.7 kWh，无储能策略 6000.0 kWh）。为比较不同策略的跨期储能消耗，表 14 另按统一参考电价折算净动用库存；折算后本文策略仍低于预测均值和无储能策略。逐月分解与费用结构见附录 A。

6.9.5 标定结果对理论分析的验证

表 17 给出 8 次滚动标定的选中参数。结果集中： α 在 8 次标定中**全部落在 0.70–0.90 之间**，其中 0.75 出现 4 次、0.70 出现 2 次、0.80 与 0.90 各 1 次。这一结果与式 (14) 的理论预测**方向一致**：由 5 倍紧急电价导出的最优分位数在 0.8 附近，而完全基于 2025 年实际数据、按真实总费用选出的参数也落在 0.70–0.90 区间。理论结果与数据标定具有一致性。

1 月联合标定选出的 $\lambda = 0.00$ 被冻结整年。这说明在 **35 天窗口的实际总费用**这一评价口径下，日末储备惩罚并未带来净收益：窗口内的时序仿真已经包含了储备不足在后续日期引发的紧急购电代价，而电池容量上限（10800 kWh）本身构成了自然的约束；在所用标定窗口内，增加储备惩罚会提高运行费用。变量 ξ 使日末储备的作用能够由数据标定。 ρ 在 8 次标定中取 0.9 五次、0.5 两次、0.8 一次，并未收敛到某个固定值，同样说明较高的储备线并不必然更省钱。

6.9.6 弃用电量的成因

全年弃用电量 2463544.98 kWh，占同期光伏发电量 19068890.0 kWh 的 12.9%。题目未对弃电设置惩罚或收益，它不带来额外扣款，但仍是日前计划与实时运行之间偏差的**指示器**：其中 71.8%（1769824 kWh）是光伏富余超出储能吸纳能力；另 28.2%（693721 kWh）发生在光伏没有富余的时段，来自实际负载低于预测值时未使用的计划购电量。该部分计划购电成本为 466267 元（占全年计划购电费的 3.54%）。敏感性分析显示， α 由 0.80 降至 0.50 时，弃电由 2628.6 降至 1493.7 MWh，紧急购电量由 177.9 增至 636.1 MWh，总费用由 1403.8 万元升至 1508.8 万元。完整分解与运营含义见附录 A 与附录 L。

6.9.7 本节小结

按时间顺序的全年时序仿真表明，风险余量、日前 MILP 与因果执行规则能够协同利用储能的跨时段调节能力，并将运行费用较无储能基准降低 24.73%。参数均由决策日前的连续历史窗口标定，完整的基准对照与成因分解见附录 L。

七、 问题三：多版本预报驱动的滚动购电与调整模型

7.1 模型框架：购电量定义与分时调整方式

问题三沿用问题二的储能、能量平衡与效率约束，并增加日内购电调整。为统一计划、调整和结算口径，定义以下三个变量：

- g_t^0 ：当天 0:00 依据当时唯一可得的 0:00 预报确定的 **初始计划购电量**，确定后作为后续调整的结算基准；
- x_t ：该时段**调整后生效的常规购电量**，表示完整购电量，不含紧急购电；
- r_t ：实际运行中额外发生的**紧急购电量**。

于是全天计划量为 $G^{\text{plan}} = \sum_t g_t^0$ ，最终生效常规量为 $G^{\text{eff}} = \sum_t x_t$ ，紧急量为 $G^{\text{emg}} = \sum_t r_t$ ，微网实际购入的总电量为 $G^{\text{recv}} = G^{\text{eff}} + G^{\text{emg}}$ 。

为明确各次调整的作用范围，将一天划分为四个六小时调整区间，每个区间在起始时刻根据最新预报确定购电量：

决策时刻	本时刻确定的调整区间	时段索引
0:00	0:00–6:00	0–35
6:00	6:00–12:00	36–71
12:00	12:00–18:00	72–107
18:00	18:00–24:00	108–143

每个十分钟时段相对于初始计划量 g_t^0 结算一次。0:00 生成全天初始计划，此后的求解同时计算剩余时段的参考方案，其中当前六小时区间的购电量立即生效，后续区间的结果仅用于规划储能轨迹，并在相应发布时刻根据新信息重新计算。

7.2 预报数据的时间口径

附件 3 的每条记录给出“预报 1 小时”至“预报 24 小时”共 24 个功率值。为判定其表示目标整点功率还是目标小时平均功率，本文将预报值与附件 2 的实际光伏功率按不同时间平移对齐，比较全天平均绝对偏差，在正式区间内得到整点对齐时 189.81 kW、前移一小时 808.24 kW、按小时平均 267.40 kW（正式区间 2025-02-01 至 2025-12-31 的十分钟平均绝对偏差）。扫描脚本与输出见支撑材料 `p3_alignment_scan.py` 与 `p3_alignment_scan.json`。整点口径的偏差最小且差距显著，故**按整点功率处理**。

插值左端点。发布时刻为 $\tau \in \{0, 6, 12, 18\}$ 的版本给出目标 $\tau + 1, \dots, \tau + 24$ 共 24 个整点功率。将其转换到十分钟网格时，以发布时刻所在时段的实际光伏功率作为左端点（0:00 版取 0，因为附件 2 中全年每天 0:00–0:10 的实际光伏恒为零），节点为 $(\tau, \tau + 1, \dots, \tau + 24)$ ，在相邻节点之间作分段线性插值，只在 $\tau < \text{时段中点} \leq \tau + 24$ 内取值，并将结果限制为非负值。该处理仅使用发布时刻之前已经获得的实测信息。

负荷预测与预报版本管理。负荷由历史实际值外推，优先采用上一周同一星期 ($d-7$) 的曲线；历史长度不足时依次采用前一日曲线和已有历史均值。第 τ 次求解仅使用发布时间不晚于 τ 的光伏预报。

图 3 把四个发布版本对同一个日期的光伏预报与事后实际值画在一起：0:00 版覆盖全天，6:00、12:00、18:00 版的覆盖时段依次缩短。后文分别从预测误差和费用变化两个角度评价各版本的作用。

7.3 计费口径与调整模型

题目规定：“计划购电量高于调整购电量的部分，违约电价是交易时刻电价的 50%；调整购电量高于计划购电量的部分，超出部分的电价是交易时刻电价的 1.5 倍”，而“总的购电费用包括计划购电费用、紧急购电费用和调整购电量的相关费用”。题目未明确调减时是否退回被取消部分的原购电费用，因此分别考虑退款和不退款两种计费口径。

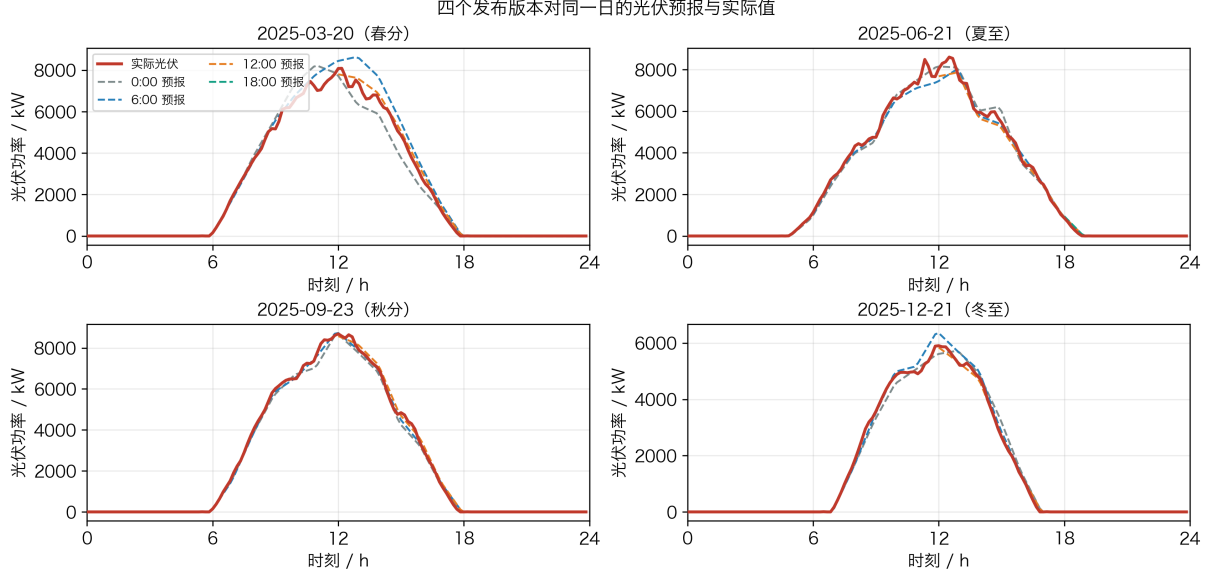


图3 同一目标日期的多版本光伏预报与事后实际值

主口径（退款：取消部分退回原价，只收 50% 违约金）。

$$J^{\text{refund}} = \sum_t \left[p_t g_t^0 + 1.5p_t(x_t - g_t^0)_+ - 0.5p_t(g_t^0 - x_t)_+ + 5p_t r_t \right], \quad (y)_+ = \max(y, 0).$$

其中，上调部分按 $1.5p_t$ 计费；调减部分退回原价并支付 $0.5p_t$ 的违约金，因此净退费为 $0.5p_t$ 。该口径允许购电量双向调整。

敏感性口径（不退款：调减仍付原价，另收 50% 违约金）。

$$J^{\text{norefund}} = \sum_t \left[p_t g_t^0 + 1.5p_t(x_t - g_t^0)_+ + 0.5p_t(g_t^0 - x_t)_+ + 5p_t r_t \right].$$

此时调减仍需支付原计划费用及违约金，因此最优方案不会主动调减购电量。两种计费口径采用相同的分段线性化方法：引入非负的增量与减量变量 $u_t, v_t \geq 0$ ，令

$$x_t - g_t^0 = u_t - v_t, \quad 0 \leq v_t \leq g_t^0,$$

则调价项为 $1.5p_t u_t - 0.5p_t v_t$ （主口径）或 $1.5p_t u_t + 0.5p_t v_t$ （不退款口径），均无需引入二元变量。退款口径下，同时增加 u_t 和 v_t 会使费用增加 $p_t \delta$ ，故最优解中二者不会同时为正。两种计费口径的全年结果见表 21，总费用相差 1.49%。

在每个调整时点，初始计划费用及已执行时段的调整费和紧急购电费均为已发生费用，不再进入剩余时段的优化目标。

7.4 计划与调整两个线性规划

0:00 初始计划。沿用问题二的风险净负荷思想，以 $\tilde{N}_t$ 为需求求解

$$\min \sum_{t=0}^{143} p_t g_t^0 + \lambda \xi, \quad \xi \geq E^{\text{tar}} - \bar{E}_{143}, \quad \xi \geq 0,$$

$$g_t^0 + \bar{d}_t = \tilde{N}_t + \bar{c}_t + \bar{w}_t, \quad \bar{E}_t = \bar{E}_{t-1} + \eta_c \bar{c}_t - \frac{\bar{d}_t}{\eta_d}, \quad 1200 \leq \bar{E}_t \leq 10800,$$

$$0 \leq \bar{c}_t \leq M, \quad 0 \leq \bar{d}_t \leq M, \quad M = 5000 \times \frac{1}{6} = \frac{2500}{3} \text{ kWh}.$$

其中 $\bar{c}_t, \bar{d}_t, \bar{E}_t$ 表示参考储能轨迹, $\bar{w}_t$ 表示规划弃电。该模型不设置紧急购电变量; 0:00 时仅使用已发布预报生成风险修正后的全天计划, 后续偏差由调整层与执行层处理。购电量 g_t^0 的有效上限由能量平衡、储能容量和购电成本共同确定。

发布时点的调整模型。 设 $\mathcal{T}_\tau$ 为当前时刻之后尚未执行的时段, 以 x_t 为候选完整购电量:

$$\min \sum_{t \in \mathcal{T}_\tau} (1.5p_t u_t - 0.5p_t v_t) + \sum_{t \in \mathcal{T}_\tau} 5p_t \bar{r}_t + \lambda \xi,$$

$$x_t = g_t^0 + u_t - v_t, \quad 0 \leq v_t \leq g_t^0, \quad x_t + \bar{r}_t + \bar{d}_t = \tilde{N}_t^{(\tau)} + \bar{c}_t + \bar{w}_t,$$

储能递推式与上下界、充放电上限与计划模型完全相同。

两个模型均为线性规划。在 $\eta_c \eta_d = 0.81 < 1$ 且购电成本非负时, 同时充放电会增加能量损失, 最优解自然满足充放电互斥。每次求解以当前实际储电量为初值; $\tilde{N}_t^{(\tau)}$ 与 g_t^0 使用不同的信息集: 前者用最新发布的预报和最新误差分位数算出, 后者在 0:00 就冻结。更新预报后, 即便不改变 x_t 也可以重新安排储能以降低未来缺口。因此, 每次求解后更新参考轨迹 $\bar{E}_t$, 仅将当前调整区间的 x_t 写入生效购电量。

执行层。 每十分钟的实际充放电与紧急补购沿用问题二的因果规则。每个决策时刻依据当前状态重新规划后续轨迹, 符合模型预测控制的基本思想 [6]。令使用储能前的盈余为 $b_t = x_t + v_t^{\text{act}} - \ell_t^{\text{act}}$, 用当前参考轨迹构造储备线 $R_t = \text{clip}(1200 + \rho(\bar{E}_t - 1200), 1200, 10800)$, 则

$$c_t = \min \left\{ (b_t)_+, M, \frac{10800 - E_{t-1}}{\eta_c} \right\}, \quad d_t = \min \left\{ (-b_t)_+, M, \eta_d(E_{t-1} - R_t)_+ \right\},$$

$$r_t = (-b_t - d_t)_+, \quad w_t = (b_t - c_t)_+, \quad E_t = E_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d},$$

并满足逐段守恒 $x_t + r_t + v_t^{\text{act}} + d_t = \ell_t^{\text{act}} + c_t + w_t$ 。规则只依赖当前与历史信息: b_t 用到当段实际负载与光伏, E_{t-1} 是上一段的实际状态。当日实际日末电量直接作为次日初值, 不要求日循环; 1 月作为预热期, 2 月 1 日的初值由 1 月的实际运行得到。

7.5 参数标定: 1 月联合标定与样本外检验

需要标定的参数包括风险分位数 α 、储备线比例 ρ 、日末储备引导权重 λ 与分位数窗口 W 。以 2025 年 1 月为预热期完成联合标定, 所得参数在 2 月 1 日至 12 月 31 日的评价区间内保持不变。

标定面为 $\alpha \in \{0.4, \dots, 0.8\} \times \rho \in \{0.35, 0.5, 0.7\} \times W \in \{14, 21, 28\}$ 的 45 点网格, 每组参数均在 1 月进行完整时序仿真, 取含 5 倍紧急费用的实际总费用最小者; λ 另行单独扫描。结果见表 22: 标定面在 α 方向有明确的极小点 $\alpha = 0.6$, 在 ρ 与 W 方向则相当平坦。

参数取值。 1 月标定面上的最优取值为 $(\alpha, \rho, W) = (0.6, 0.35, 14)$, 1 月费用 1,763,097 元, 比本文采用的 $(0.6, 0.5, 28)$ 的 1,772,822 元低 9,725 元。两组参数在完整评价区间内的费用分别为 13,461,840 元和 13,418,626 元。考虑到 1 月样本长度及标定面在 ρ 、 W 方向上的平坦性, 采用中心参数 $(\alpha, \rho, \lambda, W) = (0.6, 0.5, 1.0, 28)$, 并通过表 20 报告单参数灵敏度。 $\lambda = 1.0$ 与 $\lambda = 0.5$ 在评价区间上的费用相差 9,852 元 (0.07%); 非零权重可使日末储电量高于 $\lambda = 0$ 时的 1,806 kWh, 从而保留跨日调节能力。

题目未设置预报获取成本, 因此以各预报版本对实际总费用的影响、跨月份稳定性和版本间的替代关系评价其使用价值。

表 7 指定日期六个时段的初始计划购电量 g_t^0 、最终生效购电量 x_t 、调整量与紧急购电量（单位：kWh）

日期	时段	初始计划 g_t^0	最终生效 x_t	调整量	紧急购电 r_t
2025.3.20	10:00-10:10	0.00	0.00	0.00	0.00
	12:00-12:10	545.31	545.31	0.00	0.00
	14:00-14:10	0.00	0.00	0.00	0.00
	16:00-16:10	625.46	505.51	-119.95	0.00
	18:00-18:10	700.79	709.98	9.20	0.00
	20:00-20:10	0.00	0.00	0.00	0.00
2025.6.21	10:00-10:10	0.00	0.00	0.00	0.00
	12:00-12:10	0.00	0.00	0.00	0.00
	14:00-14:10	0.00	0.00	0.00	0.00
	16:00-16:10	164.25	157.01	-7.24	0.00
	18:00-18:10	0.00	0.00	0.00	0.00
	20:00-20:10	0.00	0.00	0.00	0.00
2025.9.23	10:00-10:10	0.00	0.00	0.00	0.00
	12:00-12:10	363.24	363.24	0.00	0.00
	14:00-14:10	0.00	0.00	0.00	0.00
	16:00-16:10	569.83	513.35	-56.49	0.00
	18:00-18:10	761.89	761.89	0.00	0.00
	20:00-20:10	0.00	0.00	0.00	0.00
2025.12.21	10:00-10:10	0.00	0.00	0.00	0.00
	12:00-12:10	964.56	919.88	-44.69	0.00
	14:00-14:10	0.00	0.00	0.00	0.00
	16:00-16:10	804.23	813.23	9.01	0.00
	18:00-18:10	738.59	719.75	-18.84	0.00
	20:00-20:10	0.00	0.00	0.00	0.00

续表：全天费用分解（单位：元）。当日计划费为 $\sum_t p_t g_t^0$ ；调整费为 $p_t g_t^0 + 1.5 p_t (x_t - g_t^0)_+ - 0.5 p_t (g_t^0 - x_t)_+$ 与计划费之差；紧急费为 $\sum_t 5 p_t r_t$ 。三项相加即当日实际总费用。

日期	初始计划量	最终常规量	紧急购电量	计划费	调整费	当日合计费用
2025.3.20	68,107.8	66,649.1	0.0	41,438	1,602	43,040
2025.6.21	35,303.3	35,603.4	0.0	19,926	451	20,376
2025.9.23	68,803.7	66,960.7	333.6	42,556	470	44,957
2025.12.21	96,566.3	95,397.7	144.7	61,811	306	63,085

表 8 储能设备在指定日期的实际充放电量（按四小时汇总，单位：kWh）

时段	2025.3.20		2025.6.21		2025.9.23		2025.12.21	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00-4:00	3,928.5	298.0	545.4	133.8	4,759.2	105.3	5,089.6	218.3
4:00-8:00	1,178.2	5,534.6	493.3	4,393.3	915.9	6,684.8	1,674.7	1,596.6
8:00-12:00	3,657.1	2,777.4	9,175.6	0.0	5,295.8	1,896.6	5,283.5	7,522.8
12:00-16:00	6,440.1	373.6	0.0	0.0	5,417.7	530.4	7,793.6	2,236.3
16:00-20:00	178.9	5,259.9	108.7	5,623.8	54.6	6,009.3	40.5	5,940.3
20:00-24:00	5,616.2	2,915.0	6,089.8	2,534.6	5,623.6	2,380.0	5,433.0	2,767.5
0:00 储电量	6,637.8		6,637.4		5,935.7		5,802.8	
24:00 储电量	6,471.9		7,313.9		6,233.2		6,050.8	

表 9 2025 年 12 月 21 日储能设备的实际充放电量与 0:00、24:00 储电量

时段	充电量/kWh	放电量/kWh
0:00-4:00	5,089.62	218.34
4:00-8:00	1,674.66	1,596.63
8:00-12:00	5,283.46	7,522.76
12:00-16:00	7,793.61	2,236.33
16:00-20:00	40.52	5,940.30
20:00-24:00	5,433.04	2,767.55
0:00 储电量/kWh	5,802.85	
24:00 储电量/kWh	6,050.81	

表 10 微网在指定日期的紧急购电事件（问题三，按同日连续时段合并）

日期	紧急购电时间段	紧急购电量/kWh	该日紧急费/元
2025.3.20	无		0.00
2025.6.21	无		0.00
2025.9.23	20:20-20:40	161.86	1,128
	20:50-21:40	171.78	
2025.12.21	7:50-8:00	34.67	201
	20:30-20:40	110.01	

注：紧急购电量逐十分钟按 $5p_t r_t$ 计费后再把同日连续时段合并成事件，合并不改变费用；合并后电量与逐段之和严格相等。

7.6 求解结果与一致性检验

以 2025 年 1 月为预热器、2 月 1 日–12 月 31 日共 334 天为正式区间，主策略（使用全部四个发布时刻）的结果为 **合计购电费** 13,418,626.35 元。按性质拆开：计划购电费 $\sum_t p_t g_t^0 = 12,583,676.62$ 元；上调增费 $\sum_t 1.5p_t u_t = 804,431.51$ 元；调减退费 $\sum_t 0.5p_t v_t = 242,417.95$ 元；两项相抵得 **调整净额** 562,013.56 元，故 **计划调整费合计** 13,145,690.17 元；**紧急购电费** 272,936.18 元。

电量方面，初始计划量 $G^{\text{plan}} = 20,762,854.17$ kWh，最终生效常规量 $G^{\text{eff}} = 20,643,456.67$ kWh（其中上调 656,263.05 kWh、调减 775,660.56 kWh），紧急购电量 $G^{\text{emg}} = 47,350.77$ kWh，故微网实际收到 $G^{\text{recv}} = 20,690,807.44$ kWh；储能充电 6,689,851.97 kWh、放电 5,419,388.14 kWh、弃电 1,481,154.12 kWh；评价区间首日 0:00 储电量 6,473.78 kWh，末日期末储电量 5,798.18 kWh。全年共正式提交调整 $334 \times 3 = 1002$ 次、求解线性规划 $334 \times 4 = 1336$ 次，与四个六小时调整区间各求解一次的设定一致。

对主策略的 334 个运行日逐日执行九项一致性检验，结果见表 23。各项检验的异常天数均为零。逐段供需平衡的最大残差为 2.27×10^{-13} kWh，储能状态递推的最大绝对偏差与两项费用按独立路径复算的偏差为 0。储电量、充放电功率、互斥关系、购电量非负性和跨日状态衔接均满足约束。

7.7 结果分析

(1) 预报组合对费用构成的影响。表 18 与图 4 给出了八种预报组合的费用。在全部 12 条”多用一个预报版本”的覆盖边上，**总费用严格不增**（由仅用 0:00 的 14,977,809 元降到全部使用的 13,418,626 元，省 1,559,183 元、10.41%），紧急费由 2,382,673 元降到 272,936 元，降幅为 88.5%。 G^{plan} 在八种方式间仅相差 0.16%，表明费用下降主要来自新增预报将部分紧急购电转化为价格较低的计划调整。计划调整费不具有相同的单调关系：它在 12 条覆盖边上有 10 条上升，却在 $\{18\} \rightarrow \{12, 18\}$ (13,187,014 $\rightarrow$ 13,033,851 元) 与 $\{6, 18\} \rightarrow \{6, 12, 18\}$ (13,259,141 $\rightarrow$ 13,145,690 元) 这两条边上下降。12:00 预报改变了储能与购电安排，使部分预期缺口由已有电量覆盖，因而调整净额与总费用同时下降。

(2) 各预报版本的边际贡献。定义 $\Delta_h(S) = C(S) - C(S \cup \{h\})$ ，表示已有预报集合 S 时加入时刻 h 预报所降低的费用（完整结果见附录第 1.8 节）。从零开始单独引入时，18:00 版省 117.82 万元、12:00 版省 94.59 万元、6:00 版省 49.78 万元；并且 18:00 版在 $\{6\}$ 、 $\{12\}$ 、 $\{6, 12\}$ 之上仍然都是三者中贡献最大（分别降低 94.69、45.17、46.23 万元）。该排序与表 19 的预测精度排序不同：在白天窗口 12:00–18:00 上，12:00 版的平均绝对误差最小 (181.3 kW)，6:00 版为 304.9 kW，0:00 版为 445.8 kW。18:00 预报作用于 18:00–24:00，该区间同时包含电价高峰和日末储能安排，因此产生较高的边际费用贡献。相比之下，12:00 版的边际价值在 $\{6, 18\}$ 之上只有 11.44 万元 (0.85%)，远小于它单独引入时的 94.59 万元：其对下午光伏功率的修正与 6:00、18:00 预报存在较多重叠。

按移除各版本后的费用增加量排序：移除 18:00 版增加 46.23 万元、移除 6:00 版增

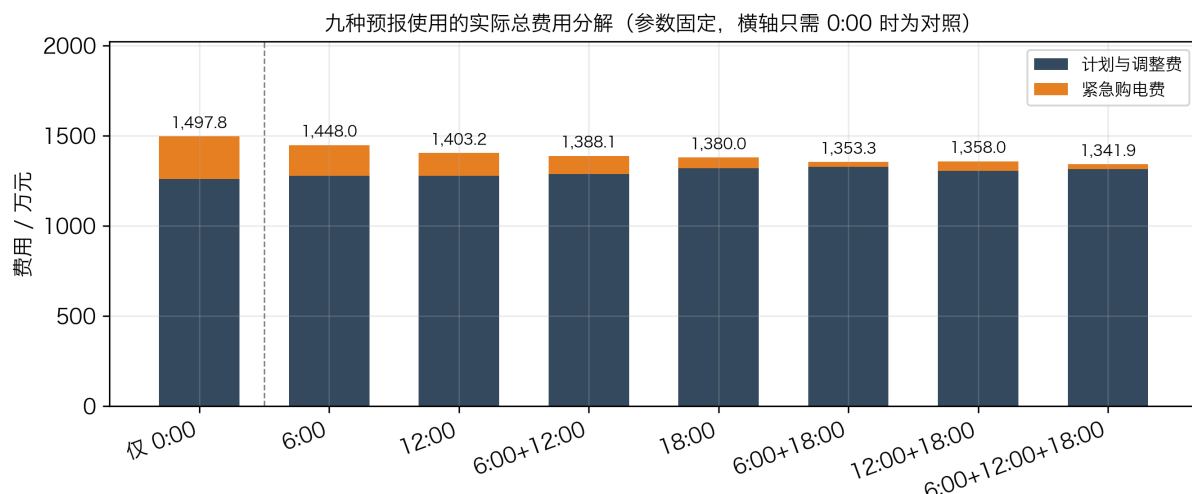


图4 八种预报使用方式的实际总费用分解（最左一项为只用 0:00 预报并关闭全天调整的对照）

加 16.16 万元、移除 12:00 版增加 11.44 万元。因此建议是使用三个时刻的全部预报；若减少一次预报更新，可省略 12:00 版，相应费用增加约为全年费用的 0.85%。

(3) **计费口径与风险分位数的影响。**表 21 给出两种结算口径下的结果。调减是否退款使全年总费用相差 24.24 万元 (1.81%)：把主策略自身的物理量按不退款口径结算，计划调整费由 13,145,690 元升至 13,388,108 元；完全按不退款口径重新优化后为 13,618,944 元，比主口径高 1.49%。两臂的物理运行差异很小，差额主要来自结算规则。将全时段合并分位数改为逐时段分位数后，总费用变化 0.25% (13,384,829 元)；分位数水平 α 的影响见表 20。

(4) **与问题二的对照。**问题二不允许日内调整，缺口按 5 倍电价补购，在同样的 334 天上紧急购电量为 140,301.24 kWh、紧急费 755,499.17 元；问题三主动付费调整，紧急购电量降到 47,350.77 kWh (33.8%)、紧急费降到 272,936.18 元 (36.1%)，合计购电费由 13,939,720.46 元降到 13,418,626.35 元，降低 521,094.11 元 (3.74%)。问题二与问题三的期末储电量分别为 1,559.72 kWh 和 5,798.18 kWh。问题三在保持较高期末储电量的同时取得更低费用，主要原因是日内预报将部分 5p 紧急购电转化为 1.5p 的计划追加。

(5) **分时调整与储能轨迹。**1002 次提交中，全年购电量发生调整的时段占 34.91%。四个指定日期的逐时段结果见表 7、表 8、表 9 与表 10；调整在三个六小时区间内的分布、储能轨迹的滚动修正与收益的月份差异，见附录第 1.16 节。

7.8 本节小结

以四个发布时刻的预报驱动滚动调整后，问题三在 334 天评价区间的购电费用为 13,418,626.35 元，较不作日内调整的对照降低 10.41%。费用下降主要来自将紧急购电转化为常规计划调整；三个日内版本均有增量价值，其中 18:00 版本贡献最大。结算口径与参数的补充比较见附录 L。

八、 问题四：波动电价下的购电调度模型

8.1 问题特征与建模思路

前三问采用已知的固定分时电价曲线，问题四改用附件 4 给出的逐日、逐十分钟波动电价。制定购电计划时，未来时段的价格需要预测，并与负荷和光伏预测共同进入优化模型。问题四主要包含以下三个方面：

- **价格预测：**各决策时点仅使用已经实现的价格，未来价格由历史数据预测；

- **联合风险**：考虑电价与净负荷预测误差的相关性，并据此修正风险分位数；
- **统一结算**：result4-2.xlsx 与 result4-3.xlsx 在同一条实际价格路径上分别进行时序仿真，以保证两类调度方式具有一致的费用比较基准。

首先分析附件 4 的价格结构，在此基础上建立价格预测和联合风险修正方法；随后分别构建计划固定与允许日内调整的两类模型，并比较其运行结果。

8.2 附件 4 的电价结构

附件 4 共 $365 \times 144 = 52,560$ 个价格，0.0076–1.7936 元/kWh，均值 0.7662 元/kWh，标准差 0.3407 元/kWh，日中位峰谷比为 4.33。价格变化可分为日内形状和日间水平两部分。以 365 天的分时段均值描述日内形状时，该曲线可解释总方差的 84.6%；逐日均价的变异系数为 0.135，对应标准差为 0.1036 元/kWh。因此，时段差异是价格变化的主要来源，日间差异则用于进一步提取星期规律。

将附件 4 价格记为 P ，附件 1 的固定曲线记为 F ，令 $P = F + R$ ，得到以下特征（数值见表 24，图 12 左）：

1. **水平相同**。 P 的 365 天逐时段均值与 F 的最大偏差只有 5.15×10^{-5} 元/kWh，两者年均值分别为 0.766198 (P ，附件 4) 与 0.766197 (F ，附件 1)，相差不足 10^{-6} 元/kWh。因此，后续对照中的费用变化主要反映价格波动的影响。
2. **残差与基准曲线近似不相关**。 $\text{corr}(R, F) = -1.5 \times 10^{-5}$ ，表明 R 基本不含固定分时段曲线中的日内结构。
3. **R 有星期结构**。把 R 按星期几分组求均值，周一至周四与周日均约为 +0.053 元/kWh，周五与周六约为 -0.137 元/kWh。两个水平相差 0.1913 元/kWh，相当于均价的 25.0%。0:00 的价格预测可利用这一星期差异。
4. **剩余波动**。扣除星期效应后， R 的方差下降 41.7%，其余 58.3% 构成日内残差波动。

记 $\bar{R}_w$ 为星期 w 的历史残差均值，则第 n 天第 t 段的价格预测为

$$\hat{p}_{n,t} = \frac{\sum_{i \in \mathcal{L}} w_i p_{n-i,t}}{\sum_{i \in \mathcal{L}} w_i}, \quad \mathcal{L} = \{7, 14, 21\}, \quad w_1 \geq w_2 \geq w_3 > 0,$$

即取最近三个同星期日价格的加权平均。该式等价于附件 1 日内曲线与同星期残差加权；在 334 天数据上的最大数值残差为 4.44×10^{-16} 元/kWh。采用随时间距离递减的权重，以提高近期样本的影响。具体权重由预热期数据确定。

8.3 价格预测模型与参数选择

将数据划分为参数选择期和评价期。参数选择期为 2025 年 1 月（第 1–30 天，30 天、4,320 个时段），权重 w 与日内修正系数 β 均由该区间确定；评价期为 2 月 1 日至 12 月 31 日（334 天、48,096 个时段），评价期间参数保持不变。

选型程序是：把常见候选（昨天同时段、上周同一天、近 7/14/28 天滚动均值、同星期等权、两组同星期加权）在选型期上比平均绝对误差，取最优者。结果见附录表 26：同星期加权 0.5/0.3/0.2 以 0.0486 元/kWh 胜出，其后是同星期等权 1/3（0.0487）与加权 0.6/0.3/0.1（0.0488），三者误差接近；跨星期滚动均值和前一日同时段方案的误差为 0.0847–0.0861 元/kWh。将参数选择期划分为三个子窗口后，0.5/0.3/0.2 均取得最低误差。

在评价区间内，0.6/0.3/0.1 和固定采用的 0.5/0.3/0.2 分别取得 0.0441 与 0.0446 元/kWh 的平均绝对误差，差异为 0.0005 元/kWh（约 1.1%）。表 27 将该结果作为样本外评价。

日内修正。0:00 定下全天预测后，随着当天价格逐段实现，可以用已实现时段对剩

余时段做一日内修正。取修正量

$$\hat{p}_{n,t} \leftarrow \hat{p}_{n,t} \exp\left(\beta \cdot \frac{1}{|\mathcal{T}_<|} \sum_{s \in \mathcal{T}_<} \ln \frac{p_{n,s}^{\text{act}}}{\hat{p}_{n,s}}\right), \quad t \geq t_0,$$

即用已完成时段的平均对数偏差外推剩余时段。 β 同样只在选型期扫描一次，且逐发布时刻单独选（各时刻可用于估计偏差的已完成样本数不同：6:00 只有 36 段、18:00 有 108 段）。1 月扫描得 $\beta = (1.0, 0.75, 0.5)$ ，并在评价期内保持不变。评价区间的参数扫描结果见表 28，其中 6:00 版本在 $\beta = 0.5$ 时的误差比固定参数低 0.0009 元/kWh。

第 h 个版本仅预测发布时刻以后的时段，四个版本的误差统计区间不同。为获得可比结果，表 29 同时报告 18:00–24:00 共同时段上的误差。该时段内，6:00 版最低 (0.0389)，其后是 18:00 版 (0.0403) 与 12:00 版 (0.0409)，0:00 版最差 (0.0428)。三版更新均优于未作日内修正的 0:00 版；12:00 版比 18:00 版高 0.0006 元/kWh。相关原因见附录 1.12 节。作为参照，不使用日内形状信息（全天套用均价）的误差为 0.2774 元/kWh，是四版的六倍以上。

8.4 电价与净负荷误差的联合风险

问题二的风险余量对净负荷预测误差 ε 取等权分位数。在波动电价下，紧急购电损失同时取决于电价与净负荷预测误差，而

$$\mathbb{E}[P\varepsilon] = \mathbb{E}[P]\mathbb{E}[\varepsilon] + \text{Cov}(P, \varepsilon).$$

因此先分析二者的协方差。结果见表 31：合并统计时相关系数为 +0.0863；固定时段后，144 个时段的相关系数均值为 +0.216（范围 +0.085–+0.402），全部为正。电价最高的 20% 时段中， ε 均值为 +40.5 kW；最低的 20% 时段中为 -74.7 kW，两端相差 115.2 kW。

该结果表明，电价与预测误差的相关性主要存在于同一时段的跨日样本中。因此，风险余量按时段分别估计，并采用价格加权的经验分位数表示高价时段的误差成本。

价格加权分位数。据此把风险余量从等权分位数改为以实际电价为权重的经验分位数：设第 n 天第 t 段的历史误差样本为 $\{\varepsilon_{i,t}\}_{i < n}$ ，对应权重取同一目标时段的实际电价 $\{p_{i,t}\}_{i < n}$ ，则

$$\tilde{N}_{n,t} = \hat{N}_{n,t} + Q_{\alpha}^P(\varepsilon_{\cdot,t}), \quad Q_{\alpha}^P = \inf \left\{ x : \frac{\sum_i p_{i,t} \mathbf{1}[\varepsilon_{i,t} \leq x]}{\sum_i p_{i,t}} \geq \alpha \right\}.$$

将样本按误差升序排列后，取累计权重比例首次达到 α 的样本值。当权重相等时，该定义与经验分布分位数一致；当权重和为零时采用普通分位数。价格较高的历史样本由此对风险余量产生更大影响。

表 30 给出价格加权分位数与等权分位数的对照。以相同下确界定义的等权分位数为基准， $\alpha = 0.6, 0.7, 0.8$ 时，价格加权风险余量分别提高 4.12、3.04、1.49 kWh/时段，相应增幅为 39.71%、11.74%、3.33%。表中另列问题二采用的线性插值等权分位数，便于区分价格权重与分位数定义的影响。该修正提高了高电价时段对应误差样本的权重；最终费用仍由计划优化及实时执行共同决定。

8.5 计费口径与两个分支的模型

8.5.1 价格信息与费用结算

两个分支均采用以下价格信息与费用结算规则：

环节	采用的价格	说明
决策 结算	决策时可获得的价格信息 附件 4 的实际价格	确定购电量与购电时段 计算实际购电费用

因此，4-2 与 4-3 均按对应交付时段的实际电价结算。4-3 的初始计划量 g_t^0 在 0:00 确定，日内可上调或下调最终生效量 x_t 。主计费口径采用与问题三相同的退款规则：上调部分按实际价格的 1.5 倍结算，被取消部分退回原价并按实际价格的 0.5 倍支付违约金。实际费用为

$$C = \sum_t p_t^{\text{act}} g_t^0 + 1.5 \sum_t p_t^{\text{act}} (x_t - g_t^0)_+ - 0.5 \sum_t p_t^{\text{act}} (g_t^0 - x_t)_+ + 5 \sum_t p_t^{\text{act}} r_t.$$

初始计划费、调整净额和紧急购电费分别统计；上调量不会重复计入计划费。上式也可写为

$$C = \sum_t p_t^{\text{act}} x_t + 0.5 \sum_t p_t^{\text{act}} |x_t - g_t^0| + 5 \sum_t p_t^{\text{act}} r_t.$$

预测阶段用当时可得的价格预测替代 p_t^{act} 作为目标函数的价格系数，结算阶段则使用附件 4 的实际价格。

若初始计划费不退回，调减部分还需额外支付 $0.5p_t^{\text{act}}$ 的违约金，则调减不具备费用优势，可令 $x_t \geq g_t^0$ 。这一不退款口径作为独立的敏感性分析，重新标定并报告其结果（附录 4.1）。其费用函数为

$$C_{\text{norefund}} = \sum_t p_t^{\text{act}} g_t^0 + 1.5 \sum_t p_t^{\text{act}} (x_t - g_t^0)_+ + 5 \sum_t p_t^{\text{act}} r_t.$$

8.5.2 分支 4-2：日前计划固定

4-2 对应交付文件 **result4-2.xlsx**：全天 144 段的购电量在 0:00 一次确定，并在日内保持不变。相对问题二的日计划 MILP，调整以下两项输入：

1. 目标函数中的价格系数由附件 1 的固定曲线换成 0:00 的**价格预测** $\hat{p}^0$ ，即 $\min \sum_t \hat{p}_t^0 g_t + \lambda \xi$ ；
2. 风险净负荷由等权分位数换成第 8.4 节的价格加权分位数 $\tilde{N}_t = \hat{N}_t + Q_\alpha^P(\varepsilon_{\cdot,t})$ 。

负荷与光伏仍采用问题二的历史外推模型，分支 4-2 不使用附件 3 的日内光伏预报。执行阶段按照实际负荷、光伏功率和参考储能轨迹逐段确定充放电及紧急购电量；常规购电量始终采用 0:00 确定的计划值。

8.5.3 分支 4-3：价格与光伏预报共同驱动的滚动调整

4-3 对应 **result4-3.xlsx**，沿用问题三的预报发布、六小时调整区间和购电量定义，并将调整模型中的固定电价替换为相应决策时刻的预测电价。设 $\mathcal{T}_\tau$ 为发布时刻 τ 到当日 24:00 的尚未执行时段，

$$\min \sum_{t \in \mathcal{T}_\tau} \hat{p}_t^{(\tau)} \left[1.5(y_t - g_t^0)_+ - 0.5(g_t^0 - y_t)_+ \right] + \sum_{t \in \mathcal{T}_\tau} 5\hat{p}_t^{(\tau)} \bar{r}_t + \lambda \xi,$$

其中 g_t^0 为 0:00 冻结的初始计划，上调和下调分别按 $1.5\hat{p}_t^{(\tau)}$ 与 $-0.5\hat{p}_t^{(\tau)}$ 计入调整净额。初始计划费在初次求解时计入，此处优化尚未执行时段的调整费用。 $\hat{p}^{(\tau)}$ 表示 τ 时刻可获得的价格预测：0:00 用 0:00 版，6:00 用第 8.3 节修正后的 6:00 版（对剩余时段）。因此，4-3 在日内同时更新光伏预测和剩余时段的价格预测。

第 τ 次求解仅读取发布时间不晚于 τ 的光伏预报 $F^{(\tau)}$ ，以及由已实现价格生成的预测 $\hat{p}^{(\tau)}$ 。初始计划按价格模式、计费口径和预报组合分别存储，以保持不同策略的输入相互独立。

执行与跨日衔接与问题三完全一致： $b_t = x_t + v_t^{\text{act}} - \ell_t^{\text{act}}$ ， $c_t = \min\{(b_t)_+, M, (10800 - E_{t-1})/\eta_c\}$ ， $d_t = \min\{(-b_t)_+, M, \eta_d(E_{t-1} - R_t)_+\}$ ， $r_t = (-b_t - d_t)_+$ ，逐段守恒 $x_t + r_t + v_t^{\text{act}} + d_t = \ell_t^{\text{act}} + c_t + w_t$ ；当日日末电量直接作为次日初值，两个分支各自独立保存实际

状态，1 月为预热期。

8.6 求解结果与一致性检验

8.6.1 两个交付文件的结果

以 2025 年 1 月为预热期、2 月 1 日–12 月 31 日共 334 天为正式区间，两个分支分别标定参数，并在同一条附件 4 实际价格路径上进行时序仿真，结果见表 11。**result4-2** 在计划固定的约束下全年合计购电费 14740813.21 元；**result4-3** 允许在 6:00、12:00、18:00 三个时刻调整计划，全年合计 14153285.53 元，比 4-2 少 587527.68 元（-3.99%）。

4-2 的全天计划在日内保持不变，预测缺口主要通过 5 倍电价的紧急购电补足。4-3 可在 6:00、12:00 和 18:00 上调或下调购电量，并根据更新后的价格与光伏预报调整剩余时段，从而降低紧急购电费用。表 11 同时报告两个分支的期初和期末储电量，用于控制跨期储能状态对费用比较的影响。

参数标定结果。 λ 由 1 月的 180 组 (α, ρ, λ) 联合标定确定，并在评价期保持不变。4-2 与 4-3 分别选择 $\lambda = 0.80$ 和 $\lambda = 0.50$ ，问题二的对应取值为 $\lambda = 0.00$ 。该结果表明，在波动电价条件下，模型对日末储能保有量赋予了更高权重。

4-2 的 (α, ρ) 随 8 个标定窗口变化，每个窗口的最优方案比次优方案低 26.9–3112.5 元。由于全天计划在 0:00 固定，且少购电需要按 5 倍价格补足，标定结果中的 α 整体偏向较高分位。

4-3 的 8 个滚动标定窗口均选择 $\alpha = 0.70, \rho = 0.5$ 。这两个参数位于当前搜索网格的下边界；本文报告已执行网格内的最优结果。逐窗口标定结果见附录。

8.6.2 逐日一致性检验

对两个分支的 334 个运行日逐日执行六项一致性检验，异常天数均为零。检验内容包括信息时序、4-2 计划固定、4-3 调整权限、物理可行性、储能状态累计和费用复算。逐项结果见表 33 及附录第 1.13 节。

8.6.3 三种价格信息条件下的策略对照

为评价价格预测的经济作用，设置三种价格信息条件。三种策略采用相同的负荷、光伏、储能和执行规则，均按同一条实际价格路径结算，其区别仅在于制定计划时使用的价格信息：

- **波动电价预测策略**（主策略）：0:00 用第 8.3 节的价格预测制定全天计划；4-3 在三个发布时刻用当天已实现的价格修正剩余时段。
- **固定电价参考策略**：采用附件 1 的分时电价曲线制定全天计划，并按附件 4 的实际价格结算；
- **未来价格已知参考**：假设当天价格在 0:00 已全部公布，作为完全信息参考，用于评价价格预测可能达到的改进空间。

定义 $\Delta J = J(\text{固定电价}) - J(\text{波动电价预测})$ 。各策略均使用相应历史信息独立标定参数。结果见表 25 与图 13：4-2 和 4-3 的 ΔJ 分别为 20139 元（0.14%）和 36231 元（0.26%）。价格预测在两个分支中均降低费用，但相对于全年总费用的比例较小。

附件 1 曲线已包含主要日内价格形状，可解释价格方差的 84.6%。因此，波动电价预测模型主要改善残差 R 中由星期差异解释的部分，该部分占残差方差的 41.7%。

在 4-3 中，波动电价预测策略的计划费比固定电价参考低 38161 元（13802989 元与 13841150 元），调整费增加 4340 元，紧急费减少 2410 元。因此，36231 元的总费用差异主要来自初始计划的时段配置。

完全信息参考在两个分支上分别为 14646668 与 14075672 元，比波动电价预测策略再低 94145 元（0.64%）与 77614 元（0.55%）。该差额反映在当前调度模型下，进一步提高价格预测精度所对应的费用改进空间。三种策略的费用分解见附录图 13。

8.6.4 八种预报组合

将 4-3 的三个额外预报时刻组成 $2^3 = 8$ 种组合，每种组合独立标定后进行全年时序仿真，结果见附录表 32。仅采用 0:00 预报时费用为 15022901 元，比主策略（三版全用，14153286 元）高 869615 元（+6.15%），三次日内预报合计降低费用 86.96 万元。单独加入 18:00、12:00 和 6:00 预报时，费用分别降低 51.91、44.54 和 41.75 万元；在已有 {6:00, 18:00} 预报的条件下，加入 12:00 版仍降低 10.56 万元。八种组合中，三个时刻全部使用时费用最低。完整的组合增量及费用构成见附录第 1.9 节。

8.6.5 本节小结

价格预测、价格加权风险余量与日内滚动调整共同构成问题四的决策链。三次日内价格预报使全年费用较仅用 0:00 预报降低 86.96 万元，且三个版本同时使用时费用最低。退款主口径下的完整费用分解及价格信息价值对照见附录 L。

表 11 两个交付文件在 2025 年 2 月 1 日–12 月 31 日（334 天）的实际运行结果

指标	result4-2（计划不得修改）	result4-3（6/12/18 可调整）
计划量（0:00 定下的量）/kWh	21,705,435	21,399,492
最终生效常规量/kWh	21,705,435	20,842,200
上调量/kWh	—	436,206
下调量/kWh	—	993,498
紧急购电量/kWh	116,391	29,325
弃电量/kWh	2,645,486	1,667,712
计划费/元	14,041,884	13,802,989
调整费/元	—	197,228
紧急费/元	698,929	153,069
合计费用/元	14,740,813	14,153,286
期初储电量/kWh	7,007	6,827
期末储电量/kWh	6,242	5,890

注：两个分支均在同一条附件 4 实际价格路径上分别进行时序仿真和参数标定，费用分别统计。两者均按对应时段的实际电价结算，并包含初始计划费。4-3 主口径允许上调和下调，调减部分退回原价并按 50% 的实际电价支付违约金；4-2 的全天计划固定，不发生调整。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型公式整理、求解代码编写与调试、结果校验脚本、论文部分文字的草拟与排版，详细使用情况见支撑材料。

参考文献

[1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题：微网与外部电网电力调控策略 [Z]. 2026.

[2] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J]. Nature Methods, 2020, 17(3): 261–272.

[3] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142.

[4] 王成山, 武震, 李鹏. 微电网关键技术研究 [J]. 电工技术学报, 2014, 29(2): 1–12.

[5] 李建林, 马会萌, 惠东. 储能技术融合分布式可再生能源的现状与发展趋势 [J]. 电工技术学报, 2016, 31(14): 1–10.

[6] 席裕庚, 李德伟, 林姝. 模型预测控制——现状与挑战 [J]. 自动化学报, 2013, 39(3): 222–236.

[7] Silver E A, Pyke D F, Peterson R. Inventory Management and Production Planning and Scheduling[M]. 3rd ed. New York: John Wiley & Sons, 1998.

A 补充结果分析与对照

本附录给出正文结论所对应的时序分析、图表和一致性检验，以便完整呈现模型的计算依据。

1.1 问题二：紧急购电的触发机制

图 5 给出四个指定日期的净负荷预测、风险余量与实际净负荷。三个层次的关系清晰可见：净负荷预测（虚线）给出基准，风险余量 Q_α （橙色阴影）在预测之上叠加一段风险修正量，得到用于日前求解的风险净负荷（橙实线），而实际净负荷（深蓝实线）则在其上下波动。

由于 Q_α 是历史误差的经验分位数，其取值可以为负：正式区间 $334 \times 144 = 48096$ 个时段中有 4069 个（8.5%）为负，最小值 -40.6 kWh 、最大值 258.5 kWh 。这些负值出现在误差分布整体偏负（长期被高估）的时段，此时分位数修正将相应降低计划购电量。该现象与风险修正整体提高计划购电量的统计趋势相容，具体调整方向由各时段的经验分位数决定。

关键在于**实际曲线落在风险净负荷之上的部分**。图 5 用倒三角标出了发生紧急购电的时段，9 月 23 日出现了 19 段紧急购电，3 月 20 日 1 段，其余两个日期则全天为零。但逐时段核对可以发现，**超出风险净负荷只是发生紧急购电的必要条件，而不是充分条件**：

- 6 月 21 日几乎被风险净负荷**完整包住**（144 段中仅 1 段超出，超出量 4.7 kWh ）；
- 3 月 20 日与 12 月 21 日分别有 50 段与 18 段超出风险净负荷，但最大超出仅 79.9 kWh 与 84.4 kWh ，且都出现在储能尚有余量的时段，可由储能吸收，因此**全天未发生紧急购电**；
- 9 月 23 日有 71 段超出，最大超出 133.7 kWh ，且**当日储能在 20:40 已降至下限 1200 kWh ，并在其后 9 个时段保持该水平**；储电量达到下界后，后续各时段的供电缺口需按 5 倍电价紧急购入。

这解释了 9 月 23 日晚间的 19 段紧急购电：其中 15 段的实际净负荷高于风险净负荷，另 4 段（19:00、19:30、19:40、20:30）的实际值甚至**低于**风险净负荷 $9.8\text{--}30.5 \text{ kWh}$ ，却仍分别紧急购电 $51.0\text{--}71.0 \text{ kWh}$ ，这是由于储能已达到放电下限。紧急购电量由实际净负荷相对风险修正值的超出量与储能剩余可放电量共同决定；只有储能无法覆盖缺口时，缺口才转化为 5 倍电价的支出。这三个日期的差别正在于此——3 月 20 日与 12 月 21 日的超出量小（超幅超过 50 kWh 的时段各仅 4 个与 2 个），9 月 23 日则达 22 个。

由此可见，预测偏低会在储能不足时形成紧急购电成本，而预测偏高主要增加弃电。这种不对称费用结构构成采用分位数计划的依据。

1.2 问题二：预测误差对储能轨迹的影响

图 6 对比了四个指定日期的日前参考储电量 $\bar{E}$ 与实际执行后的储电量 E 。两者的差异正是预测误差在执行层的体现：参考轨迹是 0:00 基于风险净负荷算出的最优路径，而实际轨迹则因实际负载与光伏的偏离而上下浮动。图中同时标出了 $[1200, 10800] \text{ kWh}$ 的运行边界与 $E^{\text{tar}} = 6000 \text{ kWh}$ 的参考水平。可以看到实际轨迹始终未越界，且四个日期的日初与日末储电量**各不相同**（例如 3 月 20 日 $1982.53 \rightarrow 1624.64 \text{ kWh}$ ，6 月 21 日 $4554.68 \rightarrow 4903.38 \text{ kWh}$ ），与问题二**跨日连续、不要求日循环**的模型设定一致。

1.3 问题二：全年费用分解与对照策略

图 7 从全年尺度展示结果。图 7(a) 给出本文策略的逐月费用分解。月费用最高的三个月是 12 月（175.9 万元）、6 月（155.2 万元）与 7 月（154.1 万元），最低的三个月是 4 月（96.6 万元）、5 月（100.4 万元）与 10 月（112.5 万元），最高月约为最低月的 1.8 倍。

图 1 指定日期的净负荷预测、风险余量与实际净负荷

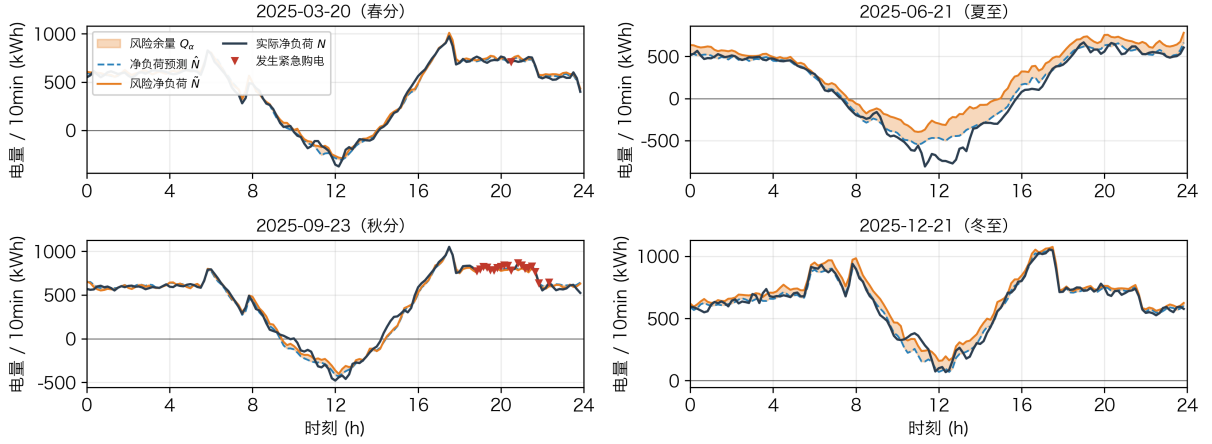


图 5 指定日期的净负荷预测、风险余量与实际净负荷

图 3 指定日期的日前参考储电量与实际储电量

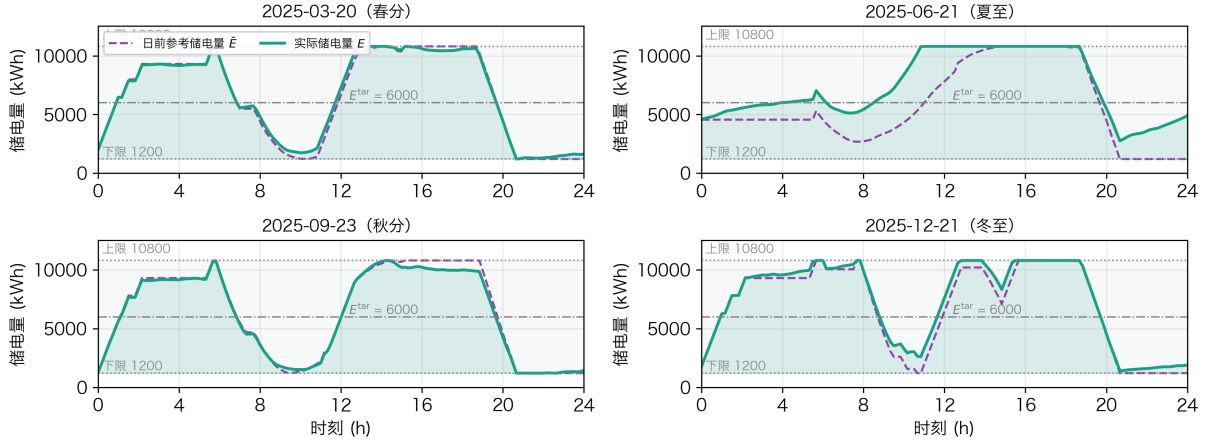


图 6 指定日期的日前参考储电量与实际储电量

这一形态主要由负载与光伏的季节错配决定：12 月光伏出力全年最弱而负载处于高位，6–7 月则因制冷负荷抬高用电水平，而春季与初秋光伏尚可、负载适中，故费用最低。

值得注意的是月度总费用的排序与紧急购电费的排序并不一致：紧急购电费在 6 月达到 190493 元，为全年最高，占全年紧急购电费的 25.2%，12 月（123830 元）与 9 月（72567 元）次之；而 11 月的紧急购电费仅 11265 元、为全年倒数第二低，其总费用却仍排在第 5 位。这说明月费用的高低主要仍由负载水平决定，紧急购电费则取决于预测偏差恰好与储能容量紧张相遇的频率，二者是两个不同的驱动因素。

为量化风险余量的价值，构造三组对照策略，全部使用同一预测模型、同一实时执行规则、同一评价区间（2–12 月共 334 天），并各自单独标定参数：

- **预测均值策略**：令 $\alpha = \text{None}$ ，即完全不加风险余量，直接用点预测制定日前计划，其余规则完全相同；
- **无储能策略**：仍提前制定购电计划、仍按 5 倍价格补缺，但不启用储能，用于单独评价电池的价值；
- **事后理想策略**：用当天实际净负荷替换预测值求解同一日前 MILP，代表“完全预知当天净负荷”的费用参照。须注意它每天仍从主策略的实际日初储电量出发、且不要求日末保留电量，因此会把电池放到下限，作为理想信息条件下的费用参照。

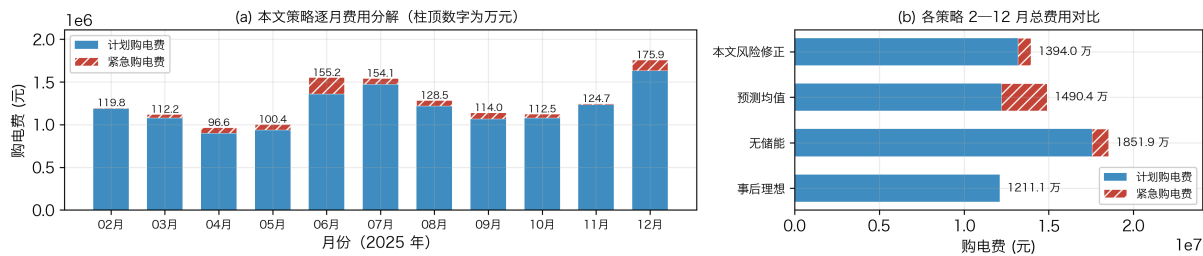
表 13 给出四者的费用对比。本文风险修正策略全年合计 13939720 元，较预测均值

策略的 14903679 元节省 963959 元、较无储能策略的 18519391 元节省 4579670 元，分别相当于**各基准费用的 6.47% 与 24.73%**（表 13 末列给出同一差额，但以本模型费用为分母，见该表注）。事后理想策略为 12110855 元，比本文策略低 1828866 元（13.1%）。该策略每天沿用主策略的日初储电量，且不设置日末储备约束；其 334 天的日末储电量均为下限 1200 kWh。因此，13.1% 的差距同时包含信息条件和期末库存口径的影响，作为理想条件下的费用参照。

费用结构显示，预测均值策略的**紧急购电费为 2708867 元，是本文策略 755499 元的 3.6 倍**，而其计划购电费反而更低（12194812 元对 13184221 元）。该费用差异体现了代价不对称性：不加风险余量时，日前计划购电量较少，但运行时会更频繁地触发 5 倍电价的紧急购电，总费用因而更高。风险余量通过增加计划购电支出，降低了高倍价紧急购电支出。

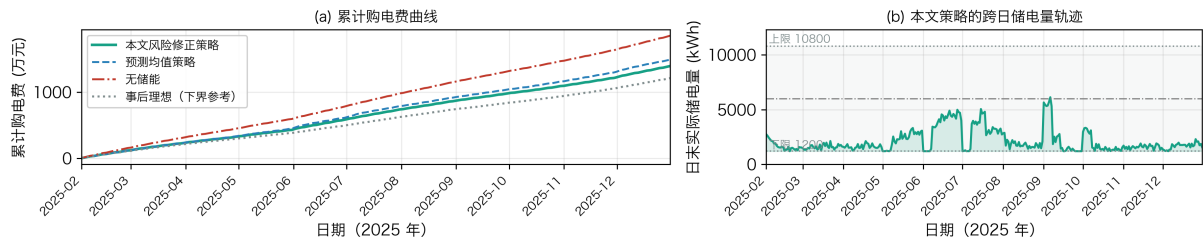
同时应注意，四个策略的**期末储电量并不相同**（本文策略 1559.7 kWh，预测均值策略 5960.7 kWh，无储能策略 6000.0 kWh）。期末储电量差异意味着库存并未被完全对等地消耗，因此本文同时报告该指标，避免把”多消耗库存”误当作算法收益。

图 4 逐月费用分解与策略费用对比



(a) 逐月费用分解与策略费用对比

图 5 各策略累计费用与跨日储电量



(b) 各策略累计费用与跨日储电量

图 7 全年费用分解与策略对照

1.4 问题二：弃用电量的成因

全年弃用电量 2463544.98 kWh，占同期光伏发电量 19068890.0 kWh 的 12.9%。题目未对弃电设置惩罚或收益，故弃电不带来额外扣款，但它仍是日前计划与实时运行之间偏差的**指示器**。逐时段分解显示弃电来自**两个性质不同的来源**：71.8%（1769824 kWh）是光伏富余超出储能吸纳能力（储能受 5000 kW 功率上限与 10800 kWh 容量上限双重约束）；另 28.2%（693721 kWh）发生在**光伏并无富余的时段**，只能来自**已按计划电价付费、却因实际负载低于预期而用不掉的计划购电量**，其当初购电成本 466267 元约占全年计划购电费的 3.54%，说明该部分弃电来自为降低紧急购电风险而增加的计划购电量。 α 敏感性验证表明这一取舍**经过权衡且最优**： α 由 0.80 降至 0.50 时弃电由 2628.6 降至 1493.7 MWh，但紧急购电量由 177.9 猛增至 636.1 MWh，**总费用反由 1403.8 万元升至 1508.8 万元**。此外，紧急购电负担在时间上**高度集中**：334 天中 143 天发生紧急购电，而 6 月 1-2 日两天即占全年紧急购电费的 13.1%，说明费用主要集中在少数连续高

负荷时段。

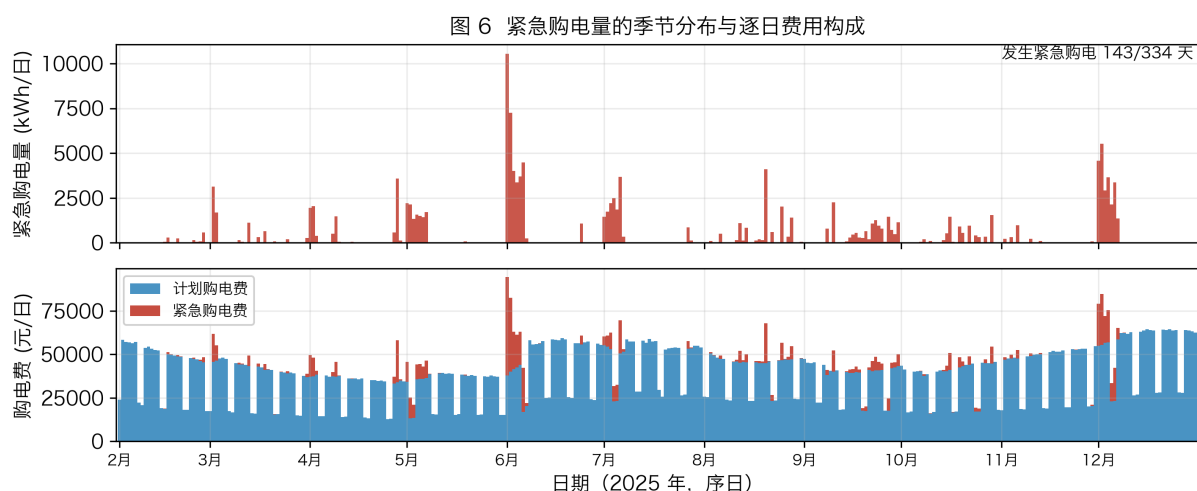


图 8 紧急购电量的季节分布与逐日费用构成

1.5 问题一：购电策略的逐段时序特征

图 9 给出全天逐时段的计划购电量与电价的对照关系。可以看到购电主要集中在低电价时段：

- 在 0:00–6:00 的夜间低谷（电价约 0.37–0.91 元/kWh），购电量维持在每 10 分钟 1400 kWh 上下，其中 0:00–0:40 因需同时供给负载并为储能充电而达到全天最大的 1406.6–1409.8 kWh/10 min；
- 在 5:30–5:40 电价降至全天最低 (0.3713 元/kWh) 时，购电量为 1383.66 kWh/10 min，仍处高位；此后随储能充满 (5:40 触及上限 10800 kWh) 迅速退出购电，5:50 起连续 31 个时段购电量降为零；
- 在 7:50–9:10 与 17:40–20:50 两个电价高峰段（电价最高达 1.3952 元/kWh，约为低谷的 3.76 倍），购电行为被显著抑制：早高峰 7:30–10:00 的购电量**恰好为零**，完全由储能放电支撑；晚高峰 17:00–21:00 购电量回落至 0–826.08 kWh/10 min（24 个时段中仅 12 个时段有购电），降幅虽大但仍保留了部分购电——因为此时储能储电量已接近下限，继续放电将破坏后续时段的供电能力；
- 在光伏出力超过小区负载的 9:30–14:20（共 30 个时段）内，购电量在 22 个时段**降为零**，微网完全由光伏供电并有盈余为储能充电；其余 8 个时段（11:00–12:30）仍有 179.28–575.17 kWh/10 min 的购电。该部分购电用于利用正午时段的局部低价（0.4113–0.5007 元/kWh，低于相邻时段）：此时储能尚有充足容量（储电量由 2971 kWh 升至 9144 kWh），故在光伏已经富余的情况下仍提前购入低价电存入储能，并在傍晚 1.3952 元/kWh 的高价时段释放。因此，表 1 中 10:00、14:00、20:00 三个时段购电量为零、而 12:00、16:00、18:00 仍保留一部分购电的原因——16:00、18:00 的购电则是因储电量已达上限 10800 kWh、光伏出力已低于负载，只能由外网补足。

1.6 问题一：储能运行的逐段特征

图 10 给出储能设备的充放电功率与储电量轨迹，呈现出典型的两充两放格局：

1. **第一充放循环**：0:00–5:40 利用夜间低谷电价充电，储电量由 6000 kWh 升至 10800 kWh（5:40 首次触及上界）；5:50–9:30 在早高峰放电，储电量一路降至全天次低点 1834.62 kWh；

图 2 全天计划购电量与电价

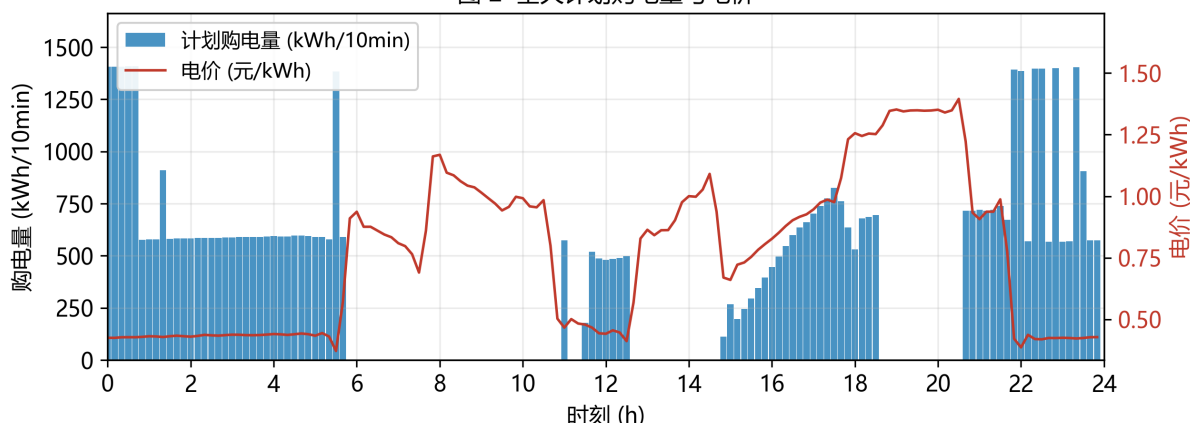


图 9 问题一：全天计划购电量与电价

2. **第二充放循环**: 9:30–16:00 吸收光伏盈余并在正午电价局部低谷时补充购电充电(储电量自 1834.62 kWh 回升, 13:50 突破 10500 kWh, 16:00 再次达到上界 10800 kWh); 18:00–20:40 在晚高峰放电, 储电量降至下界 1200 kWh (20:40 首次触及, 并保持至 21:50);
3. 21:50–24:00 再次充电, 将储电量精确送回 6000 kWh, 满足日循环条件。

图 3 储能设备充放电功率与储电量

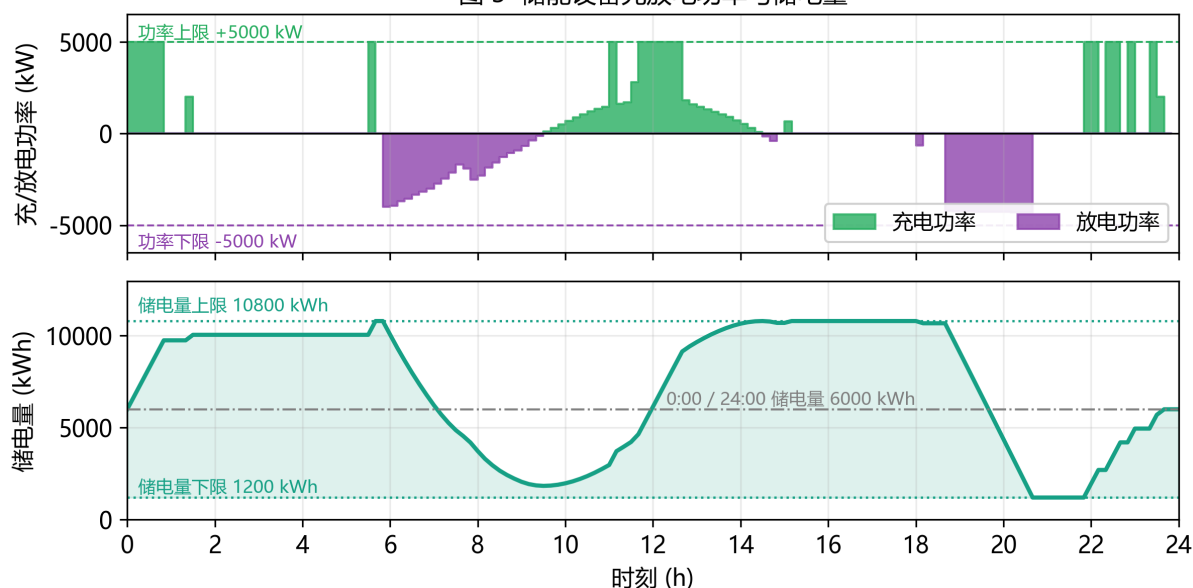


图 10 问题一：储能设备充放电功率与储电量

1.7 问题一：全天电量平衡与弃光

图 11 给出全天电量的供给与需求构成。供给侧为光伏发电 55482.84 kWh、电网购电 59482.70 kWh 与储能放电 16799.94 kWh，合计 131765.48 kWh；需求侧为小区负载 111024.81 kWh 与储能充电 20740.67 kWh，合计亦为 131765.48 kWh，两侧严格相等。

一个值得注意的结果是：本模型的**弃光电量为零**，即 $s_t^* = 0$ 对所有 t 成立。储能设备将光伏出力超过负载的全部盈余电量 (6247.96 kWh) 尽数吸纳，未造成任何浪费。这是 10 分钟精细调度粒度带来的直接收益——若采用粗粒度调度，光伏盈余与储能充电能力在时间上难以精确匹配，将不可避免地产生弃光（详见正文第 5.6 节）。

图 4 全天电量平衡

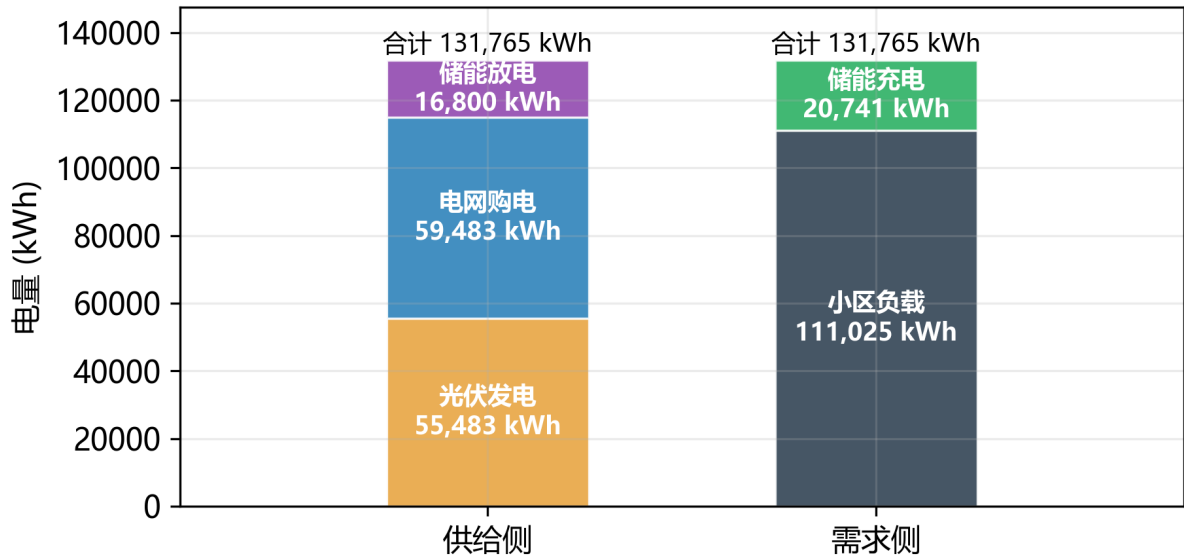


图 11 问题一：全天电量平衡构成

1.8 问题三：预报使用的边际分析

表 18 与图 4 给出八种预报使用方式的实际总费用。为回答“是否需要其他时刻的预报”，还要看**边际改善** $\Delta_h(S) = C(S) - C(S \cup \{h\})$ ，即已经用了 S 之后再加用时刻 h 能省多少。其中 $S = \emptyset$ 一列表示“只用 0:00 的预报、关闭全天调整”的对照。结果如下 (单位：万元)：

已有组合 S	$\Delta_6(S)$	$\Delta_{12}(S)$	$\Delta_{18}(S)$	参考
$\emptyset$ (仅 0:00)	49.78	94.59	117.82	单独引入
{6}	—	59.91	94.69	
{12}	15.10	—	45.17	
{18}	26.66	21.94	—	
{6, 12}	—	—	46.23	
{6, 18}	—	11.44	—	
{12, 18}	16.16	—	—	

由此得到以下三项结论。

其一，18:00 预报的边际贡献最大。单独引入时它是三者中改善最大的 (省 117.82 万元)；在 {6}、{12}、{6, 12} 之上再引入它仍分别省 94.69、45.17、46.23 万元，都是在当时所有可选时刻中最大的。该排序由预报覆盖时段的经济影响共同决定：按表 19，在四版共同覆盖的白天窗口 12:00–18:00 上 12:00 版最准 (平均绝对误差 181.3 kW，优于 6:00 版的 304.9 kW 与 0:00 版的 445.8 kW)，而 18:00 版**对这一窗口没有任何覆盖**。18:00 版提交后用于晚间 18:00–24:00，这一段既覆盖电价高峰、又是日末回充窗口，而且是全天唯一没有光伏的时段，负载预测误差及其费用影响相对突出。因此，预报价值取决于预测精度、覆盖时段和相应费用的共同作用。

其二，12:00 的边际价值在已有相邻版本后迅速衰减。它在 {18} 之上只省 21.94 万元、在 {6, 18} 之上只省 11.44 万元，远小于它单独引入时的 94.59 万元。原因是它的信息与相邻两个时刻**高度重复**：它对当天下午的修正，6:00 已经预报过一次，18:00 又会覆盖相同的目标时段 (三个版本的提交块首尾相接，12:00 版负责的第 3 块被前后两个版本”夹”在中间)。

其三，预报的边际贡献取决于已有信息集。单独引入时，12:00 优于 6:00 (94.59 对 49.78 万元)；但在已有 {18} 之后次序**反转**——此时加 6:00 省 26.66 万元，加 12:00 只省 21.94 万元。这与 $\Delta_6(\{6, 12\}) = 15.10$ 万元同源：6:00 版在 {12} 之上仍能省 15.10 万

元。该版本除提供第 2 时段的更新预报外，还以当日实际储电量和上午已实现的光伏出力为初值，重新优化剩余时段；其中部分作用也可由 18:00 版本实现。因此，预报版本应在给定已有信息集的条件下排序。

预报版本的取舍。按移除后的费用增量排序：移除 18:00 版全年增加 462,252.98 元、移除 6:00 版增加 161,579.57 元、移除 12:00 版仅增加 114,388.95 元。因此，使用三个时刻的全部预报时费用最低；若减少一个预报版本，移除 12:00 版的费用增量最小，约为全年费用的 0.85%。

仅使用 0:00 预报的对照结果。仅使用 0:00 预报并关闭全天调整时，全年费用为 14,977,809.03 元，比问题二的历史外推策略 13,939,720.46 元高 1,038,088.57 元。该对照同时改变了光伏预测来源和日内调整机制，其紧急购电费为 2,382,672.91 元（占自身费用的 15.9%），问题二的相应费用为 755,499.17 元。因此，该结果用于评价两项变化的综合影响，不用于单独比较预报质量。

信息收益与机制收益。上面格上的每一条覆盖边（例如 $\{6\} \rightarrow \{6, 12\}$ 省 59.91 万元、 $\{6, 12\} \rightarrow \{6, 12, 18\}$ 省 46.23 万元）都是在机制不变时仅更换预报集合，因此表示信息增量。从对照 $\emptyset$ 到全集的 155.92 万元差额同时包含日内调整机制和新发布预报的作用。正文中的 10.41% 改善据此解释为两者的共同贡献；各预报版本之间的横向比较用于衡量信息的边际贡献。

1.9 问题四：八种预报组合的边际分析

记 $\Delta_h(S) = J(S) - J(S \cup \{h\})$ 为在已有预报集合 S 的基础上增加 h 时刻预报所产生的费用变化。八种组合分别标定，并在 334 天评价期内进行完整时序仿真；对应的 12 个边际增量列于表 12。

表 12 4-3 八种预报组合的边际价值（波动电价，各组合独立标定；单位：元）

已有预报 S	再加一版 h	增量 $\Delta_h(S)$	折合
$\emptyset$	6:00	417,501	41.75 万元
$\emptyset$	12:00	445,421	44.54 万元
$\emptyset$	18:00	519,070	51.91 万元
$\{6\}$	12:00	234,966	23.50 万元
$\{6\}$	18:00	346,498	34.65 万元
$\{12\}$	6:00	207,046	20.70 万元
$\{12\}$	18:00	239,349	23.93 万元
$\{18\}$	6:00	244,930	24.49 万元
$\{18\}$	12:00	165,700	16.57 万元
$\{6, 12\}$	18:00	217,148	21.71 万元
$\{6, 18\}$	12:00	105,616	10.56 万元
$\{12, 18\}$	6:00	184,846	18.48 万元

注： $\Delta_h(S) = J(S) - J(S \cup \{h\})$ 表示在已有集合 S 上增加版本 h 后的费用变化。所有组合均在同一条附件 4 实际价格路径上结算，并分别标定 α, ρ, λ 。加粗行对应最小边际贡献；本口径的 12 个边际增量均为正。

边际贡献随已有信息集变化。单独加入 18:00、12:00 和 6:00 版本分别降低费用 51.91、44.54 和 41.75 万元；在已有 18:00 版本时加入 12:00，费用仍降低 16.57 万元；已有 $\{6:00, 18:00\}$ 时再加入 12:00，仍降低 10.56 万元。八种组合中，三个时刻全部使用的全年费用最低，为 14153286 元。

这些增量由各组合独立标定后的实际结算费用计算。12 个边际增量均为正，但幅度随已有预报集合变化。12:00 与 18:00 的部分信息相互重叠，因此加入 12:00 版的边际收益随已有预报增多而下降。预报集合的比较以完整时序仿真的总费用为准。

1.10 问题四：附件 4 电价的结构分解

正文第 8.2 节将附件 4 的电价分解为附件 1 的固定分时曲线与星期残差。图 12 左图比较一周内的两类价格曲线，右图给出残差 $R = P - F$ 的星期均值，用于展示同星期加权预测所利用的周期结构。

附件 4 实时电价并非另一条曲线，而是附件 1 曲线叠加星期偏移

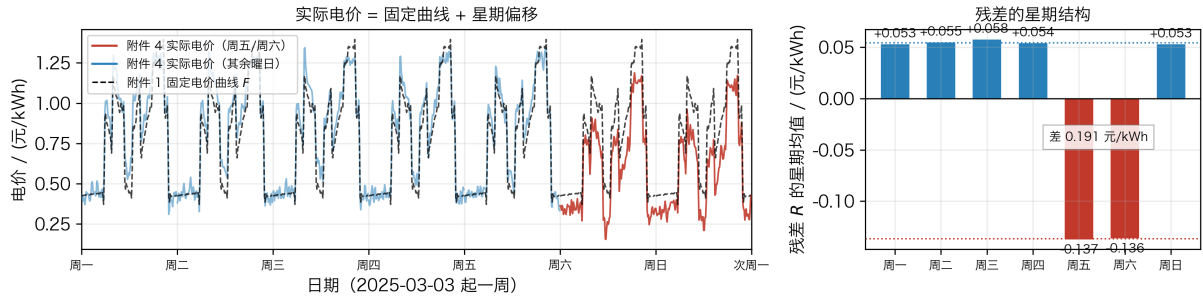


图 12 附件 4 电价的结构分解：左为实际电价与附件 1 曲线（横轴为一周），右为残差 R 的星期均值

1.11 问题四：三种价格信息条件下的策略费用分解

正文表 25 已给出三种策略在两个分支上的精确费用，这里附上其费用分解图，用于比较计划阶段和执行阶段的费用构成。

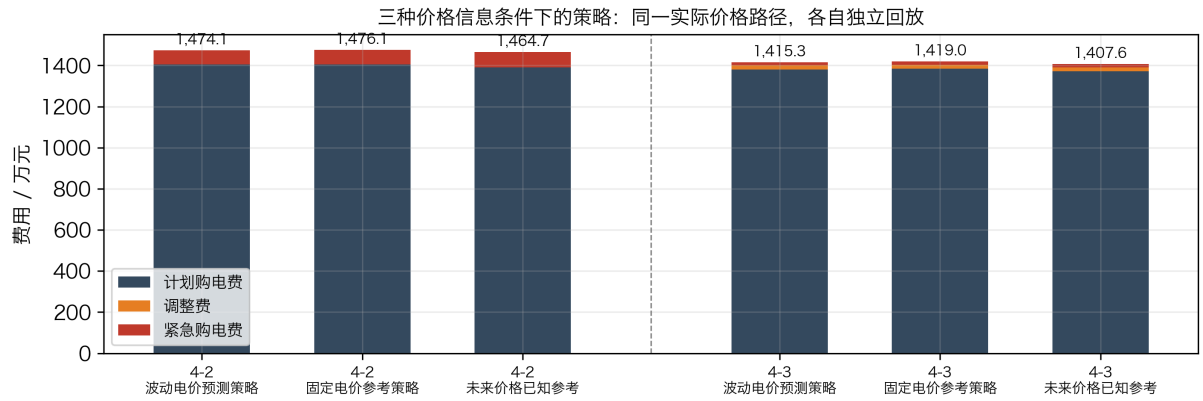


图 13 三种价格信息条件下的策略费用分解（同一实际价格路径结算）

1.12 问题四：12:00 与 18:00 版本的预测误差差异

正文表 29 的“共同时段”一列里，四个发布版本按精度排序为 6:00 (0.0389) < 18:00 (0.0403) < 12:00 (0.0409) < 0:00 (0.0428)。三个更新版本都优于不修正的 0:00 版；12:00 版比 18:00 版高 0.0006 元/kWh。下面分析这一差异的来源。

首先检验窗口长度的影响。将三个版本的 β 统一后，预测误差关系如下：

β 取法	0:00 版	6:00 版	12:00 版	18:00 版
冻结值 (1.0, 0.75, 0.5)	0.0428	0.0389	0.0409	0.0403
统一 $\beta = 0.5$	0.0428	0.0389	0.0401	0.0403
统一 $\beta = 0.75$	0.0428	0.0385	0.0409	0.0419

统一取 $\beta = 0.5$ 时得到 $0.0401 < 0.0403$ ，统一取 $\beta = 0.75$ 时得到 $0.0409 < 0.0419$ 。由于两组实验的窗口长度不变，原有差异主要来自各发布时刻采用不同的修正系数：12:00 版取 $\beta = 0.75$ ，18:00 版取 $\beta = 0.5$ ；在最后 36 个时段，较大的修正幅度对应更高的误差。

该对照说明，逐发布时刻选择 β 能够适应不同的信息量，也会引入参数估计差异。选型期扫描对窗口较为敏感：仅使用第 22–30 天时，最优值变为 (0.5, 0.5, 0.0)。为保持样本外评价的一致性，本文仅在评价期之前选择一次 β ，并在评价期内保持不变。

1.13 问题二、问题三、问题四：六项与七项一致性检验的逐条明细

1.13.1 六项一致性检验

按前述要求，对 365 天的完整运行逐日执行六项检查，表 16 给出结果，**全部通过**：

1. **信息时点**：程序逐日记录预测实际引用的历史日下标，其上界严格小于当天；误差分位数只取 $i < n$ 的已结束日期； $g_{n,t}$ 在 0:00 冻结，执行层结构中不存在修改 g 的入口。
2. **物理可行性**：365 天逐段供需平衡残差的最大值为 2.3×10^{-13} kWh（判定阈值为 1.3×10^{-3} kWh，实测值比阈值小 10 个数量级）；储电量全程落在 $[1200, 10800]$ kWh、越界量为 0； $c, d \leq M$ （越限量 0）且无同时充放电时段（0 段）。
3. **跨日连续性**：逐日校验 $E_{n,0}^{\text{actual}} = E_{n-1,144}^{\text{actual}}$ ，365 天全部成立。
4. **状态累计**：式 (27) 逐日成立， ΔE 与 $\eta_c \sum_t c_t - \eta_d^{-1} \sum_t d_t$ 的最大绝对偏差为 1.3×10^{-11} kWh。
5. **费用一致性**：用未四舍五入的计划量与紧急量重新计算费用，与记录值的最大偏差为 0 元；弃电不退款；日末储备惩罚未混入实际账单。
6. **汇总一致性**：日期完整、无跨日错位；紧急购电事件合并前后的电量总和严格相等（390 个事件全部核对）。

1.13.2 问题四的六项一致性检验

对问题四两个分支的 334 个交付日逐日执行下列六项检查，表 33 给出结果，**未通过天数全部为零**：

1. **信息时序约束**：第 τ 次求解实际读取的价格预测与光伏预报，其发布时刻均不晚于 τ ；6:00 之后的价格预测只使用当天 **已经实现**的时段，不引用任何未来价格。这一项在 4-2 与 4-3 上都按同一标准检查。
2. **计划冻结（仅 4-2）**： g_t 在整个交付日内恒等于 0:00 定下的 g_t^0 ，执行层不存在修改 g 的入口。4-3 不受这一约束，改由下一条检查它的调整权限。
3. **调整权限（仅 4-3）**：只有已发布的调整区间允许 $x_t \neq g_t^0$ ，其余块 $x_t \equiv g_t^0$ ；调整区间边界处 x_t 的取值只由该块发布时刻的信息决定，不跨块引用更晚的信息。主计费口径下 x_t 可高于或低于 g_t^0 ；本年度上调总量为 436206.27 kWh，下调总量为 993498.18 kWh（见第 8.6 节）。
4. **物理可行性**：334 天逐段供需平衡残差的最大值为 2.3×10^{-13} kWh（两个分支相同）；储电量全程落在 $[1200, 10800]$ kWh，上下界余量均为 0（即恰好触界而不越界）；不存在同时充放电的时段。
5. **状态累计**：由 c_t, d_t 从日初储电量独立递推得到的序列与记录值一致；当日日末电量严格作为次日初值，**跨日衔接最大偏差为 0 kWh**。
6. **费用一致性**：计划费、调整费、紧急费三项分别按实际价格重算，与记录值的最大偏差均为 0 元；0:00 的”预估费用”另按当时的预测价复算，同样相符。这与第 8.5 节的结算原则一致：**决策用当时的信息，结算一律用实际价格**。

1.14 问题一：最优解六项复核的逐条明细

1.14.1 求解结果的校验

为确保结果可信，对最优解独立复核了以下各项，**全部通过**：

1. **时段完整性**：144 个时段完整覆盖当天 0:00–24:00，首尾与四小时分组均无错位；附件 1 时间标签经校验为 $\{10, 20, \dots, 1440\}$ 分钟且严格递增。
2. **逐段供需平衡**： $\max_t |g_t + v_t - s_t + d_t - \ell_t - c_t| < 10^{-6}$ kWh。

3. **状态递推**: 由 c^*, d^* 从 E_0 出发独立累计递推得到的储电量序列与求解器返回的 E^* 最大偏差小于 10^{-6} kWh。
4. **物理边界**: E^* 全程落在 $[1200, 10800]$ kWh 内; $c_t^*, d_t^* \leq M = 2500/3$ kWh; $0 \leq s_t^* \leq v_t$; 且不存在同时充放电的时段。
5. **日循环**: $E_{144}^* = E_0^* = 6000$ kWh, 且按式 (10) 校验 $\sum_t d_t^* = 0.81 \sum_t c_t^*$ (实测 $16799.94/20740.67 = 0.810000$)。
6. **费用与汇总**: 用未四舍五入的数据重新计算 $\sum_t p_t g_t^*$, 与求解器返回值一致; 各分段汇总与全天明细一致。

数值复核仅设置极小的浮点容差, 未通过删除负数、裁剪越界值或修改末段结果的方式掩盖任何约束违背。

1.15 问题二：四个指定日期的逐日结构

1.15.1 指定日期的结果

表 3、表 4、表 5 与表 6 分别给出 3 月 20 日、6 月 21 日、9 月 23 日与 12 月 21 日四个指定日期的计划购电量、储能充放电量与紧急购电情况 (其中 12 月 21 日的储能充放电量因列宽原因单列一表)。四个日期的运行特征差异明显, 恰好覆盖了不同季节与不同星期的组合:

- **3 月 20 日 (周四)** 与 **9 月 23 日 (周二)** 为工作日, 负载处于高位 (117648.38 kWh 与 121182.28 kWh), 光伏出力中等 (55439.42 kWh 与 59065.67 kWh), 计划购电量分别为 67400.98 kWh 与 66102.01 kWh;
- **6 月 21 日 (周六)** 属于低负载的周末族, 全天负载仅 81408.06 kWh, 而光伏出力却是四者中最高 (62073.32 kWh), 净负荷最低, 故计划购电量仅 40911.64 kWh, 为四个日期中最低; 也是四者中弃电量最大的一天 (19087.44 kWh);
- **12 月 21 日 (周日)** 适逢冬至, 光伏出力处于全年低位 (35338.66 kWh, 在正式区间内为第 8 低), 负载则属高负载族 (124376.44 kWh, 该族约 124000 kWh/日), 净负荷在正式区间内列第 3, 全天计划购电量高达 99337.46 kWh、购电费 64146.24 元, 为四个日期中最高。

四个日期的这一组差异, 恰好覆盖了负载的星期结构与光伏的季节变化两个维度, 使得指定日期的结果具有代表性。

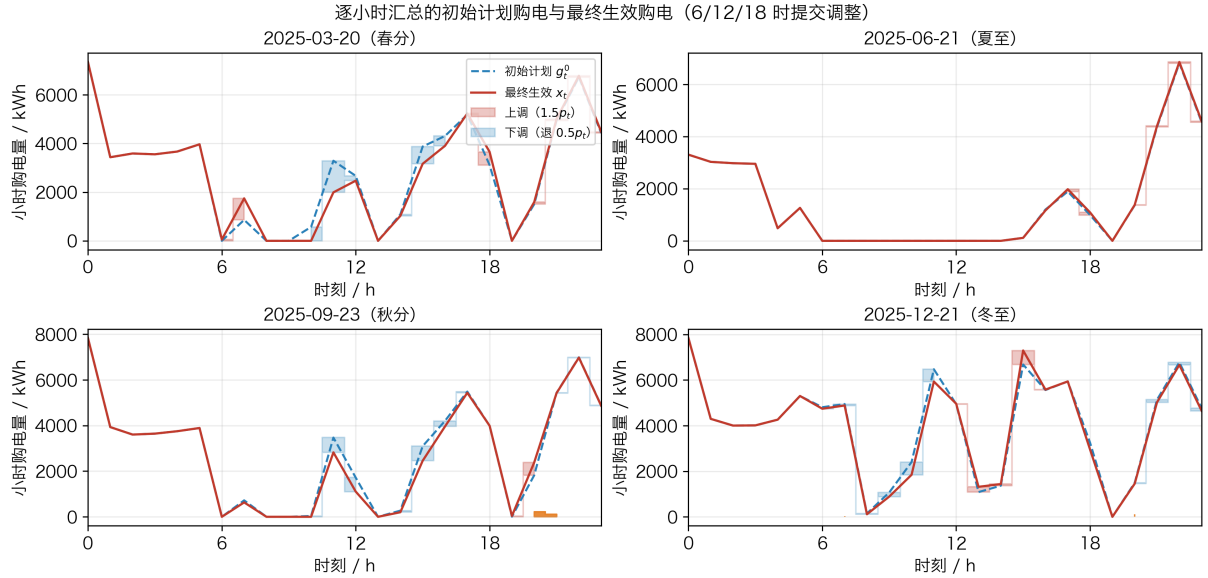
在紧急购电方面, 四个指定日期中有两个**全天无紧急购电** (6 月 21 日与 12 月 21 日), 3 月 20 日仅 1 个事件 (20:30–20:40, 14.56 kWh、101.56 元), 而 9 月 23 日发生 3 个紧急购电事件 (18:50–20:40、20:50–22:00、22:20–22:30), 合计 1253.46 kWh、7777.51 元。这些事件都出现在晚间 18:50 以后: 该日白天光伏出力偏低, 储能到傍晚已接近下限, 晚间高价时段出现预测未能覆盖的额外缺口, 只能按 5 倍电价补购。事件按”同日连续发生紧急购电的时段合并为一个事件”的规则生成, 合并后的电量与逐段之和严格相等。

1.16 问题三：调整频次、储能轨迹与逐日收益

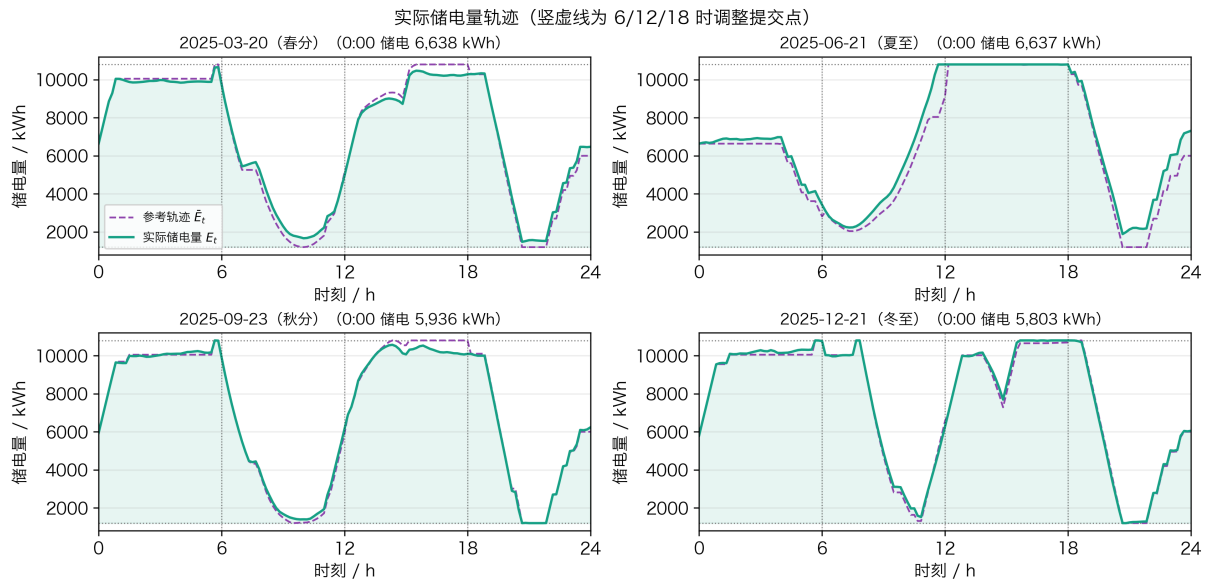
(3) 调整频次。三个发布时刻全年共正式提交 $334 \times 3 = 1002$ 次, 但其中相当一部分提交的结果是”保持不变”: 全部 $334 \times 144 = 48,096$ 个时段中, 最终生效量发生变化的共有 16,788 个, 占 34.91%。图 14a 给出四个指定日期的初始计划购电量与最终生效购电量, 两条曲线的差异只在少数时段出现。每个调整区间的调整都是**双向**的 (全年上调 656,263.05 kWh、调减 775,660.56 kWh), 这与问题二的执行规则作为对照, 问题二不允许日内调整, 缺口按 5 倍电价补购。更新预报后重新求解可以调整购电量, 也可以仅调整储能路径; 当预计减少的紧急购电费高于调整费时, 模型才改变购电量。

(4) 储能轨迹被滚动修正。图 14b 给出四个指定日期的参考轨迹与实际储电量。可

以看到每个发布时刻的参考轨迹都会重新拼接（竖虚线），而实际储电量始终连续。评价区间内日末储电量落在 5,216.7–7,966.9 kWh 之间，均值 6,284.9 kWh，均未达到储能上下限。日末位置与逐时轨迹的统计口径不同：逐时段看，储能会在光伏富余时段触及 10800 kWh 的上界（334 天中有 322 天）、在连续缺口时被压至 1200 kWh 的下界（146 天），反映了设备容量对消纳能力的约束；储备线 $R_t = \text{clip}(1200 + \rho(\bar{E}_t - 1200), 1200, 10800)$ 用于在执行层保留一定的日末储电量。四个指定日期的具体结果见表 7、表 8、表 9 与表 10。



(a) 逐小时汇总的初始计划购电量 g_t^0 与最终生效购电量 x_t



(b) 滚动参考储电量 $\bar{E}_t$ 与实际储电量 E_t

图 14 指定日期的购电量调整与储能轨迹

(5) 收益在月份间并不均匀。图 15 给出主策略逐日的调整费、紧急费与日末储电量。按月汇总，调整费在 12 月最高（164.18 万元），约为 4 月（90.60 万元）的 1.81 倍；紧急费则在 5 月最高（5.61 万元）、11 月最低（0.89 万元），跨度接近 6 倍。调整费的峰值出现在年末，与冬季光伏出力低、净负荷高、计划量对误差更敏感有关；紧急费的峰值出现在 5 月，与空调负荷开始爬升、日内波动加大有关。两项支出的月度分布并不同步，这也说明分月评价预报收益是必要的，不能只看四个指定日期。

主策略逐日的计划调整费、紧急费与日末储电量

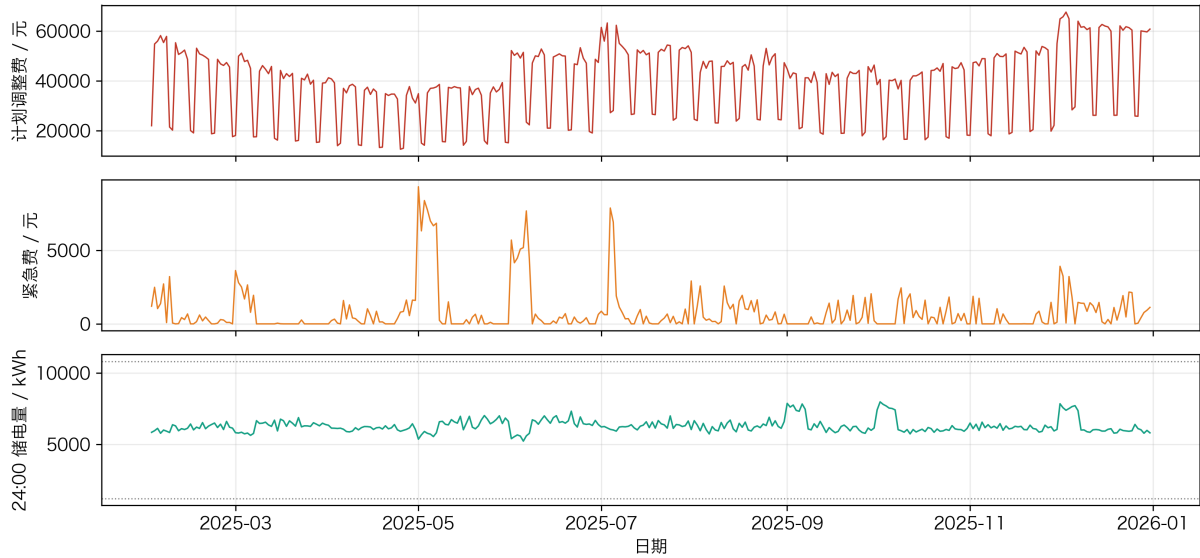


图 15 主策略逐日的调整费、紧急费与日末储电量

B 问题一求解核心代码

2.1 参数、数据载入与时段校验

```

1 N = 144 # 一天 144 个 10 分钟时段
2 TAU = 10 / 60 # 每时段小时数 = 1/6 h
3 E_MIN, E_MAX = 1200.0, 10800.0 # 储电量运行下界/上界 kWh
4 E_INIT, P_MAX, ETA = 6000.0, 5000.0, 0.90
5 M_ENERGY = P_MAX * TAU # 单时段最大充放电电量 M = 2500/3 kWh
6 ETA_SYM = float(np.sqrt(ETA)) # 若 90% 指往返效率, 单程对称取 sqrt(0.9)
7
8 def load_attach1():
9     """读取附件 1, 并校验时段标签的完整性与单调性。"""
10    df = pd.read_excel(ATTACH1, sheet_name="Sheet1")
11    df.columns = ["时间", "电价", "小区负载", "光伏发电预测功率"]
12    if len(df) != N:
13        raise ValueError(f"附件 1 应有 {N} 行, 实际 {len(df)} 行")
14    mins = np.array([to_minutes(v) for v in df["时间"]])
15    # 时间标签必须严格递增, 且恰为 {10,20,...,1440} 分钟
16    if not np.all(np.diff(mins) > 0):
17        raise ValueError("附件 1 时间标签非严格递增, 可能被重排或替换")
18    if list(mins) != list(range(10, 1441, 10)):
19        raise ValueError("附件 1 时间标签不是 {10,20,...,1440} 分钟")
20    # 第 t 行对应区间 [(t-1)*10min, t*10min)
21    df.insert(0, "区间序号", np.arange(N))
22    df["区间标签"] = [interval_label(i) for i in range(N)]
23    return df
    
```

2.2 模型装配与求解 (两种 LP 松弛与完整 MILP)

```

1 # 变量分块: [g(0:144) | c(144:288) | d(288:432) | s(432:576) | E(576:720) | z]
2 iG, iC, iD, iS, iE, iZ = 0, N, 2 * N, 3 * N, 4 * N, 5 * N
3 has_mutex = var.binary or var.relax_mutex
4 nz = N if has_mutex else 0
5 nvar = 5 * N + nz
6
7 # 目标: min J = sum_t p_t * g_t (g 已是电量, 不再乘 TAU)
8 cvec = np.zeros(nvar)
    
```

```

9 cvec[iG:iG + N] = price
10
11 # (1) 供需平衡 (等式 + 显式弃光) :
12 #  $g_t + (v_t - s_t) + d_t = l_t + c_t$ 
13 for i in range(N):
14     add_eq(i, [(iG + i, 1.0), (iD + i, 1.0),
15                (iC + i, -1.0), (iS + i, -1.0)], load_kwh[i] - pv_kwh[i])
16
17 # (2) 状态递推:  $E_t = E_{t-1} + \eta_c c_t - d_t / \eta_d$ 
18 for i in range(N):
19     coeffs = [(iE + i, 1.0)]
20     if i > 0:
21         coeffs.append((iE + i - 1, -1.0))
22     coeffs.append((iC + i, -ec))
23     coeffs.append((iD + i, 1.0 / ed))
24     add_eq(N + i, coeffs, E_INIT if i == 0 else 0.0)
25
26 # (3) 日循环:  $E_{144} = E_0 = 6000$ 
27 add_eq(2 * N, [(iE + N - 1, 1.0)], E_INIT)
28
29 # (4) LP2 与 MILP 保留互斥不等式; LP1 不引入 z
30 if has_mutex:
31     for i in range(N):
32         ub_rows += [2*i, 2*i]; ub_cols += [iC + i, iZ + i]
33         ub_vals += [1.0, -M_ENERGY]; ub_rhs.append(0.0)
34         ub_rows += [2*i+1, 2*i+1]; ub_cols += [iD + i, iZ + i]
35         ub_vals += [1.0, M_ENERGY]; ub_rhs.append(M_ENERGY)
36
37 # (5) 电量边界:  $0 \leq c, d \leq M\_ENERGY, 0 \leq s \leq pv \cdot \tau$ 
38 lb = np.concatenate([np.zeros(N), np.zeros(N), np.zeros(N),
39                      np.zeros(N), np.full(N, E_MIN), np.zeros(nz)])
40 ub = np.concatenate([np.full(N, np.inf), np.full(N, M_ENERGY), np.full(N, M_ENERGY)
41                      ,
42                      pv_kw.copy() * TAU, np.full(N, E_MAX), np.ones(nz)])
43 integrality = np.zeros(nvar)
44 if var.binary:
45     integrality[iZ:iZ + N] = 1 # 0 = 连续, 1 = 整数 (二元)
46
47 res = milp(cvec, constraints=[LinearConstraint(A_eq, b_eq, b_eq), ...],
48            integrality=integrality, bounds=Bounds(lb, ub))

```

2.3 解的正确性独立复核

```

1 def validate(sol, df, var, tol=1e-6):
2     """对最优解做独立复核, 返回问题清单 (空列表 = 全部通过)。"""
3     problems = []
4     # 1) 逐段供需平衡 (等式 + 显式弃光)
5     resid = sol.g + (pv_kwh - sol.s) + sol.d - load_kwh - sol.c
6     if np.abs(resid).max() > tol * max(1.0, load_kwh.max()):
7         problems.append(f"供需平衡残差过大: {np.abs(resid).max():.3e}")
8     # 2) 弃光量非负且不超过光伏出力
9     if sol.s.min() < -tol: problems.append("弃光量为负")
10    if (sol.s - pv_kwh).max() > tol: problems.append("弃光量超过光伏可用出力")
11    # 3) 充放电电量限值  $M = 2500/3$  kWh
12    if sol.c.max() > M_ENERGY + tol: problems.append("充电量超限")
13    if sol.d.max() > M_ENERGY + tol: problems.append("放电量超限")
14    # 4) 储电量上下界 (145 个状态点)
15    Eseq = np.concatenate([sol.E0, sol.E])
16    if Eseq.min() < E_MIN - tol: problems.append("储电量低于下界")
17    if Eseq.max() > E_MAX + tol: problems.append("储电量高于上界")
18    # 5) 状态递推自洽
19    Erec = [sol.E0]

```

```

20 for i in range(N):
21     Erec.append(Erec[-1] + ec * sol.c[i] - sol.d[i] / ed)
22 if np.abs(np.array(Erec[1:]) - sol.E).max() > tol:
23     problems.append("储电量递推不自洽")
24 # 6) 日循环:  $E_{144} = E_0$ , 且  $\text{sum}(d) = \text{eta}_c * \text{eta}_d * \text{sum}(c)$ 
25 if abs(sol.E[-1] - sol.E0) > tol: problems.append("E144 != E0")
26 if abs(sol.d.sum() - ec * ed * sol.c.sum()) > tol * max(1.0, sol.d.sum()):
27     problems.append("sum(d) != eta_c*eta_d*sum(c)")
28 # 7) 充放电互斥 (题目未直接给出、需自证的物理约束)
29 if ((sol.c > tol) & (sol.d > tol)).any():
30     problems.append("存在区间同时充放电")
31 # 8) 费用复算
32 if abs(float(np.sum(price * sol.g)) - sol.obj) > tol * max(1.0, abs(sol.obj)):
33     problems.append("目标函数复算不符")
34 return problems
35
36 # LP1 删除模式变量; LP2 放宽模式变量; MILP 保持二元模式
37 lp1_sol = solve(df, Variant(name="LP1"))
38 lp2_sol = solve(df, Variant(name="LP2", relax_mutex=True))
39 milp_sol = solve(df, Variant(name="MILP", binary=True))
40 lower_bound_order = lp1_sol.obj <= lp2_sol.obj <= milp_sol.obj + 1e-6

```

C 问题一结果文件说明

result1.xlsx 按附件 5 模板的两个工作表组织:

- **计划购电量**: 144 行, 逐 10 分钟时段的购电量 (kWh), 时间标签统一为正文口径的“起始-结束”, 自 0:00-0:10 至 23:50-0:00+1;
- **充放电量**: 6 行 4 小时时段汇总的充电量、放电量 (kWh), 末段标签按题目表 2 记为 20:00-24:00, 以及 0:00 与 24:00 的储电量 (kWh), 均为 6000.00。

此外, 程序另输出内部结果明细表 p1_detail.csv, 逐时段给出时段起止、电价、负载电量、光伏电量、购电量、充电量、放电量、弃光量、期初储电量、期末储电量与时段费用; 论文表 1、表 2 与上述结果工作簿均由该明细表汇总得到, 以避免多处分别计算导致口径不一致。

D 问题二、问题三、问题四的标定记录与校验明细

以下各表给出问题二至问题四的参数标定、汇总结果与一致性检验明细，正文中的相应结论均由这些表格支持。

表 13 各策略在 2025 年 2 月 1 日–12 月 31 日的费用对照（采用同一预测模型和实时执行规则，并分别标定参数）

策略	计划购电费/元	紧急购电费/元	合计费用/元	期末储电量/kWh	相对本模型费用
本文风险修正策略（主模型）	13,184,221	755,499	13,939,720	1,559.7	—
预测均值策略（不加风险余量）	12,194,812	2,708,867	14,903,679	5,960.7	+6.92%
无储能（仍提前计划、五倍补缺）	17,540,767	978,624	18,519,391	6,000.0	+32.85%
事后理想（完全预知当天净负荷）	12,110,855	0	12,110,855	1,200.0	-13.12%

注：“事后理想”以当天实际净负荷替换预测值求解同一日前 MILP，并沿用主策略的实际日初储电量，不设置日末储备约束，其日末储电量日均为 1,200.0 kWh。该方案作为理想信息条件下的费用参照。未列以本模型费用为分母；以各基准费用为分母时，节约率分别为 6.47%（预测均值策略）和 24.73%（无储能策略）。

表 14 问题二对照策略的储能期末差异及库存折算费用

策略	期初储电量/kWh	期末储电量/kWh	库存折算费用/元	可比费用/元
风险修正策略	2,287.7	1,559.7	557.76	13,940,278.22
预测均值策略	7,097.3	5,960.7	870.86	14,904,550.07
无储能策略	6,000.0	6,000.0	0.00	18,519,390.60
事后理想策略	2,287.7	1,200.0	833.38	12,111,688.15

注：可比费用为账单费用加上评价区间内净动用储电量按 0.766197 元/kWh 折算的价值。该指标仅用于识别不同策略期初、期末储能状态的影响，不改变实际购电账单。

表 15 问题二正式区间（2025 年 2 月 1 日–12 月 31 日）的总体结果与预测精度

项目	数值
正式区间	2025-02-01 至 2025-12-31（334 天）
计划购电量	21,463,158.37 kWh
计划购电费	13,184,221.29 元
紧急购电量	140,301.24 kWh
紧急购电费	755,499.17 元
合计购电量	21,603,459.62 kWh
合计购电费	13,939,720.46 元
紧急费占比	5.42%
弃用电量	2,463,544.98 kWh
期末储电量	1,559.72 kWh
负载预测 MAE	1,915.8 kWh/日（1.73%）
光伏预测 MAE	2,114.2 kWh/日（3.70%）
净负荷预测 MAE	3,158.8 kWh/日
净负荷被低估（逐时段）	48.9% 的时段，3,158.3 kWh/日
净负荷被高估（逐时段）	51.1% 的时段，3,167.2 kWh/日
风险余量为负的时段	8.5%（最小 -40.6、最大 258.5 kWh）

表 16 问题二六项一致性检验结果

检查项	内容	结果
信息时点	预测、误差分位数与参数仅用当天之前的数据；计划购电量在 0:00 后冻结	通过
物理可行性	逐段供需平衡；储电量在 [1200, 10800] kWh；充放电量 $\leq M$ 且同段互斥	通过
跨日连续性	次日 0:00 储电量严格等于前日 24:00 实际末值	通过
状态累计	$E_{n,144} - E_{n,0} = \eta_c \sum_t c_{n,t} - \frac{1}{\eta_d} \sum_t d_{n,t}$ 逐日成立	通过
费用一致性	未舍入数据复算计划费与紧急费；弃电不退款；储备罚不计入账单	通过
汇总一致性	日期完整无错位；紧急事件合并前后电量一致	通过

4.1 问题四：不退款计费口径的敏感性时序仿真

题目规定调减部分的违约电价为交易时刻电价的 50%，但未明确计划购电费用的计费基数。主口径允许调减：被取消部分退回原价，并按 50% 电价支付违约金。敏感性

表 17 滚动参数标定的选中结果（窗口 35 天，每 42 天重标定一次）

标定日	α	ρ	λ	窗口内选中策略
2025-02-01	0.75	0.9	0.00	$\alpha=0.75, \rho=0.9, \lambda=0.00$
2025-03-15	0.75	0.9	0.00	$\alpha=0.75, \rho=0.9, \lambda=0.00$
2025-04-26	0.80	0.5	0.00	$\alpha=0.80, \rho=0.5, \lambda=0.00$
2025-06-07	0.90	0.9	0.00	$\alpha=0.90, \rho=0.9, \lambda=0.00$
2025-07-19	0.70	0.9	0.00	$\alpha=0.70, \rho=0.9, \lambda=0.00$
2025-08-30	0.70	0.9	0.00	$\alpha=0.70, \rho=0.9, \lambda=0.00$
2025-10-11	0.75	0.8	0.00	$\alpha=0.75, \rho=0.8, \lambda=0.00$
2025-11-22	0.75	0.5	0.00	$\alpha=0.75, \rho=0.5, \lambda=0.00$

全年共标定 8 次： $\alpha = 0.70$ 2 次、 $\alpha = 0.75$ 4 次、 $\alpha = 0.80$ 1 次、 $\alpha = 0.90$ 1 次； λ 在所有标定中均取 0.00

表 18 八种预报组合在 2025 年 2 月 1 日–12 月 31 日的实际费用（参数固定为 $\alpha = 0.6, \rho = 0.5, \lambda = 1.0, W = 28$ ，评价区间、负载模型、执行规则与计费口径完全一致）

使用的预报时刻	计划调整费/元	紧急费/元	合计费用/元	相对基准	期末储电量/kWh
仅 0:00	12,595,136	2,382,673	14,977,809	—	6,007.5
6:00	12,767,094	1,712,866	14,479,959	-3.32%	6,007.5
12:00	12,767,834	1,264,057	14,031,892	-6.32%	6,007.5
6:00+12:00	12,874,840	1,006,040	13,880,879	-7.32%	6,007.5
18:00	13,187,014	612,575	13,799,589	-7.87%	5,798.2
6:00+18:00	13,259,141	273,874	13,533,015	-9.65%	5,798.2
12:00+18:00	13,033,851	546,355	13,580,206	-9.33%	5,798.2
6:00+12:00+18:00	13,145,690	272,936	13,418,626	-10.41%	5,798.2
主策略（全用）	13,145,690	272,936	13,418,626	-10.41%	5,798.2

注：所有方案都使用 0:00 发布的预报；组合列出的时刻是额外使用的版本。首行“仅 0:00”是关闭全部调整的对照，其购电量不随日内预报变化。“相对基准”以该对照为分母，负值表示比不做调整更省。合计费用已含计划部分，不能再与计划调整费、紧急费相加。

表 19 附件 3 四个发布版本在 2025 年 2 月 1 日–12 月 31 日的光伏出力预报精度（单位：kW；把整点预报因果内插到十分钟后与实际光伏逐段比较）

预报版本	当天可用时段	可用时段数	覆盖段绝对误差	其中 12:00–18:00
0:00 发布	0:00–24:00	48,096	206.6	445.8
6:00 发布	6:00–24:00	36,072	180.0	304.9
12:00 发布	12:00–24:00	24,048	98.6	181.3
18:00 发布	18:00–24:00	12,024	15.9	—

注：四个版本的覆盖时段长度不同，因此采用绝对误差。18:00 版仅覆盖光伏接近零的夜间，相对误差会受较小分母影响。在四版共同覆盖的 18:00–24:00 时段，误差均为 15.7–15.9 kW；18:00 版不覆盖白天，相应单元格留空。预报版本的经济作用还取决于其对应的调整时段。

表 20 主策略参数的单变量扰动（每次只动一个参数，其余取 $\alpha = 0.6, \rho = 0.5, \lambda = 1.0, W = 28$ ，评价区间同前）

参数取值	计划调整费/元	紧急费/元	合计费用/元	相对主策略
主策略 $\alpha = 0.6, \rho = 0.5, \lambda = 1.0, W = 28$	13,145,690	272,936	13,418,626	—
$W = 14$	13,165,617	297,334	13,462,951	+0.33%
$W = 21$	13,158,472	282,713	13,441,185	+0.17%
$\alpha = 0.5, \rho = 0.35$	13,053,920	511,608	13,565,528	+1.09%
$\alpha = 0.5, \rho = 0.5$	13,052,126	516,895	13,569,021	+1.12%
$\alpha = 0.5, \rho = 0.7$	13,040,807	545,535	13,586,342	+1.25%
$\alpha = 0.6, \rho = 0.35$	13,146,530	271,409	13,417,938	-0.01%
$\alpha = 0.6, \rho = 0.7$	13,139,824	286,336	13,426,160	+0.06%
$\alpha = 0.7, \rho = 0.35$	13,327,609	137,247	13,464,856	+0.34%
$\alpha = 0.7, \rho = 0.5$	13,327,039	137,873	13,464,912	+0.34%
$\alpha = 0.7, \rho = 0.7$	13,325,109	141,702	13,466,811	+0.36%
$\alpha = 0.8, \rho = 0.35$	13,648,143	67,335	13,715,479	+2.21%
$\alpha = 0.8, \rho = 0.5$	13,647,810	67,850	13,715,659	+2.21%
$\alpha = 0.8, \rho = 0.7$	13,646,892	69,426	13,716,318	+2.22%
$\lambda = 0.0$	13,134,961	300,666	13,435,627	+0.13%
$\lambda = 0.5$	13,135,859	272,916	13,408,774	-0.07%

注： α 为净负荷预测的风险分位数， ρ 为实时放电储备比例， λ 为日末储电量回归目标的引导权重， W 为分位数所用的历史窗口天数。表中省略了与主策略完全相同的取值行。

表 21 计费口径与风险分位数口径的敏感性（模型、参数、评价区间均与主策略相同，只换这一处口径）

口径	计划调整费/元	紧急费/元	合计费用/元	相对主策略
主策略：退款口径 + 跨时段合并分位数	13,145,690	272,936	13,418,626	—
调减不退款口径	13,357,217	261,727	13,618,944	+1.49%
逐时段分位数	13,082,057	302,772	13,384,829	-0.25%

注：题目未明确调减部分的原价是否退回。主策略采用退款口径，即调减部分按 $0.5p_t$ 结算；不退款口径仍支付原计划费用。本表并列报告两种计费结果。第二项比较跨时段合并分位数与逐时段分位数，用于衡量风险估计粒度的影响。

表 22 参数的 1 月标定记录：标定面，以及用 1 月最优参数直接外推的检验

风险分位数 α	$\rho = 0.35$	$\rho = 0.5$	$\rho = 0.7$	该行最小
0.4	1,802,432	1,811,016	1,822,471	1,802,432
0.5	1,770,321	1,778,833	1,790,196	1,770,321
0.6	1,764,349	1,772,822	1,784,039	1,764,349
0.7	1,782,106	1,790,579	1,801,805	1,782,106
0.8	1,830,510	1,838,983	1,850,209	1,830,510
窗口 $W = 14$	1,771,570			(采用)
窗口 $W = 21$	1,772,674			
窗口 $W = 28$	1,772,822			
$\lambda = 0.0$	1,774,575			(日末储电仅 1,806 kWh)
$\lambda = 0.5$	1,769,148			
$\lambda = 1.0$	1,772,822			

上半部分为 1 月标定面（ $W = 28$ ，单位：元），下半部分为 W 与 λ 的单变量扫描。标定面在 α 方向有明确的极小点 $\alpha = 0.6$ ，在 ρ 与 W 方向则相当平坦： $\rho \in [0.35, 0.5]$ 、 $W \in [14, 28]$ 的全部取值都落在同一水平内，彼此相差不到 1%。**样本外检验：**1 月标定面上的最小值为 $(\alpha, \rho, W) = (0.6, 0.35, 14)$ ，1 月费用 1,763,097 元；本文采用的 $(0.6, 0.5, 28)$ 对应 1,772,822 元，相差 9,725 元。在完整评价区间内，两组参数的费用分别为 13,461,840 元和 13,418,626 元，后者低 43,213 元。考虑标定面在 ρ 、 W 方向较平坦，采用中心取值 $(0.6, 0.5, 28)$ 。 $\lambda = 1.0$ 与 $\lambda = 0.5$ 在评价区间内相差 0.07%（9,852 元）； $\lambda = 0$ 时日末储电量为 1,806 kWh，因此保留非零日末储备权重。

表 23 九项检查在主策略全部 334 个交付日上的实测结果

检查项	实测最坏值	未通过天数
逐段能量平衡	$2.27e - 13 \text{ kWh}$	0
储能状态递推	$0.00e + 00 \text{ kWh}$	0
计划调整费复算	$0.00e + 00 \text{ 元}$	0
紧急购电费复算	$0.00e + 00 \text{ 元}$	0
储电量上下界	结构上恒成立	0
充放电功率上限	结构上恒成立	0
同时充放电	结构上恒成立	0
购电量非负	结构上恒成立	0
跨日状态衔接	结构上恒成立	0
合计	—	0

注：九项检查在全部交付日上一次通过，无任何残差、越界或复算不符。前四项由独立复算得到最坏值，后五项由线性规划的约束边界与实时执行规则结构性地保证，故逐日实测值恒为零。

表 24 附件 4 实时电价的结构分解 ($P = F + R$, F 为附件 1 的固定曲线, R 为残差)

项目	数值	说明
逐年逐段均值 $\bar{P}$ 与 F 的最大偏差	$5.15e - 05$	两者骨架与水平相同
F 的年平均	0.7662	元/kWh
$\bar{P}$ 的年平均	0.7662	元/kWh, 与 F 相差不足 10^{-6}
R 与 F 的相关系数	-0.0000	正交, 故 R 就是“波动”本身
R 的周一均值	+0.0529	元/kWh
R 的周二均值	+0.0549	元/kWh
R 的周三均值	+0.0576	元/kWh
R 的周四均值	+0.0542	元/kWh
R 的周五均值	-0.1372	元/kWh
R 的周六均值	-0.1364	元/kWh
R 的周日均值	+0.0530	元/kWh
周五、周六平均	-0.1368	元/kWh, 偏低
其余五天平均	+0.0545	元/kWh
两者之差	+0.1913	元/kWh, 相当于均价的 25.0%
星期结构解释的 R 方差	41.7%	其余为无规律的日内噪声
“同星期加权”恒等式最大残差	0.00	见式 (8.2)

注：附件 4 的电价可表示为附件 1 的 144 时段基准曲线与星期周期残差之和。两者年平均值相同，因此对照中的费用差异主要来自价格波动。周五、周六残差均值较低，据此采用同星期加权预测模型。

表 25 三种价格信息条件下的策略在同一实际价格路径上的独立时序仿真（见第 6.1 节）

分支	策略	计划费/元	调整费/元	紧急费/元	合计费用/元
4-2	波动电价预测策略	14,041,884	0	698,929	14,740,813
4-2	固定电价参考策略	14,044,327	0	716,626	14,760,952
4-2	未来价格已知参考	13,911,363	0	735,304	14,646,668
4-2	ΔJ (固定电价 - 波动电价)	—	—	—	20,139 (+0.14%)
4-3	波动电价预测策略	13,802,989	197,228	153,069	14,153,286
4-3	固定电价参考策略	13,841,150	192,888	155,479	14,189,517
4-3	未来价格已知参考	13,726,007	176,274	173,391	14,075,672
4-3	ΔJ (固定电价 - 波动电价)	—	—	—	36,231 (+0.26%)

注：三种策略均在同一条实际价格路径上结算。 ΔJ 定义为“固定电价参考”费用减去“波动电价预测”费用，用于衡量价格预测的经济效益。“未来价格已知参考”假设当天价格在 0:00 已全部公布，作为完全信息参照，不计入 ΔJ 。三种策略分别依据各自的信息集独立标定。

表 26 价格预测模型选择：在 2025 年 1 月预热期（评价区间之外）比较候选模型，并在子窗口上检验稳定性

候选预测器	全窗口（30 天）后段（9 天）后半（16 天）		
同星期加权 0.5/0.3/0.2	0.0486	0.0351	0.0341
同星期等权 1/3	0.0487	0.0353	0.0343
同星期加权 0.6/0.3/0.1	0.0488	0.0355	0.0344
同星期（上周同一天）	0.0512	—	—
近 7 天滚动均值	0.0847	—	—
近 14 天滚动均值	0.0848	—	—
近 28 天滚动均值	0.0852	—	—
昨天同时段	0.0861	—	—
选中的一组：权重 0.5/0.3/0.2， $\beta = (1, 0.75, 0.5)$			

注：单位为元/kWh，指标为各候选在全天 144 个时段的平均绝对误差。“全窗口”为第 1–30 天；“后段”为滞后 7/14/21 天样本均齐备的第 22–30 天；“后半”为第 15–30 天。三个窗口的最优候选一致，因此评价期内固定该模型。 β 按发布时刻分别选择，取值见上表最后一行。

表 27 价格预测候选方案在 2025 年 2 月 1 日–12 月 31 日的平均绝对误差（样本外复核，不用于选型）

预测方案	平均绝对误差/(元/kWh)	相对均价
同星期加权 0.6/0.3/0.1	0.0441	5.82%
同星期加权 0.5/0.3/0.2（本文冻结使用）	0.0446	5.88%
同星期等权 1/3	0.0466	6.15%
同星期（上周同一天）	0.0472	6.22%
近 7 天滚动均值	0.0827	10.91%
近 14 天滚动均值	0.0838	11.07%
昨天同时段	0.0843	11.13%
近 28 天滚动均值	0.0867	11.45%
全样本固定日内形状（事后口径，仅作下界）	0.0964	12.72%

注：“相对均价”以评价区间 334 天的均价 0.7575 元/kWh 为基准，全年均价为 0.7662 元/kWh。“全样本固定日内形状”使用评价区间均值，包含未来信息，仅作为误差参照。近 N 天滚动均值未区分星期类型，因此未能提取星期偏移。评价区间内 0.6/0.3/0.1 的误差为 0.0441，预先选定的 0.5/0.3/0.2 为 0.0446，相差 0.0005 元/kWh。

表 28 日内对数偏差修正系数 β 的扫描（对剩余时段的平均绝对误差，评价区间）

β	6:00 版	12:00 版	18:00 版
0	0.0488	0.0472	0.0428
0.25	0.0470	0.0448	0.0405
0.5	0.0461	0.0436	0.0403
0.75	0.0462	0.0436	0.0419
1	0.0470	0.0448	0.0447
1.25	0.0485	0.0468	0.0485

注： $\beta = 0$ 表示不作日内修正。加粗行 $\beta = (1, 0.75, 0.5)$ 由 1 月预热期选定，并在评价区间内固定；若在评价区间内选择，结果为 $\beta = 0.5$ 。当 $\beta \geq 1.25$ 时，修正幅度增大，预测误差随之上升。

表 29 价格预测的四个发布版本在各自发布时间更新后的精度

发布版本	覆盖时段	平均绝对误差	相对均价	共同时段误差
0 点版本	0–144	0.0446	5.88%	0.0428
6 点版本	36–144	0.0470	6.20%	0.0389
12 点版本	72–144	0.0436	5.76%	0.0409
18 点版本	108–144	0.0403	5.33%	0.0403
对照：全天用同一个均价	全天	0.2774	36.62%	—

注：“覆盖时段”为时段索引范围。各版本仅修正发布后的时段，因此平均绝对误差的原始统计区间不同。最后一列统一在 18:00–24:00 上计算误差，用于比较四个版本；加粗者为该口径下的最小值。最后一行使用全天统一均价，不含日内形状信息，其误差高于四个分时预测版本。

表 30 风险余量口径对照：以实际电价为权重的分位数 vs 等权分位数（正式区间按每 7 天抽样求均值）

α	价格加权/(kWh)	等权·线性/(kWh)	等权·下确界/(kWh)	比线性插值	比下确界
0.50	1.322	-1.461	-3.806	—	—
0.60	14.503	11.362	10.380	+27.65%	+39.71%
0.70	28.953	25.367	25.911	+14.14%	+11.74%
0.80	46.264	41.997	44.774	+10.16%	+3.33%

注：“等权·线性”采用 `numpy` 的线性插值分位数；“等权·下确界”与“价格加权”均采用下确界定义，因此末列仅反映权重变化，倒数第二列还包含分位数定义差异。 $\alpha = 0.50$ 时余量为负，故相对变化以“—”表示。

表 31 净负荷预测误差 ε 与实际电价的协方差检验

项目	数值
样本数	48 096
合并口径相关系数	+0.0863
逐时段相关系数均值	+0.216
逐时段相关系数范围	+0.085 +0.402
逐时段相关为正的比率	100.0%
电价最高 20% 时段的 ε 均值	+40.5 kW
电价最低 20% 时段的 ε 均值	-74.7 kW
两者之差	+115.2 kW

注：合并口径同时包含时段间价格差异；固定时段后，相关系数由 +0.086 升至 +0.216，且全部时段均为正。因此， $\text{Cov}(P, \varepsilon) > 0$ 主要体现为同一时段的跨日相关性，风险余量据此按时段分别计算。

表 32 波动电价下 4-3 的八种预报使用组合（各组合单独标定参数，评价区间与计费口径完全一致）

使用的预报时刻	初始计划费/元	调整费/元	紧急费/元	合计费用/元	相对 $\varnothing$	期末储电量/kWh
6,12,18	13,802,989	197,228	153,069	14,153,286	-5.79%	5,889.6
6,18	13,802,526	302,659	153,716	14,258,902	-5.09%	5,889.6
12,18	13,880,759	83,222	374,151	14,338,131	-4.56%	5,889.6
6,12	14,076,549	16,306	277,579	14,370,434	-4.34%	6,062.4
18	13,880,487	236,489	386,855	14,503,831	-3.46%	5,889.6
12	14,242,219	-77,058	412,319	14,577,480	-2.96%	6,234.6
6	13,974,367	82,343	548,690	14,605,400	-2.78%	6,242.3
仅 0:00	14,209,197	0	813,704	15,022,901	—	6,242.3

注：与问题三的同一张表对照可以看出价格波动改变了什么——在固定电价下“用不用新预报”主要体现在调整费与紧急费的此消彼长；在波动电价下，更新预报同时改善了电量与价格两项判断（6:00 起既能看到当天更准的负载与光伏，也能用当天已实现的价格修正剩余时段的电价预测），因此增量收益的来源更宽。所有方案都使用 0:00 发布的预报，组合列出的时刻是额外使用的版本。

口径要求初始计划费全额支付，调减部分另付 50% 违约金，因此不执行调减，对应约束 $x_t \geq g_t^0$ 。本节在不退款口径下重新标定 4-3 并进行时序仿真。由于费用函数和可行域均发生变化，两种计费口径分别标定参数。

表 33 两个分支在 334 个交付日上的检查结果（未通过天数均为零）

检查项	result4-2 未通过天数	result4-3 未通过天数
信息边界	0	0
计划冻结	0	0
物理可行性	0	0
状态累计	0	0
费用一致性	0	0
调整权限	0	0
数值口径的最坏值		
逐段电量平衡最大残差/kWh	0.00	0.00
计划费重算最大偏差/元	0.00	0.00
调整费重算最大偏差/元	—	0.00
紧急费重算最大偏差/元	0.00	0.00
跨日衔接最大偏差/kWh	0.00	0.00
全局储电量范围/kWh	1,200.0 10,800.0	1,200.0 10,800.0
储电下界余量/kWh	0.000	0.000
储电上界余量/kWh	0.000	0.000

注：残差均小于 10^{-12} kWh。储电量上下界余量为零，表明个别日期的容量约束达到边界。result4-2 的全天计划在 0:00 确定，执行阶段不发生第二次常规购电。

表 34 4-3 退款主口径与不退款敏感性口径的结果对照（两种口径分别标定）

指标	主口径（退款）	不退款口径	差异（不退款 - 主口径）
合计费用	14,153,285.53	14,402,403.87	+249,118.34
其中计划费	13,802,988.74	13,778,495.35	-24,493.39
其中调整费	197,228.01	502,334.29	+305,106.28
其中紧急费	153,068.79	121,574.23	-31,494.56
计划量（初始）	21,399,492.24	21,344,252.60	-55,239.64
最终常规量	20,842,200.33	21,757,381.40	+915,181.06
上调量	436,206.27	413,128.80	-23,077.48
下调量	993,498.18	0.00	-993,498.18
紧急购电量	29,324.65	23,843.42	-5,481.23
弃电量	1,667,711.93	2,601,538.42	+933,826.49
期末储电量	5,889.57	6,099.71	+210.14
调整提交次数	1,002	1,002	—

注：退款口径下，被取消部分退回原价并按 50% 计收违约金；不退款口径下，计划购电费全额计收，50% 为额外违约金。两种口径分别标定参数并求解，因此表中差异同时包含计费规则与参数选择的影响。两种口径下 4-3 相对 4-2 的费用关系一致。

E 问题二求解核心代码

问题二的主计算由 03_ 代码/p2_microgrid.py 完成，共约 1560 行；表格由 03_ 代码/p2_tables.py 从结果文件自动生成， α 敏感性验证由 03_ 代码/p2_alphasweep.py 完成，三者均为可独立运行的全部源码，随支撑材料一并提交。下面按“预测层 → 风险层 → 计划层 → 执行层 → 标定层 → 校验层”依次摘录 p2_microgrid.py 中的核心片段。

5.1 因果预测模型与净负荷预测误差表

预测模型只允许访问第 n 天之前的历史日，因此“信息时点”要求由数据结构本身保证，而不是靠调用约定。净负荷预测误差表 $\varepsilon_{n,t}$ 与 α, ρ, λ 无关，故全流程只预计算一次。

```

1 class ForecastParams:
2     """近期加权历史预测的超参数（框架文档 3.2 节）。"""
3     window: int = 15          # 回看窗口天数
4     decay: float = 0.30       # 每 7 天的权重衰减因子
5     weekday_only: bool = False # 是否只用同一星期的历史
6     name: str = "负荷"
7 
```

```

8 # 在正式区间开始前选定并冻结的预测超参数，时序仿真中不再调整。
9 LOAD_FORECAST = ForecastParams(window=28, decay=0.30, weekday_only=True, name="负荷
   ")
10 PV_FORECAST    = ForecastParams(window=7, decay=0.30, weekday_only=False, name="光伏
   ")
11
12 def forecast_weights(n, fp):
13     """返回第 n 天预测所用的（历史日下标，归一化权重），只取 i < n。"""
14     lo = max(0, n - fp.window)
15     idx = np.arange(lo, n)
16     if len(idx) == 0:
17         return idx, np.zeros(0)
18     w = fp.decay ** ((n - 1 - idx) / 7.0)
19     if fp.weekday_only:
20         same = np.array([weekday_of(i) == weekday_of(n) for i in idx])
21         if same.any():
22             w = w * same
23     s = w.sum()
24     if s <= 0:
25         return idx, np.full(len(idx), 1.0 / len(idx))
26     return idx, w / s
27
28 def build_error_table(LOAD, PV, fo):
29     """预计算  $\text{eps}[n, t] = N[n, t] - \text{Nhat}[n, t]$  | 日期 < n]。
30
31     这是"逐日向前预测后得到的误差"，不是模型在训练数据上的拟合残差。
32     第 0 天无历史，记为 NaN 并永不使用。
33     """
34     eps = np.full((N_DAY, N), np.nan)
35     for n in range(1, N_DAY):
36         _, _, nhat = fo.energy(n)
37         eps[n, :] = (LOAD[n, :] - PV[n, :]) * TAU - nhat
38     return eps

```

5.2 风险余量：误差的经验分位数

```

1 def risk_margin(eps, n, rp):
2     """第 n 天各时段的风险余量  $Q_{\{\alpha, n, t\}}$  (kWh) 。
3
4     只取 n 之前已结束日期的误差；样本不足时合并相邻时段（±2 个 10 分钟），
5     仍然只使用历史数据，不引入同日或未来信息。
6     """
7     if rp.alpha is None or n == 0:
8         return np.zeros(N)
9     lo = max(0, n - rp.window)
10    hist = eps[lo:n, :]
11    hist = hist[~np.isnan(hist).all(axis=1)]
12    k = hist.shape[0]
13    if k >= rp.min_samples:
14        return np.quantile(hist, rp.alpha, axis=0)
15    if k >= 3:
16        pooled = np.empty((k, N))
17        for t in range(N):
18            a, b = max(0, t - 2), min(N, t + 3)
19            pooled[:, t] = hist[:, a:b].mean(axis=1)
20        return np.quantile(pooled, rp.alpha, axis=0)
21    return np.zeros(N)

```

5.3 日前 MILP 的装配与求解

决策变量 $x = (g, c, d, w, \bar{E}, z, \xi)$ 共 $6N + 1 = 865$ 个，其中 $z \in \{0, 1\}^{144}$ 。系数矩阵与日期无关，故在模块加载时装配一次 (`_precomp`)，逐日只替换右端项，单日求解耗时约 46 ms。

```

1 def _precomp():
2     """装配与日期无关的 MILP 结构，逐日只更换右端项，显著加速。"""
3     nvar = 6 * N + 1 # g | c | d | w | E | z | xi
4     iG, iC, iD, iW, iE, iZ, iXi = 0, N, 2*N, 3*N, 4*N, 5*N, 6*N
5
6     # (1) 风险净负荷平衡: g_t + d_t - c_t - w_t = Ntilde_t 行 0..N-1
7     # (2) 状态递推: E_t - E_{t-1} - eta_c c_t + d_t/eta_d = 0 行 N..2N-1
8     # (3) 充放电互斥: c_i - M z_i <= 0, d_i + M z_i <= M 2N 条
9     # (4) 日末储备: -xi - E_{N-1} <= -E_tar 1 条
10    A_eq = csr_matrix((vals, (rows, cols)), shape=(2 * N, nvar))
11    A_ub = csr_matrix((uvals, (urows, ucols)), shape=(2 * N + 1, nvar))
12
13    lb = np.concatenate([np.zeros(N), np.zeros(N), np.zeros(N), np.zeros(N),
14                          np.full(N, E_MIN), np.zeros(N), [0.0]])
15    ub = np.concatenate([np.full(N, np.inf), np.full(N, M_ENERGY),
16                          np.full(N, M_ENERGY), np.full(N, np.inf),
17                          np.full(N, E_MAX), np.ones(N), [np.inf]])
18    integ = np.zeros(nvar)
19    integ[iZ:iZ + N] = 1 # 只有 z 是整数
20    return {"nvar": nvar, "A_eq": A_eq, "A_ub": A_ub, "lb": lb, "ub": ub,
21           "integ": integ, "iG": iG, "iC": iC, "iD": iD, "iW": iW,
22           "iE": iE, "iXi": iXi}
23
24    _PC = _precomp()
25
26    def solve_dayahead(ntilde, e0, price, rp, etar=E_TAR):
27        """求解某一天的日前 MILP:
28        min sum_t p_t g_t + lambda * xi, xi >= E_tar - Ebar_144, xi >= 0
29        """
30        b_eq = np.empty(2 * N)
31        b_eq[:N] = ntilde # Ntilde_t (风险净负荷)
32        b_eq[N] = e0 # E_0 = 当日实际日初储电量
33        b_eq[N + 1:] = 0.0
34
35        b_ub = np.empty(2 * N + 1)
36        b_ub[:2 * N] = np.tile([0.0, M_ENERGY], N)
37        b_ub[2 * N] = -etar # 日末储备约束的右端项
38
39        cvec = np.zeros(nvar)
40        cvec[iG:iG + N] = price # 计划购电按分时电价计费
41        cvec[iXi] = rp.lam # 储备不足的规划惩罚 (不计入账单)
42
43        res = milp(cvec,
44                  constraints=[LinearConstraint(_PC["A_eq"], b_eq, b_eq),
45                               LinearConstraint(_PC["A_ub"], -np.inf, b_ub)],
46                  integrality=_PC["integ"], bounds=Bounds(_PC["lb"], _PC["ub"]))
47        if not res.success:
48            raise RuntimeError(f"日前 MILP 求解失败: {res.message}")
49        ...

```

5.4 实时执行规则

执行层的核心特征是因果性：第 t 段的动作只依赖本段内已实时测量到的实际负载与光伏、以及当前储电量，不依赖任何未来时段的信息；购电量 g_t 由日前冻结、执行层无修改入口。

```

1 def execute_day(plan_g, Ebar, l_act, v_act, e0, price, rp,
2                 rule="greedy", allow_storage=True):
3     """实时执行：购电计划固定，储能按当前已观测情况因果响应。
4
5     b_t = g_t + v_t - l_t 为使用储能前的电量盈余 (kWh)。
6     b > 0 用盈余充电（充不进去的弃用）；b < 0 在储备线之上放电，
7     剩余缺口按 5 倍电价紧急购电。
8     """
9     for t in range(N):
10        b = plan_g[t] + v_act[t] - l_act[t]
11        R = E_MIN + rp.rho * (Ebar[t] - E_MIN) # 0:00 即可算出的储备线
12
13        if b > 0:
14            bpos = min(b, M_ENERGY)
15            c[t] = min(bpos, (E_MAX - Ep) / ETA) # 盈余全部用于充电
16            w[t] = b - c[t] # 充不进去的弃用（无收益）
17        else:
18            head = max(0.0, Ep - R) # 电量不得低于储备线
19            d[t] = min(-b, M_ENERGY, ETA * head)
20            r[t] = max(0.0, -b - d[t]) # 仍缺的部分紧急购电
21
22        Ep = Ep + ETA * c[t] - d[t] / ETA
23        E[t] = Ep
24
25    cost_plan = float(np.sum(price * plan_g))
26    cost_emg = float(np.sum(5.0 * price * r))
27    return DayExec(g=plan_g.copy(), c=c, d=d, r=r, w=w, E=E, E0=float(e0),
28                  cost_plan=cost_plan, cost_emg=cost_emg)

```

5.5 滚动参数标定

标定在第 n_0 天的 0:00 进行，只用窗口内已经结束的历史日时序仿真，比较包含五倍紧急购电费的实际总费用，取最小者。

```

1 class CalConfig:
2     alphas: tuple = (0.50, 0.60, 0.70, 0.80, 0.90)
3     rhos: tuple = (0.0, 1.0)
4     lams: tuple = (0.0, 0.60)
5     window: int = 35 # 标定时序仿真窗口天数
6     every: int = 42 # 每隔多少天重新标定一次
7     start: int = REPORT_START # 首次标定发生在正式区间第一天
8
9     def grid(self):
10        return [RiskParams(alpha=a, rho=r, lam=l)
11               for a in self.alphas for r in self.rhos for l in self.lams]
12
13 def calibrate(sim, n0, e_at_window_start, cal):
14     """在第 n0 天 0:00 用**此前**数据选择参数。"""
15     lo = max(0, n0 - cal.window)
16     best, best_cost = None, np.inf
17     for rp in cal.grid():
18         cost, _, _ = sim.replay(lo, n0, rp, e_at_window_start, cal.rule,
19                                cal.allow_storage)
20         if cost < best_cost - 1e-9:
21             best_cost, best = cost, rp
22     return best, rows
23
24 def run_strategy(sim, cal, verbose=True):
25     """从 2025-01-01 的 6000 kWh 出发跑完整年，期间滚动标定参数。"""
26     for n in range(N_DAY):
27         if n >= cal.start and (n - cal.start) % cal.every == 0:
28             params, _ = calibrate(

```

```

29         sim, n, e_at_day_start[max(0, n - cal.window)], cal)
30     rec = sim.run_day(n, params, e, cal.rule, cal.allow_storage)
31     records.append(rec)
32     e = rec.E_end # 跨日连续: 次日 0:00 = 前日 24:00
33     e_at_day_start[n + 1] = e
34     return records, cal_rows

```

5.6 六项一致性检验的独立复核

结果不依赖求解器自报的可行性，而是用未四舍五入的数据逐日独立复算。

```

1 def validate_day(rec, tol=1e-6):
2     """逐日复核实际运行结果，返回问题清单（空 = 通过）。框架文档 7.2 节。"""
3     p = []
4     ex, l, v = rec.exec, rec.l_act, rec.v_act
5
6     # 1) 实际逐段供需平衡:  $g + r + v + d = l + c + w$ 
7     resid = ex.g + ex.r + v + ex.d - l - ex.c - ex.w
8     scale = max(1.0, float(l.max()))
9     if np.abs(resid).max() > tol * scale:
10         p.append(f"供需平衡残差 {np.abs(resid).max():.3e} kWh")
11
12     # 2) 物理边界: 非负、不超 M、同段不同时充放电
13     if ((ex.c > tol) & (ex.d > tol)).any():
14         p.append("存在同时充放电的时段")
15
16     # 3) 储电量区间 (含日初, 共 145 个状态点)
17     Eseq = np.concatenate([[ex.E0], ex.E])
18     if Eseq.min() < E_MIN - tol or Eseq.max() > E_MAX + tol:
19         p.append("储电量越界")
20
21     # 4) 状态累计 (问题二不能套用问题一的  $\sum d = 0.81 \sum c$ )
22     lhs = ex.E[-1] - ex.E0
23     rhs = ETA * ex.c.sum() - ex.d.sum() / ETA
24     if abs(lhs - rhs) > tol * max(1.0, abs(rhs)):
25         p.append(f"状态累计不符: dE={lhs:.6f} vs {rhs:.6f}")
26
27     # 5) 费用一致性: 弃电不退款, 储备罚不混入实际账单
28     cp = float(np.sum(_PRICE_REF * ex.g))
29     ce = float(np.sum(5.0 * _PRICE_REF * ex.r))
30     if abs(cp - ex.cost_plan) > tol * max(1.0, abs(cp)):
31         p.append("计划费复算不符")
32     if abs(ce - ex.cost_emg) > tol * max(1.0, abs(ce)):
33         p.append("紧急费复算不符")
34
35     # 6) 计划购电量在 0:00 后冻结 (由架构保证, 此处做存在性校验)
36     if ex.g.shape != (N,) or ex.g.min() < -tol:
37         p.append("计划购电量非法")
38     return p

```

F 问题二结果文件说明

result2.xlsx 按附件 5 模板的三张表组织，并增加一张全天汇总表：

- **表 1 (计划购电量)**：334 行 × 144 列，逐 10 分钟时段的计划购电量 (kWh)，覆盖 2025 年 2 月 1 日至 12 月 31 日；
- **表 2 (充放电量)**：按模板的长表口径排列，334 天 × 6 个 4 小时时段共 2004 行，给出充电量、放电量 (kWh) 以及 0:00 与 24:00 的储电量 (kWh)；
- **表 3 (紧急购电量)**：按“同日连续发生紧急购电的时段合并为一个事件”的规则生

成，共 390 个事件行；

- **全天汇总**（额外）：逐日给出计划购电量、紧急购电量、弃用电量、期初/期末储电量与计划费、紧急费，便于核对正文表 15。

时间标签统一采用正文口径的”起始–结束”，自 0:00–0:10 至 23:50–0:00+1。模板原始标签比该口径整体偏移一格，故结果工作簿由程序从模板复制后**重写**标签列，而非直接填充模板原列。

此外，程序另输出四个内部结果文件，论文中的每个数字均可追溯到其中之一：

- **p2_detail.csv**：逐时段明细（ $334 \times 144 = 48096$ 行），含电价、预测负载/光伏、净负荷预测、风险余量、风险净负荷、参考储电量、实际负载/光伏、计划购电、实际充放电、紧急购电、弃电、期初期末储电量与时段费用；
- **p2_daily.csv**：逐日汇总（334 行），含全天电量、费用、储电量、紧急事件个数与当日生效参数；
- **p2_calibration.csv**：8 次滚动标定 $\times$ 30 个参数组合的全部时序仿真费用，即表 17 的原始记录；
- **p2_results.json**：正文全部汇总数字与六项检查结果的机器可读版本。表 15–17 由 **p2_tables.py** 直接读取该文件与 **p2_detail.csv** 自动生成 (**p2_tables.tex**)，使论文表格与结果文件采用同一数据源；
- **p2_alpha_sensitivity.json**：正文 §6.9.6 弃电成因讨论中 α 敏感性验证的逐点结果（ $\alpha \in \{0.50, 0.70, 0.80, 0.90\}$ 各自的弃电量、紧急购电量与总费用），由 **p2_alphasweep.py** 在与主策略完全相同的预测模型、执行规则与评价区间下另行重跑得到。

G 问题三求解核心代码

问题三的计算链路由三个脚本组成：**03_ 代码/p3_microgrid.py** 给出模型与全部求解器（附件载入、因果插值、负载外推、计划 LP、调整 LP、实时执行规则），**03_ 代码/p3_backtest.py** 做全年因果滚动时序仿真与误差分位数的维护，**03_ 代码/p3_export.py** 写出 **result3.xlsx** 并逐日执行九项校验。论文所用的全部量化分析另由 **03_ 代码/p3_analysis.py**（主结果、八种预报使用方式、两种口径敏感性、单参数扫描、九项校验汇总、预报精度）与 **03_ 代码/p3_calibration.py**（1 月标定面与样本外检验）产生；表格与插图分别由 **p3_tables.py**、**p3_figures.py** 从这些结果文件自动排版，不含任何手工填数。下面按“预报口径 → 两个 LP → 实时执行 → 滚动时序仿真 → 校验”依次摘录核心片段。

7.1 附件 3 的载入与因果分段线性插值

附件 3 的 24 个值是目标整点的瞬时功率，索引 j 对应发布后 j 小时。插值的左端节点不能用“预报 0 小时”（附件没有给出），而取发布时刻所在时段的实际功率，并截断到非负；发布时刻之前的时段返回空值，避免把零填进分位数样本。

```
1 def pv_interp_for_issue(anchor_h:int, anchor_v_kw:float, fc24:np.ndarray):
2     """返回当天0-24h内144个slot的PV功率预测(kw)，因果插值。超出预报范围的返回nan。"""
3     t_nodes=np.arange(anchor_h, anchor_h+25) # anchor, +1..+24
4     v_nodes=np.concatenate([[anchor_v_kw], fc24])
5     slot_mid=(np.arange(N)+0.5)/6.0 # 0.083..23.917
6     out=np.full(N, np.nan)
7     mask=(slot_mid>anchor_h)&(slot_mid<=anchor_h+24)
8     out[mask]=np.interp(slot_mid[mask], t_nodes, v_nodes)
9     return np.maximum(out,0.0)
```

7.2 负载的因果外推

负载没有外部预报，只用历史实际值按“同星期优先”外推：先取 $d-7$ ，退化时取 $d-1$ ，再退化时取已有历史均值。任何一支都只用 d 之前的数据。

```
1 def load_predict(day_idx:int, load_kw:np.ndarray, dates)-> np.ndarray:
2     """同星期优先: d-7; 无则 d-1; 再无则已有均值。因果。返回kw 144."""
3     if day_idx-7>=0:
4         return load_kw[day_idx-7,:].copy()
5     if day_idx-1>=0:
6         return load_kw[day_idx-1,:].copy()
7     return np.full(N, load_kw[:max(1,day_idx),:].mean() if day_idx>0 else 4000.0)
```

7.3 计划 LP: 0:00 的风险修正日计划

目标只有购电费与日末储备的引导项， ξ 是日末储电量低于目标值的缺口。模型不含紧急购电变量，也不含整数变量；购电量只有非负下界，其有效上限由平衡式与储能容量自然给出。

```
1 def solve_lp_plan(price, net_risk_kwh, E0, E_tar=6000.0, lam=1.0):
2     prob=pulp.LpProblem("plan", pulp.LpMinimize)
3     g=[pulp.LpVariable(f"g{i}", lowBound=0) for i in range(N)]
4     c=[pulp.LpVariable(f"c{i}", lowBound=0, upBound=M_ENERGY) for i in range(N)]
5     d=[pulp.LpVariable(f"d{i}", lowBound=0, upBound=M_ENERGY) for i in range(N)]
6     w=[pulp.LpVariable(f"w{i}", lowBound=0) for i in range(N)]
7     E=[pulp.LpVariable(f"E{i}", lowBound=E_MIN, upBound=E_MAX) for i in range(N)]
8     xi=pulp.LpVariable("xi", lowBound=0)
9     prob+=pulp.lpSum([price[i]*g[i] for i in range(N)])+lam*xi
10    for i in range(N):
```

```

11 prob+=g[i]+d[i]==net_risk_kwh[i]+c[i]+w[i]
12 for i in range(N):
13     prev=E0 if i==0 else E[i-1]
14     prob+=E[i]==prev+EC*c[i]-d[i]/ED
15 prob+=xi>=E_tar-E[N-1]
16 prob.solve(pulp.PULP_CBC_CMD(msg=0))

```

7.4 调整 LP：退款口径的调价项

问题三主策略采用退款口径，调整项为 $1.5p_t u_t - 0.5p_t v_t$ ；下调量以初始计划量为上界，保证生效购电量非负。此处两个方向的调整变量可用线性规划处理：同时抬高 u_t, v_t 会多付 $p_t \delta$ ，因此最优解中二者不会同时为正。

```

1 def solve_lp_adjust(price, net_risk_kwh, E0, gP, idx_from, E_tar=6000.0, lam=1.0):
2     R=list(range(idx_from,N))
3     gA={i:pulp.LpVariable(f"gA{i}", lowBound=0) for i in R}
4     up={i:pulp.LpVariable(f"up{i}", lowBound=0) for i in R}
5     dn={i:pulp.LpVariable(f"dn{i}", lowBound=0) for i in R}
6     ...
7     prob+=pulp.lpSum([1.5*price[i]*up[i]-0.5*price[i]*dn[i] for i in R])+lam*xi
8     for i in R:
9         prob+=gA[i]==gP[i]+up[i]-dn[i]
10        prob+=dn[i]<=gP[i]+1e-9
11        prob+=gA[i]+d[i]==net_risk_kwh[i]+c[i]+w[i]
12    for k,i in enumerate(R):
13        prev=E0 if k==0 else E[R[k-1]]
14        prob+=E[i]==prev+EC*c[i]-d[i]/ED
15    prob+=xi>=E_tar-E[R[-1]]
16    prob.solve(pulp.PULP_CBC_CMD(msg=0))

```

7.5 实时执行层：储备线与因果充放电规则

储备线把参考轨迹 $\bar{E}_t$ 映射为一个“至少留这么多电”的下限，执行层据此决定放电深度。规则只用到当段的实际负载与光伏、以及上一段的实际储电量，属于严格的因果规则。

```

1 def realtime(gA_kwh, load_kwh_actual, pv_kwh_actual, E0, Ebar_ref, rho=0.5):
2     E_prev=E0; c_act=np.zeros(N); d_act=np.zeros(N); r_act=np.zeros(N); w_act=np.
3     zeros(N); E_arr=np.zeros(N)
4     for t in range(N):
5         b=gA_kwh[t]+pv_kwh_actual[t]-load_kwh_actual[t]
6         Rline=E_MIN+rho*(Ebar_ref[t]-E_MIN)
7         Rline=min(max(Rline,E_MIN),E_MAX)
8         if b>=0:
9             c_act[t]=min(b, M_ENERGY, (E_MAX-E_prev)/EC if EC>0 else M_ENERGY)
10            c_act[t]=max(c_act[t],0); d_act[t]=0.0
11        else:
12            need=-b
13            maxd=max(0.0, ED*(E_prev-Rline))
14            d_act[t]=min(need, M_ENERGY, maxd); c_act[t]=0.0
15            r_act[t]=max(0.0, -b-d_act[t])
16            w_act[t]=max(0.0, b-c_act[t])
17            E_new=E_prev+EC*c_act[t]-d_act[t]/ED
18            E_new=min(max(E_new,E_MIN-1e-6),E_MAX+1e-6)
19            E_arr[t]=E_new; E_prev=E_new
20    return c_act,d_act,r_act,w_act,E_arr

```

7.6 滚动时序仿真：误差分位数与调整区间提交

每个发布时刻做三件事：用已经完成的时段修正负载预测（观测偏差加到剩余时段上）、以发布时刻的实际光伏为锚点重新插值光伏预报、以当段的实际储电量为初值重解调整 LP。误差分位数取最近 W 天、全部时段合并后的标量分位数；样本不足 7 天时不做修正。只把当前调整区间的 x_t 写回生效购电量，其余时段一律丢弃。

```
1 for si,Tk in enumerate([36,72,108]):
2     if not adjust_mask[si]: continue
3     # 观测偏差修正负载预测
4     bias_kw=float(((load_kw[d,0:Tk]-load_pred_kw[0:Tk]).mean()))
5     load_rem_pred=load_pred_kw.copy(); load_rem_pred[Tk:]+=bias_kw
6     # 以发布时刻的实际光伏为锚点重新插值
7     issue_h=ISSUE_H[si+1]
8     anchor_actual_kw=float(pv_kw[d,Tk-1])
9     pv_rem_kw=pv_interp_for_issue(issue_h, anchor_actual_kw, fc[d,si+1,:])
10    pv_rem_kw=np.where(np.isnan(pv_rem_kw), pv_pred_kw, pv_rem_kw)
11    ...
12    net_risk_new=net_pred_new.copy(); net_risk_new[Tk:]+=Qadj
13    # 用实际储电量作初值，重解调整 LP
14    E_at_Tk=prefix_sim(...)
15    gA_new,...=solve_lp_adjust(price, net_risk_new, E_at_Tk, gP, Tk, E_tar, lam)
16    next_Tk=Tks[si+1] if si+1<len(Tks) else N
17    gA_final[Tk:next_Tk]=gA_new[Tk:next_Tk] # 只提交当前调整区间
18    Ebar_final[Tk:next_Tk]=Ebar_rem[Tk:next_Tk]
```

7.7 九项一致性检验的独立复核

校验函数不复用求解时的任何中间量，而是从最终费用记录重新按定义计算，逐日返回问题清单；只要任一天出现任一条，全年结果即视为不可用。

```
1 def validate_day(d, price, load_kw, pv_kw, out):
2     probs=[]
3     # 1 逐段能量平衡：实时层由构造保证 gA + PV + 放电 + 紧急 = 负荷 + 充电 + 弃电
4     bal=gA+pv_kw[d,:]*TAU+dd+r-load_kw[d,:]*TAU-c-w
5     if np.abs(bal).max(>1e-6: probs.append(f"实际平衡残差{np.abs(bal).max():.2e}")
6     # 2 storage bounds
7     Eseq=np.concatenate([E0],E)
8     if Eseq.min(<E_MIN-1e-3: probs.append(f"E越下界{Eseq.min():.2f}")
9     if Eseq.max(>E_MAX+1e-3: probs.append(f"E越上界{Eseq.max():.2f}")
10    # 3 power
11    if c.max(>M_ENERGY+1e-6: probs.append("充电超限")
12    if dd.max(>M_ENERGY+1e-6: probs.append("放电超限")
13    # 4 simultaneous
14    if ((c>1e-6)&(dd>1e-6)).any(): probs.append("同时充放电")
15    # 5 dynamics
16    Erec=[E0]
17    for i in range(N): Erec.append(Erec[-1]+EC*c[i]-dd[i]/ED)
18    if np.abs(np.array(Erec[1:])-E).max(>1e-4: probs.append("递推不一致")
19    # 6 购电量非负
20    if (gA<-1e-6).any(): probs.append("gA负")
21    # 7 cost recompute
22    up=(gA-gP).clip(min=0); down=(gP-gA).clip(min=0)
23    c1=float(np.sum(price*gP+1.5*price*up-0.5*price*down))
24    if abs(c1-out["cost_plan_adj"]>1e-3: probs.append("计划调整费复算不符")
25    c2=float(np.sum(5*price*r))
26    if abs(c2-out["cost_emg"]>1e-3: probs.append("紧急费复算不符")
27    return probs
```

7.8 完整源程序

以上各段是为便于对照讲解而作的分段摘录；规范第五条要求附录给出“建模所用到的全部完整、可运行的源程序代码”。下面采用 \lstinputlisting 引入当前目录 code/ 下的六个脚本。附录列出完整程序，因此包含前文已经说明的代码片段。问题一、问题二、问题四所用的源代码同样随支撑材料一并提交。

(1) p3_microgrid.py: 模型与求解器

```
1  # -*- coding: utf-8 -*-
2  r"""
3  2026 CUMCM C题 问题三 —— 日前计划 + 日内滚动调整 + 实时执行
4
5  口径（与题面及附件5模板一致）：
6  * 10分钟时段 i=0..143, 区间 [i*10min,(i+1)*10min], Δt=1/6h。
7  附件1/2 的列标签 0:10 代表区间 0:00-0:10（右端点命名），与问题一框架一致。
8  * 变量统一为电量 kWh/10min（模板单位），功率限值 M=5000*1/6=833.333 kWh。
9  * 储能母线侧口径：E_t = E_{t-1}+0.9*c_t - d_t/0.9, 1200<=E<=10800, E0(Jan1)=6000 跨日
    连续。
10 * 电价 p_t 取附件1（全天相同 deterministic），问题三不用附件4。
11 * 附件3预报：每天 0/6/12/18 时发布未来24个整点功率（kW）。
12 映射：预报k小时 = 发布时刻+k小时的整点功率。
13 当天使用部分：0时报 1-24时；6时报 7-24时（前18个）；12时报13-24时（前12个）；18时报19-24时（
    前6个）。
14 小时→10分钟采用因果分段线性插值（锚点为发布时刻已知实际功率），论文已验证优于阶梯保持
    （全年MAE 201 vs 362 kW）。
15 * 负荷无外部预报，仅用历史实际（附件2）因果预测：同星期往期优先（周五/六低谷约3265 vs 平日5165
    kW）。
16 * 费用口径（论文需显式声明的唯一合理解释）：
17 设 gP=计划购电，gA=最终调整后可用购电（分段提交：0-6时用计划，6-12用6时调整，12-18用12
    时，18-24用18时）。
18 每时段总购电费（计划+调整）= p*gA + 0.5p*(gP-gA)_+ （gA<=gP, 退50%）
19 = p*gP + 1.5p*(gA-gP)_+ （gA>=gP, 追购1.5倍）
20 即 = p*gP + 1.5p*up - 0.5p*down, 其中 up=(gA-gP)_+,
21 down=(gP-gA)_+。
22 若按字面“计划费+0.5p*下调+1.5p*上调”累加（p*gP+0.5p*down+1.5p*up），下调反而多付费，
23 最优调整永不下调，与题设“可调整”矛盾，故该加法解读不成立，特此证伪。
24 紧急购电 r 按 5p 计。总费用 = 计划调整费 + 紧急费。日末储备惩罚 lam*xi 仅用于优化引导，不
    计入账单。
25 """
26 from __future__ import annotations
27 import argparse, json
28 from pathlib import Path
29 import numpy as np
30 import pandas as pd
31 import openpyxl
32 import pulp
33
34 # 仓库根目录：本脚本位于 <仓库>/03_代码/ 下，取上一级即仓库根。
35 # 这样脚本无论放在仓库哪个位置（03_代码/ 或根目录）都能正确定位
36 # 01_题目 / 06_支撑材料，不再依赖硬编码的绝对路径。
37 ROOT = Path(__file__).resolve().parents[1]
38 ATT1 = ROOT/"01_题目"/"C题"/"附件"/"附件1.xlsx"
39 ATT2 = ROOT/"01_题目"/"C题"/"附件"/"附件2.xlsx"
40 ATT3 = ROOT/"01_题目"/"C题"/"附件"/"附件3.xlsx"
41 TEMPLATE3 = ROOT/"01_题目"/"C题"/"附件"/"附件5"/"result3.xlsx"
42 OUT_DIR = ROOT/"06_支撑材料"
43 OUT_XLSX = OUT_DIR/"result3.xlsx"
44 OUT_JSON = OUT_DIR/"p3_results.json"
45
46 N=144; TAU=10/60
47 E_MIN=1200.0; E_MAX=10800.0; E_INIT=6000.0
48 P_MAX=5000.0; M_ENERGY=P_MAX*TAU
```

```

49 EC=0.9; ED=0.9
50 TK=[0,36,72,108] # 0,6,12,18时对应的起始slot
51 ISSUE_H=[0,6,12,18]
52
53 def load_all():
54     a1=pd.read_excel(ATT1, sheet_name="Sheet1")
55     price=a1.iloc[:,1].to_numpy(float)
56     assert len(price)==N
57     a2load=pd.read_excel(ATT2, sheet_name=0, header=0)
58     a2pv=pd.read_excel(ATT2, sheet_name=1, header=0)
59     dates=pd.to_datetime(a2load.iloc[:,0])
60     load_kw=a2load.iloc[:,1:].to_numpy(float)
61     pv_kw=a2pv.iloc[:,1:].to_numpy(float)
62     assert load_kw.shape==(365,144) and pv_kw.shape==(365,144)
63     import openpyxl as _oxl
64     wb=_oxl.load_workbook(ATT3, read_only=True, data_only=True)
65     ws=wb["Sheet1"]
66     rows=list(ws.iter_rows(values_only=True))
67     fc=np.zeros((365,4,24))
68     for d in range(365):
69         base=1+d*4
70         for k in range(4):
71             fc[d,k,:]=np.array([float(x) for x in rows[base+k][2:26]])
72     return price, dates, load_kw, pv_kw, fc
73
74 def price_row(price, n: int) -> np.ndarray:
75     """从电价数组取第n天的144时段电价：1D固定电价直接返回，2D波动电价取第n行。"""
76     a = np.asarray(price, dtype=float)
77     if a.ndim == 2:
78         return a[n].copy()
79     return a.copy()
80
81 def pv_interp_for_issue(anchor_h:int, anchor_v_kw:float, fc24:np.ndarray):
82     """返回当天0-24h内144个slot的PV功率预测(kw)，因果插值。超出预报范围的返回nan。"""
83     # 时间轴: anchor_h -> anchor_h+24, 值: anchor_v -> fc24[0..23]
84     t_nodes=np.arange(anchor_h, anchor_h+25) # anchor, +1..+24
85     # fc24[j] 对应 anchor_h+1+j
86     v_nodes=np.concatenate([anchor_v_kw], fc24)
87     slot_mid=(np.arange(N)+0.5)/6.0 # 0.083..23.917
88     out=np.full(N, np.nan)
89     mask=(slot_mid>anchor_h)&(slot_mid<=anchor_h+24)
90     out[mask]=np.interp(slot_mid[mask], t_nodes, v_nodes)
91     out=np.maximum(out,0.0)
92     return out
93
94 def load_predict(day_idx:int, load_kw:np.ndarray, dates) -> np.ndarray:
95     """同星期优先: d-7; 无则 d-1; 再无则已有均值。因果。返回kw 144."""
96     if day_idx-7>=0:
97         return load_kw[day_idx-7,:].copy()
98     if day_idx-1>=0:
99         return load_kw[day_idx-1,:].copy()
100     return np.full(N, load_kw[:max(1,day_idx),:].mean() if day_idx>0 else 4000.0)
101
102 def solve_lp_plan(price, net_risk_kwh, E0, E_tar=6000.0, lam=1.0):
103     prob=pulp.LpProblem("plan", pulp.LpMinimize)
104     g=[pulp.LpVariable(f"g{i}", lowBound=0) for i in range(N)]
105     c=[pulp.LpVariable(f"c{i}", lowBound=0, upBound=M_ENERGY) for i in range(N)]
106     d=[pulp.LpVariable(f"d{i}", lowBound=0, upBound=M_ENERGY) for i in range(N)]
107     w=[pulp.LpVariable(f"w{i}", lowBound=0) for i in range(N)]
108     E=[pulp.LpVariable(f"E{i}", lowBound=E_MIN, upBound=E_MAX) for i in range(N)]
109     xi=pulp.LpVariable("xi", lowBound=0)
110     prob+=pulp.lpSum([price[i]*g[i] for i in range(N)])+lam*xi
111     for i in range(N):
112         prob+=g[i]+d[i]==net_risk_kwh[i]+c[i]+w[i]

```

```

113     for i in range(N):
114         prev=E0 if i==0 else E[i-1]
115         prob+=E[i]==prev+EC*c[i]-d[i]/ED
116     prob+=xi>=E_tar-E[N-1]
117     prob.solve(pulp.PULP_CBC_CMD(msg=0))
118     assert pulp.LpStatus[prob.status]=="Optimal", prob.status
119     gv=np.array([v.value() for v in g]); cv=np.array([v.value() for v in c])
120     dv=np.array([v.value() for v in d]); wv=np.array([v.value() for v in w])
121     Ev=np.array([v.value() for v in E])
122     return gv,cv,dv,wv,Ev,float(pulp.value(prob.objective))
123
124 def solve_lp_adjust(price, net_risk_kwh, E0, gP, idx_from, E_tar=6000.0, lam=1.0):
125     R=list(range(idx_from,N))
126     m=len(R)
127     prob=pulp.LpProblem("adj", pulp.LpMinimize)
128     gA={i:pulp.LpVariable(f"gA{i}", lowBound=0) for i in R}
129     up={i:pulp.LpVariable(f"up{i}", lowBound=0) for i in R}
130     dn={i:pulp.LpVariable(f"dn{i}", lowBound=0) for i in R}
131     c={i:pulp.LpVariable(f"c{i}", lowBound=0, upBound=M_ENERGY) for i in R}
132     d={i:pulp.LpVariable(f"d{i}", lowBound=0, upBound=M_ENERGY) for i in R}
133     w={i:pulp.LpVariable(f"w{i}", lowBound=0) for i in R}
134     E={i:pulp.LpVariable(f"E{i}", lowBound=E_MIN, upBound=E_MAX) for i in R}
135     xi=pulp.LpVariable("xi", lowBound=0)
136     prob+=pulp.lpSum([1.5*price[i]*up[i]-0.5*price[i]*dn[i] for i in R])+lam*xi
137     for i in R:
138         prob+=gA[i]==gP[i]+up[i]-dn[i]
139         prob+=dn[i]<=gP[i]+1e-9
140         prob+=gA[i]+d[i]==net_risk_kwh[i]+c[i]+w[i]
141     for k,i in enumerate(R):
142         prev=E0 if k==0 else E[R[k-1]]
143         prob+=E[i]==prev+EC*c[i]-d[i]/ED
144     prob+=xi>=E_tar-E[R[-1]]
145     prob.solve(pulp.PULP_CBC_CMD(msg=0))
146     assert pulp.LpStatus[prob.status]=="Optimal", prob.status
147     gA_full=gP.copy(); c_full=np.zeros(N); d_full=np.zeros(N); E_full=np.zeros(N)
148     for i in R:
149         gA_full[i]=gA[i].value(); c_full[i]=c[i].value(); d_full[i]=d[i].value();
150     E_full[i]=E[i].value()
151     up_arr=np.array([up[i].value() for i in R]); dn_arr=np.array([dn[i].value() for
152         i in R])
153     return gA_full,c_full,d_full,E_full,up_arr,dn_arr,float(pulp.value(prob.
154     objective))
155
156 def realtime(gA_kwh, load_kwh_actual, pv_kwh_actual, E0, Ebar_ref, rho=0.5):
157     E_prev=E0; c_act=np.zeros(N); d_act=np.zeros(N); r_act=np.zeros(N); w_act=np.
158     zeros(N); E_arr=np.zeros(N)
159     for t in range(N):
160         b=gA_kwh[t]+pv_kwh_actual[t]-load_kwh_actual[t]
161         Rline=E_MIN+rho*(Ebar_ref[t]-E_MIN)
162         Rline=min(max(Rline,E_MIN),E_MAX)
163         if b>=0:
164             c_act[t]=min(b, M_ENERGY, (E_MAX-E_prev)/EC if EC>0 else M_ENERGY)
165             c_act[t]=max(c_act[t],0); d_act[t]=0.0
166         else:
167             need=-b
168             maxd=max(0.0, ED*(E_prev-Rline))
169             d_act[t]=min(need, M_ENERGY, maxd); c_act[t]=0.0
170             r_act[t]=max(0.0, -b-d_act[t])
171             w_act[t]=max(0.0, b-c_act[t])
172             E_new=E_prev+EC*c_act[t]-d_act[t]/ED
173             E_new=min(max(E_new,E_MIN-1e-6),E_MAX+1e-6)
174             E_arr[t]=E_new; E_prev=E_new
175     return c_act,d_act,r_act,w_act,E_arr

```

```

173 if __name__=="__main__":
174     price,dates,load_kw,pv_kw,fc=load_all()
175     print("loaded",price.shape,load_kw.shape,fc.shape)

```

(2) p3_backtest.py: 全年因果滚动时序仿真

```

1  # -*- coding: utf-8 -*-
2  """问题三回溯：因果滚动、Jan调参、Feb-Dec评价、消融对比。"""
3  from __future__ import annotations
4  import numpy as np
5  from pathlib import Path
6  import sys
7  sys.path.insert(0, str(Path(__file__).resolve().parent))
8  from p3_microgrid import *
9
10 def run_backtest(price_in, dates, load_kw, pv_kw, fc, alpha=0.8, lam=1.0, rho=0.5,
11                 E_tar=6000.0, W=28,
12                 use_adjust=True, adjust_mask=(True,True,True), day_range=None,
13                 verbose=False,
14                 bill_price=None):
15     n_days=load_kw.shape[0]
16     if day_range is None: day_range=range(n_days)
17     # histories of errors (kWh) for quantile: store per-day full vectors of (
18     # actual_net - pred_net) at plan/adj times
19     # For simplicity pooled scalar quantile across all slots & recent W days (
20     # causal).
21     plan_err_hist=[] # list of arrays
22     adj_err_hists={36:[],72:[],108:[]} # per Tk
23     E0=E_INIT
24     # need to carry E0 across all days from Jan1 even if day_range starts later?
25     # Caller must ensure warmup.
26     # Here we simulate only day_range but E0 passed in? Simplify: simulate from 0..
27     # max(day_range) always, record only day_range.
28     maxd=max(day_range)
29     results={}
30     # storage for E0 per day
31     E0_by_day={}
32     E0_cur=E_INIT
33     for d in range(0, maxd+1):
34         E0_by_day[d]=E0_cur
35         price=price_row(price_in, d) # 支持2D波动电价：取当天144时段（计划价）
36         # 结算价：默认与计划价相同；传入 bill_price 时计划/结算分离（合规：计划用预测、结算用实
37         # 际）
38         bill_d=price_row(bill_price, d) if bill_price is not None else price
39         if d not in day_range:
40             # still need to step storage? For days before range (warmup), we must
41             # simulate with same policy to get correct E0.
42             # To avoid complexity, require day_range starts at 0. We'll just
43             # simulate all from 0.
44             pass
45         # ---- 0:00 plan prediction
46         load_pred_kw=load_predict(d, load_kw, dates)
47         anchor_v=0.0 # midnight PV=0
48         pv_pred_kw=pv_interp_for_issue(0, anchor_v, fc[d,0,:])
49         # nan beyond 24h? all 144 should be valid for 0:00 (covers full day)
50         pv_pred_kw=np.where(np.isnan(pv_pred_kw), 0.0, pv_pred_kw)
51         net_pred_kwh=(load_pred_kw-pv_pred_kw)*TAU
52         # quantile from history
53         if len(plan_err_hist)>=7:
54             pool=np.concatenate(plan_err_hist[-W:])
55             Q=np.quantile(pool, alpha)
56         else:

```

```

48     Q=0.0
49     net_risk=net_pred_kwh+Q
50     gP,,_,_,Ebar_plan,_=solve_lp_plan(price, net_risk, E0_cur, E_tar, lam)
51     # adjustments
52     gA_final=gP.copy()
53     Ebar_final=np.zeros(N); Ebar_final[:]=Ebar_plan # provisional, will
overwrite segments
54     # track per-segment solutions for reserve
55     seg_Ebars={}
56     seg_Ebars[(0,36)]=Ebar_plan[0:36]
57     if use_adjust:
58         # ordered Tk
59         Tks=[36,72,108]
60         for si,Tk in enumerate(Tks):
61             if not adjust_mask[si]: continue
62             # observed prefix bias for load: mean(actual-pred) over [0,Tk)
63             l_actual_pre=load_kw[d,0:Tk]; p_pred_pre=load_pred_kw[0:Tk]
64             bias_kw=float((l_actual_pre-p_pred_pre).mean()) if Tk>0 else 0.0
65             # load remaining pred = base + bias
66             load_rem_pred=load_pred_kw.copy()
67             load_rem_pred[Tk:]+=bias_kw
68             # PV remaining pred from latest forecast anchored at actual at Tk
69             issue_h=ISSUE_H[si+1]
70             anchor_actual_kw=float(pv_kw[d,Tk-1]) # power at Tk:00 (slot
ending)
71             fc24=fc[d,si+1,:]
72             pv_rem_kw=pv_interp_for_issue(issue_h, anchor_actual_kw, fc24)
73             pv_rem_kw=np.where(np.isnan(pv_rem_kw), pv_pred_kw, pv_rem_kw)
74             # for t<Tk keep old, for t>=Tk use new
75             pv_pred_new=pv_pred_kw.copy(); pv_pred_new[Tk:]=pv_rem_kw[Tk:]
76             load_pred_new=load_pred_kw.copy(); load_pred_new[Tk:]=load_rem_pred
[Tk:]
77             net_pred_new=(load_pred_new-pv_pred_new)*TAU
78             # quantile for this Tk
79             hist=adj_err_hists[Tk]
80             if len(hist)>=7:
81                 pool=np.concatenate(hist[-W:])
82                 Qadj=np.quantile(pool, alpha)
83             else:
84                 Qadj=Q # fallback to plan Q
85             net_risk_new=net_pred_new.copy(); net_risk_new[Tk:]+= (Qadj) #
only remaining matters
86             # E0 at Tk = actual storage after executing prefix with final gA so
far?
87             # To get E0_actual at Tk, we need to have simulated prefix actuals
with committed gA.
88             # Commit logic: segments before Tk already frozen. Simulate prefix
[0,Tk) with current gA_final (which includes prior adjustments) + actuals to get
E at Tk-1.
89             # Use provisional Ebar for reserve? For prefix simulation use rho
rule with Ebar_final provisional? Simpler: simulate prefix exactly as final
realtime will do, using Ebar_final current (which for prefix segments is already
final).
90             # Do prefix simulation:
91             E_tmp=E0_cur
92             # need Ebar_ref for prefix: Ebar_final[0:Tk] (already committed)
93             # run realtime prefix only
94             # quick inline prefix sim
95             E_prev=E0_cur
96             for t in range(Tk):
97                 b=gA_final[t]+pv_kw[d,t]*TAU-load_kw[d,t]*TAU
98                 Rline=E_MIN+rho*(Ebar_final[t]-E_MIN); Rline=min(max(Rline,
E_MIN),E_MAX)
99             if b>=0:

```

```

100         c_ = min(b, M_ENERGY, (E_MAX - E_prev) / EC); dd_ = 0.0
101     else:
102         dd_ = min(-b, M_ENERGY, max(0.0, ED * (E_prev - Rline))); c_ = 0.0
103         E_prev = E_prev + EC * c_ - dd_ / ED
104         E_at_Tk = E_prev
105         gA_new, _, _, Ebar_rem, _, _, _ = solve_lp_adjust(price, net_risk_new,
106         E_at_Tk, gP, Tk, E_tar, lam)
107         # commit only until next Tk (or end)
108         next_Tk = Tks[si+1] if si+1 < len(Tks) else N
109         gA_final[Tk:next_Tk] = gA_new[Tk:next_Tk]
110         Ebar_final[Tk:next_Tk] = Ebar_rem[Tk:next_Tk]
111         # ---- realtime full day with final gA (裁剪求解容差负值)
112         gA_final = np.maximum(gA_final, 0.0); gP = np.maximum(gP, 0.0)
113         load_act_kwh = load_kw[d,:]*TAU; pv_act_kwh = pv_kw[d,:]*TAU
114         c_act, d_act, r_act, w_act, E_arr = realtime(gA_final, load_act_kwh, pv_act_kwh,
115         E0_cur, Ebar_final, rho)
116         # costs
117         up = (gA_final - gP).clip(min=0); down = (gP - gA_final).clip(min=0)
118         cost_plan_adj = float(np.sum(bill_d * gP + 1.5 * bill_d * up - 0.5 * bill_d * down))
119         cost_emg = float(np.sum(5 * bill_d * r_act))
120         cost_total = cost_plan_adj + cost_emg
121         # update error histories (causal for future): plan error = actual_net -
122         pred_net (without Q)
123         actual_net = (load_kw[d,:] - pv_kw[d,:]) * TAU
124         pred_net_plan = (load_pred_kw - pv_pred_kw) * TAU
125         plan_err_hist.append(actual_net - pred_net_plan)
126         # adj errors: for each Tk, error of that Tk's prediction vs actual for t >=
127         Tk
128         # reconstruct adj predictions used (need to store)? Simplify: error of
129         final PV/load combo? For quantile we need distribution of (actual - pred_at_Tk).
130         # Recompute adj preds as above (without Q) for each Tk to record error.
131         for si, Tk in enumerate([36, 72, 108]):
132             issue_h = ISSUE_H[si+1]
133             anchor_actual_kw = float(pv_kw[d, Tk-1])
134             pv_rem = pv_interp_for_issue(issue_h, anchor_actual_kw, fc[d, si+1,:])
135             pv_rem = np.where(np.isnan(pv_rem), pv_pred_kw, pv_rem)
136             # load with bias as used
137             l_actual_pre = load_kw[d, 0: Tk]; p_pred_pre = load_pred_kw[0: Tk]
138             bias_kw = float((l_actual_pre - p_pred_pre).mean()) if Tk > 0 else 0.0
139             load_rem = load_pred_kw.copy(); load_rem[Tk:] += bias_kw
140             pv_full = pv_pred_kw.copy(); pv_full[Tk:] = pv_rem[Tk:]
141             load_full = load_pred_kw.copy(); load_full[Tk:] = load_rem[Tk:]
142             pred_net_adj = (load_full - pv_full) * TAU
143             err_full = actual_net - pred_net_adj
144             adj_err_hists[Tk].append(err_full[Tk:]) # only remaining matters
145         E0_cur = E_arr[-1]
146         if d in day_range:
147             results[d] = {"gP": gP.copy(), "gA": gA_final.copy(), "Ebar": Ebar_final.copy
148             (),
149             "c_act": c_act, "d_act": d_act, "r": r_act, "w": w_act, "E": E_arr.
150             copy(),
151             "E0": E0_by_day[d], "cost_plan_adj": cost_plan_adj, "cost_emg":
152             cost_emg, "cost_total": cost_total,
153             "Q": float(Q)}
154         return results
155
156 if __name__ == "__main__":
157     price, dates, load_kw, pv_kw, fc = __import__("p3_microgrid").load_all()
158     # quick smoke: Jan first 3 days
159     res = run_backtest(price, dates, load_kw, pv_kw, fc, day_range=range(0, 3), verbose=True
160     )
161     for d, v in res.items():
162         print(d, round(v["cost_total"], 2), "emg", round(v["cost_emg"], 2), "Q",
163         round(v["Q"], 2))

```

(3) p3_export.py: 结果导出与九项校验

```
1  # -*- coding: utf-8 -*-
2  """问题三最终求解: 全量回测 + 九项校验 + result3.xlsx导出 + 指定日期汇总。"""
3  from __future__ import annotations
4  import json
5  from pathlib import Path
6  import sys
7  import numpy as np, pandas as pd, openpyxl
8  sys.path.insert(0, str(Path(__file__).resolve().parent))
9  from p3_microgrid import *
10 from p3_backtest import run_backtest
11
12 BEST=dict(alpha=0.6, lam=1.0, rho=0.5, E_tar=6000.0, W=28)
13 SPEC=["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
14
15 def fmt_slot(i):
16     a=i*10; b=(i+1)*10
17     def f(m):
18         if m==1440: return "0:00+1"
19         return f"{m//60}:{m%60:02d}"
20     return f"{f(a)}-{f(b)}"
21
22 BLOCKS=[(0,24),(24,48),(48,72),(72,96),(96,120),(120,144)]
23 def block_label(a,z):
24     sh=a*10//60; eh=24 if z*10==1440 else z*10//60
25     return f"{sh}:00-{eh}:00"
26
27 def validate_day(d, price, load_kw, pv_kw, out):
28     probs=[]
29     gP=out["gP"]; gA=out["gA"]; c=out["c_act"]; dd=out["d_act"]; r=out["r"]; w=out["w"]; E=out["E"]; E0=out["E0"]
30     # 1 逐段能量平衡: 实时层由构造保证 gA + PV + 放电 + 紧急 = 负荷 + 充电 + 弃电
31     bal=gA+pv_kw[d,:]*TAU+dd+r-load_kw[d,:]*TAU-c-w
32     if np.abs(bal).max()>1e-6: probs.append(f"实际平衡残差{np.abs(bal).max():.2e}")
33     # 2 storage bounds
34     Eseq=np.concatenate([E0],E)
35     if Eseq.min()<E_MIN-1e-3: probs.append(f"E越下界{Eseq.min():.2f}")
36     if Eseq.max()>E_MAX+1e-3: probs.append(f"E越上界{Eseq.max():.2f}")
37     # 3 power
38     if c.max()>M_ENERGY+1e-6: probs.append("充电超限")
39     if dd.max()>M_ENERGY+1e-6: probs.append("放电超限")
40     # 4 simultaneous
41     if ((c>1e-6)&(dd>1e-6)).any(): probs.append("同时充放电")
42     # 5 dynamics
43     Erec=[E0]
44     for i in range(N): Erec.append(Erec[-1]+EC*c[i]-dd[i]/ED)
45     if np.abs(np.array(Erec[1:])-E).max()>1e-4: probs.append("递推不一致")
46     # 6 购电量非负 (下调幅度已由 up/down 定义约束, 无需另行判据)
47     if (gA<-1e-6).any(): probs.append("gA负")
48     # 7 cost recompute
49     up=(gA-gP).clip(min=0); down=(gP-gA).clip(min=0)
50     c1=float(np.sum(price*gP+1.5*price*up-0.5*price*down))
51     if abs(c1-out["cost_plan_adj"])>1e-3: probs.append("计划调整费复算不符")
52     c2=float(np.sum(5*price*r))
53     if abs(c2-out["cost_emg"])>1e-3: probs.append("紧急费复算不符")
54     return probs
55
56 def main():
57     price,dates,load_kw,pv_kw,fc=load_all()
58     print("backtest full year with BEST",BEST,flush=True)
59     res=run_backtest(price,dates,load_kw,pv_kw,fc,day_range=range(0,365),**BEST,
60         use_adjust=True,adjust_mask=(True,True,True))
61     # validate Feb-Dec
```

```

61 bad=0
62 for d in range(31,365):
63     p=validate_day(d,price,load_kw,pv_kw,res[d])
64     if p: bad+=1; print(dates[d].date(),p)
65 print(f"校验: 334天中{bad}天有问题",flush=True)
66 # totals
67 tot=sum(res[d]["cost_total"] for d in range(31,365))
68 pa=sum(res[d]["cost_plan_adj"] for d in range(31,365))
69 emg=sum(res[d]["cost_emg"] for d in range(31,365))
70 print(f"Feb-Dec Total={tot:.2f} plan_adj={pa:.2f} emg={emg:.2f}",flush=True)
71 # specified dates: 把 SPEC 的日期字符串映射到 dates 数组下标
72 import datetime
73 spec_idx={}
74 for s in SPEC:
75     dt=pd.to_datetime(s).date()
76     for i,dd in enumerate(dates):
77         if dd.date()==dt: spec_idx[s]=i; break
78 print(spec_idx)
79 payload={"BEST":BEST,"Feb-Dec":{"total":tot,"plan_adj":pa,"emg":emg},"spec":{}}
80 for s,i in spec_idx.items():
81     o=res[i]
82     want_idx=[60,72,84,96,108,120] # 10:00-10:10 is slot60? slot0=0:00-0:10,
slot60=10:00-10:10 yes
83     t1=[{"slot":fmt_slot(k),"gP":float(o["gP"][k]),"gA":float(o["gA"][k])} for
k in want_idx]
84     t2=[]
85     for a,z in BLOCKS:
86         t2.append({"block":block_label(a,z),"chg":float(o["c_act"][a:z].sum()),
"dis":float(o["d_act"][a:z].sum())})
87     # emergency events merged
88     r=o["r"]; ev=[]; t=0
89     while t<N:
90         if r[t]>1e-6:
91             s0=t
92             while t<N and r[t]>1e-6: t+=1
93             s1=t
94             ev.append({"start":fmt_slot(s0).split("-")[0],"end":fmt_slot(s1-1).
split("-")[1],"kwh":float(r[s0:s1].sum()),"cost":float(np.sum(5*price[s0:s1]*r[
s0:s1]))})
95         else: t+=1
96     payload["spec"][s]={"table1":t1,"Gplan":float(o["gP"].sum()),"Gadj":float(o
["gA"].sum()),
97         "Gemg":float(o["r"].sum()),"cost_plan_adj":float(o["cost_plan_adj"]),
cost_emg":float(o["cost_emg"]),
98         "cost_total":float(o["cost_total"]),"table2":t2,"E0":float(o["E0"]),
"E144":float(o["E"][-1]),"emg_events":ev}
99     print(s, payload["spec"][s])
100 OUT_DIR.mkdir(parents=True,exist_ok=True)
101 (OUT_DIR/"p3_spec.json").write_text(json.dumps(payload,ensure_ascii=False,
indent=2),encoding="utf-8")
102 # export result3.xlsx
103 export_result3(price,dates,res)
104 print("done",flush=True)
105
106 def export_result3(price,dates,res):
107     # 每次强制从官方模板重新拷贝: 确保表头标签、列口径、残留行都来自模板,
108     # 不沿用上一次输出文件里的历史内容 (否则旧表头/脏数据会被带入交付件)。
109     import shutil
110     assert TEMPLATE3.exists(), f"官方模板不存在: {TEMPLATE3}"
111     shutil.copyfile(TEMPLATE3, OUT_XLSX)
112     wb=openpyxl.load_workbook(OUT_XLSX)
113     # correct slot labels
114     correct=[fmt_slot(i) for i in range(N)]
115     for sheet in ["计划购电量","调整购电量"]:

```

```

116     ws=wb[sheet]
117     # header row1: keep first cell, overwrite slot labels, keep last two
118     for j,lab in enumerate(correct, start=2):
119         ws.cell(row=1,column=j,value=lab)
120     # rows 2..335 correspond to Feb1..Dec31 (indices31..364)
121     for r,d in enumerate(range(31,365), start=2):
122         ws.cell(row=r,column=1,value=dates[d].to_pydatetime().replace(tzinfo=
None))
123         arr=res[d]["gP"] if sheet=="计划购电量" else res[d]["gA"]
124         for j,v in enumerate(arr, start=2):
125             ws.cell(row=r,column=j,value=round(float(v),6))
126         ws.cell(row=r,column=146,value=round(float(arr.sum()),6))
127         # 全天购电费: 计划表填计划费  $\sum p \cdot gP$ ; 调整表填计划调整总费  $\sum (p \cdot gP + 1.5p \cdot up - 0.5$ 
128         # 紧急购电费不在本列, 单独在论文正文与 p3_spec.json 中列出。
129         up=(res[d]["gA"]-res[d]["gP"]).clip(min=0)
130         down=(res[d]["gP"]-res[d]["gA"]).clip(min=0)
131         if sheet=="计划购电量":
132             cost_pa=float(np.sum(price*res[d]["gP"]))
133         else:
134             cost_pa=float(np.sum(price*res[d]["gP"])+1.5*price*up-0.5*price*down
))
135         ws.cell(row=r,column=147,value=round(cost_pa,6))
136     # 充放电: 展开全部334天
137     ws=wb["充放电"]
138     # clear existing rows beyond header
139     ws.delete_rows(2, ws.max_row)
140     row=2
141     for d in range(31,365):
142         o=res[d]
143         for bi,(a,z) in enumerate(BLOCKS):
144             ws.cell(row=row,column=1,value=dates[d].to_pydatetime().replace(tzinfo=
None) if bi==0 else None)
145             ws.cell(row=row,column=2,value=block_label(a,z))
146             ws.cell(row=row,column=3,value=round(float(o["c_act"][a:z].sum()),6))
147             ws.cell(row=row,column=4,value=round(float(o["d_act"][a:z].sum()),6))
148             if bi==0:
149                 ws.cell(row=row,column=5,value="0:00"); ws.cell(row=row,column=6,
value=round(float(o["E0"]),6))
150             elif bi==1:
151                 ws.cell(row=row,column=5,value="24:00"); ws.cell(row=row,column=6,
value=round(float(o["E"][-1]),6))
152             row+=1
153     # 紧急购电量: 合并连续事件
154     ws=wb["紧急购电量"]
155     ws.delete_rows(2, ws.max_row)
156     row=2
157     for d in range(31,365):
158         r=res[d]["r"]; ev=[]; t=0
159         while t<N:
160             if r[t]>1e-6:
161                 s0=t
162                 while t<N and r[t]>1e-6: t+=1
163                 ev.append((s0,t,float(r[s0:t].sum())))
164             else: t+=1
165         if not ev: continue
166         for ei,(s0,s1,kwh) in enumerate(ev):
167             ws.cell(row=row,column=1,value=dates[d].to_pydatetime().replace(tzinfo=
None) if ei==0 else None)
168             ws.cell(row=row,column=2,value=f"{fmt_slot(s0).split('-')[0]}-{fmt_slot
(s1-1).split('-')[1]}")
169             ws.cell(row=row,column=3,value=round(kwh,6))
170             row+=1
171     wb.save(OUT_XLSX)

```

```

172     print("saved",OUT_XLSX,flush=True)
173
174 if __name__=="__main__":
175     main()

```

(4) p3_analysis.py: 主结果、消融、敏感性与精度分析

```

1  # -*- coding: utf-8 -*-
2  """问题三：在已完成模型 (p3_microgrid / p3_backtest / p3_export) 之上做全套量化分析。
3
4  论文所需的每一个数字都从本脚本产生, `p3_tables.py` 与 `p3_figures.py` 只做排版,
5  不再自行计算。分析分五组:
6
7  1. 主结果          全年因果时序仿真 (1 月预热、2 月 1 日--12 月 31 日评价)
8  2. 预报组合消融    6:00 / 12:00 / 18:00 三个发布时刻的  $2^3 = 8$  种取法,
9                    外加"只用 0:00 预报"的对照 (共 9 条策略)
10 3. 口径敏感性      计费口径 (退款 / 不退款) 与风险分位数口径 (跨时段合并 / 逐时段)
11 4. 参数敏感性      alpha、rho、lambda、W 四个参数的单变量扰动
12 5. 逐日校验与精度  九项逐日一致性检查、附件 3 四个发布版本的预报精度
13
14 后两组中的"不退款口径"与"逐时段分位数"两个变体, 由本脚本按最小改动从
15 `p3_microgrid.py` / `p3_backtest.py` 自动生成到 `_03_代码/_p3_variants/`,
16 生成规则写在 `build_variants()` 里, 改动只有一行目标函数或两处分位数语句。
17 """
18 from __future__ import annotations
19
20 import json
21 import sys
22 from pathlib import Path
23
24 import numpy as np
25 import pandas as pd
26
27 HERE = Path(__file__).resolve().parent
28 sys.path.insert(0, str(HERE))
29 ROOT = HERE.parent
30 OUT = ROOT / "06_支撑材料"
31 OUT.mkdir(exist_ok=True)
32 VAR = HERE / "_p3_variants"
33 VAR.mkdir(exist_ok=True)
34
35 START, END = 31, 365          # 评价区间: 2 月 1 日 -- 12 月 31 日 (334 天)
36 BEST = dict(alpha=0.6, lam=1.0, rho=0.5, E_tar=6000.0, W=28)
37 TKS = (36, 72, 108)
38 ADJ_LABELS = ("6:00", "12:00", "18:00")
39 COMMON = slice(108, 144)      # 四个发布版本共同覆盖的时段: 18:00--24:00
40
41 # ----- 变体生成
42 def build_variants() -> None:
43     """把两个口径变体写成独立模块, 供消融实验导入。"""
44     (VAR / "__init__.py").write_text("", encoding="utf-8")
45     src_m = (HERE / "p3_microgrid.py").read_text(encoding="utf-8")
46     src_b = (HERE / "p3_backtest.py").read_text(encoding="utf-8")
47
48     # 变体一: 不退款口径。调减不产生贷方, 于是调整 LP 的目标去掉 -0.5*p*dn。
49     m_c = src_m.replace(
50         "prob+=pulp.lpSum([1.5*price[i]*up[i]-0.5*price[i]*dn[i] for i in R])+lam*",
51         "prob+=pulp.lpSum([1.5*price[i]*up[i] for i in R])+lam*",
52     )
53     assert m_c != src_m, "不退款口径的目标函数替换失败"
54

```

```

55 (VAR / "p3_microgrid_c.py").write_text(m_c, encoding="utf-8")
56
57 # backtest 的其余名字 (常量、预测、实时规则) 在 _c 模块里是同一份拷贝,
58 # 所以只要把 import 改指到 _c, 它调用的 solve_lp_adjust 就自动换成不退款的那一版。
59 b_c = src_b.replace(
60     "from p3_microgrid import *", "from p3_microgrid_c import *"
61 ).replace(
62     "cost_plan_adj=float(np.sum(bill_d*gP+1.5*bill_d*up-0.5*bill_d*down))",
63     "cost_plan_adj=float(np.sum(bill_d*gP+1.5*bill_d*up))",
64 )
65 assert "from p3_microgrid_c import *" in b_c, "不退款口径的 import 替换失败"
66 assert "-0.5*bill_d*down" not in b_c, "不退款口径的结算式替换失败"
67 (VAR / "p3_backtest_c.py").write_text(b_c, encoding="utf-8")
68
69 # 变体二: 逐时段分位数。把"跨全部时段合并成一个标量"换成"每段各自一个分位数"。
70 old_plan = """
71     if len(plan_err_hist)>=7:
72         pool=np.concatenate(plan_err_hist[-W:])
73         Q=np.quantile(pool, alpha)
74     else:
75         Q=0.0
76     net_risk=net_pred_kwh+Q"""
77 new_plan = """
78     if len(plan_err_hist)>=7:
79         pool=np.stack(plan_err_hist[-W:])
80         Q=np.quantile(pool, alpha, axis=0)
81     else:
82         Q=np.zeros(N)
83     net_risk=net_pred_kwh+Q"""
84 old_adj = """
85     hist=adj_err_hists[Tk]
86     if len(hist)>=7:
87         pool=np.concatenate(hist[-W:])
88         Qadj=np.quantile(pool, alpha)
89     else:
90         Qadj=Q # fallback to plan Q"""
91 new_adj = """
92     hist=adj_err_hists[Tk]
93     if len(hist)>=7:
94         pool=np.stack(hist[-W:])
95         Qadj=np.quantile(pool, alpha, axis=0)
96     else:
97         Qadj=Q[Tk:] # 样本不足时退回 0:00 版的逐时段分位数"""
98 b_ps = src_b.replace(old_plan, new_plan).replace(old_adj, new_adj)
99 b_ps = b_ps.replace('Q':float(Q)}', 'Q':float(np.mean(Q))}')
100 assert b_ps != src_b, "逐时段分位数变体的替换失败"
101 (VAR / "p3_backtest_ps.py").write_text(b_ps, encoding="utf-8")
102
103 # 变体模块放在 _p3_variants/ 下, 要能 import 到主模块与彼此。路径插入必须
104 # 落在 `from __future__` 之后, 否则会破坏“future 导入须在最前”的语法要求。
105 for name in ("p3_microgrid_c.py", "p3_backtest_c.py", "p3_backtest_ps.py"):
106     p = VAR / name
107     lines = p.read_text(encoding="utf-8").splitlines(keepends=True)
108     at = 0
109     for i, ln in enumerate(lines):
110         if ln.startswith("from __future__"):
111             at = i + 1
112             break
113     if i > 80: # 文件头 docstring 可能很长, 但不会超过 80 行
114         break
115     lines.insert(at,
116                 "import sys as _sys\n"
117                 f"_sys.path.insert(0, r'{HERE}'))\n"
118                 f"_sys.path.insert(0, r'{VAR}'))\n")
119     p.write_text("".join(lines), encoding="utf-8")

```

----- 汇总

```

119 def summarize(res) -> dict:
120     """汇总一次全年时序仿真的主要结果。"""
121     N = len(res[END - 1]["gA"])
122     tot = pa = em = 0.0
123     gP_s = gA_s = r_s = c_s = d_s = w_s = up_s = dn_s = 0.0
124     n_days_chg = 0
125     n_emg_events = 0
126     E_min, E_max = np.inf, -np.inf
127     daily = []
128     for d in range(START, END):
129         o = res[d]
130         up = np.clip(o["gA"] - o["gP"], 0, None)
131         dn = np.clip(o["gP"] - o["gA"], 0, None)
132         tot += o["cost_total"]; pa += o["cost_plan_adj"]; em += o["cost_emg"]
133         gP_s += o["gP"].sum(); gA_s += o["gA"].sum(); r_s += o["r"].sum()
134         c_s += o["c_act"].sum(); d_s += o["d_act"].sum(); w_s += o["w"].sum()
135         up_s += up.sum(); dn_s += dn.sum()
136         n_days_chg += int((np.abs(o["gA"] - o["gP"]) > 1e-6).any())
137         r = o["r"]; t = 0
138         while t < N:
139             if r[t] > 1e-6:
140                 n_emg_events += 1
141                 while t < N and r[t] > 1e-6:
142                     t += 1
143             else:
144                 t += 1
145         seq = np.concatenate([o["E0"], o["E"]])
146         E_min = min(E_min, float(seq.min())); E_max = max(E_max, float(seq.max()))
147         daily.append(dict(
148             day=d, total=o["cost_total"], plan_adj=o["cost_plan_adj"], emg=o["cost_emg"],
149             gP=float(o["gP"].sum()), gA=float(o["gA"].sum()), up=float(up.sum()),
150             dn=float(dn.sum()), r=float(r.sum()), w=float(o["w"].sum()),
151             charge=float(o["c_act"].sum()), discharge=float(o["d_act"].sum()),
152             E0=float(o["E0"]), E144=float(o["E"][-1]),
153             Emin=float(seq.min()), Emax=float(seq.max()),
154             nsub=int((np.abs(o["gA"] - o["gP"]) > 1e-6).sum()),
155         ))
156     return dict(
157         total=tot, plan_adj=pa, emg=em, gP=gP_s, gA=gA_s, r=r_s,
158         charge=c_s, discharge=d_s, curtail=w_s, up=up_s, dn=dn_s,
159         n_days_adjusted=n_days_chg, n_emg_events=n_emg_events,
160         E_min=E_min, E_max=E_max,
161         E0_first=float(res[START]["E0"]), E144_last=float(res[END - 1]["E"][-1]),
162         daily=daily,
163     )
164
165
166 def worker(cfg):
167     """每个参数配置对应一次全年时序仿真，并在子进程中执行。"""
168     sys.path.insert(0, str(HERE))
169     sys.path.insert(0, str(VAR))
170     if cfg["kind"] == "norefund":
171         import p3_backtest_c as B
172     elif cfg["kind"] == "perslot":
173         import p3_backtest_ps as B
174     else:
175         import p3_backtest as B
176     import p3_microgrid as M
177
178     price, dates, load_kw, pv_kw, fc = M.load_all()
179     kw = dict(cfg["params"])
180     if cfg["kind"] == "ctrl":
181         res = B.run_backtest(price, dates, load_kw, pv_kw, fc,

```

```

182                                     day_range=range(0, 365), use_adjust=False, **kw)
183     else:
184         res = B.run_backtest(price, dates, load_kw, pv_kw, fc,
185                               day_range=range(0, 365), use_adjust=True,
186                               adjust_mask=cfg["mask"], **kw)
187     s = summarize(res)
188     if cfg.get("dump"):
189         np.savez_compressed(
190             OUT / "p3_main_arrays.npz",
191             **{"gP_{d}": res[d]["gP"] for d in range(START, END)},
192             **{"gA_{d}": res[d]["gA"] for d in range(START, END)},
193             **{"r_{d}": res[d]["r"] for d in range(START, END)},
194             **{"c_{d}": res[d]["c_act"] for d in range(START, END)},
195             **{"d_{d}": res[d]["d_act"] for d in range(START, END)},
196             **{"E_{d}": res[d]["E"] for d in range(START, END)},
197             **{"Ebar_{d}": res[d]["Ebar"] for d in range(START, END)},
198         )
199     return cfg["label"], s
200
201
202 def build_configs():
203     cfgs = [
204         dict(label="主策略", kind="main", mask=(True, True, True), params=BEST, dump
205             =True),
206         dict(label="对照：只用0:00预报", kind="ctrl", mask=None, params=BEST),
207         dict(label="敏感性：不退款口径", kind="norefund", mask=(True, True, True),
208             params=BEST),
209         dict(label="敏感性：逐时段分位数", kind="perslot", mask=(True, True, True),
210             params=BEST),
211     ]
212     for i in range(1, 8):
213         mask = tuple(bool(i >> k & 1) for k in range(3))
214         name = "+".join(ADJ_LABELS[k] for k in range(3) if mask[k])
215         cfgs.append(dict(label=name, kind="combo", mask=mask, params=BEST))
216     return cfgs
217
218
219 def param_grid():
220     g = []
221     for a in (0.5, 0.6, 0.7, 0.8):
222         for r in (0.35, 0.5, 0.7):
223             g.append(dict(label=f"alpha={a}, rho={r}", kind="combo",
224                             mask=(True, True, True), params=dict(BEST, alpha=a, rho=r
225                                 )))
226     for lam in (0.0, 0.5):
227         g.append(dict(label=f"lam={lam}", kind="combo", mask=(True, True, True),
228                         params=dict(BEST, lam=lam)))
229     for w in (14, 21):
230         g.append(dict(label=f"W={w}", kind="combo", mask=(True, True, True),
231                         params=dict(BEST, W=w)))
232     return g
233
234 # ----- 逐日校验
235 def validation():
236     """九项逐日一致性检查（与 p3_export.py 中的校验完全同一套判据）。"""
237     from p3_export import validate_day
238     import p3_microgrid as M
239     import p3_backtest as B
240     price, dates, load_kw, pv_kw, fc = M.load_all()
241     res = B.run_backtest(price, dates, load_kw, pv_kw, fc,
242                           day_range=range(0, 365), use_adjust=True,
243                           adjust_mask=(True, True, True), **BEST)
244     n_days_bad = 0

```

```

242 problems: set[str] = set()
243 worst = dict(balance=0.0, dynamics=0.0, cost_adj=0.0, cost_emg=0.0)
244 for d in range(START, END):
245     o = res[d]
246     bal = np.abs(o["gA"] + pv_kw[d, :] * M.TAU + o["d_act"] + o["r"]
247               - load_kw[d, :] * M.TAU - o["c_act"] - o["w"])
248     worst["balance"] = max(worst["balance"], float(bal.max()))
249     rec = [o["E0"]] + list(o["E"])
250     dyn = np.abs(np.array([rec[i] + M.EC * o["c_act"][i] - o["d_act"][i] / M.ED
251               for i in range(len(o["c_act"]))]) - np.array(o["E"]))
252     worst["dynamics"] = max(worst["dynamics"], float(dyn.max()))
253     up = np.clip(o["gA"] - o["gP"], 0, None); down = np.clip(o["gP"] - o["gA"],
254     0, None)
255     c1 = float((price * o["gP"] + 1.5 * price * up - 0.5 * price * down).sum())
256     c2 = float((5 * price * o["r"]).sum())
257     worst["cost_adj"] = max(worst["cost_adj"], abs(c1 - o["cost_plan_adj"]))
258     worst["cost_emg"] = max(worst["cost_emg"], abs(c2 - o["cost_emg"]))
259     probs = validate_day(d, price, load_kw, pv_kw, o)
260     if probs:
261         n_days_bad += 1
262         problems.update(probs)
263     return dict(n_days_bad=n_days_bad, problems=sorted(problems),
264               n_days=END - START, worst=worst)
265
266 # ----- 预报精度
267 def forecast_skill():
268     """附件 3 四个发布版本的光伏预报精度。
269
270     只报"绝对误差 (kW)"而不报相对误差：各版本的覆盖时段长度不同，18:00 版只覆盖
271     18:00--24:00 这一段光伏趋零的夜间，任何以"该时段平均功率"为分母的相对指标都会
272     被极小的分母放大成无意义的数字。因此改为并列三个绝对口径：
273
274     mae_kw          该版本自己覆盖到的全部时段
275     mae_noon_kw     12:00--18:00 (白天最后一段，光伏最强、方案差异最大)
276     mae_eve_kw      18:00--24:00 (四版共同覆盖的时段，唯一可横向比较的窗口)
277
278     """
279     import p3_microgrid as M
280     price, dates, load_kw, pv_kw, fc = M.load_all()
281     NOON = slice(72, 108)
282     rows = []
283     for k, h in enumerate([0, 6, 12, 18]):
284         err_all, err_noon, err_eve = [], [], []
285         for d in range(START, END):
286             anchor = 0.0 if h == 0 else float(pv_kw[d, int(h * 6) - 1])
287             f = M.pv_interp_for_issue(h, anchor, fc[d, k, :])
288             m = ~np.isnan(f)
289             act = pv_kw[d, :]
290             err_all.append(np.abs(f[m] - act[m]))
291             for sl, buf in ((NOON, err_noon), (COMMON, err_eve)):
292                 mm = np.zeros_like(m)
293                 mm[sl] = m[sl]
294                 if mm.any():
295                     buf.append(np.abs(f[mm] - act[mm]))
296             rows.append(dict(
297                 release=f"{h}:00",
298                 n=int(np.concatenate(err_all).size),
299                 mae_kw=float(np.concatenate(err_all).mean()),
300                 mae_noon_kw=float(np.concatenate(err_noon).mean()) if err_noon else
301                 None,
302                 mae_eve_kw=float(np.concatenate(err_eve).mean()),
303             ))
304     pd.DataFrame(rows).to_csv(OUT / "p3_forecast_skill.csv", index=False)

```

```

303     return rows
304
305
306 # ----- 主流程
307 def main():
308     from multiprocessing import Pool
309
310     build_variants()
311     with Pool(5) as pool:
312         out = pool.map(worker, build_configs())
313     main_res = dict(out)
314     for k, v in main_res.items():
315         print(f"{k:>22s} total={v['total']:.2f} "
316               f"计划调整={v['plan_adj']:.2f} 紧急={v['emg']:.2f}")
317
318     with Pool(5) as pool:
319         gout = pool.map(worker, param_grid())
320
321     d = pd.DataFrame(main_res["主策略"]["daily"])
322     d.to_csv(OUT / "p3_main_daily.csv", index=False)
323
324     payload = dict(
325         configs={k: {kk: vv for kk, vv in v.items() if kk != "daily"}
326                  for k, v in main_res.items()},
327         daily=main_res["主策略"]["daily"],
328         params={k: {kk: vv for kk, vv in v.items() if kk != "daily"}
329                for k, v in dict(gout).items()},
330         skill=forecast_skill(),
331         validation=validation(),
332         best=BEST,
333         start=START, end=END,
334     )
335     json.dump(payload, open(OUT / "p3_analysis.json", "w"),
336               ensure_ascii=False, indent=1)
337     print("已写出 06_支撑材料/p3_analysis.json")
338
339
340 if __name__ == "__main__":
341     main()

```

(5) p3_calibration.py: 1 月标定与样本外检验

```

1  # -*- coding: utf-8 -*-
2  r"""问题三：参数的滚动标定记录。
3
4  题目要求"每天 0:00 制定当天计划购电策略"，因此任何参数都只能由\textbf{已经过去}
5  的数据决定。本文的做法是：把 2025 年 1 月当作预热月，在 1 月上做一次联合标定，
6  把选出的  $(\alpha, \rho, \lambda, W)$  冻结，此后 2 月 1 日--12 月 31 日的评价区间内
7  \textbf{不再回头修改}。本脚本把这个标定过程完整跑一遍并留档，回答三个问题：
8
9  1. 1 月最优的参数是哪一组？与最终采用的  $(\alpha, \rho, \lambda, W)=(0.6, 0.5, 1.0, 28)$ 
10  差多少？
11  2. 如果直接把 1 月最优参数拿去跑 2 月--12 月，会比现在更好还是更差？
12  3. 每个参数单独看，1 月的费用曲线是不是"平底"——即结论对参数是否敏感？
13
14  结果写入 `06_支撑材料/p3_calibration.json`。
15  """
16  from __future__ import annotations
17
18  import json
19  import sys
20  from pathlib import Path

```

```

21
22 import numpy as np
23
24 HERE = Path(__file__).resolve().parent
25 sys.path.insert(0, str(HERE))
26 OUT = HERE.parent / "06_支撑材料"
27
28 JAN = range(0, 31) # 预热月: 1 月 1 日--1 月 31 日
29 BEST = dict(alpha=0.6, lam=1.0, rho=0.5, E_tar=6000.0, W=28)
30 ALPHAS = (0.4, 0.5, 0.6, 0.7, 0.8)
31 RHOS = (0.35, 0.5, 0.7)
32 WS = (14, 21, 28)
33 LAMS = (0.0, 0.5, 1.0)
34
35
36 def jan_cost(cfg):
37     """在 1 月数据上进行时序仿真, 返回该月实际总费用 (计划调整费与紧急费之和)。"""
38     import p3_backtest as B
39     import p3_microgrid as M
40     price, dates, load_kw, pv_kw, fc = M.load_all()
41     res = B.run_backtest(price, dates, load_kw, pv_kw, fc,
42                         day_range=JAN, use_adjust=True,
43                         adjust_mask=(True, True, True), **cfg)
44     days = sorted(res)
45     total = sum(res[d]["cost_total"] for d in days)
46     return dict(
47         total=total,
48         plan_adj=sum(res[d]["cost_plan_adj"] for d in days),
49         emg=sum(res[d]["cost_emg"] for d in days),
50         E144=float(res[days[-1]]["E"][-1]),
51     )
52
53
54 def worker(cfg):
55     return cfg["label"], jan_cost(cfg["params"])
56
57
58 def grid():
59     g = []
60     for a in ALPHAS:
61         for r in RHOS:
62             for w in WS:
63                 g.append(dict(label=f"a{a}_r{r}_W{w}",
64                             params=dict(BEST, alpha=a, rho=r, W=w)))
65     return g
66
67
68 def lam_scan():
69     return [dict(label=f"lam{l}", params=dict(BEST, lam=l)) for l in LAMS]
70
71
72 def main() -> None:
73     from multiprocessing import Pool
74
75     with Pool(5) as pool:
76         out = dict(pool.map(worker, grid()))
77     with Pool(5) as pool:
78         lam = dict(pool.map(worker, lam_scan()))
79
80     best_key = min(out, key=lambda k: out[k]["total"])
81     best_cfg = dict(zip(("alpha", "rho", "W"),
82                       (float(best_key.split("_")[0][1:]),
83                        float(best_key.split("_")[1][1:]),
84                        int(best_key.split("_")[2][1:]))))

```

```

85 jan_best = dict(BEST, **best_cfg)
86 jan_best_lam = min(lam, key=lambda k: lam[k]["total"])
87
88 # 问题 2: 把 1 月最优参数拿去跑评价区间, 看是否会更好
89 from p3_analysis import summarize, BEST as _B, START, END
90 import p3_backtest as B
91 import p3_microgrid as M
92 price, dates, load_kw, pv_kw, fc = M.load_all()
93
94 def full(cfg):
95     res = B.run_backtest(price, dates, load_kw, pv_kw, fc, day_range=range(0,
96     365),
97     use_adjust=True, adjust_mask=(True, True, True), **cfg
98 )
99     s = summarize(res)
100     return {k: v for k, v in s.items() if k != "daily"}
101
102 payload = dict(
103     jan=out, lam_scan=lam,
104     jan_best_key=best_key, jan_best_jan=out[best_key],
105     jan_best_cfg=jan_best,
106     jan_best_lam=jan_best_lam, jan_best_lam_jan=lam[jan_best_lam],
107     frozen=BEST, frozen_jan=out.get(
108         f"a{BEST['alpha']}_r{BEST['rho']}_w{BEST['W']}",
109     best_febdec=full(BEST),
110     jan_best_febdec=full(jan_best),
111     jan_best_lam_febdec=full(dict(BEST, lam=float(jan_best_lam[3:])),
112 )
113 (OUT / "p3_calibration.json").write_text(
114     json.dumps(payload, ensure_ascii=False, indent=1), encoding="utf-8")
115
116 print(f"1 月最优: {best_key} 费用={out[best_key]['total']:, .2f}")
117 print(f"采用值 : a0.6_r0.5_W28 费用={payload['frozen_jan']['total']:, .2f}")
118 print(f"1 月最优 lambda: {jan_best_lam} 费用={lam[jan_best_lam]['total']:, .2f}")
119 print(f"评价区间 (冻结 BEST) : {payload['best_febdec']['total']:, .2f}")
120 print(f"评价区间 (1 月最优参数) : {payload['jan_best_febdec']['total']:, .2f}")
121 print(f"评价区间 (1 月最优 λ) : {payload['jan_best_lam_febdec']['total']:, .2f}")
122 print("已写出 06_支撑材料/p3_calibration.json")
123
124 if __name__ == "__main__":
125     main()

```

(6) p3_alignment_scan.py: 附件 3 时间口径判定

```

1 # -*- coding: utf-8 -*-
2 """附件 3 的时间口径判定: 整点瞬时功率, 还是小时平均功率?
3
4 附件 3 的每条记录给出"预报 1 小时"至"预报 24 小时"共 24 个功率值, 但没有说明
5 它对应目标整点的瞬时功率, 还是目标小时内的平均功率。这个口径决定了整点之间该
6 如何取值 (分段线性插值 vs 整小时内取常值), 必须先判定。
7
8 判定方法: 把两种口径分别铺成十分钟功率序列, 与附件 2 的实际光伏按不同时间平移
9 对齐, 比较正式区间 (2025-02-01 至 2025-12-31) 内的全天平均绝对偏差 (MAE, kW)。
10 偏差最小的口径即为正确口径; 同时报告"前移一小时"作为反例, 用以说明对齐方式确实
11 被区分开了。
12
13 输出: 06_支撑材料/p3_alignment_scan.json
14
15 论文正文引用的三个数字即由本脚本产生。运行:
16 python3 03_代码/p3_alignment_scan.py
17 """

```

```

18 from __future__ import annotations
19
20 import json
21 import sys
22 from pathlib import Path
23
24 import numpy as np
25
26 sys.path.insert(0, str(Path(__file__).resolve().parent))
27 import p3_microgrid as m # noqa: E402
28 from p1_microgrid import N, TAU, interval_label # noqa: E402
29 from p2_microgrid import (N_DAY, REPORT_END, REPORT_START, # noqa: E402
30                          load_attach2)
31 # 附件 3 的载入与版本索引在滚动脚本里 (p3_microgrid 只负责单版本求解,
32 # 不导出 load_attach3)。此处必须从滚动脚本取, 否则 AttributeError。
33 from p3_rolling_microgrid import load_attach3 # noqa: E402
34
35 OUT = m.OUT_DIR / "p3_alignment_scan.json"
36 RELEASE_HOURS = (0, 6, 12, 18)
37
38
39 def _interp(A: np.ndarray, minutes: float) -> float:
40     """在整点功率序列 A[0..24] 上按分钟作分段线性插值。"""
41     x = minutes / 60.0
42     if x <= 0:
43         return float(A[0])
44     if x >= 24:
45         return float(A[24])
46     k = int(np.floor(x))
47     theta = x - k
48     return float((1 - theta) * A[k] + theta * A[k + 1])
49
50
51 def _step(A: np.ndarray, minutes: float) -> float:
52     """小时平均口径: A[j] 是第 j 个小时 ((j-1)*60 ~ j*60 分钟) 内的平均功率。"""
53     if minutes <= 0:
54         return float(A[0])
55     if minutes >= 24 * 60:
56         return float(A[24])
57     j = int(np.ceil(minutes / 60.0))
58     return float(A[min(max(j, 1), 24)])
59
60
61 def scan() -> dict:
62     LOAD, PV, dates = load_attach2()
63     fp = load_attach3(PV)
64
65     # load_attach2 返回的 (LOAD, PV) 单位已经是 kW (求解器用 PV * TAU 换算成
66     # 十分钟电量), 故这里直接就是十分钟平均功率, 不要再除 TAU。
67     pv_kw = PV
68
69     variants = {
70         "整点对齐 (分段线性插值)": (_interp, 0),
71         "前移一小时": (_interp, -60),
72         "按小时平均 (整小时内取常值)": (_step, 0),
73     }
74     acc = {k: [] for k in variants}
75     per_release = {k: {h: [] for h in RELEASE_HOURS} for k in variants}
76
77     for n in range(REPORT_START, min(REPORT_END, N_DAY)):
78         for hi, rel_h in enumerate(RELEASE_HOURS):
79             A = fp.F[hi, n, :]
80             t0 = 36 * hi # 发布时刻之前的时段该版本不预测
81             for t in range(t0, N):

```

```

82         ra = t * 10.0 - 60.0 * rel_h # 相对发布时刻的分钟数
83         actual = pv_kw[n, t]
84         if not np.isfinite(actual):
85             continue
86         for name, (fn, shift) in variants.items():
87             if ra + shift < 0:
88                 continue # 平移后落到发布前, 无定义
89             dev = abs(fn(A, ra + shift) - actual)
90             acc[name].append(dev)
91             per_release[name][rel_h].append(dev)
92
93     rep = {}
94     for name in variants:
95         rep[name] = {
96             "MAE_kw": float(np.mean(acc[name])),
97             "样本数": int(len(acc[name])),
98             "分版本MAE_kw": {
99                 f"{h}:00": round(float(np.mean(per_release[name][h])), 2)
100                 for h in RELEASE_HOURS},
101         }
102
103     best = min(rep, key=lambda k: rep[k]["MAE_kw"])
104     return {
105         "区间": f"{dates[REPORT_START]} 至 {dates[REPORT_END - 1]}",
106         "各口径MAE_kw": {k: round(v["MAE_kw"], 2) for k, v in rep.items()},
107         "分版本MAE_kw": {k: v["分版本MAE_kw"] for k, v in rep.items()},
108         "判定": best,
109         "明细": rep,
110     }
111
112
113 def main() -> None:
114     res = scan()
115     OUT.write_text(json.dumps(res, ensure_ascii=False, indent=2), encoding="utf-8")
116     print(f"区间: {res['区间']}")
117     print(f"{'口径':<28}{'MAE/kw':>10}")
118     for k, v in res["各口径MAE_kw"].items():
119         mark = "    <-- 最小, 判定为该口径" if k == res["判定"] else ""
120         print(f"{'k':<28}{'v':>10.2f}{mark}")
121     print("\n分发布版本 MAE/kw: ")
122     for k, d in res["分版本MAE_kw"].items():
123         print(f"    {'k':<28}" + " ".join(f"{h}:00={v:.2f}" for h, v in d.items()))
124     print(f"\n判定: 附件 3 为{res['判定']}")
125     print(f"写入 {OUT}")
126
127
128 if __name__ == "__main__":
129     main()

```

H 问题三结果文件说明

结果按题目要求的 result3.xlsx 与支撑材料中的内部文件两套输出。result3.xlsx 由 p3_export.py 从官方模板 01_ 题目/C 题/附件/附件 5/result3.xlsx 逐次重新拷贝后再写入, 以确保每次导出均以原始模板为起点。

(1) result3.xlsx (按附件 5 指定格式)

- 工作表 计划购电量: 逐日逐时段 (334 天 $\times$ 144 段) 的初始计划购电量 g_t^0 , 覆盖 2025-02-01 至 2025-12-31; 末两列给出当日计划总量与计划购电费 $\sum_t p_t g_t^0$;
- 工作表 调整购电量: 同一口径的最终生效购电量 x_t , 与上表逐格对照即可复现全部调整量 $x_t - g_t^0$; 末两列给出调整总量与计划调整费 $\sum_t (p_t g_t^0 + 1.5p_t u_t - 0.5p_t v_t)$;

- 工作表 充放电量：按附件 5 的长表口径排列，334 天 $\times$ 6 个四小时时段共 2004 行，给出充电量、放电量与储电量 (kWh)，每天首行记 0:00 储电量、第二行记 24:00 储电量；
- 工作表 紧急购电量：按“同日连续发生紧急购电的时段合并为一个事件”的规则生成，共 391 个事件行。合并不改变费用，合并后的电量与逐段之和相等；
- 两个购电表的时段列已按左端点改写为 0:00–0:10, ..., 23:50–0:00：官方模板的表列比数据错开一格（第 2 列标为 0:10–0:20、末列标为 0:00–0:10+1），本表按“列 2 对应 $t = 0$ ”的口径重新标注，使列名与时段索引一一对应。

(2) 内部结果文件（支撑材料 06_ 支撑材料/）

- **p3_spec.json**：四个指定日期的逐时段明细 (g_t^0 、 x_t 、调整量、紧急购电量、分块充放电量、0:00 与 24:00 储电量、当日三项费用与紧急购电事件)，即表 7–10 的原始记录；
- **p3_analysis.json**：八种预报使用方式的全年汇总、两种口径与逐时段分位数两处敏感性、14 个单参数扰动的结果、九项校验汇总与四个发布版本的预报精度，即表 18–23 的原始记录；
- **p3_main_daily.csv**：主策略逐日汇总 (334 行)，含全天电量、三项费用、储电量极值、调整时段数与紧急购电事件数；
- **p3_main_arrays.npz**：主策略逐时段数组 (g_t^0 、 x_t 、 r_t 、实际充放电、实际储电量与参考储电量)，供 **p3_tables.py** 与 **p3_figures.py** 复算表格中的分块量与作图；
- **p3_forecast_skill.csv**：四个发布版本的预报精度（覆盖段数、覆盖段平均绝对误差、12:00–18:00 与 18:00–24:00 两个窗口的分窗口误差）；
- **p3_calibration.json**：1 月标定面 (45 点)、 λ 与 W 的单独扫描、以及“用 1 月最优参数直接外推”的样本外检验。附表的 18–23 由 **p3_tables.py** 直接读取上述文件自动生成 (**p3_tables.tex** 与 **p3_tables_appendix.tex**)。

I 问题四求解核心代码

问题四的主计算由 03_ 代码/p4_microgrid.py 完成,产出 result4-2.xlsx、result4-3.xlsx 与 p4_results.json 等结果文件。论文主口径对应程序的 --refund-main 选项; 价格策略对照与预报组合须在同一退款口径下生成。论文表格由 03_ 代码/p4_tables.py 从结果文件自动生成,插图由 03_ 代码/p4_figures.py 生成。三个脚本及其依赖随支撑材料一并提交。下面按” 价格结构 → 价格预测 → 联合风险 → 信息时序约束 → 组合比较” 依次摘录核心片段。

9.1 价格分解与星期结构

正文第 8.2 节的预测式由分解 $P = F + R$ 的四项统计特征确定。代码计算这些特征并写入结果文件, 其中星期结构用于确定同星期加权预测形式。

```
1 def verify_price_structure(PRICE, FIXED, pp=PRICE_FORECAST):
2     R = PRICE - FIXED
3     wd = np.array([(datetime.date(2025, 1, 1)
4                     + datetime.timedelta(days=n)).weekday()
5                     for n in range(N_DAY)])
6     means = {int(w): float(R[wd == w].mean()) for w in range(7)}
7     eff = np.array([means[int(w)] for w in wd][:, None])
8     return {
9         "逐时段均值最大偏差": float(np.abs(PRICE.mean(axis=0)-FIXED).max()),
10        "残差与固定曲线相关": float(
11            np.corrcoef(R.ravel(), np.tile(FIXED, N_DAY))[0, 1]),
12        "残差星期均值": means,
13        "星期效应解释比例": float(1-(R-eff).var()/R.var()),
14    }
```

9.2 价格预测模型：同星期加权与日内对数修正

0:00 版本是最近三个同星期几价格的加权平均; 发布时刻 $\tau > 0$ 的版本再用已完成时段对剩余时段做一次平均对数偏差外推。两个版本都只读取发布时刻之前的数据, 这一约束由切片显式写死, 不依赖调用方的自觉。

```
1 class PriceForecaster:
2     def forecast_at(self, n: int, hi: int) -> np.ndarray:
3         """第 n 天、第 hi 个发布时刻可用的价格预测 ($\tau$ 之后的时段)。"""
4         base = self.forecast0(n) # 同星期加权, 只用 < n 的天
5         b = self.pp.beta_at(hi) # $\beta$ 逐发布时刻取值
6         if hi == 0 or b == 0.0:
7             return base
8         t0 = 36 * hi # 发布时刻对应的时段索引
9         obs, pred = self.PRICE[n, :t0], base[:t0]
10        m = (obs > 0) & (pred > 0)
11        if m.sum() < 6: # 已完成样本过少则不修正
12            return base
13        # 只用已经实现的 [0, t0) 段估计平均对数偏差, 外推到 [t0, 144)
14        bias = float(np.mean(np.log(obs[m] / pred[m])))
15        out = base.copy()
16        out[t0:] = base[t0:] * np.exp(b * bias)
17        return np.maximum(out, 1e-6)
```

9.3 联合风险：以实际电价为权的经验分位数

正文第 8.4 节把风险余量从等权分位数换成价格加权分位数。实现上是一段带权重分位数函数: 权重为同一目标时段的历史实际电价, 权重和为零时退回普通分位数, 避免样本缺失导致静默出错。

```

1 def _weighted_quantile(x: np.ndarray, q: float, w: np.ndarray) -> np.ndarray:
2     # x、w 均为 (历史日期, 目标时段) 矩阵, 按列取经验分位数
3     o = np.argsort(x, axis=0)
4     xs = np.take_along_axis(x, o, axis=0)
5     ws = np.take_along_axis(w, o, axis=0)
6     cw = np.cumsum(ws, axis=0)
7     tot = cw[-1]
8     plain = np.quantile(x, q, axis=0)
9     ok = np.isfinite(tot) & (tot > 0)
10    tgt = np.where(ok, q * tot, np.inf)[None, :]
11    reached = cw >= tgt
12    hit = reached.any(axis=0)
13    idx = reached.argmax(axis=0)
14    out = xs[idx, np.arange(x.shape[1])]
15    return np.where(hit, out, plain)

```

9.4 信息时序约束与计划缓存

为保证 0:00 计划仅使用当时可得的信息, 代码将初始计划按 (价格模式, 计费口径, 预报组合) 三元组分别缓存: 不同价格模式分别生成 g^0 , 从数据结构上隔离各策略的计划结果。

```

1 _PLAN0_CACHE4: dict[tuple, tuple[np.ndarray, np.ndarray]] = {}
2
3 def initial_plan4(n, ntilde0, e0, price, rp, tag):
4     key = (tag, n, rp.alpha, rp.lam, round(float(e0), 6))
5     hit = _PLAN0_CACHE4.get(key)
6     if hit is not None:
7         return hit[0].copy(), hit[1].copy()
8     plan = solve_dayahead(ntilde0, e0, price, rp)
9     _PLAN0_CACHE4[key] = (plan.g.copy(), plan.Ebar.copy())
10    return plan.g, plan.Ebar
11
12 # run_day43 中分别编码价格模式、计费口径和预报组合
13 tag = f"{mode}|{convention}|{S}"
14 g0, Ebar = initial_plan4(n, n_risk0, e0, price_hat0, params, tag)

```

9.5 组合比较与独立标定

附录表 32 的八个组合分别标定参数, 使标定与运行阶段采用相同的预报集合 S :

```

1 def run_combo4(S, cal43, book, LOAD, PV, fp, eps3, fo,
2               weighted=True, warmup=None):
3     cal = _replace(cal43, S=tuple(S))
4     recs_all, cal_rows = run_strategy43(
5         cal, book, "forecast", LOAD, PV, fp, eps3, fo,
6         S=S, weighted=weighted, warmup=warmup)
7     recs = recs_all[REPORT_START:REPORT_END]
8     totals = totals43(recs)
9     totals["使用预报"] = combo_name(S)
10    totals["标定记录数"] = len(cal_rows)
11    return totals

```

9.6 完整源程序

以上各段给出核心实现，下面用 \lstinputlisting 引入当前目录 code/ 下的三个完整脚本，因此包含前文已经说明的代码片段。

(1) p4_microgrid.py: 问题四主求解器

```
1 # -*- coding: utf-8 -*-
2 r"""
3 2026 CUMCM C 题 问题四 —— 实时波动电价下的购电策略
4
5 问题四要求在实时波动的外网电价下重新建立当天购电策略模型，并**重新计算
6 问题二和问题三**，分别输出 result4-2.xlsx 与 result4-3.xlsx。两个分支不是
7 同一套策略的两种写法，而是各自独立回放的策略：
8
9 4-2 沿用问题二规则：0:00 定全天计划 g，日内不得修改常规购电量
10 4-3 沿用问题三规则：0:00 定初始计划 g0，6/12/18 时可按新预报滚动调整
11
12 主线是**用决策当时能知道的信息预测价格来做决策，再用实际价格结算真实
13 运行结果**。这条分离是本题的核心：把问题二、三已经得到的购电量直接乘附件 4
14 的电价，只能得到旧策略在新价格下的账单，不能代表已经适应波动电价的新策略。
15
16 价格信息边界（框架 2.2 节）
17 题目只说电价实时波动，没有说当天价格在 0:00 已经公布。基础方案因此采用
18 明确假设：**未来实际价格在决策时不可见，只能用此前已观测价格预测；电量
19 提前确定，费用按对应交付时段的实际电价结算。** 不额外假设 0:00 已锁定
20 全天成交价。若题目补充说明价格曲线在 0:00 已全部公开，该情形应作为另一种
21 信息条件**单独报告**——本文件的 "oracle" 价格模式就是这个对照，它量化
22 "更准确的价格信息值多少钱"，但绝不作为主策略。
23
24 三个价格模式（框架 6.1 节的三种策略）
25 forecast 决策用当时的历史与预测价，结算用实际价 ← 本文主策略
26 fixed 决策用附件 1 的固定分时电价，结算用实际价 ← 原有价格判断直接
27 搬到新环境会怎样的参照
28 oracle 决策用当天未来实际价，结算用实际价 ← 信息增强对照
29 三者都在**同一条实际价格路径**上各自独立回放，改善量
30  $\Delta J = J(\text{fixed}) - J(\text{forecast})$  才是有意义的。绝不能用"旧策略在固定价下的
31 费用"减"新策略在实际价下的费用"再称其为优化收益（框架 6.1 节明令禁止）。
32
33 价格预测（框架 3.1 节）
34 先做简单、可追踪的预测，不引入复杂网络。本实现用**同星期加权**：
35  $\hat{p}^0_{\{n,t\}} = \sum_k w_k \cdot p_{\{n-7k, t\}}$ 
36 历史不足时退化为可取历史日的等权均值。日内按已完成时段的**对数价格
37 偏差**修正后续时段：
38  $\hat{p}^{\{(\tau)\}}_{\{n,t\}} = \hat{p}^0_{\{n,t\}} \cdot \exp(\beta \cdot \text{mean}_{\{s<\tau\}} \log(p_{\{n,s\}} / \hat{p}^0_{\{n,s\}}))$ 
39 滞后集合、权重与  $\beta$  由 select_forecast_params() 在**1 月预热期**（第 1--30
40 天，落在正式评价区间之外）标定一次，随后整段评价区间冻结复用——参数不在
41 评价区间上再调，避免样本内选型。正式区间上的 price_forecast_compare() 与
42 price_beta_sweep() 只做样本外复核展示，不参与选型。
43
44 联合风险（框架 3.2 与 3.3 节）
45 框架 3.2 节强调  $E[\text{PR}] = E[\text{P}]E[\text{R}] + \text{Cov}(\text{P}, \text{R})$ ，**不能**把两个期望直接相乘，
46 并且明确要求"不能在未分析数据前就认定其一定存在或一定为正"。本文件的
47 risk_price_correlation() 就是这一步，实测（正式区间，48096 个时段样本）：
48
49  $\text{corr}(\text{净负荷预测误差, 实际电价}) = +0.0863$  → 为正
50 按电价分档（合并全部时段）：最高 20% 误差均值 +40.5 kW，
51 最低 20% 为 -74.7 kW，两端差 +115.2 kW
52
53 但合并口径会**低估**这种关系：把误差按目标时段固定住再看，逐时段相关系数
54 均值达 **+0.216**（区间 0.083~0.401），明显高于合并口径的 0.086。原因是
55 电价的主要变动来自"时段之间"（傍晚贵、凌晨便宜），合并口径里这一层掩盖了
56 "同一天同一时段、价格偏高时负载也偏高"这一层。**协方差为正且是同时段内
```

的性质**，因此风险余量必须逐时段计算，不能对全年做一次汇总——这也正是框架 3.4 节要求按 t 单独求分位数的原因。

据此，风险余量取**价格加权分位数**（框架 3.3 节 $F_P(z) = E[P \cdot 1\{N \leq z\}]/E[P]$ 的经验版本，框架 3.4 节），权重是同一目标时段的历史实际电价：

$\hat{N} = \hat{N} + Q^{\alpha} P_{\alpha}(\varepsilon)$
`risk_weight_effect()` 给出加权与不加权的对照。效应是**方向正确、幅度温和**的：
 $\alpha=0.6$ 抬高 3.00 kWh (+25.6%)， $\alpha=0.7$ 抬高 3.57 kWh (+13.8%)， $\alpha=0.8$ 抬高 4.31 kWh (+10.0%)——即每时段约 +3~4 kWh。之所以不算大，是因为同一时段的价格在日与日之间变动有限（日内同一时段的日间变异系数均值仅 0.17）；但方向上它把余量推向高代价时段，且实现成本极低，故保留为主口径。--unweighted 可退回普通分位数供对照。

注意对照口径：加权一侧按**加权经验分布的下确界**取值（论文问题四“价格加权分位数”一节的公式），不加权一侧用 `np.quantile` 的默认线性插值约定（与问题二一致）。因此表中的“抬高”同时含**加权**与**取值约定**两层差别，不是纯粹的加权效应；两者的量级由 `test_weighted_quantile()` 与 `risk_weight_effect()` 分别检验。

计费（框架 5.3 节）

全部系数乘**对应交付时段的实际电价**，不乘 0:00 的预测价：

退款口径（主口径，与问题三一致）

$$C = \sum_t p_t g_t^0 + 1.5 \sum_t p_t (x_t - g_t^0)_+ - 0.5 \sum_t p_t (g_t^0 - x_t)_+ + 5 \sum_t p_t r_t$$

不退款口径（敏感性）

$$C = \sum_t p_t g_t^0 + 1.5 \sum_t p_t (x_t - g_t^0)_+ + 5 \sum_t p_t r_t$$

首项是**初始计划 g^0 **，不是最终生效量 x ：1.5 p_t 是上调部分的**总价**，不是“先付 p_t 再加 1.5 p_t ”成为 2.5 p_t （问题三 7.3 节已就此说明）。

主口径取退款，是题面“计划购电量高于调整购电量的部分，违约电价是交易时刻电价的 50%”的字面读法：被取消的部分按 0.5 p 计，故 $(g^0 - x)_+$ 的系数是 -0.5。

它等价于 $C = \sum p_t x_t + 0.5 \sum p_t |x_t - g_t^0| + 5 \sum p_t r_t$ ，两侧对称。

不退款口径把“计划购电费用”理解为无条件全额、50% 只作**额外**违约金，此时调减每度净付 1.5 p ，最优解不会调减，可外加 $x_t \geq g_t^0$ 的简化约束。它与退款口径**由同一份代码的 convention 参数切换**（见 `CalConfig3/run_strategy43`），两者各自独立标定、独立回放，不与主口径混合；--refund-main 可把 4-3 的主结果与配套的策略对照、预报组合一并切到退款口径重算。

.....

```
from __future__ import annotations
```

```
import argparse
```

```
import copy
```

```
import dataclasses
```

```
import datetime
```

```
import json
```

```
import re
```

```
import shutil
```

```
import time
```

```
from concurrent.futures import ThreadPoolExecutor
```

```
from dataclasses import dataclass
```

```
from itertools import combinations
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
from p1_microgrid import ( # noqa: E402
```

```
    N, TAU, E_MIN, E_MAX, E_INIT, M_ENERGY, ETA, interval_label, block_label,
    BLOCKS, load_attach1,
```

```
)
```

```
from p2_microgrid import ( # noqa: E402
```

```
    ROOT, N_DAY, REPORT_START, REPORT_END, EMG_EPS,
    Forecaster, RiskParams, load_attach2, solve_dayahead, execute_day,
    build_error_table, LOAD_FORECAST, PV_FORECAST, CalConfig, LAMBDA_DEFAULT,
```

```

121 ForecastParams, JIA_LOAD_FORECAST, JIA_PV_FORECAST, JIA_ALPHAS, JIA_RHOS,
122 JIA_LAMS, JIA_MIN_SAMPLES,
123 )
124 from p3_rolling_microgrid import ( # noqa: E402
125     ForecastPanel, load_attach3, forecast_energy_slots, load_forecast_at,
126     build_eps3, solve_adjust, DayRunner, CalConfig3, ALL_S,
127     RELEASE_HOURS, BLOCK_BOUNDS, COEF_UP, COEF_DOWN, COEF_EMG,
128 )
129
130 # ===== 路径与常量
131 ATTACH4 = ROOT / "01_题目" / "C题" / "附件" / "附件4.xlsx"
132 TEMPLATE42 = ROOT / "01_题目" / "C题" / "附件" / "附件5" / "result4-2.xlsx"
133 TEMPLATE43 = ROOT / "01_题目" / "C题" / "附件" / "附件5" / "result4-3.xlsx"
134 OUT_DIR = ROOT / "06_支撑材料"
135 OUT_XLSX42 = OUT_DIR / "result4-2.xlsx"
136 OUT_XLSX43 = OUT_DIR / "result4-3.xlsx"
137 OUT_JSON4 = OUT_DIR / "p4_results.json"
138 OUT_DETAIL42 = OUT_DIR / "p4_detail_42.csv"
139 OUT_DETAIL43 = OUT_DIR / "p4_detail_43.csv"
140 OUT_DAILY42 = OUT_DIR / "p4_daily_42.csv"
141 OUT_DAILY43 = OUT_DIR / "p4_daily_43.csv"
142 OUT_COMBOS4 = OUT_DIR / "p4_combinations.csv"
143 OUT_STRAT4 = OUT_DIR / "p4_strategies.csv"
144 OUT_PFCAND = OUT_DIR / "p4_price_forecast_candidates.csv"
145 OUT_PBETA = OUT_DIR / "p4_price_beta.csv"
146 OUT_RWEIGHT = OUT_DIR / "p4_risk_weight.csv"
147 CACHE_DIR = OUT_DIR / "p4_cache"
148
149 # 计费口径的两个取值。"main" 是**不退款**公式 (4-2 无口径, 该参数对它无效);
150 # 论文主口径取 "refund", 由 --refund-main 把 4-3 主结果、策略对照与预报组合
151 # 一并切过去。字符串本身不改名, 以免动到计划缓存键与既有 JSON 的键名。
152 CONVENTIONS = ("main", "refund")
153 PRICE_MODES = ("forecast", "fixed", "oracle")
154
155 MODE_LABEL = {
156     "forecast": "波动电价预测策略",
157     "fixed": "固定电价参考策略",
158     "oracle": "未来价格已知参考",
159 }
160
161
162 # ===== 问题二内核
163 # 4-2 / 4-3 的骨架就是问题二那一套 (预测 → 风险余量 → 日前 LP → 因果执行 + 滚动
164 # 标定), 差别只在价格与调整时点。所以问题二换内核时, 两问必须一起重算: 否则
165 # 三问的参数选择逻辑不一致, 跨问对照 (4-2 与 4-3 的 ΔJ) 就失去意义。
166 #
167 # legacy —— 本仓旧行为, 仅用于复现历史数字 (4-2 = 14,763,159.83 元):
168 #     预测器取模块默认、预热期参数硬编码、第 0 天冷启动、min_samples=10。
169 # jia —— 对照实现 origin/q2-jia 最终版的配方, 也是当前默认:
170 #     * 预测器写死 负荷近 28 天/0.30、光伏近 7 天/0.30;
171 #     * 预热期参数不再硬编码, 改在 5×6×6 = 180 组 (α,ρ,λ) 上按 1 月
172 #       总费用联合选出;
173 #     * λ 在 1 月选定后**整年冻结**, 正式区间只在 (α,ρ) 的 30 组上重选;
174 #     * min_samples=8。
175 # 各问的 1 月联合标定都用**各自的**日回放与计费口径进行, 所以 α/ρ/λ 会按本问的
176 # 代价结构自然取到不同的值——移植的是配方, 不是他那一组数字。
177 #
178 # **不移植的一项是第 0 天走日 LP 的口径。** 那一项在问题二里能让计划层照常
179 # 求解, 是因为电价是题面给定的固定曲线, 第 0 天也有价可用; 问题四的电价要靠
180 # 历史外推 (附件 1 那条曲线只是"固定电价参考策略", 不是预测), 第 0 天既无价格
181 # 历史也无负载/光伏历史, 日前 LP 拿不到任何可用的决策价格。因此 4-2 / 4-3 的
182 # 第 0 天一律按冷启动作业 (计划量为零、储能待机、缺口 5 倍补购), 三种价格模式
183 # 用同一口径, ΔJ 才只反映评价区间内的价格信息之差。
184

```

```

185
186 @dataclass(frozen=True)
187 class CorePreset:
188     """问题二内核的配置包：预测器与风险参数网格。
189
190     不含第 0 天口径——见上方说明，问题四第 0 天一律冷启动。
191     """
192
193     name: str
194     load_fp: ForecastParams
195     pv_fp: ForecastParams
196     risk_window: int
197     min_samples: int
198     warmup_alphas: tuple
199     warmup_rhos: tuple
200     warmup_lams: tuple
201     warmup_n_days: int = REPORT_START
202
203     def warmup_grid(self) -> list[RiskParams]:
204         """1 月联合标定的  $(\alpha, \rho, \lambda)$  网格。"""
205         return [RiskParams(alpha=a, rho=r, lam=l, window=self.risk_window,
206                             min_samples=self.min_samples)
207                 for a in self.warmup_alphas for r in self.warmup_rhos
208                 for l in self.warmup_lams]
209
210
211 CORE_LEGACY = CorePreset(
212     name="legacy", load_fp=LOAD_FORECAST, pv_fp=PV_FORECAST, risk_window=28,
213     min_samples=10, warmup_alphas=(), warmup_rhos=(), warmup_lams=())
214
215 CORE_JIA = CorePreset(
216     name="jia", load_fp=JIA_LOAD_FORECAST, pv_fp=JIA_PV_FORECAST,
217     risk_window=28, min_samples=JIA_MIN_SAMPLES,
218     warmup_alphas=JIA_ALPHAS, warmup_rhos=JIA_RHOS, warmup_lams=JIA_LAMS)
219
220 CORES = {c.name: c for c in (CORE_LEGACY, CORE_JIA)}
221
222
223 # ===== 模块 1: 数据对齐
224 def load_attach4() -> np.ndarray:
225     """读取附件 4 的全年实际电价，返回 (365, 144) 的元/kWh 数组。
226
227     附件 4 的版式与附件 2 一致：首列为日期，其后 144 列是 "0:10" ... "23:50"
228     与末尾的 "0:00+1"。按前文统一的右端点命名口径，第 j 列 (0 起) 对应区间
229     [10j, 10(j+1)) 分钟，与附件 1、2 的第 j 行逐段对齐——论文 3.1 节已就该
230     时间口径作过说明，此处沿用同一假设。
231
232     同时做框架 8.1 节要求的数据核对：无空值、无非数值、电价全部为正、
233     日期序列与 2025-01-01 起的连续日期一致。
234     """
235     raw = pd.read_excel(ATTACH4, header=0)
236     if raw.shape[0] != N_DAY:
237         raise ValueError(f"附件 4 应有 {N_DAY} 天，实际 {raw.shape[0]} 天")
238     if raw.shape[1] != N + 1:
239         raise ValueError(f"附件 4 应有 {N + 1} 列，实际 {raw.shape[1]} 列")
240     last = str(raw.columns[-1]).strip()
241     if last not in ("0:00+1", "0:00:00+1", "24:00"):
242         raise ValueError(f"附件 4 末列标签异常: {last!r}, 预期 0:00+1")
243
244     price = raw.iloc[:, 1:].to_numpy(dtype=float)
245     if price.shape != (N_DAY, N):
246         raise ValueError(f"附件 4 数据区形状异常: {price.shape}")
247     if np.isnan(price).any():
248         raise ValueError("附件 4 存在空值或非数值")

```

```

249 if (price <= 0).any():
250     raise ValueError("附件 4 出现非正电价, 最小 {price.min():.4f}")
251
252 dates = pd.to_datetime(raw.iloc[:, 0])
253 expect = pd.date_range("2025-01-01", periods=N_DAY, freq="D")
254 if not (dates.values == expect.values).all():
255     raise ValueError("附件 4 日期序列与 2025-01-01 起的连续日期不一致")
256 return price
257
258
259 def shape_ratio(PRICE: np.ndarray) -> float:
260     """单一固定日内形状能解释的总方差比例, 衡量价格的可预测性。"""
261     shape = PRICE.mean(axis=0)
262     return float(1.0 - (PRICE - shape).var() / PRICE.var())
263
264
265 def price_summary(PRICE: np.ndarray) -> dict:
266     """附件 4 的价格统计, 供论文说明"实时波动"到底有多大。"""
267     return {
268         "最小_元每kWh": float(PRICE.min()),
269         "最大_元每kWh": float(PRICE.max()),
270         "均值_元每kWh": float(PRICE.mean()),
271         "标准差_元每kWh": float(PRICE.std()),
272         "日中位峰谷比": float(np.median(PRICE.max(axis=1) / PRICE.min(axis=1))),
273         "日内形状解释方差比": shape_ratio(PRICE),
274         "日间水平离散度": float(PRICE.mean(axis=1).std() / PRICE.mean()),
275     }
276
277
278 # ===== 模块 2: 价格预测
279 @dataclass(frozen=True)
280 class PriceForecastParams:
281     """框架 3.1 节的简单可追踪价格预测参数。
282
283     这个预测器不是试出来的, 而是从一个可验证的价格结构直接推出来的。把附件 4
284     记为 P、附件 1 的固定曲线记为 F、残差 R = P - F, 实测 (见 verify_price_
285     structure()), 四个断言都被逐年数据证实):
286
287     断言 1 附件 4 的 365 天逐段均值 == 附件 1 的曲线 (最大偏差 5.2e-05)。
288         即波动电价与固定电价**年平均均值相同**, 日内的 144 段骨架也相同。
289     断言 2 R 与 F 正交 (相关 -0.0000), 故 R 就是"波动"本身。
290     断言 3 R 有明显的**星期结构**, 且是两水平的: 周五、周六 -0.1368 元/kWh,
291         其余五天 +0.0545 元/kWh, 相差 0.1913 元/kWh, 相当于均价 (0.766)
292         的 25.0%。周五周六便宜符合用电侧需求走低的常识。
293     断言 4 这个星期结构解释了 R 总方差的 41.7%。
294
295     于是正确的预测式不是"对近期价格做平滑", 而是**把骨架和星期偏移分开**:
296
297         
$$\hat{p}_n^o\{n, t\} = F_t + \sum_k w_k \cdot R_{\{n-7k, t\}}$$

298
299     断言 2 保证这一步是恒等式, 实测偏差 4.4e-16。也就是说: 同星期加权 ==
300     附件 1 曲线 + 同星期残差的加权平均。滞后取 7/14/21 天 (三个同星期样本),
301     权重偏向最近一期——因为星期偏移本身是缓慢漂移的, 越近样本越可信。这也
302     解释了为什么"近 N 天滚动均值" (0.0827~0.0867) 反而比不动脑筋用固定曲线
303     (0.0961) 只好一点点: 把不同星期混在一起平均, 恰恰把这个 0.19 元/kWh
304     的偏移平均掉了。候选对比见 price_forecast_compare()。
305
306     日内再用对数偏差修正 (β, 见 price_beta_sweep()): 当天已完成时段的实际
307     价格高于预测, 说明当天整体水平偏高, 按比例外推到剩余时段。
308
309     这里的 lags/weights/beta 只是**候选模板的默认值**; 正式计算用的是
310     select_forecast_params() 在 1 月预热期选出的冻结参数。
311     """
312     lags: tuple = (7, 14, 21)

```

```

313 weights: tuple = (0.6, 0.3, 0.1)
314 beta: float = 0.5
315 name: str = "同星期加权"
316 # ( $\beta_{@6:00}$ ,  $\beta_{@12:00}$ ,  $\beta_{@18:00}$ )。None 表示三个时点一律取 beta。允许逐时点取值
317 # 是因为各时点可用于估计偏差的已完成时段数不同 (6:00 只有 36 段、18:00 有
318 # 108 段), 最优收缩系数本就未必相同。
319 beta_by_hour: tuple | None = None
320
321 def beta_at(self, hi: int) -> float:
322     """第 hi 个发布时刻的  $\beta$ ; hi = 0/1/2/3 对应 0:00/6:00/12:00/18:00。"""
323     if self.beta_by_hour is None or not 1 <= hi <= len(self.beta_by_hour):
324         return self.beta
325     return float(self.beta_by_hour[hi - 1])
326
327 def label(self) -> str:
328     w = "/".join(f"{x:g}" for x in self.weights)
329     if self.beta_by_hour is None:
330         b = f" $\beta$ ={self.beta:g}"
331     else:
332         b = " $\beta$ =" + "/".join(f"{x:g}" for x in self.beta_by_hour)
333     return (f"{self.name}(滞后{'', '.join(map(str, self.lags))});"
334           f"权重{w};{b}")
335
336 PRICE_FORECAST = PriceForecastParams()
337
338
339 class PriceForecaster:
340     """因果价格预测器: 只用第 n 天之前已完整观测的历史价格。"""
341
342     def __init__(self, PRICE: np.ndarray,
343                  pp: PriceForecastParams = PRICE_FORECAST):
344         self.PRICE, self.pp = PRICE, pp
345         self._f0 = [self._compute0(n) for n in range(N_DAY)]
346
347     def _compute0(self, n: int) -> np.ndarray | None:
348         """同星期加权; 不足一个滞后时退化为已有历史的等权均值。
349
350         第 0 天没有任何已观测价格, 返回 None —— 该日按冷启动处理 (计划量为
351         零、缺口全部紧急补购), 与问题二、三的预热日约定一致; 它也不在
352         2025-02-01 起的正式评价区间内。
353         """
354         if n == 0:
355             return None
356         idx, w = [], []
357         for lag, wt in zip(self.pp.lags, self.pp.weights):
358             if n - lag >= 0:
359                 idx.append(n - lag)
360                 w.append(wt)
361         if not idx:
362             return self.PRICE[:n].mean(axis=0)
363         w = np.asarray(w, dtype=float)
364         w = w / w.sum()
365         return (self.PRICE[idx] * w[:, None]).sum(axis=0)
366
367     def forecast0(self, n: int) -> np.ndarray:
368         f = self._f0[n]
369         if f is None:
370             raise ValueError(f"第 {n} 天无历史价格, 不能给出 0:00 预测")
371         return f.copy()
372
373     def forecast_at(self, n: int, hi: int) -> np.ndarray:
374         """第 hi 个发布时刻的价格预测 (框架 3.1 式的对数偏差修正) 。
375
376

```

```

377     hi = 0 直接返回 0:00 预测; hi > 0 时只用当天 s < 36 · hi 的**实际**
378     价格算偏差, 绝不用到尚未交付时段的价格。已完成样本过少时不修正。
379     """
380     base = self.forecast0(n)
381     b = self.pp.beta_at(hi)
382     if hi == 0 or b == 0.0:
383         return base
384     t0 = 36 * hi
385     obs, pred = self.PRICE[n, :t0], base[:t0]
386     m = (obs > 0) & (pred > 0)
387     if m.sum() < 6:
388         return base
389     bias = float(np.mean(np.log(obs[m] / pred[m])))
390     out = base.copy()
391     out[t0:] = base[t0:] * np.exp(b * bias)
392     return np.maximum(out, 1e-6)
393
394 # ----- 预测器选型
395 def _cand_forecast(PRICE: np.ndarray, n: int, lags: tuple,
396                   weights: tuple) -> np.ndarray:
397     idx, w = [], []
398     for lag, wt in zip(lags, weights):
399         if n - lag >= 0:
400             idx.append(n - lag)
401             w.append(wt)
402     if not idx:
403         return PRICE[:n].mean(axis=0)
404     w = np.asarray(w, float)
405     return (PRICE[idx] * (w / w.sum()))[:, None].sum(axis=0)
406
407
408 def price_forecast_compare(PRICE: np.ndarray) -> pd.DataFrame:
409     """价格预测候选方案在**正式评价区间**上的逐一对表 (样本外复核) 。
410
411     框架 3.1 节要求"先做简单、可追踪的预测, 并明确误差评价方式", 同时允许
412     "只有在给出明确理由时才引入更复杂模型"。这张表展示各候选在正式区间上
413     的样本外表现, **不参与选型**——选型由 select_forecast_params() 在 1 月
414     预热期完成并冻结。候选包含:
415
416         昨天同时段          最朴素的持续预测
417         同星期 (上周同一天)  捕捉星期效应, 但只用一个样本
418         近 7/14/28 天滚动均值 平滑但抹掉了星期结构
419         同星期等权 1/3        用于确认"加权"本身有没有用
420         同星期加权 0.5/0.3/0.2 / 0.6/0.3/0.1
421                             用于确认权重怎么给
422         全样本固定日内形状    把形状与水平分开、水平取全样本均值 (含未来信息,
423                             是事后口径, 仅作下界参照, 不是可用策略)
424
425     """
426     lo, hi = REPORT_START, REPORT_END
427     rows = []
428     for name, lags, w in CANDIDATES:
429         err = [float(np.abs(PRICE[n] - _cand_forecast(PRICE, n, lags, w)).mean())
430               for n in range(lo, hi)]
431         rows.append({"预测器": name, "平均绝对误差_元每kWh": float(np.mean(err))})
432     shape = PRICE[lo:hi].mean(axis=0)
433     rows.append({"预测器": "全样本固定日内形状 (事后口径, 仅作下界)",
434               "平均绝对误差_元每kWh": float(np.abs(PRICE[lo:hi] - shape).mean())})
435     df = pd.DataFrame(rows)
436     df["相对均价"] = df["平均绝对误差_元每kWh"] / float(PRICE[lo:hi].mean())
437     return df.sort_values("平均绝对误差_元每kWh").reset_index(drop=True)
438
439
440 CANDIDATES = [

```

```

441 ("昨天同时段", (1,), (1.0,)),
442 ("同星期(上周同一天)", (7,), (1.0,)),
443 ("近 7 天滚动均值", tuple(range(1, 8)), (1.0,) * 7),
444 ("近 14 天滚动均值", tuple(range(1, 15)), (1.0,) * 14),
445 ("近 28 天滚动均值", tuple(range(1, 29)), (1.0,) * 28),
446 ("同星期等权 1/3", (7, 14, 21), (1 / 3, 1 / 3, 1 / 3)),
447 ("同星期加权 0.5/0.3/0.2", (7, 14, 21), (0.5, 0.3, 0.2)),
448 ("同星期加权 0.6/0.3/0.1", (7, 14, 21), (0.6, 0.3, 0.1)),
449 ]
450
451
452 def select_forecast_params(PRICE: np.ndarray, lo: int, hi: int
453                           ) -> tuple[PriceForecastParams, dict]:
454     """在**预期间** [lo, hi) 上一次性选定价格预测器的权重与  $\beta$ 。
455
456     框架第八节第 2 条要求"选参同样只用过去数据"。因此预测器超参数**只在正式
457     评价区间之前的窗口上选一次**，随后整段评价区间冻结使用；评价区间上报出的
458     精度与费用因此是真正的样本外结果，不存在"用考试答案选方法"的问题。
459
460      $\beta$  逐发布时刻单独选：6:00 只有 36 段已完成样本、18:00 有 108 段，可用于
461     估计偏差的样本量不同，最优收缩系数本就未必相同。选择准则统一为"该版本对
462     剩余时段的 MAE 最小"。
463
464     返回 (选中参数，选型记录)；选型记录原样进入结果文件，供论文报告选型窗口、
465     候选对照与最终取值。
466     """
467     scored = [(
468         name, lags, w,
469         float(np.mean([np.abs(PRICE[n] - _cand_forecast(PRICE, n, lags, w)).mean()
470                     for n in range(lo, hi)])),
471     ) for name, lags, w in CANDIDATES]
472     scored.sort(key=lambda r: r[3])
473     rows = [{"预测器": s[0], "平均绝对误差_元每kWh": s[3]} for s in scored]
474     best_name, lags, weights, best_err = scored[0]
475
476     # 稳健性：1 月只有 30 天，且前 21 天的滞后 14/21 天样本尚不可用 (_cand_forecast
477     # 会按可用样本重归一权重)。为确认结论不是这段"退化期"造成的，把预热期再切成
478     # 三个子窗口重跑候选对照；若排序不变，选型就可以放心冻结。
479     robust = []
480     for rlo, rhi, rlab in ((lo, hi, f"全窗口 (第 {lo}--{hi - 1} 天)"),
481                           (lo + 21, hi, f"后段 (第 {lo + 21}--{hi - 1} 天, "
482                                     f"滞后样本齐备)"),
483                           (lo + 14, hi, f"后半 (第 {lo + 14}--{hi - 1} 天)")):
484         if rhi - rlo < 5:
485             continue
486         rr = sorted(
487             (float(np.mean([np.abs(PRICE[n]
488                             - _cand_forecast(PRICE, n, l, w)).mean()
489                             for n in range(rlo, rhi)])), nm)
490             for nm, l, w in CANDIDATES)
491         robust.append({"子窗口": rlab, "天数": rhi - rlo,
492                      "最优预测器": rr[0][1],
493                      "最优平均绝对误差_元每kWh": rr[0][0],
494                      "前三": [{"预测器": nm, "平均绝对误差_元每kWh": e}
495                               for e, nm in rr[:3]]})
496
497     #  $\beta$  在**同一预期间**上对表，且用**选中的权重**——此前写死 0.6/0.3/0.1,
498     # 权重一旦改变选型就与预测器不一致。
499     def _beta_sweep(wlo: int, whi: int) -> tuple[list, dict]:
500         rws = []
501         for beta in (0.0, 0.25, 0.5, 0.75, 1.0, 1.25):
502             pf = PriceForecaster(PRICE, PriceForecastParams(lags, weights, beta))
503             perl = []
504             for h in (0, 1, 2, 3):

```

```

505         t0 = 36 * h
506         perl.append(float(np.mean(
507             [np.abs(PRICE[n][t0:] - pf.forecast_at(n, h)[t0:]).mean()
508                 for n in range(wlo, whi)])))
509         rws.append({"β": beta, "各版本_元每kWh": perl})
510         bb = {int(h): min(rws, key=lambda r: r["各版本_元每kWh"][h])["β"]}
511         for h in (1, 2, 3)}
512     return rws, bb
513
514 beta_rows, best_beta = _beta_sweep(lo, hi)
515 # β 的稳健性: 滞后样本齐备的后段窗口上重扫一次, 看最优 β 是否漂移。
516 if hi - (lo + 21) >= 5:
517     _, rbb = _beta_sweep(lo + 21, hi)
518     robust.append({"子窗口": f"后段 β 重扫 (第 {lo + 21}--{hi - 1} 天)",
519                  "天数": hi - (lo + 21), "最优预测器": " (β 逐时刻)",
520                  "最优平均绝对误差_元每kWh": None,
521                  "前三": [{"预测器": f"{RELEASE_HOURS[h]} 版本 β",
522                           "平均绝对误差_元每kWh": rbb[h]}
523                          for h in (1, 2, 3)]})
524 pp = PriceForecastParams(
525     lags, weights, beta=0.5,
526     beta_by_hour=tuple(best_beta[h] for h in (1, 2, 3)))
527 record = {
528     "窗口": f"第 {lo}--{hi - 1} 天 (正式评价区间之外)",
529     "候选": rows,
530     "最优预测器": best_name,
531     "最优平均绝对误差_元每kWh": best_err,
532     "选中滞后": list(lags),
533     "选中权重": list(weights),
534     "β对照": beta_rows,
535     "β最优": {RELEASE_HOURS[h]: b for h, b in best_beta.items()},
536     "稳健性": robust,
537     "选中的参数": pp.label(),
538 }
539 return pp, record
540
541
542 def verify_price_structure(PRICE: np.ndarray, FIXED: np.ndarray,
543                           pp: PriceForecastParams = PRICE_FORECAST) -> dict:
544     """验证价格分解  $P = F + R$  的四个断言 (PriceForecastParams 的推导依据)。
545
546     这不是自动动作: 框架 8.1/8.2 节要求对价格数据的结构做检查并如实报告, 而
547     "波动电价与固定电价年平均值相同"这一条直接决定了策略对照该怎么解读——
548     如果附件 4 只是把附件 1 的曲线整体抬高或压低, 那么费用差异里就混了水平
549     效应; 实测两者水平相同, 费用差异**纯粹来自波动**, 对照才是干净的。
550
551     同时报告"同星期加权 == 附件1曲线 + 同星期残差加权"这个恒等式的数值残差。
552     """
553     R = PRICE - FIXED
554     d0 = datetime.date(2025, 1, 1)
555     wd = np.array([(d0 + datetime.timedelta(days=n)).weekday()
556                    for n in range(N_DAY)]) # 0=周一 ... 6=周日
557     means = {int(w): float(R[wd == w].mean()) for w in range(7)}
558     eff = np.array([means[int(w)] for w in wd][:, None])
559     # 恒等式残差
560     gaps = []
561     for n in range(REPORT_START, REPORT_END):
562         idx = [n - lag for lag in pp.lags if n - lag >= 0]
563         w = np.array(pp.weights[:len(idx)], float)
564         w = w / w.sum()
565         lhs = (PRICE[idx] * w[:, None]).sum(axis=0)
566         rhs = FIXED + (R[idx] * w[:, None]).sum(axis=0)
567         gaps.append(float(np.abs(lhs - rhs).max()))
568     return {

```

```

569     "断言1_附件4逐段均值与附件1最大偏差": float(
570         np.abs(PRICE.mean(axis=0) - FIXED).max()),
571     "断言1_年平均值_附件1": float(FIXED.mean()),
572     "断言1_年平均值_附件4": float(PRICE.mean()),
573     "断言2_残差与固定曲线相关": float(
574         np.corrcoef(R.ravel(), np.tile(FIXED, N_DAY))[0, 1]),
575     "断言3_残差星期均值_元每kWh": {
576         ["周一", "周二", "周三", "周四", "周五", "周六", "周日"][k]: v
577         for k, v in means.items()},
578     "断言3_周五六均值": float((means[4] + means[5]) / 2),
579     "断言3_其余五天均值": float(np.mean([means[k] for k in
580         (0, 1, 2, 3, 6)])),
581     "断言3_两水平差_元每kWh": float(np.mean([means[k] for k in
582         (0, 1, 2, 3, 6)])
583         - (means[4] + means[5]) / 2),
584     "断言3_相对均价": float((np.mean([means[k] for k in (0, 1, 2, 3, 6)])
585         - (means[4] + means[5]) / 2) / FIXED.mean()),
586     "断言4_残差总方差": float(R.var()),
587     "断言4_扣除星期效应后方差": float((R - eff).var()),
588     "断言4_星期效应解释比例": float(1 - (R - eff).var() / R.var()),
589     "断言2_同星期加权恒等式最大残差": float(max(gaps)),
590 }
591
592 def price_beta_sweep(PRICE: np.ndarray, pp: PriceForecastParams) -> pd.DataFrame:
593     """日内对数偏差修正系数  $\beta$  的扫描。
594
595      $\beta = 0$  相当于不做日内修正。修正用当天已完成时段的平均对数偏差外推，捕捉的
596     是**水平的日内漂移**； $\beta$  过大则把已实现偏差过度外推，在偏差只是噪声时反而
597     变差。表中每个  $\beta$  下各列是"在对应发布时刻更新后、对剩余时段的 MAE"。
598     """
599     rows = []
600     for beta in (0.0, 0.25, 0.5, 0.75, 1.0, 1.25):
601         pf = PriceForecaster(PRICE, PriceForecastParams(pp.lags, pp.weights,
602             beta))
603         row = {" $\beta$ ": beta}
604         for hi in (0, 1, 2, 3):
605             t0 = 36 * hi
606             e = [float(np.abs(PRICE[n, t0:] - pf.forecast_at(n, hi)[t0:]).mean())
607                 for n in range(REPORT_START, REPORT_END)]
608             row[f"{RELEASE_HOURS[hi]}点版本_元每kWh"] = float(np.mean(e))
609         rows.append(row)
610     return pd.DataFrame(rows)
611
612 def price_forecast_skill(PRICE: np.ndarray, pf: PriceForecaster) -> dict:
613     """价格预测精度：0:00 版本与各发布时刻更新后的版本（框架 6.3 节）。
614
615     除平均绝对误差外，单独报告**高价时段**的识别情况——框架 6.3 节要求
616     "除平均误差外，检查对高价时段及高费用日期的识别情况"。高费用才是真正
617     影响账单的部分，平均误差小不代表高价时段看得准。
618
619     另设**共同时段**一列：第 hi 个版本只能修正它发布之后的时段，故四个版本
620     的"平均绝对误差"覆盖的时段互不相同（0:00 版覆盖全天 144 段，18:00 版只
621     覆盖最后 36 段），直接比大小是错的。共同时段一列把四个版本都限制在
622     18:00--24:00（索引 108--143，四个版本都覆盖）上求平均绝对误差，才是可比的。
623     两列都保留：前者说明各版本在自己负责的时段上的绝对水平，后者才是版本间排序。
624     """
625     mae = {hi: [] for hi in (0, 1, 2, 3)}
626     hi_mae = {hi: [] for hi in (0, 1, 2, 3)}
627     comm = {hi: [] for hi in (0, 1, 2, 3)}
628     T_COMMON = 108 # 18:00 对应的时段索引
629     for n in range(REPORT_START, REPORT_END):
630         act = PRICE[n]

```

```

633     thr = float(np.quantile(act, 0.8))          # 当天实际价格的高价阈值
634     for hi in (0, 1, 2, 3):
635         t0 = 36 * hi
636         seg_a = act[t0:]
637         seg_h = pf.forecast_at(n, hi)[t0:]
638         mae[hi].append(float(np.abs(seg_a - seg_h).mean()))
639         m = seg_a >= thr
640         if m.sum() > 0:
641             hi_mae[hi].append(float(np.abs(seg_a[m] - seg_h[m]).mean()))
642             comm[hi].append(float(np.abs(act[T_COMMON:] -
643                                     pf.forecast_at(n, hi)[T_COMMON:].mean())))
644     mean_price = float(PRICE[REPORT_START:REPORT_END].mean())
645     out = {}
646     for hi in (0, 1, 2, 3):
647         out[f"{RELEASE_HOURS[hi]}点版本"] = {
648             "平均绝对误差_元每kWh": float(np.mean(mae[hi])),
649             "相对均价": float(np.mean(mae[hi])) / mean_price,
650             "高价时段平均绝对误差_元每kWh": (
651                 float(np.mean(hi_mae[hi])) if hi_mae[hi] else None,
652             "共同时段平均绝对误差_元每kWh": float(np.mean(comm[hi])),
653         }
654     # 对照：把价格预测换成"全天用同一个均价"，衡量形状信息有没有用
655     flat = [float(np.abs(PRICE[n] - PRICE[n].mean()).mean())
656             for n in range(REPORT_START, REPORT_END)]
657     out["对照_全天均价"] = {
658         "平均绝对误差_元每kWh": float(np.mean(flat)),
659         "相对均价": float(np.mean(flat)) / mean_price,
660         "高价时段平均绝对误差_元每kWh": None,
661     }
662     return out
663
664 # ===== 模块 3: 联合风险
665 def _weighted_quantile(x: np.ndarray, q: float, w: np.ndarray) -> np.ndarray:
666     """按列的价格加权分位数 Q^P_q (框架 3.3/3.4 节) 。
667
668     x, w 形状均为 (k, m): k 个历史样本、m 个目标时段。权重是同一目标时段的
669     历史**实际电价**，因此加权后的分布  $F_P(z) = E[P \cdot 1\{N \leq z\}] / E[P]$  在
670     "价格高  $\Rightarrow$  缺电代价大"的方向上给误差更大的权重。权重全为零的列退回
671     普通分位数，避免除零。
672
673     定义取**加权经验分布的下确界**（与论文问题四"价格加权分位数"一节的公式
674     逐字一致）：
675
676         
$$Q^P_q = \inf\{z : \sum_i w_i \cdot 1[x_i \leq z] / \sum_i w_i \geq q\}$$

677
678     即排序后累积权重比例首次达到 q 的那一个**样本值**。这是临界比规则所要的
679     分位数：按此取值，加权意义下的覆盖率恰好达到 q。权重全相等时它精确退化
680     为经验分布分位数 (np.quantile 的 inverted_cdf 约定)。
681
682     注意不要退回"在累积权重轴上线性插值" (np.interp) 的写法：插值出的值可能
683     落在两个样本之间，其加权覆盖率并不等于 q（例如样本 [0,10] 等权、q=0.8 时
684     插值给 6，而加权覆盖率在该点只有 0.5），且权重全相等时也不退回任何标准
685     分位数。见本文件 test_weighted_quantile() 的自检。
686     """
687     o = np.argsort(x, axis=0)
688     xs = np.take_along_axis(x, o, axis=0)
689     ws = np.take_along_axis(w, o, axis=0)
690     cw = np.cumsum(ws, axis=0)
691     tot = cw[-1]
692     plain = np.quantile(x, q, axis=0)
693     ok = np.isfinite(tot) & (tot > 0)
694     # 权重和无效的列目标阈值取 +inf，于是 reached 恒为 False、走普通分位数；
695     # argmax 在整列为 False 时返回 0，由 hit 掩掉，不会误取第 0 个样本。

```

```

697     tgt = np.where(ok, q * tot, np.inf)[None, :]
698     reached = cw >= tgt
699     hit = reached.any(axis=0)
700     idx = reached.argmax(axis=0)
701     out = xs[idx, np.arange(x.shape[1])]
702     return np.where(hit, out, plain)
703
704
705 def test_weighted_quantile() -> list[str]:
706     """加权分位数定义的自检，返回失败项列表（空 = 全部通过）。
707
708     三条性质，缺一不可：
709     1) 权重全相等时精确退回经验分布分位数（inverted_cdf 约定）；
710     2) 取值必是某个样本值（下确界定义不允许落在样本之间）；
711     3) 该点处的加权覆盖率 >= q，且是满足此条件的最小样本值。
712     """
713     bad: list[str] = []
714     checks = []
715     for x, w, q in [
716         (np.array([[0.0], [10.0]]), np.array([[1.0], [1.0]]), 0.8),
717         (np.array([[0.0], [10.0]]), np.array([[1.0], [1.0]]), 0.5),
718         (np.array([[0.0], [10.0]]), np.array([[3.0], [1.0]]), 0.8),
719         (np.arange(28.0).reshape(-1, 1), np.ones((28, 1)), 0.9),
720     ]:
721         checks.append((x, w, q))
722     for i, (x, w, q) in enumerate(checks):
723         got = _weighted_quantile(x, q, w)[0]
724         xs = np.sort(x[:, 0])
725         ws = w[np.argsort(x[:, 0]), 0]
726         # 1) 等权退回 inverted_cdf
727         if np.allclose(w[:, 0], w[0, 0]):
728             ref = float(np.quantile(x, q, axis=0, method="inverted_cdf")[0])
729             if not np.isclose(got, ref):
730                 bad.append(f"自检{i}: 等权未退回 inverted_cdf ({got} vs {ref}) ")
731         # 2) 取值必是样本值
732         if not np.any(np.isclose(got, xs)):
733             bad.append(f"自检{i}: 取值 {got} 不在样本中")
734         # 3) 覆盖率 >= q 且最小
735         cw = np.cumsum(ws)
736         cov = cw[np.isclose(xs, got)][0] / cw[-1]
737         if cov < q - 1e-12:
738             bad.append(f"自检{i}: 覆盖率 {cov:.4f} < q={q}")
739         smaller = xs[xs < got - 1e-12]
740         if smaller.size:
741             cws = np.cumsum(ws)[np.isclose(xs, smaller[-1])][0] / cw[-1]
742             if cws >= q - 1e-12:
743                 bad.append(f"自检{i}: 存在更小的满足点 {smaller[-1]}")
744         # 4) 权重和为零退回普通分位数（不除零、不出 NaN）
745         z = _weighted_quantile(np.arange(28.0).reshape(-1, 1), 0.8,
746                                np.zeros((28, 1)))
747         if not np.isfinite(z[0]):
748             bad.append("自检4: 零权重未退回普通分位数（出现 NaN）")
749     return bad
750
751
752 def _pool_slots(hist: np.ndarray, wp: np.ndarray | None, span: int = 2):
753     """样本不足时合并相邻 ±span 个十分钟时段，再取分位数。
754
755     合并的是**同一批历史日期**的相邻时段，不引入未来信息。
756     """
757     m = hist.shape[1]
758     pooled = np.empty_like(hist)
759     pooled_w = None if wp is None else np.empty_like(wp)
760     for s in range(m):

```

```

761     a, b = max(0, s - span), min(m, s + span + 1)
762     pooled[:, s] = hist[:, a:b].mean(axis=1)
763     if wp is not None:
764         pooled_w[:, s] = wp[:, a:b].mean(axis=1)
765     return pooled, pooled_w
766
767
768 def _margin_from(hist: np.ndarray, wp: np.ndarray | None, alpha: float,
769                 min_samples: int, weighted: bool, span: int = 2
770                 ) -> np.ndarray:
771     """在给定历史残差上取（价格加权）分位数，样本不足则先合并相邻时段。"""
772     if hist.shape[0] >= min_samples:
773         if weighted:
774             return _weighted_quantile(hist, alpha, wp)
775         return np.quantile(hist, alpha, axis=0)
776     pooled, pooled_w = _pool_slots(hist, wp if weighted else None, span)
777     if weighted:
778         return _weighted_quantile(pooled, alpha, pooled_w)
779     return np.quantile(pooled, alpha, axis=0)
780
781
782 def risk_margin4(eps: np.ndarray, n: int, alpha: float | None,
783                 PRICE: np.ndarray, window: int = 28, min_samples: int = 10,
784                 weighted: bool = True) -> np.ndarray:
785     """4-2 的风险余量：价格加权分位数（框架 3.4 节）。
786
787     eps 是 (365,144) 的净负荷预测残差（问题二口径：光伏也靠历史外推，
788     因为框架 2.1 节规定附件 3 只进入 4-3）。权重取同一目标时段的历史实际
789     电价 PRICE[lo:n, t]，全部是第 n 天之前已观测完的数据，满足信息边界。
790     """
791     out = np.zeros(N)
792     if alpha is None or n == 0:
793         return out
794     lo = max(0, n - window)
795     hist = eps[lo:n, :]
796     keep = ~np.isnan(hist).all(axis=1)
797     hist = hist[keep]
798     if hist.shape[0] < 3:
799         return out
800     wp = PRICE[lo:n, :][keep] if weighted else None
801     return _margin_from(hist, wp, alpha, min_samples, weighted)
802
803
804 def risk_margin4_ver(eps3: np.ndarray, hi: int, n: int, alpha: float | None,
805                     PRICE: np.ndarray, window: int = 28,
806                     min_samples: int = 10, weighted: bool = True
807                     ) -> np.ndarray:
808     """4-3 的按发布版本价格加权风险余量。
809
810     与 risk_margin4 同构，只是残差取 eps3[hi] 且只在  $t \geq 36 \cdot hi$  上有定义。
811     逻辑与 p3 的 risk_margin3 一致，只把“分位数”换成“价格加权分位数”。
812     """
813     t0 = 36 * hi
814     out = np.zeros(N)
815     if alpha is None or n == 0:
816         return out
817     lo = max(0, n - window)
818     hist = eps3[hi, lo:n, t0:]
819     keep = ~np.isnan(hist).all(axis=1)
820     hist = hist[keep]
821     if hist.shape[0] < 3:
822         return out
823     wp = PRICE[lo:n, t0:][keep] if weighted else None
824     out[t0:] = _margin_from(hist, wp, alpha, min_samples, weighted)

```

```

825 return out
826
827
828 def risk_weight_effect(eps: np.ndarray, PRICE: np.ndarray,
829                       alphas=(0.5, 0.6, 0.7, 0.8)) -> pd.DataFrame:
830     """价格加权与普通分位数的对照表，支撑框架 3.3 节的判断。
831
832     在正式区间内抽样逐日求两种口径的风险余量，报告均值与被加权口径抬高的
833     幅度。若加权与不加权几乎一样，框架 3.3 节就可以一句话带过；实测差别
834     显著，故必须作为模型的一部分写进论文。
835     """
836     rows = []
837     ones = np.ones_like(PRICE)
838     ns = range(REPORT_START, REPORT_END, 7)
839     for a in alphas:
840         w = [risk_margin4(eps, n, a, PRICE, weighted=True) for n in ns]
841         p = [risk_margin4(eps, n, a, PRICE, weighted=False) for n in ns]
842         # 与加权口径**同约定**的等权基线：权重恒为 1 的加权分位数精确等于经验
843         # 分布分位数 (inverted_cdf)，且走完全相同的合并时段逻辑。用它作分母，
844         # "相对抬升"才是纯粹的加权效应；用 p (问题二的线性插值约定) 作分母则
845         # 会混入取值约定之差。两列都报，读者可自行核对。
846         q = [risk_margin4(eps, n, a, ones, weighted=True) for n in ns]
847         wm, pm = float(np.mean(w)), float(np.mean(p))
848         qm = float(np.mean(q))
849         rows.append({"α": a, "价格加权均值_kWh": wm,
850                    "普通分位数均值_kWh": pm, "抬高_kWh": wm - pm,
851                    "相对抬升": (wm / pm - 1.0) if pm > 1e-9 else None,
852                    "等权下确界均值_kWh": qm,
853                    "纯加权相对抬升": (wm / qm - 1.0) if qm > 1e-9 else None})
854     # pm 可能在 α≤0.5 时为负（误差中位数本身为负），此时"相对抬升"无意义，
855     # 上面已置 None，避免 NaN 进入 JSON。
856     return pd.DataFrame(rows)
857
858
859 def risk_price_correlation(eps: np.ndarray, PRICE: np.ndarray) -> dict:
860     """框架 3.2 节的协方差检查：残差与电价到底有没有关系。
861
862     这是框架明确要求"不能在未分析数据前就认定"的那一步，因此报告两种口径：
863
864         合并口径 把全部（日期，时段）样本混在一起算。这一口径会被"时段之间的
865                 电价差异"主导——傍晚电价高、误差也大——看起来相关，但分不清
866                 是"同时段内价格高伴随误差大"还是单纯"贵时段误差本来就大"。
867         逐时段口径 对每个目标时段单独算相关，再看这 144 个系数的分布。这才是
868                 风险余量真正的建模对象，因为余量是逐时段加的。
869
870     实测逐时段的相关系数明显高于合并口径，说明这种关系**是同时段内的性质**，
871     不是时段间的成分。eps 是 kWh/时段，此处换算成 kW 再报告，与负载同量纲。
872     """
873     e_all = eps[REPORT_START:REPORT_END, :] / TAU # kWh/时段 → kW
874     p_all = PRICE[REPORT_START:REPORT_END, :]
875     m = np.isfinite(e_all)
876     e, p = e_all[m], p_all[m]
877     hi = e[p >= float(np.quantile(p, 0.8))]
878     lo = e[p <= float(np.quantile(p, 0.2))]
879     r = float(np.corrcoef(e, p)[0, 1])
880     # 逐时段相关：只保留两端都有变异的时段，避免常数序列产生 NaN
881     cs = []
882     for t in range(N):
883         a, b = e_all[:, t], p_all[:, t]
884         ok = np.isfinite(a)
885         if ok.sum() > 10 and np.std(a[ok]) > 1e-9 and np.std(b[ok]) > 1e-9:
886             cs.append(float(np.corrcoef(a[ok], b[ok])[0, 1]))
887     cs = np.array(cs)
888     return {

```

```

889         "样本数": int(e.size),
890         "合并相关系数": r,
891         "合并协方差_kW元每kWh": float(np.cov(e, p)[0, 1]),
892         "逐时段相关系数均值": float(cs.mean()),
893         "逐时段相关系数最小": float(cs.min()),
894         "逐时段相关系数最大": float(cs.max()),
895         "逐时段相关为正的比率": float((cs > 0).mean()),
896         "低价20%_误差均值_kW": float(lo.mean()),
897         "低价20%_正误差概率": float((lo > 0).mean()),
898         "高价20%_误差均值_kW": float(hi.mean()),
899         "高价20%_正误差概率": float((hi > 0).mean()),
900         "两端误差均值差_kW": float(hi.mean() - lo.mean()),
901         "结论": ("协方差为正, 忽略会低估紧急购电风险" if r > 0
902                  else "协方差为负, 忽略会高估风险"),
903     }
904
905
906 # ===== 决策价格
907 class PriceBook:
908     """按价格模式给出决策时刻应当使用的价格（框架 6.1 节的三种策略）。
909
910     三种模式在**结算**时都使用附件 4 的实际价格，区别只在于**做决策时**看得
911     到什么价格。这正是"用当时能知道的信息决策、用实际价格结算"的具体落实。
912     """
913
914     def __init__(self, PRICE: np.ndarray, FIXED: np.ndarray,
915                  pf: PriceForecaster):
916         self.PRICE, self.FIXED, self.pf = PRICE, FIXED, pf
917
918     def decide(self, n: int, hi: int, mode: str) -> np.ndarray:
919         if mode == "oracle":
920             # 信息增强对照：假设当天价格已完全公布。仍不读取未来负载、光伏
921             # 或未发布的预报（框架 6.1 节末段）。
922             return self.PRICE[n].copy()
923         if mode == "fixed":
924             # 沿用附件 1 的固定分时电价——"把原有价格判断直接搬到新环境"
925             return self.FIXED.copy()
926         if mode == "forecast":
927             return self.pf.forecast_at(n, hi)
928         raise ValueError(f"未知价格模式 {mode!r}")
929
930
931 # ===== 模块 4: 4-2
932 @dataclass
933 class DayRecord42:
934     """4-2 的单日记录。price_hat 是决策用价，price_act 是结算用价。"""
935
936     n: int
937     date: str
938     g: np.ndarray
939     c: np.ndarray
940     d: np.ndarray
941     r: np.ndarray
942     w: np.ndarray
943     E: np.ndarray
944     E0: float
945     Ebar: np.ndarray
946     price_hat: np.ndarray
947     price_act: np.ndarray
948     l_hat: np.ndarray
949     v_hat: np.ndarray
950     n_hat: np.ndarray
951     margin: np.ndarray
952     n_risk: np.ndarray

```

```

953 l_act: np.ndarray
954 v_act: np.ndarray
955 mode: str
956
957 @property
958 def E_end(self) -> float:
959     return float(self.E[-1])
960
961 @property
962 def cost_plan(self) -> float:
963     return float(np.sum(self.price_act * self.g))
964
965 @property
966 def cost_emg(self) -> float:
967     return float(np.sum(COEF_EMG * self.price_act * self.r))
968
969 @property
970 def cost_total(self) -> float:
971     return self.cost_plan + self.cost_emg
972
973 @property
974 def cost_plan_hat(self) -> float:
975     """同一个计划量若按**当时预测的价格**计费会是多少（框架 8 节第 7 条）。
976
977     优化器的目标值与真实账单是两个不同的量：前者用预测价，后者用实际价。
978     两者之差就是"预计便宜"与"实际便宜"的差距，必须在账本里能直接看出来。
979     """
980     return float(np.sum(self.price_hat * self.g))
981
982 @property
983 def cost_total_hat(self) -> float:
984     return self.cost_plan_hat + float(
985         np.sum(COEF_EMG * self.price_hat * self.r))
986
987
988 def forecast_energy_slots_p2(fo: Forecaster, n: int) -> np.ndarray:
989     """4-2 的光伏预测：问题二口径的历史外推，不用附件 3。"""
990     if n == 0:
991         return np.zeros(N)
992     _, v_hat, _ = fo.energy(n)
993     return v_hat
994
995
996 def run_day42(n: int, e0: float, params: RiskParams, book: PriceBook,
997             mode: str, LOAD: np.ndarray, PV: np.ndarray, eps: np.ndarray,
998             fo: Forecaster, weighted: bool = True) -> DayRecord42:
999     """回放 4-2 的第 n 天：0:00 用预测价定计划，日内按实际价结算。
1000
1001     框架 4.1 节：负载与光伏都按历史外推预测（**不使用附件 3**）；把原固定
1002     价格系数替换为当时预测的价格，并使用价格加权的风险净负荷。计划一旦
1003     定下，日内不因价格变化而修改。
1004
1005     第 0 天一律冷启动：既无价格历史（日前 LP 拿不到决策价格）也无负载/光伏
1006     历史，故计划购电为零、储能不越储备线，缺口全部紧急补购。三种价格模式
1007     同此口径，ΔJ 才只反映评价区间内的价格信息之差。
1008     """
1009     l_act = LOAD[n, :] * TAU
1010     v_act = PV[n, :] * TAU
1011     date = _date_of(n)
1012     price_act = book.PRICE[n]
1013
1014     if n == 0:
1015         z = np.zeros(N)
1016         runner = DayRunner(l_act, v_act, e0, params.rho)

```

```

1017     runner.run(0, N, z, np.full(N, E_INIT))
1018     return DayRecord42(
1019         n=n, date=date, g=z.copy(), c=runner.c, d=runner.d, r=runner.r,
1020         w=runner.w, E=runner.E, E0=float(e0), Ebar=np.full(N, E_INIT),
1021         price_hat=z.copy(), price_act=price_act, l_hat=z.copy(),
1022         v_hat=z.copy(), n_hat=z.copy(), margin=z.copy(), n_risk=z.copy(),
1023         l_act=l_act, v_act=v_act, mode=mode)
1024
1025     l_hat_day, _, _ = fo.energy(n)
1026     l_hat = load_forecast_at(l_hat_day, l_act, 0)
1027     v_hat = forecast_energy_slots_p2(fo, n)
1028     n_hat = l_hat - v_hat
1029     margin = risk_margin4(eps, n, params.alpha, book.PRICE,
1030                          window=params.window, min_samples=params.min_samples,
1031                          weighted=weighted)
1032     n_risk = n_hat + margin
1033
1034     price_hat = book.decide(n, 0, mode)
1035     plan = solve_dayahead(n_risk, e0, price_hat, params)
1036     # 执行层不接受价格输入：储能按当前已观测的供需因果响应（问题二 5.2 节），
1037     # 价格只影响 0:00 的计划与随后的费用统计。
1038     ex = execute_day(plan.g, plan.Ebar, l_act, v_act, e0, price_act, params)
1039
1040     return DayRecord42(
1041         n=n, date=date, g=plan.g, c=ex.c, d=ex.d, r=ex.r, w=ex.w, E=ex.E,
1042         E0=float(e0), Ebar=plan.Ebar, price_hat=price_hat,
1043         price_act=price_act, l_hat=l_hat, v_hat=v_hat, n_hat=n_hat,
1044         margin=margin, n_risk=n_risk, l_act=l_act, v_act=v_act, mode=mode)
1045
1046
1047 def replay42(n0: int, n1: int, e0: float, params: RiskParams, book: PriceBook,
1048            mode: str, LOAD: np.ndarray, PV: np.ndarray, eps: np.ndarray,
1049            fo: Forecaster, weighted: bool = True
1050            ) -> tuple[list[DayRecord42], float]:
1051     """从 e0 出发回放 [n0, n1) 天，返回（逐日记录，末日储电量）。"""
1052     recs, e = [], float(e0)
1053     for n in range(n0, n1):
1054         rec = run_day42(n, e, params, book, mode, LOAD, PV, eps, fo, weighted)
1055         recs.append(rec)
1056         e = rec.E_end
1057     return recs, e
1058
1059
1060 def calibrate42(n0: int, e_at_window_start: float, cal: CalConfig,
1061               book: PriceBook, mode: str, LOAD: np.ndarray, PV: np.ndarray,
1062               eps: np.ndarray, fo: Forecaster, weighted: bool = True
1063               ) -> tuple[RiskParams, list[dict]]:
1064     """4-2 的滚动参数标定：在窗口内回放，取**实际结算总费用**最小者。
1065
1066     窗口初值取主运行在窗口起点日 0:00 的实际储电量（不是第 n0 天，第 n0 天
1067     在窗口末尾），避免把未来状态信息带入。标定目标是实际价格结算的费用，
1068     不是预测目标值——框架 8.7 节要求“预测费用、规划中的补购量和日末储备惩罚
1069     不能直接当作真实结果”。
1070     """
1071     lo = max(0, n0 - cal.window)
1072     rows, best, best_cost = [], None, np.inf
1073     for rp in cal.grid():
1074         recs, _ = replay42(lo, n0, e_at_window_start, rp, book, mode,
1075                          LOAD, PV, eps, fo, weighted)
1076         cost = sum(r.cost_total for r in recs)
1077         rows.append({"标定日": _date_of(n0), "窗口起": _date_of(lo),
1078                    "参数": rp.label(), "窗口实际总费用_元": cost,
1079                    "选中": False})
1080     if cost < best_cost - 1e-9:

```

```

1081         best, best_cost = rp, cost
1082     assert best is not None
1083     for row in rows:
1084         row["选中"] = row["参数"] == best.label()
1085     return best, rows
1086
1087
1088 def select_warmup42(core: CorePreset, book: PriceBook, mode: str,
1089                   LOAD: np.ndarray, PV: np.ndarray, eps: np.ndarray,
1090                   fo: Forecaster, weighted: bool = True
1091                   ) -> tuple[RiskParams, list[dict]]:
1092     """4-2 的 1 月联合标定: 只用 1 月数据在 ( $\alpha, \rho, \lambda$ ) 上选预热期参数。
1093
1094     准则与滚动标定一致——1 月 31 天上回放、取**实际结算总费用**最小者;
1095     第 0 天对所有候选相同, 故不影响比较。选出的  $\lambda$  随后整年冻结。
1096     """
1097     rows: list[dict] = []
1098     best: RiskParams | None = None
1099     best_cost = np.inf
1100     for rp in core.warmup_grid():
1101         cost = 0.0
1102         e = float(E_INIT)
1103         for n in range(core.warmup_n_days):
1104             rec = run_day42(n, e, rp, book, mode, LOAD, PV, eps, fo, weighted)
1105             cost += rec.cost_total
1106             e = rec.E_end
1107             rows.append({"参数": rp.label(), "1月总费用_元": cost,
1108                        "2月1日储电量_kWh": e, "选中": False})
1109             if cost < best_cost - 1e-9:
1110                 best, best_cost = rp, cost
1111     assert best is not None, "4-2 的 1 月联合标定: 没有任何可行组合"
1112     for row in rows:
1113         row["选中"] = row["参数"] == best.label()
1114     return best, rows
1115
1116
1117 def run_strategy42(cal: CalConfig, book: PriceBook, mode: str,
1118                  LOAD: np.ndarray, PV: np.ndarray, eps: np.ndarray,
1119                  fo: Forecaster, e_init: float = E_INIT,
1120                  weighted: bool = True, verbose: bool = False,
1121                  warmup: RiskParams | None = None
1122                  ) -> tuple[list[DayRecord42], list[dict]]:
1123     """4-2 全年滚动: 预热 1 月, 正式区间内每隔 cal.every 天重新标定一次。
1124
1125     `warmup` 给出 1 月预热期参数, 由 `select_warmup42` 在 1 月数据上离线选出;
1126     传 None 时退回旧的硬编码值 ( $\alpha=0.80/\rho=1.0/\lambda=0.60$ ), 仅供复现历史数字。
1127     """
1128     recs, cal_rows = [], []
1129     e = float(e_init)
1130     e_at = np.full(N_DAY + 1, np.nan)
1131     e_at[0] = e
1132     params = (warmup if warmup is not None
1133              else RiskParams(alpha=0.80, rho=1.0, lam=LAMBDA_DEFAULT))
1134     for n in range(N_DAY):
1135         if n >= cal.start and (n - cal.start) % cal.every == 0:
1136             params, rows = calibrate42(n, e_at[max(0, n - cal.window)], cal,
1137                                       book, mode, LOAD, PV, eps, fo, weighted)
1138             cal_rows.extend(rows)
1139             if verbose:
1140                 print(f"    [4-2 {cal.date_of(n)}] 选中 {params.label()}",
1141                       flush=True)
1142             rec = run_day42(n, e, params, book, mode, LOAD, PV, eps, fo, weighted)
1143             recs.append(rec)
1144             e = rec.E_end

```

```

1145     e_at[n + 1] = e
1146     return recs, cal_rows
1147
1148
1149 # ===== 模块 5: 4-3
1150 @dataclass
1151 class DayRecord43:
1152     n: int
1153     date: str
1154     g0: np.ndarray          # 0:00 冻结的初始计划
1155     x: np.ndarray          # 最终生效的完整购电量
1156     c: np.ndarray
1157     d: np.ndarray
1158     r: np.ndarray
1159     w: np.ndarray
1160     E: np.ndarray
1161     E0: float
1162     Ebar: np.ndarray
1163     price_hat0: np.ndarray  # 0:00 决策价
1164     price_act: np.ndarray   # 结算价
1165     l_hat0: np.ndarray
1166     v_hat0: np.ndarray
1167     n_hat0: np.ndarray
1168     margin0: np.ndarray
1169     n_risk0: np.ndarray
1170     l_act: np.ndarray
1171     v_act: np.ndarray
1172     n_solve: int
1173     n_submit: int
1174     S: tuple
1175     mode: str
1176     convention: str
1177
1178     @property
1179     def E_end(self) -> float:
1180         return float(self.E[-1])
1181
1182     @property
1183     def cost_plan(self) -> float:
1184         return float(np.sum(self.price_act * self.g0))
1185
1186     @property
1187     def cost_adj(self) -> float:
1188         up = np.maximum(self.x - self.g0, 0.0)
1189         if self.convention == "main":
1190             return float(np.sum(COEF_UP * self.price_act * up))
1191         dn = np.maximum(self.g0 - self.x, 0.0)
1192         return float(np.sum(COEF_UP * self.price_act * up
1193                             - COEF_DOWN * self.price_act * dn))
1194
1195     @property
1196     def cost_emg(self) -> float:
1197         return float(np.sum(COEF_EMG * self.price_act * self.r))
1198
1199     @property
1200     def cost_total(self) -> float:
1201         return self.cost_plan + self.cost_adj + self.cost_emg
1202
1203     @property
1204     def cost_plan_hat(self) -> float:
1205         """初始计划与最终量若都按 **0:00 的预测价**计费（框架 8 节第 7 条）。
1206
1207         与 cost_plan + cost_adj 的差别：那里用实际价结算，这里用当时预测价，
1208         故差额 = ``预测觉得自己省了多少``与``账本上真的省了多少``的差。

```

```

1209         """
1210         up = np.maximum(self.x - self.g0, 0.0)
1211         dn = np.maximum(self.g0 - self.x, 0.0)
1212         base = float(np.sum(self.price_hat0 * self.g0))
1213         if self.convention == "main":
1214             return base + float(np.sum(COEF_UP * self.price_hat0 * up))
1215         return base + float(np.sum(COEF_UP * self.price_hat0 * up
1216                                   - COEF_DOWN * self.price_hat0 * dn))
1217
1218     @property
1219     def cost_total_hat(self) -> float:
1220         return self.cost_plan_hat + float(
1221             np.sum(COEF_EMG * self.price_hat0 * self.r))
1222
1223     @property
1224     def g_initial(self) -> float:
1225         return float(self.g0.sum())
1226
1227     @property
1228     def g_effective(self) -> float:
1229         return float(self.x.sum())
1230
1231     @property
1232     def g_emg(self) -> float:
1233         return float(self.r.sum())
1234
1235     @property
1236     def up(self) -> float:
1237         return float(np.maximum(self.x - self.g0, 0.0).sum())
1238
1239     @property
1240     def down(self) -> float:
1241         return float(np.maximum(self.g0 - self.x, 0.0).sum())
1242
1243
1244 _PLAN0_CACHE4: dict[tuple, tuple[np.ndarray, np.ndarray]] = {}
1245
1246
1247 def initial_plan4(n: int, ntilde0: np.ndarray, e0: float,
1248                  price: np.ndarray, rp: RiskParams, tag: str
1249                  ) -> tuple[np.ndarray, np.ndarray]:
1250     """0:00 初始计划的带缓存求解。
1251
1252     缓存键比 p3 多一个 tag: 问题四里**同一天、同一个  $(\alpha, \lambda, e0)$  在不同价格模式
1253     下会得到不同的计划** (预测价 vs 固定价 vs 已知未来价), 若沿用 p3 的
1254      $(n, \alpha, \lambda, e0)$  键会把三种策略的计划互相污染。
1255     """
1256     key = (tag, n, rp.alpha, rp.lam, round(float(e0), 6))
1257     hit = _PLAN0_CACHE4.get(key)
1258     if hit is not None:
1259         return hit[0].copy(), hit[1].copy()
1260     if len(_PLAN0_CACHE4) > 60000:
1261         _PLAN0_CACHE4.clear()
1262     plan = solve_dayahead(ntilde0, e0, price, rp)
1263     _PLAN0_CACHE4[key] = (plan.g.copy(), plan.Ebar.copy())
1264     return plan.g, plan.Ebar
1265
1266
1267 def run_day43(n: int, e0: float, S: tuple, params: RiskParams,
1268              book: PriceBook, mode: str, LOAD: np.ndarray, PV: np.ndarray,
1269              fp: ForecastPanel, eps3: np.ndarray, fo: Forecaster,
1270              convention: str = "main", weighted: bool = True) -> DayRecord43:
1271     """回放 4-3 的第 n 天: 0:00 定初始计划, 6/12/18 时按新预报调整。
1272

```

与问题三的差别集中在"价格"二字上,规则本身不变(框架 5.1 节:"实时电价每十分钟波动,并不自动增加普通购电调整时点"):

- * 0:00 用当时的预测价定 g^0 ;
- * 第 hi 个交付块若在使用集合 S 内,用**该时刻更新的价格预测**重解剩余时段的调整问题,只正式提交当前交付块,更远时段只是内部展望;
- * 全部费用按实际价格结算,包括初始计划费(框架 5.3 节)。

第 0 天一律冷启动,理由同 4-2 (见 `run\_day42`): 无价格历史即无决策价格。

```
l_act = LOAD[n, :] * TAU
v_act = PV[n, :] * TAU
date = _date_of(n)
price_act = book.PRICE[n]

if n == 0:
    z = np.zeros(N)
    runner = DayRunner(l_act, v_act, e0, params.rho)
    runner.run(0, N, z, np.full(N, E_INIT))
    return DayRecord43(
        n=n, date=date, g0=z.copy(), x=z.copy(), c=runner.c, d=runner.d,
        r=runner.r, w=runner.w, E=runner.E, E0=float(e0),
        Ebar=np.full(N, E_INIT), price_hat0=z.copy(), price_act=price_act,
        l_hat0=z.copy(), v_hat0=z.copy(), n_hat0=z.copy(),
        margin0=z.copy(), n_risk0=z.copy(), l_act=l_act, v_act=v_act,
        n_solve=0, n_submit=0, S=(), mode=mode, convention=convention)

l_hat_day, _, _ = fo.energy(n)
l_hat0 = load_forecast_at(l_hat_day, l_act, 0)
v_hat0 = forecast_energy_slots(fp, 0, n)
margin0 = risk_margin4_ver(eps3, 0, n, params.alpha, book.PRICE,
                           window=params.window,
                           min_samples=params.min_samples,
                           weighted=weighted)

n_hat0 = l_hat0 - v_hat0
n_risk0 = n_hat0 + margin0
price_hat0 = book.decide(n, 0, mode)
tag = f"{mode}|{convention}|{S}" # 计划缓存必须区分策略
g0, Ebar = initial_plan4(n, n_risk0, e0, price_hat0, params, tag)
x = g0.copy()

runner = DayRunner(l_act, v_act, e0, params.rho)
n_solve, n_submit = 1, 0

for hi, (a, b) in enumerate(BLOCK_BOUNDS):
    if hi > 0 and RELEASE_HOURS[hi] in S:
        l_hat_h = load_forecast_at(l_hat_day, l_act, hi)
        v_hat_h = forecast_energy_slots(fp, hi, n)
        margin_h = risk_margin4_ver(eps3, hi, n, params.alpha,
                                    book.PRICE, window=params.window,
                                    min_samples=params.min_samples,
                                    weighted=weighted)
        n_risk_h = (l_hat_h - v_hat_h) + margin_h
        # 关键: 调整目标用**该发布时刻更新的价格预测**, 而不是 0:00 的
        # 预测价; 结算仍走 price_act。
        price_h = book.decide(n, hi, mode)
        res = solve_adjust(n_risk_h[a:], g0[a:], runner.Ep, price_h[a:],
                           params, convention)
        x[a:b] = res.y[b - a:] # 只正式提交当前交付块
        Ebar[a:] = res.Ebar # 参考轨迹整体刷新 (含展望部分)
        n_solve += 1
        n_submit += 1
    runner.run(a, b, x, Ebar)

return DayRecord43(
```

```

1337     n=n, date=date, g0=g0, x=x, c=runner.c, d=runner.d, r=runner.r,
1338     w=runner.w, E=runner.E, E0=float(e0), Ebar=Ebar,
1339     price_hat0=price_hat0, price_act=price_act, l_hat0=l_hat0,
1340     v_hat0=v_hat0, n_hat0=n_hat0, margin0=margin0, n_risk0=n_risk0,
1341     l_act=l_act, v_act=v_act, n_solve=n_solve, n_submit=n_submit,
1342     S=tuple(S), mode=mode, convention=convention)
1343
1344
1345 def replay43(n0: int, n1: int, e0: float, S: tuple, params: RiskParams,
1346             book: PriceBook, mode: str, LOAD: np.ndarray, PV: np.ndarray,
1347             fp: ForecastPanel, eps3: np.ndarray, fo: Forecaster,
1348             convention: str = "main", weighted: bool = True
1349             ) -> tuple[list[DayRecord43], float]:
1350     recs, e = [], float(e0)
1351     for n in range(n0, n1):
1352         rec = run_day43(n, e, S, params, book, mode, LOAD, PV, fp, eps3, fo,
1353                        convention, weighted)
1354         recs.append(rec)
1355         e = rec.E_end
1356     return recs, e
1357
1358
1359 def calibrate43(n0: int, e_at_window_start: float, cal: CalConfig3,
1360               book: PriceBook, mode: str, LOAD: np.ndarray, PV: np.ndarray,
1361               fp: ForecastPanel, eps3: np.ndarray, fo: Forecaster,
1362               convention: str = "main", weighted: bool = True
1363               ) -> tuple[RiskParams, list[dict]]:
1364     """4-3 的滚动标定，目标同样是窗口内的**实际结算总费用**。
1365
1366     mode / convention 必须原样转交 replay43：标定是在**该策略自己的信息集与
1367     计费口径**下进行的，否则固定电价对照与不退款口径敏感性会用另一套策略的
1368     参数，对照就失去意义（沿用 p3 的同一原则）。
1369     """
1370     lo = max(0, n0 - cal.window)
1371     rows, best, best_cost = [], None, np.inf
1372     for rp in cal.grid():
1373         recs, _ = replay43(lo, n0, e_at_window_start, cal.S, rp, book, mode,
1374                           LOAD, PV, fp, eps3, fo, convention, weighted)
1375         cost = sum(r.cost_total for r in recs)
1376         rows.append({"标定日": _date_of(n0), "窗口起": _date_of(lo),
1377                     "参数": rp.label(), "窗口实际总费用_元": cost,
1378                     "选中": False})
1379         if cost < best_cost - 1e-9:
1380             best, best_cost = rp, cost
1381     assert best is not None
1382     for row in rows:
1383         row["选中"] = row["参数"] == best.label()
1384     return best, rows
1385
1386
1387 def select_warmup43(core: CorePreset, book: PriceBook, mode: str,
1388                   LOAD: np.ndarray, PV: np.ndarray, fp: ForecastPanel,
1389                   eps3: np.ndarray, fo: Forecaster, convention: str = "main",
1390                   weighted: bool = True
1391                   ) -> tuple[RiskParams, list[dict]]:
1392     """4-3 的 1 月联合标定：只用 1 月数据在  $(\alpha, \rho, \lambda)$  上选预热期参数。
1393
1394     S 固定取空集：1 月预热期不触发调整（REPORT_START 之前没有标定，也没有
1395     调整时点的信息可用），故预热期参数与后续使用集合无关，可以离线一次选定。
1396     """
1397     rows: list[dict] = []
1398     best: RiskParams | None = None
1399     best_cost = np.inf
1400     for rp in core.warmup_grid():

```

```

1401     cost = 0.0
1402     e = float(E_INIT)
1403     for n in range(core.warmup_n_days):
1404         rec = run_day43(n, e, (), rp, book, mode, LOAD, PV, fp, eps3, fo,
1405                        convention, weighted)
1406         cost += rec.cost_total
1407         e = rec.E_end
1408         rows.append({"参数": rp.label(), "1月总费用_元": cost,
1409                    "2月1日 储电量_kWh": e, "选中": False})
1410         if cost < best_cost - 1e-9:
1411             best, best_cost = rp, cost
1412     assert best is not None, "4-3 的 1 月联合标定: 没有任何可行组合"
1413     for row in rows:
1414         row["选中"] = row["参数"] == best.label()
1415     return best, rows
1416
1417
1418 def run_strategy43(cal: CalConfig3, book: PriceBook, mode: str,
1419                  LOAD: np.ndarray, PV: np.ndarray, fp: ForecastPanel,
1420                  eps3: np.ndarray, fo: Forecaster, S: tuple | None = None,
1421                  e_init: float = E_INIT, convention: str = "main",
1422                  weighted: bool = True, verbose: bool = False,
1423                  warmup: RiskParams | None = None
1424                  ) -> tuple[list[DayRecord43], list[dict]]:
1425     """整年滚动: 预热 1 月, 正式区间内每隔 cal.every 天重新标定一次参数。
1426
1427     `warmup` 给出 1 月预热期参数, 由 `select_warmup43` 在 1 月数据上离线选出;
1428     传 None 时退回旧的硬编码值 ( $\alpha=0.60/\rho=0.5/\lambda=0.60$ ), 仅供复现历史数字。
1429     """
1430     S = cal.S if S is None else tuple(S)
1431     recs, cal_rows = [], []
1432     e = float(e_init)
1433     e_at = np.full(N_DAY + 1, np.nan)
1434     e_at[0] = e
1435     params = (warmup if warmup is not None
1436              else RiskParams(alpha=0.60, rho=0.5, lam=LAMBDA_DEFAULT))
1437     for n in range(N_DAY):
1438         if n >= cal.start and (n - cal.start) % cal.every == 0:
1439             params, rows = calibrate43(n, e_at[max(0, n - cal.window)], cal,
1440                                       book, mode, LOAD, PV, fp, eps3, fo,
1441                                       convention, weighted)
1442             cal_rows.extend(rows)
1443             if verbose:
1444                 print(f"    [4-3 {cal.date_of(n)}] 选中 {params.label()}",
1445                       flush=True)
1446             rec = run_day43(n, e, S, params, book, mode, LOAD, PV, fp, eps3, fo,
1447                            convention, weighted)
1448             recs.append(rec)
1449             e = rec.E_end
1450             e_at[n + 1] = e
1451     return recs, cal_rows
1452
1453
1454 # ===== 模块 6: 汇总与校验
1455 def _sum(recs, attr: str) -> float:
1456     return float(sum(getattr(r, attr) for r in recs))
1457
1458
1459 def _wavg(recs, qty: str, price: str) -> float:
1460     """按电量加权的均价 (框架 6.3 节: 要能看出充放电发生在什么价位)。
1461
1462     不能用简单平均: 均价必须按各时段的电量加权, 否则低压低谷时段的零星
1463     充放电会与高峰时段的大量充放电等权, 掩盖"低价充电、高价放电"的实际情况。
1464     """

```

```

1465 q = sum(float(getattr(r, qty).sum()) for r in recs)
1466 if q <= 1e-9:
1467     return 0.0
1468 num = sum(float((getattr(r, price) * getattr(r, qty)).sum()) for r in recs)
1469 return float(num / q)
1470
1471
1472 def totals42(recs: list[DayRecord42]) -> dict:
1473     cp, ce = _sum(recs, "cost_plan"), _sum(recs, "cost_emg")
1474     return {
1475         "天数": len(recs),
1476         "计划量_kWh": float(sum(r.g.sum() for r in recs)),
1477         "紧急购电量_kWh": float(sum(r.r.sum() for r in recs)),
1478         "收到总电量_kWh": float(sum((r.g + r.r).sum() for r in recs)),
1479         "计划费_元": cp,
1480         "调整费_元": 0.0,
1481         "紧急费_元": ce,
1482         "合计费用_元": cp + ce,
1483         "弃电量_kWh": float(sum(r.w.sum() for r in recs)),
1484         "充电量_kWh": float(sum(r.c.sum() for r in recs)),
1485         "放电量_kWh": float(sum(r.d.sum() for r in recs)),
1486         "期初储电量_kWh": float(recs[0].E0),
1487         "期末储电量_kWh": float(recs[-1].E[-1]),
1488         "调整提交次数": 0,
1489         "MILP求解次数": len(recs),
1490         # 框架第八节第 7 条: 优化器的目标值 (按预测价) 与真实账单 (按实际价)
1491         "预估费用_元": _sum(recs, "cost_total_hat"),
1492         # 框架 6.3 节: 充放电与紧急补购各自发生在什么价位
1493         "充电加权均价_元每kWh": _wavg(recs, "c", "price_act"),
1494         "放电加权均价_元每kWh": _wavg(recs, "d", "price_act"),
1495         "紧急购电加权均价_元每kWh": _wavg(recs, "r", "price_act"),
1496     }
1497
1498
1499 def totals43(recs: list[DayRecord43]) -> dict:
1500     cp, ca, ce = (_sum(recs, "cost_plan"), _sum(recs, "cost_adj"),
1501                  _sum(recs, "cost_emg"))
1502     return {
1503         "天数": len(recs),
1504         "计划量_kWh": float(sum(r.g0.sum() for r in recs)),
1505         "初始计划量_kWh": float(sum(r.g0.sum() for r in recs)),
1506         "最终常规量_kWh": float(sum(r.x.sum() for r in recs)),
1507         "上调量_kWh": _sum(recs, "up"),
1508         "下调量_kWh": _sum(recs, "down"),
1509         "紧急购电量_kWh": float(sum(r.r.sum() for r in recs)),
1510         "收到总电量_kWh": float(sum((r.x + r.r).sum() for r in recs)),
1511         "计划费_元": cp,
1512         "调整费_元": ca,
1513         "紧急费_元": ce,
1514         "合计费用_元": cp + ca + ce,
1515         "弃电量_kWh": float(sum(r.w.sum() for r in recs)),
1516         "充电量_kWh": float(sum(r.c.sum() for r in recs)),
1517         "放电量_kWh": float(sum(r.d.sum() for r in recs)),
1518         "期初储电量_kWh": float(recs[0].E0),
1519         "期末储电量_kWh": float(recs[-1].E[-1]),
1520         "调整提交次数": sum(r.n_submit for r in recs),
1521         "MILP求解次数": sum(r.n_solve for r in recs),
1522         # 框架第八节第 7 条: 优化器的目标值 (按预测价) 与真实账单 (按实际价)
1523         "预估费用_元": _sum(recs, "cost_total_hat"),
1524         # 框架 6.3 节: 充放电与紧急补购各自发生在什么价位
1525         "充电加权均价_元每kWh": _wavg(recs, "c", "price_act"),
1526         "放电加权均价_元每kWh": _wavg(recs, "d", "price_act"),
1527         "紧急购电加权均价_元每kWh": _wavg(recs, "r", "price_act"),
1528     }

```

```

1529
1530
1531 CHECK_KEYS_42 = ("信息边界", "计划冻结", "物理可行性", "状态累计", "费用一致性")
1532 CHECK_KEYS_43 = ("信息边界", "调整权限", "物理可行性", "状态累计", "费用一致性")
1533
1534
1535 def check_day42(rec: DayRecord42, tol: float = 1e-6) -> dict[str, list[str]]:
1536     """4-2 单日校验：框架第八节八项要求中可逐日核验的部分。
1537
1538     返回的是**具体问题清单**而非布尔值，便于论文如实报告失败天数与原因。
1539     """
1540     p: dict[str, list[str]] = {k: [] for k in CHECK_KEYS_42}
1541     if rec.n == 0:
1542         return p
1543     if not np.isfinite(rec.price_hat).all() or (rec.price_hat <= 0).any():
1544         p["信息边界"].append("决策价格非正或非有限")
1545     if not np.isfinite(rec.n_risk).all():
1546         p["信息边界"].append("风险净负荷出现非有限值")
1547     # 计划冻结的可核验内容就是逐段电量平衡：执行层唯一的外部购电来源是 0:00
1548     # 定下的 g (加上日内紧急补购 r)，日内不存在别的常规购电。平衡式为
1549     #  $g + v + d + r = l + c + w$ 
1550     # 即"来源 = 用途"：购电、光伏、放电、紧急购电 供给 负载、充电、弃电。
1551     # 注意 r 与 d 同为供给侧，符号为正——写成减号会把"缺口用储能与紧急购电
1552     # 共同补足"误判为不平衡。
1553     resid = (rec.g + rec.v_act + rec.d + rec.r
1554             - rec.c - rec.w - rec.l_act)
1555     if np.abs(resid).max() > 1e-5:
1556         p["计划冻结"].append(f"逐段电量平衡残差 {np.abs(resid).max():.3e} kWh")
1557     E = rec.E
1558     if E.min() < E_MIN - 1e-4 or E.max() > E_MAX + 1e-4:
1559         p["物理可行性"].append(f"储电量越界 [{E.min():.4f},{E.max():.4f}]")
1560     if (rec.r < -tol).any() or (rec.w < -tol).any():
1561         p["物理可行性"].append("紧急量或弃电量出现负值")
1562     if (rec.c > M_ENERGY + 1e-4).any() or (rec.d > M_ENERGY + 1e-4).any():
1563         p["物理可行性"].append("充放电功率超限")
1564     Ep = rec.E0
1565     for t in range(N):
1566         Ep = Ep + ETA * rec.c[t] - rec.d[t] / ETA
1567         if abs(Ep - rec.E[t]) > 1e-4:
1568             p["状态累计"].append(f"第 {t} 段储电量递推不一致")
1569             break
1570     cp = float(np.sum(rec.price_act * rec.g))
1571     ce = float(np.sum(COEF_EMG * rec.price_act * rec.r))
1572     if abs(cp - rec.cost_plan) > 1e-4 or abs(ce - rec.cost_emg) > 1e-4:
1573         p["费用一致性"].append("费用重算不符")
1574     return p
1575
1576
1577 def check_day43(rec: DayRecord43, tol: float = 1e-6) -> dict[str, list[str]]:
1578     p: dict[str, list[str]] = {k: [] for k in CHECK_KEYS_43}
1579     if rec.n == 0:
1580         return p
1581     if not np.isfinite(rec.price_hat0).all() or (rec.price_hat0 <= 0).any():
1582         p["信息边界"].append("0:00 决策价格非正或非有限")
1583     # 调整权限：不退款口径（敏感性）下恒有  $x \geq g^0$ ，因为调减每度净付 1.5p；且只有
1584     # S 内的交付块允许  $x \neq g^0$ 。退款主口径两侧都放开，故不设 x 与  $g^0$  的大小约束。
1585     if rec.convention == "main" and (rec.x < rec.g0 - 1e-6).any():
1586         p["调整权限"].append("不退款口径出现调减")
1587     for hi, (a, b) in enumerate(BLOCK_BOUNDS):
1588         if np.abs(rec.x[a:b] - rec.g0[a:b]).max() <= 1e-6:
1589             continue
1590         if not (hi > 0 and RELEASE_HOURS[hi] in rec.S):
1591             p["调整权限"].append(f"第 {hi} 块发生未授权的调整")
1592     if float(np.max(np.abs(rec.x - rec.g0))) > tol and rec.S == ():

```

```

1593     p["调整权限"].append("未使用任何更新时刻却调整了购电量")
1594     # 平衡式同 check_day42, 只是常规购电改用最终生效的 x:
1595     #     x + v + d + r = l + c + w
1596     resid = (rec.x + rec.v_act + rec.d + rec.r
1597              - rec.c - rec.w - rec.l_act)
1598     if np.abs(resid).max() > 1e-5:
1599         p["物理可行性"].append(f"逐段电量平衡残差 {np.abs(resid).max():.3e} kWh")
1600     E = rec.E
1601     if E.min() < E_MIN - 1e-4 or E.max() > E_MAX + 1e-4:
1602         p["物理可行性"].append(f"储电量越界 [{E.min():.4f},{E.max():.4f}]")
1603     if (rec.r < -tol).any() or (rec.w < -tol).any():
1604         p["物理可行性"].append("紧急量或弃电量出现负值")
1605     Ep = rec.E0
1606     for t in range(N):
1607         Ep = Ep + ETA * rec.c[t] - rec.d[t] / ETA
1608         if abs(Ep - rec.E[t]) > 1e-4:
1609             p["状态累计"].append(f"第 {t} 段储电量递推不一致")
1610             break
1611     # 费用一致性: 全部系数乘**对应交付时段的实际电价**
1612     cp = float(np.sum(rec.price_act * rec.g0))
1613     up = np.maximum(rec.x - rec.g0, 0.0)
1614     ca = (float(np.sum(COEF_UP * rec.price_act * up))
1615          if rec.convention == "main" else
1616          float(np.sum(COEF_UP * rec.price_act * up
1617                      - COEF_DOWN * rec.price_act
1618                      * np.maximum(rec.g0 - rec.x, 0.0))))
1619     ce = float(np.sum(COEF_EMG * rec.price_act * rec.r))
1620     if (abs(cp - rec.cost_plan) > 1e-4 or abs(ca - rec.cost_adj) > 1e-4
1621         or abs(ce - rec.cost_emg) > 1e-4):
1622         p["费用一致性"].append("费用重算不符")
1623     return p
1624
1625
1626 def check_all4(recs, kind: str) -> dict[str, int]:
1627     """返回每一类检查的**失败天数**; 全为零才说明通过。"""
1628     fn = check_day42 if kind == "42" else check_day43
1629     keys = CHECK_KEYS_42 if kind == "42" else CHECK_KEYS_43
1630     out = {k: 0 for k in keys}
1631     for r in recs:
1632         for k, v in fn(r).items():
1633             if v:
1634                 out[k] += 1
1635     return out
1636
1637
1638 def validation_metrics4(recs, kind: str) -> dict:
1639     """数值口径的最坏情形指标, 供论文以数字而非布尔值报告校验结果。
1640
1641     布尔"通过"掩盖差异的量级; 这里给出最大残差与储电量极值, 读者可以据此
1642     判断约束是被满足到浮点精度, 还是被近似满足。
1643     """
1644     e_lo = min(float(r.E.min()) for r in recs)
1645     e_hi = max(float(r.E.max()) for r in recs)
1646     out = {
1647         "全局最小储电量_kWh": e_lo,
1648         "全局最大储电量_kWh": e_hi,
1649         "储电下界余量_kWh": e_lo - E_MIN,
1650         "储电上界余量_kWh": E_MAX - e_hi,
1651         "跨日衔接最大偏差_kWh": _cross_day_gap(recs),
1652     }
1653     if kind == "42":
1654         out["逐段平衡最大残差_kWh"] = _max_abs(
1655             np.abs(r.g + r.v_act + r.d + r.r - r.c - r.w - r.l_act).max()
1656             for r in recs)

```

```

1657     out["计划费重算最大偏差_元"] = _max_abs(
1658         np.sum(r.price_act * r.g) - r.cost_plan for r in recs)
1659     out["调整费重算最大偏差_元"] = 0.0
1660 else:
1661     out["逐段平衡最大残差_kWh"] = _max_abs(
1662         np.abs(r.x + r.v_act + r.d + r.r - r.c - r.w - r.l_act).max()
1663         for r in recs)
1664     out["计划费重算最大偏差_元"] = _max_abs(
1665         np.sum(r.price_act * r.g0) - r.cost_plan for r in recs)
1666     out["调整费重算最大偏差_元"] = _max_abs(
1667         (np.sum(COEF_UP * r.price_act * np.maximum(r.x - r.g0, 0.0))
1668          - (0.0 if r.convention == "main" else
1669             np.sum(COEF_DOWN * r.price_act
1670                   * np.maximum(r.g0 - r.x, 0.0)))) - r.cost_adj
1671         for r in recs)
1672     out["紧急费重算最大偏差_元"] = _max_abs(
1673         np.sum(COEF_EMG * r.price_act * r.r) - r.cost_emg for r in recs)
1674     return out
1675
1676 def _max_abs(it) -> float:
1677     vals = [abs(float(v)) for v in it]
1678     return max(vals) if vals else 0.0
1679
1680
1681 def _cross_day_gap(recs) -> float:
1682     """框架 8.5 节：每个分支独立保存实际状态，次日日初 = 前日日末。"""
1683     if len(recs) < 2:
1684         return 0.0
1685     return float(max(abs(recs[i + 1].E0 - recs[i].E[-1])
1686                     for i in range(len(recs) - 1)))
1687
1688
1689 def emergency_events4(rec, eps: float = EMG_EPS) -> list[dict]:
1690     """合并相邻的紧急购电时段为一个事件，事件费用逐十分钟求和。
1691
1692     框架 7.3 节明令：事件费用必须  $\sum 5 \cdot p_{\text{actual}} \cdot r$  逐段累加，**不能**用事件
1693     起点价格、全天平均价格或预测价格乘整个事件电量。
1694     """
1695     r, price, out, i = rec.r, rec.price_act, [], 0
1696     while i < N:
1697         if r[i] > eps:
1698             j = i
1699             while j + 1 < N and r[j + 1] > eps:
1700                 j += 1
1701             out.append({
1702                 "时段": f"{interval_label(i).split('-')[0]}-"
1703                        f"{interval_label(j).split('-')[1]}",
1704                 "电量_kWh": float(r[i:j + 1].sum()),
1705                 "费用_元": float(np.sum(COEF_EMG * price[i:j + 1]
1706                                         * r[i:j + 1])),
1707             })
1708             i = j + 1
1709         else:
1710             i += 1
1711     return out
1712
1713
1714 # ===== 模块 7：导出
1715 def summaries42(recs: list[DayRecord42]) -> list[dict]:
1716     out = []
1717     for r in recs:
1718         out.append({
1719             "日期": r.date, "计划量_kWh": float(r.g.sum()),
1720

```

```

1721         "紧急电量_kWh": float(r.r.sum()), "计划费_元": r.cost_plan,
1722         "调整费_元": 0.0, "紧急费_元": r.cost_emg,
1723         "合计费用_元": r.cost_total, "弃电量_kWh": float(r.w.sum()),
1724         "期初储电量_kWh": float(r.E0), "期末储电量_kWh": float(r.E[-1]),
1725         "block_charge": [float(r.c[a:b].sum()) for a, b in BLOCKS],
1726         "block_discharge": [float(r.d[a:b].sum()) for a, b in BLOCKS],
1727         "events": emergency_events4(r),
1728         "plan_arr": r.g, "g0_arr": r.g, "x_arr": r.g,
1729     })
1730     return out
1731
1732
1733 def summaries43(recs: list[DayRecord43]) -> list[dict]:
1734     out = []
1735     for r in recs:
1736         out.append({
1737             "日期": r.date, "计划量_kWh": r.g_initial,
1738             "初始计划量_kWh": r.g_initial, "最终常规量_kWh": r.g_effective,
1739             "上调量_kWh": r.up, "下调量_kWh": r.down,
1740             "紧急电量_kWh": r.g_emg, "计划费_元": r.cost_plan,
1741             "调整费_元": r.cost_adj, "紧急费_元": r.cost_emg,
1742             "合计费用_元": r.cost_total, "弃电量_kWh": float(r.w.sum()),
1743             "期初储电量_kWh": float(r.E0), "期末储电量_kWh": float(r.E[-1]),
1744             "调整提交次数": r.n_submit,
1745             "block_charge": [float(r.c[a:b].sum()) for a, b in BLOCKS],
1746             "block_discharge": [float(r.d[a:b].sum()) for a, b in BLOCKS],
1747             "events": emergency_events4(r),
1748             "plan_arr": r.g0, "g0_arr": r.g0, "x_arr": r.x,
1749         })
1750     return out
1751
1752
1753 def _slot_labels() -> tuple[list[str], list[str]]:
1754     """时段起止标签，与问题二、三的明细表列名保持一致（便于交叉核对）。"""
1755     lab = [interval_label(t) for t in range(N)]
1756     return ([s.split("-")[0] for s in lab], [s.split("-")[1] for s in lab])
1757
1758
1759 def detail_frame42(rec: DayRecord42) -> pd.DataFrame:
1760     a, b = _slot_labels()
1761     return pd.DataFrame({
1762         "日期": rec.date,
1763         "时段起": a,
1764         "时段止": b,
1765         "预测电价_元每kWh": rec.price_hat,
1766         "实际电价_元每kWh": rec.price_act,
1767         "实际负载_kWh": rec.l_act,
1768         "实际光伏_kWh": rec.v_act,
1769         "预测负载_kWh": rec.l_hat,
1770         "预测光伏_kWh": rec.v_hat,
1771         "净负荷预测_kWh": rec.n_hat,
1772         "价格加权风险余量_kWh": rec.margin,
1773         "风险净负荷_kWh": rec.n_risk,
1774         "参考储电量_kWh": rec.Ebar,
1775         "计划购电_kWh": rec.g,
1776         "实际充电_kWh": rec.c,
1777         "实际放电_kWh": rec.d,
1778         "紧急购电_kWh": rec.r,
1779         "弃电_kWh": rec.w,
1780         "储电量_kWh": rec.E,
1781         "计划费_元": rec.price_act * rec.g,
1782         "紧急费_元": COEF_EMG * rec.price_act * rec.r,
1783         # 同一计划量若按当时预测价计费（框架第八节第 7 条：目标与账单分开）
1784         "预估计划费_元": rec.price_hat * rec.g,

```

```

1785     })
1786
1787
1788 def detail_frame43(rec: DayRecord43) -> pd.DataFrame:
1789     a, b = _slot_labels()
1790     return pd.DataFrame({
1791         "日期": rec.date,
1792         "时段起": a,
1793         "时段止": b,
1794         "0点预测电价_元每kWh": rec.price_hat0,
1795         "实际电价_元每kWh": rec.price_act,
1796         "实际负载_kWh": rec.l_act,
1797         "实际光伏_kWh": rec.v_act,
1798         "预测负载_kWh": rec.l_hat0,
1799         "预测光伏_kWh": rec.v_hat0,
1800         "净负荷预测_kWh": rec.n_hat0,
1801         "价格加权风险余量_kWh": rec.margin0,
1802         "风险净负荷_kWh": rec.n_risk0,
1803         "参考储电量_kWh": rec.Ebar,
1804         "初始计划购电_kWh": rec.g0,
1805         "最终购电_kWh": rec.x,
1806         "上调量_kWh": np.maximum(rec.x - rec.g0, 0.0),
1807         "下调量_kWh": np.maximum(rec.g0 - rec.x, 0.0),
1808         "实际充电_kWh": rec.c,
1809         "实际放电_kWh": rec.d,
1810         "紧急购电_kWh": rec.r,
1811         "弃电_kWh": rec.w,
1812         "储电量_kWh": rec.E,
1813         "计划费_元": rec.price_act * rec.g0,
1814         "紧急费_元": COEF_EMG * rec.price_act * rec.r,
1815         # 同一初始计划与最终量若都按 0:00 预测价计费 (框架第八节第 7 条)
1816         "预估计划费_元": rec.price_hat0 * rec.g0
1817         + COEF_UP * rec.price_hat0 * np.maximum(rec.x - rec.g0, 0.0),
1818     })
1819
1820
1821 # ----- Excel 写出
1822 def _clear(ws) -> None:
1823     if ws.max_row > 1:
1824         ws.delete_rows(2, ws.max_row)
1825
1826
1827 def _write_slot_sheet(ws, arr: np.ndarray, summaries: list[dict],
1828                      total_key: str, fee_key: str) -> None:
1829     """144 列 × 每日一行的购电表，表头沿用论文统一的区间标签。"""
1830     _clear(ws)
1831     ws.cell(row=1, column=1, value="日期\\时间")
1832     for t in range(N):
1833         ws.cell(row=1, column=2 + t, value=interval_label(t))
1834     ws.cell(row=1, column=2 + N, value="全天购电量")
1835     ws.cell(row=1, column=3 + N, value="全天费用")
1836     for k, s in enumerate(summaries):
1837         ws.cell(row=2 + k, column=1, value=s["日期"])
1838         for t in range(N):
1839             ws.cell(row=2 + k, column=2 + t, value=float(arr[k, t]))
1840         ws.cell(row=2 + k, column=2 + N, value=s[total_key])
1841         ws.cell(row=2 + k, column=3 + N, value=s[fee_key])
1842
1843
1844 def _write_block_sheet(wb, name: str, summaries: list[dict]) -> None:
1845     """6 个 4 小时区间的充放电电量：每天 6 行，首行写日期与 0:00 储电量。"""
1846     ws = wb[name]
1847     _clear(ws)
1848     for c, v in enumerate(("日期", "时间段", "充电量", "放电", "时刻",

```

```

1849         "储电量"), start=1):
1850     ws.cell(row=1, column=c, value=v)
1851     for k, s in enumerate(summaries):
1852         for j, (a, b) in enumerate(BLOCKS):
1853             r = 2 + 6 * k + j
1854             if j == 0:
1855                 ws.cell(row=r, column=1, value=s["日期"])
1856                 ws.cell(row=r, column=5, value="0:00")
1857                 ws.cell(row=r, column=6, value=round(s["期初储电量_kWh"], 4))
1858             ws.cell(row=r, column=2, value=block_label(a, b))
1859             ws.cell(row=r, column=3, value=round(s["block_charge"][j], 4))
1860             ws.cell(row=r, column=4, value=round(s["block_discharge"][j], 4))
1861             if j == 5:
1862                 ws.cell(row=r, column=5, value="24:00")
1863                 ws.cell(row=r, column=6, value=round(s["期末储电量_kWh"], 4))
1864
1865
1866 def _write_emg_sheet(wb, name: str, summaries: list[dict]) -> None:
1867     """紧急购电事件表：同一天连续非零的十分钟时段合并为一个事件。"""
1868     ws = wb[name]
1869     _clear(ws)
1870     for c, v in enumerate(("日期", "购电时间段", "购电量"), start=1):
1871         ws.cell(row=1, column=c, value=v)
1872     row = 2
1873     for s in summaries:
1874         for j, ev in enumerate(s["events"]):
1875             ws.cell(row=row, column=1, value=s["日期"] if j == 0 else None)
1876             ws.cell(row=row, column=2, value=ev["时段"])
1877             ws.cell(row=row, column=3, value=round(ev["电量_kWh"], 4))
1878             row += 1
1879
1880
1881 def _write_summary_sheet(wb, title: str, summaries: list[dict],
1882                          cols: list[tuple[str, str]]) -> None:
1883     if title in wb.sheetnames:
1884         del wb[title]
1885     ws = wb.create_sheet(title)
1886     for c, (hdr, _) in enumerate(cols, start=1):
1887         ws.cell(row=1, column=c, value=hdr)
1888     for k, s in enumerate(summaries):
1889         for c, (_, key) in enumerate(cols, start=1):
1890             v = s.get(key)
1891             ws.cell(row=2 + k, column=c,
1892                    value=round(v, 4) if isinstance(v, float) else v)
1893
1894
1895 SUMMARY_COLS_42 = [
1896     ("日期", "日期"), ("计划购电量_kWh", "计划量_kWh"),
1897     ("紧急购电量_kWh", "紧急电量_kWh"), ("计划购电费_元", "计划费_元"),
1898     ("紧急购电费_元", "紧急费_元"), ("合计购电费_元", "合计费用_元"),
1899     ("弃电量_kWh", "弃电量_kWh"), ("0:00储电量_kWh", "期初储电量_kWh"),
1900     ("24:00储电量_kWh", "期末储电量_kWh"),
1901 ]
1902
1903 SUMMARY_COLS_43 = SUMMARY_COLS_42[:1] + [
1904     ("初始计划量_kWh", "初始计划量_kWh"), ("最终常规量_kWh", "最终常规量_kWh"),
1905     ("上调量_kWh", "上调量_kWh"), ("下调量_kWh", "下调量_kWh"),
1906     ("紧急购电量_kWh", "紧急电量_kWh"), ("计划费_元", "计划费_元"),
1907     ("调整费_元", "调整费_元"), ("紧急费_元", "紧急费_元"),
1908     ("合计费用_元", "合计费用_元"), ("弃电量_kWh", "弃电量_kWh"),
1909     ("0:00储电量_kWh", "期初储电量_kWh"),
1910     ("24:00储电量_kWh", "期末储电量_kWh"),
1911     ("调整提交次数", "调整提交次数"),
1912 ]

```

```

1913
1914
1915 def _open_template(template: Path, path: Path):
1916     from openpyxl import Workbook, load_workbook
1917     path.parent.mkdir(parents=True, exist_ok=True)
1918     if template.exists():
1919         shutil.copyfile(template, path)
1920         return load_workbook(path)
1921     wb = Workbook()
1922     wb.remove(wb.active)
1923     for name in ("计划购电量", "调整购电量", "充放电量", "紧急购电量",
1924                 "全天汇总"):
1925         wb.create_sheet(name)
1926     return wb
1927
1928
1929 def write_result42(summaries: list[dict], path: Path = OUT_XLSX42) -> None:
1930     """写 result4-2.xlsx: 计划购电量 / 充放电量 / 紧急购电量 / 全天汇总。"""
1931     wb = _open_template(TEMPLATE42, path)
1932     _write_slot_sheet(wb["计划购电量"],
1933                      np.array([s["plan_arr"] for s in summaries]),
1934                      summaries, "计划量_kWh", "计划费_元")
1935     _write_block_sheet(wb, "充放电量", summaries)
1936     _write_emg_sheet(wb, "紧急购电量", summaries)
1937     _write_summary_sheet(wb, "全天汇总", summaries, SUMMARY_COLS_42)
1938     wb.save(path)
1939
1940
1941 def write_result43(summaries: list[dict], path: Path = OUT_XLSX43) -> None:
1942     """写 result4-3.xlsx: 计划购电量 / 调整购电量 / 充放电量 / 紧急购电量。
1943
1944     框架 7.2 节: 调整购电表填**每段最终生效的完整 x**, 不是正负调整幅度;
1945     未调整的时段仍然填 x = g°, 不能填零。
1946     """
1947     wb = _open_template(TEMPLATE43, path)
1948     _write_slot_sheet(wb["计划购电量"],
1949                      np.array([s["g0_arr"] for s in summaries]),
1950                      summaries, "初始计划量_kWh", "计划费_元")
1951     _write_slot_sheet(wb["调整购电量"],
1952                      np.array([s["x_arr"] for s in summaries]),
1953                      summaries, "最终常规量_kWh", "合计费用_元")
1954     _write_block_sheet(wb, "充放电量", summaries)
1955     _write_emg_sheet(wb, "紧急购电量", summaries)
1956     _write_summary_sheet(wb, "全天汇总", summaries, SUMMARY_COLS_43)
1957     wb.save(path)
1958
1959
1960 def _daily_frame(summaries: list[dict]) -> pd.DataFrame:
1961     return pd.DataFrame([k: v for k, v in s.items()
1962                          if not k.startswith("block_") and k != "events"
1963                          and not k.endswith("_arr")]
1964                          for s in summaries])
1965
1966
1967 def _snapshot_if_flipped() -> None:
1968     """默认路径会把 JSON 写回"不退款为主口径"的旧布局, 静默撤掉 --refund-main。
1969
1970     在覆盖之前, 若磁盘上的 JSON 已经是翻转后的布局, 先留一份快照, 免得一次
1971     误跑就把费了小时级算力的退款臂连同对照、组合一起抹掉。
1972     """
1973     if not OUT_JSON4.exists():
1974         return
1975     try:
1976         cur = json.loads(OUT_JSON4.read_text(encoding="utf-8"))

```

```

1977 except (OSError, ValueError):
1978     return
1979 if "4-3 不退款口径" not in cur:
1980     return
1981 bak = OUT_DIR / ".backup_pre_refund"
1982 bak.mkdir(exist_ok=True)
1983 dst = bak / "p4_results.flipped.json"
1984 if not dst.exists():
1985     shutil.copyfile(OUT_JSON4, dst)
1986     print(f"!!! 磁盘上的 {OUT_JSON4.name} 已是翻转后的布局；本次按默认路径"
1987           f"重算会把它写回旧布局。\\n    已先快照到 "
1988           f"{dst.relative_to(OUT_DIR.parent)}。")
1989
1990
1991 def write_side_files(recs42, recs43, sm42, sm43, result: dict) -> None:
1992     _snapshot_if_flipped()
1993     if recs42:
1994         pd.concat([detail_frame42(r) for r in recs42],
1995                   ignore_index=True).to_csv(OUT_DETAIL42, index=False)
1996         _daily_frame(sm42).to_csv(OUT_DAILY42, index=False)
1997     if recs43:
1998         pd.concat([detail_frame43(r) for r in recs43],
1999                   ignore_index=True).to_csv(OUT_DETAIL43, index=False)
2000         _daily_frame(sm43).to_csv(OUT_DAILY43, index=False)
2001     OUT_JSON4.write_text(json.dumps(result, ensure_ascii=False, indent=1,
2002                                     default=float), encoding="utf-8")
2003
2004
2005 # ===== 模块 8: 组合比较
2006 def combo_name(S: tuple) -> str:
2007     return "ø" if not S else "{" + ",".join(str(h) for h in S) + "}"
2008
2009
2010 def all_combos() -> list[tuple]:
2011     out = []
2012     for k in range(len(ALL_S) + 1):
2013         out.extend(combinations(ALL_S, k))
2014     return out
2015
2016
2017 def run_combo4(S: tuple, cal43: CalConfig3, book: PriceBook,
2018               LOAD, PV, fp, eps3, fo, weighted: bool = True,
2019               warmup: RiskParams | None = None,
2020               convention: str = "main") -> dict:
2021     """4-3 的一个预报使用组合在波动电价下的整年回放（框架 6.2 节）。
2022
2023     `convention` 必须与主结果所用的计费口径一致：组合之间比的是预报信息的
2024     价值，若主结果是退款口径而组合是不退款口径，差额里就混进了计费歧义。
2025
2026     **每个组合必须用"它自己的信息集"标定参数。** CalConfig3 带着一个 S 字段，
2027     calibrate43 是按 cal.S 回放标定窗口的；若只把 S 传给部署用的
2028     run_strategy43、却让标定继续用 cal43 原来的 S（默认 {6,12,18}），那么
2029     "仅 0:00"这类组合就会被强加上"全套预报"选出的参数，组合之间比出来的差额
2030     里混进了调参差异——这正是问题三 7.5 节明确禁止的做法。故此处先把 cal 的
2031     S 换成当前组合，再同时用于标定与部署。
2032     """
2033     cal = _replace(cal43, S=tuple(S))
2034     recs_all, cal_rows = run_strategy43(cal, book, "forecast", LOAD, PV,
2035                                       fp, eps3, fo, S=S, weighted=weighted,
2036                                       warmup=warmup, convention=convention)
2037     recs = recs_all[REPORT_START:REPORT_END]
2038     t = totals43(recs)
2039     t["使用预报"] = combo_name(S)
2040     t["标定记录数"] = len(cal_rows)

```

```

2041 t["校验失败天数"] = sum(1 for v in check_all4(recs, "43").values() if v > 0)
2042 return t
2043
2044
2045 # ===== 缓存
2046 def _safe_tag(tag: str) -> str:
2047     return "".join("_" if ch in '<>:"/\|?*:' else ch for ch in tag)
2048
2049
2050 def save_cache(tag: str, payload: dict) -> None:
2051     CACHE_DIR.mkdir(parents=True, exist_ok=True)
2052     (CACHE_DIR / f"{_safe_tag(tag)}.json").write_text(
2053         json.dumps(payload, ensure_ascii=False, default=float),
2054         encoding="utf-8")
2055
2056
2057 def _date_of(n: int) -> str:
2058     return str(np.datetime64("2025-01-01") + np.timedelta64(int(n), "D"))
2059
2060
2061 # ===== 主流程
2062 def build_inputs(core: CorePreset = CORE_JIA) -> dict:
2063     """装载全部输入，并构造问题四新增的价格面。
2064
2065     预测器超参数直接取自问题二内核（`core.load_fp` / `core.pv_fp`）：问题四的
2066     负载与光伏预测沿用问题二口径，不另立一套。
2067     """
2068     LOAD, PV, dates = load_attach2()
2069     PRICE = load_attach4()
2070     FIXED = load_attach1()["电价"].to_numpy(float)
2071     if FIXED.shape != (N,):
2072         raise ValueError(f"附件 1 电价长度异常: {FIXED.shape}")
2073     fp = load_attach3()
2074     fo = Forecaster(LOAD, PV, core.load_fp, core.pv_fp)
2075     eps = build_error_table(LOAD, PV, fo)
2076     eps3 = build_eps3(LOAD, PV, fp, fo)
2077     # 预测器超参数**只在预期间（1 月，第 1--30 天）选一次**，随后整段评价区间
2078     # 冻结。这样评价区间上的精度与费用才是样本外结果。
2079     pf_params, pf_select = select_forecast_params(PRICE, 1, REPORT_START)
2080     pf = PriceForecaster(PRICE, pf_params)
2081     return dict(LOAD=LOAD, PV=PV, dates=dates, PRICE=PRICE, FIXED=FIXED,
2082                fp=fp, fo=fo, eps=eps, eps3=eps3, pf=pf,
2083                pf_params=pf_params, pf_select=pf_select,
2084                book=PriceBook(PRICE, FIXED, pf))
2085
2086
2087 def _replace(cal, **kw):
2088     """字段级复制：标定时要临时换掉网格/窗口，但不能改到主策略的配置。
2089
2090     CalConfig / CalConfig3 是 frozen dataclass，不能 setattr；用
2091     dataclasses.replace 生成新实例，原配置对象保持不被修改。
2092     """
2093     return dataclasses.replace(cal, **kw)
2094
2095
2096 def run_main_strategies(S: dict, cal42: CalConfig, cal43: CalConfig3,
2097                        weighted: bool, result: dict,
2098                        warmup42: RiskParams | None = None,
2099                        warmup43: RiskParams | None = None
2100                        ) -> tuple[list, list, list[dict], list[dict]]:
2101     """跑 4-2 与 4-3 的主策略，写两个交付 xlsx。"""
2102     recs42_all, cal42_rows = run_strategy42(
2103         cal42, S["book"], "forecast", S["LOAD"], S["PV"], S["eps"], S["fo"],
2104         weighted=weighted, verbose=True, warmup=warmup42)

```

```

2105 recs42 = recs42_all[REPORT_START:REPORT_END]
2106 sm42 = summaries42(recs42)
2107 result["4-2 主策略"] = totals42(recs42)
2108 result["4-2 校验失败天数"] = check_all4(recs42, "42")
2109 result["4-2 校验数值口径"] = validation_metrics4(recs42, "42")
2110 result["4-2 标定记录"] = cal42_rows
2111 write_result42(sm42)
2112 print(f"[4-2] 合计 {result['4-2 主策略']['合计费用_元']:.2f} 元 "
2113       f"→ {OUT_XLSX42.name}", flush=True)
2114
2115 recs43_all, cal43_rows = run_strategy43(
2116     cal43, S["book"], "forecast", S["LOAD"], S["PV"], S["fp"], S["eps3"],
2117     S["fo"], weighted=weighted, verbose=True, warmup=warmup43)
2118 recs43 = recs43_all[REPORT_START:REPORT_END]
2119 sm43 = summaries43(recs43)
2120 result["4-3 主策略"] = totals43(recs43)
2121 result["4-3 校验失败天数"] = check_all4(recs43, "43")
2122 result["4-3 校验数值口径"] = validation_metrics4(recs43, "43")
2123 result["4-3 标定记录"] = cal43_rows
2124 write_result43(sm43)
2125 print(f"[4-3] 合计 {result['4-3 主策略']['合计费用_元']:.2f} 元 "
2126       f"→ {OUT_XLSX43.name}", flush=True)
2127 return recs42, recs43, sm42, sm43
2128
2129
2130 def run_strategy_comparison(S: dict, cal42: CalConfig, cal43: CalConfig3,
2131                            weighted: bool,
2132                            warmup42: RiskParams | None = None,
2133                            warmup43: RiskParams | None = None,
2134                            convention: str = "main",
2135                            branches: tuple = ("4-2", "4-3")) -> pd.DataFrame:
2136     """框架 6.1 节的三种策略对照：同一条实际价格路径，各自独立回放。
2137
2138      $\Delta J = J(\text{固定电价参考}) - J(\text{波动电价预测})$ ，即“适应波动电价值多少钱”。
2139     绝不用“旧策略在固定价下的账单”减“新策略在实际价下的账单”。
2140
2141     三个价格模式共用同一组 1 月预热期参数：预热期的信息集与价格模式无关，
2142     固定住它， $\Delta J$  才只反映评价区间内价格信息之差，而不掺入预热期的漂移。
2143
2144     `convention` 只作用于 4-3 那一支，且必须与 4-3 主结果所用的计费口径一致
2145     ——否则  $\Delta J$  里混进了计费歧义。`branches` 用来只重算一支（4-2 没有计费口径，
2146     切口径时无需重跑）。
2147     """
2148     rows = []
2149     if "4-2" in branches:
2150         with ThreadPoolExecutor(max_workers=3) as ex:
2151             futs = {m: ex.submit(run_strategy42, cal42, S["book"], m, S["LOAD"],
2152                                S["PV"], S["eps3"], S["fo"], E_INIT, weighted,
2153                                False, warmup42)
2154                     for m in PRICE_MODES}
2155             t42 = {m: totals42(f.result()[0][REPORT_START:REPORT_END])
2156                    for m, f in futs.items()}
2157             for m in PRICE_MODES:
2158                 rows.append({"分支": "4-2", "策略": MODE_LABEL[m], "模式": m,
2159                             **t42[m]})
2160             d42 = t42["fixed"]["合计费用_元"] - t42["forecast"]["合计费用_元"]
2161             rows.append({"分支": "4-2", "策略": " $\Delta J$  (固定电价 - 波动电价)",
2162                         "模式": "", "合计费用_元": d42,
2163                         "相对降幅": d42 / t42["fixed"]["合计费用_元"]})
2164
2165     if "4-3" not in branches:
2166         return pd.DataFrame(rows)
2167
2168     t43 = {}

```

```

2169 for m in PRICE_MODES:
2170     recs, _ = run_strategy43(cal43, S["book"], m, S["LOAD"], S["PV"],
2171                             S["fp"], S["eps3"], S["fo"], weighted=weighted,
2172                             warmup=warmup43, convention=convention)
2173     t43[m] = totals43(recs[REPORT_START:REPORT_END])
2174     rows.append({"分支": "4-3", "策略": MODE_LABEL[m], "模式": m, **t43[m]})
2175 d43 = t43["fixed"]["合计费用_元"] - t43["forecast"]["合计费用_元"]
2176 rows.append({"分支": "4-3", "策略": "ΔJ (固定电价 - 波动电价)", "模式": "",
2177             "合计费用_元": d43,
2178             "相对降幅": d43 / t43["fixed"]["合计费用_元"]})
2179 return pd.DataFrame(rows)
2180
2181 # ===== 主口径切换: 退款 (与问题三一致)
2182 def risk_from_label(label: str, core: CorePreset) -> RiskParams:
2183     """把 JSON 里记录的标定结果 (`RiskParams.label()`) 的输出) 还原成参数。
2184
2185     ``label()`` 只含  $\alpha/p/\lambda$ , **不含** window 与 min_samples。这两项是内核口径的
2186     一部分, 必须从 core 取, 不能沿用 RiskParams 的默认值——jia 内核的
2187     min_samples 是 8, 而 RiskParams 的默认是 10。用错会让第 1 月预热的误差
2188     分位数换一套取法, 进而改变 2 月 1 日的日初储电量, 全年数字都对不上。
2189     """
2190     nums = dict(re.findall(r"([\alpha\lambda])\s*=\s*([0-9.]+)", label))
2191     missing = {" $\alpha$ ", "p", " $\lambda$ "} - set(nums)
2192     if missing:
2193         raise ValueError(f"无法从 {label!r} 还原出 {sorted(missing)}")
2194     return RiskParams(alpha=float(nums[" $\alpha$ "]), rho=float(nums["p"]),
2195                      lam=float(nums[" $\lambda$ "]), window=core.risk_window,
2196                      min_samples=core.min_samples)
2197
2198
2199 def recompute_refund_main(S: dict, core: CorePreset, weighted: bool, result: dict,
2200                          do_strategies: bool, do_sweep: bool) -> int:
2201     """把 4-3 的主计费口径定为**退款** (与问题三一致), 只重算 4-3 相关部分。
2202
2203     为什么单开一条快路径: 一次完整求解要先跑 1 月的两组 180 组联合标定, 再跑
2204     4-2、4-3、三模式对照与八组合扫描, 耗时以小时计。而切换口径只改变 4-3 的
2205     **费用函数**——价格预测器、风险口径、1 月标定结果、4-2 全部与口径无关,
2206     可以原样复用既有 p4_results.json 里已记录的值。因此这里只做:
2207
2208     1. 4-3 主结果在退款口径下重放一次, 落 result4-3.xlsx 与逐时段/逐日明细;
2209     2. 把原来的不退款臂降级为敏感性 (键 ``4-3 不退款口径``);
2210     3. 可选地把三模式对照与八组合扫描也换成退款口径。
2211
2212     口径只影响 4-3: DayRecord42 里没有 ``x`` 字段, 日内不得修改计划, 结构上
2213     产生不了调减量, 故 4-2 的两个口径数值完全重合, 无需重算。
2214
2215     返回 0 表示成功。任何一处校验不过就返回非 0 且**不落盘**。
2216     """
2217     if not OUT_JSON4.exists():
2218         print(f"!!! 找不到 {OUT_JSON4}, 无法进入 --refund-main 快路径。"
2219               "请先完整跑一次 p4_microgrid.py。")
2220         return 1
2221     old = json.loads(OUT_JSON4.read_text(encoding="utf-8"))
2222     if "4-3 不退款口径" in old:
2223         print("!!! 既有 JSON 已经是翻转后的布局 (含 '4-3 不退款口径')。"
2224               "本开关**不幂等**:\n"
2225               "再跑一次会把已是退款臂的 '4-3 主策略' 误降级成不退款臂,"
2226               "静默毁掉真正的退款臂。"
2227               "若要重来, 请先从 06_支撑材料/.backup_pre_refund/ 恢复 "
2228               "p4_results.json (或先完整重跑一次默认流程), 再执行本开关。")
2229         return 1
2230     lab = old.get("4-3 1月联合标定", {}).get("选中")

```

```

2233 if not lab:
2234     print("!!! 既有 JSON 里没有 '4-3 1月联合标定', 无法复用标定结果。")
2235     return 1
2236 warmup43 = risk_from_label(lab, core)
2237 cal43 = CalConfig3(min_samples=core.min_samples,
2238                    alphas=core.warmup_alphas or CalConfig3.alphas,
2239                    rhos=core.warmup_rhos or CalConfig3.rhos,
2240                    lams=(warmup43.lam,))
2241 print(f"\n复用 1 月标定: warmup43 = {warmup43.label()}"
2242       f" (window={warmup43.window}, min_samples={warmup43.min_samples}) ")
2243 print(f"cal43: alphas={cal43.alphas} rhos={cal43.rhos} lams={cal43.lams} "
2244       f"window={cal43.window} every={cal43.every} S={cal43.S}")
2245
2246 # ---- 1. 4-3 主结果: 退款口径
2247 t0 = time.time()
2248 print("\n" + "=" * 78)
2249 print("4-3 主结果: 退款计费口径 (与问题三一致)")
2250 recs_all, cal_rows = run_strategy43(
2251     cal43, S["book"], "forecast", S["LOAD"], S["PV"], S["fp"], S["eps3"],
2252     S["fo"], weighted=weighted, warmup=warmup43, convention="refund",
2253     verbose=True)
2254 recs = recs_all[REPORT_START:REPORT_END]
2255 tot = totals43(recs)
2256 print(f"[4-3 退款主口径] 合计 {tot['合计费用_元']:.2f} 元, "
2257       f"计划费 {tot['计划费_元']:.2f}、调整费 {tot['调整费_元']:.2f}、"
2258       f"紧急费 {tot['紧急费_元']:.2f}; 下调量 {tot['下调量_kWh']:.2f} kWh"
2259       f" (用时 {time.time() - t0:.1f}s) ", flush=True)
2260
2261 # 既有 JSON 里若已记录同口径结果, 先逐项核对——复现不了就说明口径之外的
2262 # 某处 (标定、内核、预报) 与当初不同, 此时必须停手而不是照写。
2263 ref = old.get("4-3 退款口径")
2264 if ref:
2265     bad = [(k, tot.get(k), ref.get(k)) for k, v in ref.items()
2266            if isinstance(v, (int, float)) and k in tot
2267            and abs(tot[k] - v) > max(1e-6, 1e-11 * abs(v))]
2268     if bad:
2269         print("\n!!! 与既有 JSON 记录的 4-3 退款口径对不上, 停止落盘:")
2270         for k, got, want in bad:
2271             print(f"    {k}: 本次 {got:.6f} 记录 {want:.6f} "
2272                   f"Δ {got - want:+.6f}")
2273         return 1
2274     print("    与既有 JSON 记录的退款臂逐项一致 (含合计 "
2275           f"{ref['合计费用_元']:.2f} 元) ")
2276
2277 # ---- 2. 角色互换: 退款臂成为主口径, 原不退款臂降为敏感性
2278 # 旧 JSON 的布局是"4-3 主策略"=不退款臂 + 顶层 "4-3 校验*"/"4-3 标定记录",
2279 # 而 "4-3 退款口径" 把校验项嵌在自己里面。互换时要把这些附属键一并搬对位置,
2280 # 否则 p4_tables.py 读到的校验表还是不退款臂的。
2281 no_ref = dict(old.get("4-3 主策略") or {})
2282 if not no_ref:
2283     print("!!! 既有 JSON 里没有 '4-3 主策略', 无法把不退款臂降级为敏感性。")
2284     return 1
2285 for k in ("校验失败天数", "校验数值口径"):
2286     if f"4-3 {k}" in old:
2287         no_ref.setdefault(k, old[f"4-3 {k}"])
2288
2289 result.update(old) # 保留 4-2、诊断、价格结构等全部旧键
2290 result["4-3 不退款口径"] = no_ref
2291 if "4-3 标定记录" in old:
2292     result["4-3 不退款标定记录"] = old["4-3 标定记录"]
2293 result.pop("4-3 退款口径", None) # 已升为主口径, 避免同一臂留两个键
2294 result["4-3 主策略"] = tot
2295 result["4-3 校验失败天数"] = check_all4(recs, "43")
2296 result["4-3 校验数值口径"] = validation_metrics4(recs, "43")

```

```

2297 result["4-3 标定记录"] = cal_rows
2298 print(f"      九项校验失败天数: {result['4-3 校验失败天数']}")
2299 print(f"      4-3 不退款口径 (敏感性) = "
2300       f"{result['4-3 不退款口径']['合计费用_元']:, .2f} 元")
2301
2302 # 暂存: 全部算完、全部核对通过之前不碰交付文件
2303 stage = OUT_DIR / ".refund_stage"
2304 stage.mkdir(exist_ok=True)
2305 sm = summaries43(recs)
2306 write_result43(sm, stage / "result4-3.xlsx")
2307 pd.concat([detail_frame43(r) for r in recs],
2308           ignore_index=True).to_csv(stage / "p4_detail_43.csv", index=False)
2309 _daily_frame(sm).to_csv(stage / "p4_daily_43.csv", index=False)
2310
2311 # ---- 3. 三模式对照 (只重算 4-3 那一支)
2312 if do_strategies:
2313     print("\n" + "=" * 78)
2314     print("三种价格信息条件对照: 4-3 改用退款口径重算 (4-2 无口径、沿用既有) ")
2315     t0 = time.time()
2316     new43 = run_strategy_comparison(S, None, cal43, weighted,
2317                                   warmup43=warmup43,
2318                                   convention="refund",
2319                                   branches=("4-3",))
2320     print(f" 用时 {time.time() - t0:.1f}s")
2321     keep = [r for r in old.get("策略对照", []) if r.get("分支") != "4-3"]
2322     merged = keep + new43.to_dict("records")
2323     result["策略对照"] = merged
2324     # 交付件要与 JSON 同步: 否则 p4_strategies.csv 里 4-3 那一支还是旧口径。
2325     pd.DataFrame(merged).to_csv(OUT_STRAT4, index=False)
2326     print(new43.to_string(index=False, float_format=lambda v: f"{v:,.2f}"))
2327
2328 # ---- 4. 八种预报组合 (必须在同一口径下重扫, 否则组合价值里混入口径差)
2329 if do_sweep:
2330     print("\n" + "=" * 78)
2331     print("八种预报组合: 改用退款口径重扫 (每个组合仍用它自己的信息集标定) ")
2332     t0 = time.time()
2333     combos = all_combos()
2334     with ThreadPoolExecutor(max_workers=4) as ex:
2335         futs = [ex.submit(run_combo4, c, cal43, S["book"], S["LOAD"],
2336                           S["PV"], S["fp"], S["eps3"], S["fo"], weighted,
2337                           warmup43, "refund")
2338                 for c in combos]
2339         newc = []
2340         for c, f in zip(combos, futs):
2341             row = f.result()
2342             newc.append(row)
2343             print(f"      {combo_name(c):>12s} "
2344                   f"{row['合计费用_元']:>14,.0f} 元", flush=True)
2345         pd.DataFrame(newc).to_csv(OUT_COMBOS4, index=False)
2346         result["预报组合"] = newc
2347         print(f" 用时 {time.time() - t0:.1f}s")
2348
2349 OUT_JSON4.write_text(json.dumps(result, ensure_ascii=False, indent=1,
2350                                default=float), encoding="utf-8")
2351
2352 write_result43(sm)
2353 shutil.copyfile(stage / "p4_detail_43.csv", OUT_DETAIL43)
2354 shutil.copyfile(stage / "p4_daily_43.csv", OUT_DAILY43)
2355 print(f"\n已写出 {OUT_XLSX43.name}、{OUT_DETAIL43.name}、{OUT_DAILY43.name}"
2356       f" 与 {OUT_JSON4.name}")
2357
2358 return 0
2359
2360 def main() -> None:
2361     ap = argparse.ArgumentParser(

```

```

2361     description="2026 CUMCM C题 问题四 求解（实时波动电价）")
2362 ap.add_argument("--quick", action="store_true",
2363                 help="缩短标定窗口，快速自检")
2364 ap.add_argument("--no-cal", action="store_true", help="跳过标定，用默认参数")
2365 ap.add_argument("--strategies", action="store_true",
2366                 help="额外跑三种价格模式对照（框架 6.1 节）")
2367 ap.add_argument("--sweep", action="store_true",
2368                 help="额外跑 4-3 的 8 种预报组合（框架 6.2 节）")
2369 ap.add_argument("--unweighted", action="store_true",
2370                 help="把价格加权分位数退回普通分位数（对照用）")
2371 ap.add_argument("--refund", action="store_true",
2372                 help="额外跑 4-3 的退款计费口径，写入 JSON 的 "
2373                      "'4-3 退款口径' 键（框架 5.3 节）。退款已是论文主口径，"
2374                      "经 --refund-main 后该键会被换名保存，故本开关主要用于"
2375                      "重建原始退款臂")
2376 ap.add_argument("--refund-main", action="store_true",
2377                 help="把 4-3 的主计费口径定为退款（与问题三一致），只重算 "
2378                      "'4-3 相关部分：复用 JSON 里记录的 1 月标定，重放 4-3、"
2379                      "重出 result4-3.xlsx 与明细，并把原不退款臂降级为敏感性。"
2380                      "配 --strategies/--sweep 可连同对照与组合一起换口径")
2381 ap.add_argument("--report-only", action="store_true",
2382                 help="只重算价格预测与风险统计，不跑寻优")
2383 ap.add_argument("--core", default="jia", choices=sorted(CORES),
2384                 help="问题二内核（4-2/4-3 随之重算）。jia（默认）=对照实现 "
2385                      "'origin/q2-jia 最终版：1 月联合标定（ $\alpha, \rho, \lambda$ ）、 $\lambda$  整年冻结、"
2386                      "预测器写死、第 0 天走日 LP、min_samples=8；"
2387                      "legacy=本仓旧行为，仅用于复现历史数字")
2388 args = ap.parse_args()
2389
2390 t_start = time.time()
2391 core = CORES[args.core]
2392 S = build_inputs(core)
2393 PRICE, weighted = S["PRICE"], not args.unweighted
2394
2395 print("=" * 78)
2396 print(f"附件 4 载入：{PRICE.min():.4f}~{PRICE.max():.4f} 元/kWh, "
2397       f"均值 {PRICE.mean():.4f}, 标准差 {PRICE.std():.4f}")
2398 print(f"  单日内形状解释方差 {shape_ratio(PRICE):.1%}; "
2399       f"日中位峰谷比 {np.median(PRICE.max(axis=1) / PRICE.min(axis=1)):.2f}")
2400 print(f"价格预测器：{S['pf'].pp.label()}")
2401
2402 result: dict = {
2403     "问题二内核": core.name,
2404     "价格数据": price_summary(PRICE),
2405     "价格预测器": S["pf"].pp.label(),
2406     "评价区间": f"2025-02-01 至 2025-12-31"
2407                 f"({REPORT_END - REPORT_START} 天)",
2408     "风险口径": "价格加权分位数" if weighted else "普通分位数（对照）",
2409 }
2410 print(f"问题二内核：{core.name}"
2411       f"({core.load_fp.label()}; {core.pv_fp.label()}; "
2412       f"min_samples={core.min_samples}; 第 0 天冷启动)")
2413
2414 # ---- 价格加权分位数定义的自检（论文公式 ↔ 实现一致性）
2415 wq_bad = test_weighted_quantile()
2416 result["风险_加权分位数自检"] = {"失败项": wq_bad, "通过": not wq_bad}
2417 print("\n价格加权分位数定义自检（下确界定义 ↔ 实现）：")
2418 if wq_bad:
2419     for b in wq_bad:
2420         print(f"  [失败] {b}")
2421 else:
2422     print("  等权退回 inverted_cdf、取值必为样本值、覆盖率单调 —— 通过")
2423
2424 # ---- 价格结构分解：P = 附件1曲线 + 星期偏移（预测器的推导依据）

```

```

2425 ps = verify_price_structure(PRICE, S["FIXED"], S["pf"].pp)
2426 result["价格结构"] = ps
2427 print("\n价格结构分解 P = 附件1曲线 + 残差:")
2428 print(f" 附件1 年平均 {ps['断言1_年平均_附件1']:.6f} 元/kWh, "
2429       f"附件4 年平均 {ps['断言1_年平均_附件4']:.6f} 元/kWh"
2430       f" (逐段均值最大偏差 {ps['断言1_附件4逐段均值与附件1最大偏差']:.2e}) ")
2431 print(f" 残差星期均值: "
2432       + " ".join(f"{k}={v:+.4f}"
2433                  for k, v in ps["断言3_残差星期均值_元每kWh"].items()))
2434 print(f" 周五六 {ps['断言3_周五六均值']:+.4f} vs 其余五天 "
2435       f"{ps['断言3_其余五天均值']:+.4f}, 相差 "
2436       f"{ps['断言3_两水平差_元每kWh']:+.4f} 元/kWh"
2437       f" ({ps['断言3_相对均价']:.1%}), 解释残差方差 "
2438       f"{ps['断言4_星期效应解释比例']:.1%}")
2439 print(f" '同星期加权 == 附件1曲线 + 同星期残差加权' 最大残差 "
2440       f"{ps['断言2_同星期加权恒等式最大残差']:.2e}")
2441
2442 # ---- 预测器与风险的诊断 (不依赖寻优, 先跑, 供论文取数)
2443 cmp_df = price_forecast_compare(PRICE)
2444 cmp_df.to_csv(OUT_PFCAND, index=False)
2445 print("\n价格预测候选 (正式区间 MAE, 元/kWh) : ")
2446 print(cmp_df.to_string(index=False, float_format=lambda v: f"{v:.4f}"))
2447
2448 beta_df = price_beta_sweep(PRICE, S["pf"].pp)
2449 beta_df.to_csv(OUT_PBETA, index=False)
2450 print("\n日内对数修正系数  $\beta$  扫描 (对剩余时段的 MAE) : ")
2451 print(beta_df.to_string(index=False, float_format=lambda v: f"{v:.4f}"))
2452
2453 result["价格预测精度"] = price_forecast_skill(PRICE, S["pf"])
2454 result["价格预测候选"] = cmp_df.to_dict("records")
2455 result[" $\beta$ 扫描"] = beta_df.to_dict("records")
2456
2457 # 选型窗口与选中的参数: 预测器超参数只在 1 月 (评价区间之外) 选一次,
2458 # 上面 价格预测候选 /  $\beta$ 扫描 两张表因此是**样本外**复核, 不是选型依据。
2459 sel = S["pf_select"]
2460 result["预测器选型"] = sel
2461 print("\n预测器选型——只在 1 月预热期 (第 1--30 天, 评价区间之外) 选一次, "
2462       "随后整段评价区间冻结: ")
2463 print(f" 选中 {sel['选中的参数']} (1 月 MAE "
2464       f"{sel['最优平均绝对误差_元每kWh']:.4f} 元/kWh) ")
2465 print(" 正式区间上的 价格预测候选 /  $\beta$ 扫描 两表是样本外复核, 不参与选型。")
2466 corr = risk_price_correlation(S["eps"], PRICE)
2467 result["风险_价格协方差"] = corr
2468 rw = risk_weight_effect(S["eps"], PRICE)
2469 rw.to_csv(OUT_RWEIGHT, index=False)
2470 result["风险_加权对照"] = rw.to_dict("records")
2471 print("\n风险余量: 价格加权 vs 普通分位数 (正式区间抽样均值) ")
2472 print(rw.to_string(index=False, float_format=lambda v: f"{v:.3f}"))
2473 print(f" 协方差检查: {corr['结论']}")
2474 print(f" 合并口径 corr={corr['合并相关系数']:+.4f}; "
2475       f"逐时段 corr 均值 {corr['逐时段相关系数均值']:+.3f}"
2476       f" ({corr['逐时段相关系数最小']:+.3f}~{corr['逐时段相关系数最大']:+.3f}, "
2477       f"为正占 {corr['逐时段相关为正的比率']:.1%}) ")
2478 print(f" 电价高档 20% 误差 {corr['高价20%_误差均值_kW']:+.1f} kW "
2479       f"vs 低档 {corr['低价20%_误差均值_kW']:+.1f} kW, "
2480       f"两端差 {corr['两端误差均值差_kW']:+.1f} kW")
2481
2482 if args.report_only:
2483     # 诊断部分很便宜 (<1s), 而寻优很贵。因此 --report-only 是**增量更新**:
2484     # 若已有求解结果, 只覆盖诊断相关的键, 保留主策略、标定、策略对照等
2485     # 需要长时间才能重算的内容。
2486     if OUT_JSON4.exists():
2487         try:
2488             old = json.loads(OUT_JSON4.read_text(encoding="utf-8"))

```

```

2489         old.update(result)
2490         result = old
2491         print(f"\n--report-only: 已合并进既有 {OUT_JSON4.name}"
2492               f" (保留 {sum(1 for k in old if k.startswith(('4-2', '4-3'))))}
"
2493               f" 项求解结果) ")
2494     except (json.JSONDecodeError, OSError) as e:
2495         print(f" 既有 {OUT_JSON4.name} 不可读 ({e}), 改为只写诊断")
2496     OUT_JSON4.write_text(json.dumps(result, ensure_ascii=False, indent=1,
2497                                   default=float), encoding="utf-8")
2498     print(f"--report-only 完成, 用时 {time.time() - t_start:.1f}s")
2499     return
2500
2501 if args.refund_main:
2502     # 切口只影响 4-3 的费用函数: 4-2、价格预测器、风险口径、1 月标定
2503     # 都与口径无关, 直接复用既有 JSON, 不必付一次完整求解的代价。
2504     rc = recompute_refund_main(S, core, weighted, result,
2505                               args.strategies, args.sweep)
2506     print(f"\n--refund-main 结束, 用时 {time.time() - t_start:.1f}s")
2507     return rc
2508
2509 # ---- 1 月联合标定: 预热期参数在正式区间开始前离线选定, 此后整年冻结。
2510 # 只用 1 月数据, 第 0 天对所有候选相同;  $\lambda$  选定后不再由滚动标定改动。
2511 warmup42 = warmup43 = None
2512 warmup42_rows: list[dict] = []
2513 warmup43_rows: list[dict] = []
2514 if core.warmup_grid():
2515     print("\n" + "=" * 78)
2516     print(f"1 月联合标定 (内核 {core.name}, "
2517           f"{len(core.warmup_grid())} 组 ( $\alpha, \rho, \lambda$ ), 只用 1 月数据) ")
2518     t_w = time.time()
2519     warmup42, warmup42_rows = select_warmup42(
2520         core, S["book"], "forecast", S["LOAD"], S["PV"], S["eps"], S["fo"],
2521         weighted)
2522     print(f" [4-2] 选中 {warmup42.label()}, 1 月费用 "
2523           f"{warmup42_rows[[r['选中'] for r in warmup42_rows].index(True)]['1月总
费用_元']:, .2f} 元, "
2524           f"2 月 1 日储电量 "
2525           f"{warmup42_rows[[r['选中'] for r in warmup42_rows].index(True)]['2月1
日储电量_kWh']:, .2f} kWh",
2526           flush=True)
2527     warmup43, warmup43_rows = select_warmup43(
2528         core, S["book"], "forecast", S["LOAD"], S["PV"], S["fp"],
2529         S["eps3"], S["fo"], "main", weighted)
2530     print(f" [4-3] 选中 {warmup43.label()}, 1 月费用 "
2531           f"{warmup43_rows[[r['选中'] for r in warmup43_rows].index(True)]['1月总
费用_元']:, .2f} 元, "
2532           f"2 月 1 日储电量 "
2533           f"{warmup43_rows[[r['选中'] for r in warmup43_rows].index(True)]['2月1
日储电量_kWh']:, .2f} kWh"
2534           f" (用时 {time.time() - t_w:.1f}s) ", flush=True)
2535     result["4-2 1月联合标定"] = {"选中": warmup42.label(),
2536                                   "明细": warmup42_rows}
2537     result["4-3 1月联合标定"] = {"选中": warmup43.label(),
2538                                   "明细": warmup43_rows}
2539 else:
2540     print("\n内核 legacy: 预热期参数沿用硬编码值, 不做 1 月联合标定")
2541
2542 base42 = CalConfig(min_samples=core.min_samples,
2543                   alphas=core.warmup_alphas or CalConfig.alphas,
2544                   rhos=core.warmup_rhos or CalConfig.rhos,
2545                   lams=(warmup42.lam,) if warmup42 else CalConfig.lams)
2546 base43 = CalConfig3(min_samples=core.min_samples,
2547                    alphas=core.warmup_alphas or CalConfig3.alphas,

```

```

2548         rhos=core.warmup_rhos or CalConfig3.rhos,
2549         lams=(warmup43.lam,) if warmup43 else CalConfig3.lams)
2550 cal42 = _replace(base42, window=10, every=42) if args.quick else base42
2551 cal43 = _replace(base43, window=10, every=42) if args.quick else base43
2552 if args.no_cal:
2553     # 网格塌缩成一个固定参数，等价于不做标定
2554     cal42 = _replace(cal42, alphas=(None,), rhos=(1.0,),
2555                     lams=(LAMBDA_DEFAULT,))
2556     cal43 = _replace(cal43, alphas=(None,), rhos=(1.0,),
2557                     lams=(LAMBDA_DEFAULT,))
2558
2559 recs42, recs43, sm42, sm43 = run_main_strategies(S, cal42, cal43, weighted,
2560                                                result, warmup42, warmup43)
2561
2562 if args.strategies:
2563     print("\n" + "=" * 78)
2564     print("三种价格模式对照（同一实际价格路径，各自独立标定）")
2565     strat = run_strategy_comparison(S, cal42, cal43, weighted,
2566                                   warmup42, warmup43)
2567     strat.to_csv(OUT_STRAT4, index=False)
2568     result["策略对照"] = strat.to_dict("records")
2569     print(strat[["分支", "策略", "计划费_元", "调整费_元", "紧急费_元",
2570                "合计费用_元"]].to_string(
2571           index=False, float_format=lambda v: f"{v:,.0f}"))
2572
2573 if args.sweep:
2574     print("\n" + "=" * 78)
2575     print("4-3 八种预报组合（波动电价下重算，框架 6.2 节）")
2576     combos = all_combos()
2577     with ThreadPoolExecutor(max_workers=4) as ex:
2578         futs = [ex.submit(run_combo4, c, cal43, S["book"], S["LOAD"],
2579                           S["PV"], S["fp"], S["eps3"], S["fo"], weighted,
2580                           warmup43)
2581                 for c in combos]
2582     rows = []
2583     for c, f in zip(combos, futs):
2584         row = f.result()
2585         rows.append(row)
2586         print(f"      {combo_name(c):>12s} "
2587               f"{{row['合计费用_元']:>14,.0f}} 元", flush=True)
2588     combos_df = pd.DataFrame(rows)
2589     combos_df.to_csv(OUT_COMBOS4, index=False)
2590     result["预报组合"] = combos_df.to_dict("records")
2591
2592 # 先落盘一次：本函数的重头在标定与全年回放，万一后面某一臂出错，也不该让
2593 # 几个小时的结果付诸东流。（--refund 之前的落盘内容会由它自己覆盖补齐。）
2594 write_side_files(recs42, recs43, sm42, sm43, result)
2595 print(f"\n主结果已写出，用时 {{time.time() - t_start:.1f}}s → {OUT_JSON4.name}")
2596
2597 if args.refund:
2598     # 框架 5.3 节：调减部分的原价是否退回，题面没有写死。这里把 4-3 在退款
2599     # 口径下**独立标定并重新回放**——同一套模型、同一套执行规则，只换费用
2600     # 函数。--refund-main 会把该臂提为主口径，本支路则保持原始布局。
2601     print("\n" + "=" * 78)
2602     print("退款计费口径（4-3，独立标定与回放）")
2603     recs_rf, _ = run_strategy43(cal43, S["book"], "forecast", S["LOAD"],
2604                                S["PV"], S["fp"], S["eps3"], S["fo"],
2605                                convention="refund", weighted=weighted,
2606                                verbose=True, warmup=warmup43)
2607     trf = totals43(recs_rf[REPORT_START:REPORT_END])
2608     trf["校验失败天数"] = check_all4(recs_rf[REPORT_START:REPORT_END], "43")
2609     # validation_metrics4 自己按 r.convention 选公式，无需额外传参
2610     trf["校验数值口径"] = validation_metrics4(
2611         recs_rf[REPORT_START:REPORT_END], "43")

```

```

2612     result["4-3 退款口径"] = trf
2613     print(f"[4-3 退款口径] 合计 {trf['合计费用_元']:, .2f} 元, "
2614           f"其中计划费 {trf['计划费_元']:, .2f}、"
2615           f"调整费 {trf['调整费_元']:, .2f}、"
2616           f"紧急费 {trf['紧急费_元']:, .2f}; "
2617           f"下调量 {trf['下调量_kWh']:, .2f} kWh")
2618     OUT_JSON4.write_text(json.dumps(result, ensure_ascii=False, indent=1,
2619                                     default=float), encoding="utf-8")
2620
2621     print(f"\n全部完成, 用时 {time.time() - t_start:.1f}s")
2622
2623
2624 if __name__ == "__main__":
2625     main()

```

(2) p4_tables.py: 论文表格生成

```

1  # -*- coding: utf-8 -*-
2  r"""
3  2026 CUMCM C 题 问题四 —— 由求解结果直接生成论文表格片段。
4
5  写出 05_论文/CUMCMThesis/p4_tables.tex (正文用) 与 p4_tables_appendix.tex
6  (附录用), 由 main.tex 以 \input 引入, 使论文中每个数字都与
7  result4-2/result4-3/p4_results.json 严格同源, 避免手工转录错误。
8
9  正文只有 30 页额度, 问题四能占的篇幅有限, 因此按"哪些数字是回答题目所必需"
10  来分: 题目明确要求两个交付文件, 故两个分支的费用汇总表与策略对照表留在正文;
11  价格预测的选型过程、风险口径的对照、八种预报组合与校验明细属于论据, 放附录;
12  计费口径的敏感性另写 p4_tables_refund.tex, 由 main.tex 在附录单独 \input:
13  主口径取题面字面读法的退款 (被取消部分按 50% 违约电价计), "不退款、只追加"
14  作为独立敏感性并列报告。
15
16  运行: python3 03_代码/p4_tables.py --split (需先运行 p4_microgrid.py)
17  """
18
19  from __future__ import annotations
20
21  import json
22  import math
23  import sys
24  from pathlib import Path
25
26  sys.path.insert(0, str(Path(__file__).resolve().parent))
27
28  ROOT = Path(__file__).resolve().parents[1]
29  SUP = ROOT / "06_支撑材料"
30  THESIS = ROOT / "05_论文" / "CUMCMThesis"
31  OUT_TEX = THESIS / "p4_tables.tex"
32  OUT_TEX_TAIL = THESIS / "p4_tables_appendix.tex"
33  OUT_TEX_REFUND = THESIS / "p4_tables_refund.tex"
34  OUT_TEX_COMBO = THESIS / "p4_combo_detail.tex"
35
36  SPECIAL = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
37  DISP = {"2025-03-20": "2025.3.20", "2025-06-21": "2025.6.21",
38          "2025-09-23": "2025.9.23", "2025-12-21": "2025.12.21"}
39
40
41  def fmt(v: float, nd: int = 2) -> str:
42      return f"{v:,.{nd}f}"
43
44
45  def sci(v: float) -> str:

```

```

46     return "0.00" if abs(v) < 1e-9 else f"{v:.2e}"
47
48
49 def texlab(s: str) -> str:
50     return s.replace(":", "{:}")
51
52
53 def pct(v: float, nd: int = 1) -> str:
54     """百分数。LaTeX 里 `%` 是注释符，直接写 `41.7%` 会把该行后面的
55     `&` 和 `\\` 一起注释掉，表格随即报 "Extra alignment tab",
56     所以这里必须输出 `\\%`。"""
57     return f"{v * 100:.{nd}f}\\%"
58
59
60 def signed(pct: float) -> str:
61     return f"{pct:+.2f}\\%".replace("+-", "-")
62
63
64 def signed_pct(v: float, nd: int = 2) -> str:
65     """带正负号的百分数（同样要转义 `%`）。"""
66     return f"{v * 100:+.{nd}f}\\%".replace("+-", "-")
67
68
69 def has(res: dict, key: str) -> bool:
70     return key in res
71
72
73 # ----- 表：价格结构
74 def tab_price_struct(res: dict) -> str:
75     """表：附件 4 的价格分解 P = 附件 1 曲线 + 星期偏移，及预测器选型。"""
76     ps = res["价格结构"]
77     wd = ps["断言3_残差星期均值_元每kWh"]
78     order = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
79     lines = [
80         r"\begin{table}[htbp]",
81         r" \centering",
82         r" \caption{附件 4 实时电价的结构分解（$P=F+R$，$F$ 为附件 1 的固定"
83         r"曲线，$R$ 为残差）}",
84         r" \label{tab:p4-struct}",
85         r" \footnotesize",
86         r" \begin{tabular}{lrr}",
87         r" \toprule",
88         r" 项目 & 数值 & 说明 \\",
89         r" \midrule",
90         f" 逐年逐段均值 $\overline{{P}}$ 与 $F$ 的最大偏差"
91         f" & ${sci(ps['断言1_附件4逐段均值与附件1最大偏差'])}$",
92         r" & 两者\textbf{骨架与水平相同} \\",
93         f" $F$ 的年平均 & {fmt(ps['断言1_年平均_附件1'], 4)}"
94         r" & 元/kWh \\",
95         f" $\overline{{P}}$ 的年平均 & {fmt(ps['断言1_年平均_附件4'], 4)}"
96         # 年均值之差只有 ~4e-7; $10^{-5}$ 是逐段均值最大偏差的量级，两者不是
97         # 同一个量，写混会与正文"相差不足 $10^{-6}$"自相矛盾。
98         r" & 元/kWh，与 $F$ 相差不足 $10^{-6}$ \\",
99         f" $R$ 与 $F$ 的相关系数 & ${ps['断言2_残差与固定曲线相关']:+.4f}$",
100         r" & 正交，故 $R$ 就是`波动`本身 \\",
101         r" \midrule",
102     ]
103     for w in order:
104         lines.append(f" $R$ 的{w}均值 & {wd[w]:+.4f} & 元/kWh \\\\"
105     lines += [
106         r" \midrule",
107         f" 周五、周六平均 & {ps['断言3_周五六均值']:+.4f}"
108         r" & 元/kWh，\textbf{偏低} \\",
109         f" 其余五天平均 & {ps['断言3_其余五天均值']:+.4f}"

```

```

110     r" & 元/kWh \\",
111     f"     两者之差 & {ps['断言3_两水平差_元每kWh']:+.4f}"
112     f" & 元/kWh, 相当于均价的 {pct(ps['断言3_相对均价'], 1)} \\\\",
113     f"     星期结构解释的 $R$ 方差 & {pct(ps['断言4_星期效应解释比例'], 1)}"
114     r" & 其余为无规律的日内噪声 \\",
115     f"     ``同星期加权``恒等式最大残差"
116     f" & ${sci(ps['断言2_同星期加权恒等式最大残差'])}$"
117     r" & 见式~\eqref{eq:p4-price} \\",
118     r"     \bottomrule",
119     r" \end{tabular}",
120     r" \par\vspace{2pt}",
121     r" \begin{minipage}{.92\textwidth}\footnotesize",
122     r"     注：附件 4 的电价并非与附件 1 无关的另一条曲线，而是\textbf{在附件 1}"
123     r"的 144 段骨架上叠加了一个以星期为周期、均值为零的偏移}。这一条决定了"
124     r"后文对照实验的读法：两者年平均均值相同，因此任何费用差异都来自\textbf{波动}"
125     r"本身}，不掺杂价格水平的变化。周五、周六电价系统性偏低，是残差里唯一"
126     r"可利用的规律，也正是同星期加权预测器的依据。",
127     r" \end{minipage}",
128     r"\end{table}",
129     "",
130 ]
131 return "\n".join(lines)
132
133 # ----- 表：预测器选型
134 def tab_select(res: dict) -> str:
135     """表：预测器超参数在 1 月预热期上的选型，及子窗口稳健性复核。
136
137     这张表存在的唯一理由是回应"参数是不是在评价区间上挑的"：它把选型窗口、
138     候选对照与子窗口复核一并摊开，读者可以自行判断选型是否可复现。
139     """
140
141     sel = res.get("预测器选型")
142     if not sel or "候选" not in sel:
143         return ("% 选型表需要预热期选型的结果文件，"
144                 "请先运行：python3 03_代码/p4_microgrid.py --report-only\n")
145     cand = sel["候选"]
146     robust = sel.get("稳健性", [])
147     # 稳健性条目里既有三个候选子窗口、也有一条  $\beta$  重扫。候选子窗口的第一条就是
148     # 全窗口本身，与第一列重复，故丢掉； $\beta$  重扫的结果在 tab_beta 里另有表，
149     # 这里也不列。
150     cols = [r for r in robust
151             if r["最优预测器"] != " ( $\beta$  逐时刻) "
152             and not r["子窗口"].startswith("全窗口")]
153     names = [r["预测器"] for r in cand]
154     lookup = [{c["预测器"]: c["平均绝对误差_元每kWh"] for c in r["前三"]}
155              for r in cols]
156     lines = [
157         r"\begin{table}[htbp]",
158         r" \centering",
159         r" \caption{预测器选型：在 2025 年 1 月预热期（评价区间之外）比较候选，"
160         r"并在子窗口上复核}",
161         r" \label{tab:p4-select}",
162         r" \footnotesize",
163         r" \begin{tabular}{l" + "r" * (1 + len(cols)) + "r}",
164         r" \toprule",
165         r"     候选预测器 & 全窗口 (30 天) "
166         + "".join(f" & {r['子窗口'].split(' ')[0]} ({r['天数']} 天) "
167                  for r in cols) + r" \\",
168         r" \midrule",
169     ]
170     for i, nm in enumerate(names):
171         cell = [f"{cand[i]['平均绝对误差_元每kWh']:.4f}"]
172         for lk in lookup:
173             v = lk.get(nm)

```

```

174         cell.append("----" if v is None else f"{v:.4f}")
175     # 加粗只标全窗口第一名，避免给读者"选型用了多个窗口"的错觉。
176     if i == 0:
177         cell[0] = rf"\textbf{{{cell[0]}}}"
178         nm = rf"\textbf{{{nm}}}"
179     lines.append(f"        {nm} & " + " & ".join(cell) + r" \\\")
180 betas = sel.get("β最优", {})
181 lines += [
182     r"        \midrule",
183     r"        \multicolumn{" + str(2 + len(cols))
184 + r"}{\l}{\textit{选中的一组：权重 "
185 + "/"}.join(f"{w:g}" for w in sel["选中权重"])}
186 + r", }\ $ \beta=( " + r", \, ".join(
187     f"{betas.get(h, '---'):g}" if isinstance(betas.get(h), (int, float))
188     else "----" for h in ("6", "12", "18")) + r")$} \\\",
189     r"        \bottomrule",
190     r"        \end{tabular}",
191     r"        \par\vspace{2pt}",
192     r"        \begin{minipage}{.95\textwidth}\footnotesize",
193     r"        注：单位为元/kWh，为各候选对全天 $144$ 段的平均绝对误差。``全窗口''
194 r"即第 $1$--$30$ 天；``后段''只含滞后 $7/14/21$ 天样本齐备的日子（第
195 r"$22$--$30$ 天），用于排除早期滞后退化对排序的影响；``后半''为第
196 r"$15$--$30$ 天。三个窗口的第一名一致，故选型结果冻结后不再改动。"
197 r"$ \beta$ 逐发布时刻单独选，取值见上表最后一行。",
198     r"        \end{minipage}",
199     r"    \end{table}",
200     "",
201 ]
202 return "\n".join(lines)
203
204 # ----- 表：候选复核
205 def tab_forecast(res: dict) -> str:
206     """表：评价区间上的候选复核（样本外，不参与选型）。"""
207     cand = res["价格预测候选"]
208     frozen_w = res.get("预测器选型", {}).get("选中权重")
209     lines = [
210         r"\begin{table}[htbp]",
211         r"    \centering",
212         r"    \caption{价格预测候选方案在 2025 年 2 月 1 日--12 月 31 日的平均"
213         r"绝对误差 (\textbf{样本外复核}, 不用于选型)}",
214         r"    \label{tab:p4-forecast}",
215         r"    \footnotesize",
216         r"    \begin{tabular}{lrr}",
217         r"        \toprule",
218         r"        预测方案 & 平均绝对误差/(元/kWh) & 相对均价 \\\",
219         r"        \midrule",
220     ]
221     for r in cand:
222         name = r["预测器"]
223         # 加粗的是"本文冻结使用的那一个"，不是"本区间上最好的那一个"。
224         # 两者不同正是这张表想说明的事：按评价区间挑会挑到 0.6/0.3/0.1。
225         frozen = bool(frozen_w) and name.split("(")[0].strip().endswith(
226             "/"}.join(f"{w:g}" for w in frozen_w))
227         nm = rf"\textbf{{{name}}}" if frozen else name
228         lines.append(f"        {nm} & {r['平均绝对误差_元每kWh']:.4f}"
229             f" & {pct(r['相对均价'], 2)} \\\\")
230     lines += [
231         r"        \bottomrule",
232         r"        \end{tabular}",
233         r"        \par\vspace{2pt}",
234         r"        \begin{minipage}{.92\textwidth}\footnotesize",
235         r"        注：``相对均价''以\textbf{评价区间（334 天）的均价} $0.7575$~元/kWh
236         r"为基准，而非全年均价 $0.7662$~元/kWh。``全样本固定日内形状''使用了评价"
237

```

```

238     r"区间本身的均值，含未来信息，只是作为误差下界列出，不是可用策略。``近 "
239     r"$N$ 天滚动均值"系列表现不佳，原因见正文：把不同星期几混在一起平均，"
240     r"恰好抹掉了电价里唯一可利用的星期偏移。\\textbf{本表若作选型依据，第一名"
241     r"会是 $0.6/0.3/0.1$ ($0.0441$)；本文冻结使用的 $0.5/0.3/0.2$ 是 "
242     r"$0.0446$，差 $0.0005$ 元/kWh——这点差异就是``在评价区间上选参"能换来的"
243     r"虚高精度，本文刻意不取。}",
244     r" \end{minipage}",
245     r"\\end{table}",
246     "",
247 ]
248 return "\\n".join(lines)
249
250 # ----- 表：β 扫描
251 def tab_beta(res: dict) -> str:
252     """表：日内修正系数 β 在评价区间上的扫描（样本外，不参与选型）。"""
253     beta = res["β扫描"]
254     sel = res.get("预测器选型", {})
255     frozen = sel.get("β最优", {})
256     keys = ["6点版本_元每kWh", "12点版本_元每kWh", "18点版本_元每kWh"]
257     heads = ["6:00 版", "12:00 版", "18:00 版"]
258     fz = [frozen.get("6"), frozen.get("12"), frozen.get("18")]
259     lines = [
260         r"\\begin{table}[htbp]",
261         r" \centering",
262         r" \caption{日内对数偏差修正系数 $\beta$ 的扫描（对剩余时段的平均绝对"
263         r"误差，评价区间）}",
264         r" \label{tab:p4-beta}",
265         r" \footnotesize",
266         r" \begin{tabular}{lrrr}",
267         r" \toprule",
268         r" $\beta$ & " + " & ".join(heads) + r" \\",
269         r" \midrule",
270     ]
271     for r in beta:
272         b = r["β"]
273         cells = []
274         for k, f in zip(keys, fz):
275             v = r[k]
276             # 加粗"本文冻结使用的 β"那一行，而不是"本区间最优"那一行。
277             cells.append(rf"\\textbf{{{v:.4f}}}" if f is not None
278                         and abs(b - f) < 1e-9 else f"{v:.4f}")
279         lines.append(f" {b:g} & " + " & ".join(cells) + r" \\\n")
280     lines += [
281         r" \bottomrule",
282         r" \end{tabular}",
283         r" \par\vspace{2pt}",
284         r" \begin{minipage}{.92\textwidth}\footnotesize",
285         r" 注：$\beta=0$ 即不做日内修正。本表是\\textbf{样本外复核}：加粗行是"
286         r"在 $1$ 月预热器上选出的冻结值 $\beta=( " + r",\\",
287         r" f"{v:g}" if isinstance(v, (int, float)) else "----" for v in fz)
288         r"$, \\textbf{不是} 本区间的最佳行——按本区间挑会指向 $\beta=0.5$。"
289         r"$\beta$ 在评价区间上更``好看"只是区间自身的偏好，本文不据此改动参数。"
290         r"$\beta\ge1.25$ 时把已实现的偏差过度外推，误差反而回升。",
291         r" \end{minipage}",
292         r"\\end{table}",
293         "",
294     ]
295     return "\\n".join(lines)
296
297 # ----- 表：预测精度
298 def tab_skill4(res: dict) -> str:
299     """表：四个发布版本的预测精度（含共同时段可比口径）。"""
300
301

```

```

302 sk = res["价格预测精度"]
303 keys = ["0点版本", "6点版本", "12点版本", "18点版本"]
304 lines = [
305     r"\begin{table}[htbp]",
306     r" \centering",
307     r" \caption{价格预测的四个发布版本在各自动态时刻更新后的精度}"
308     r" (框架 6.3 节)",
309     r" \label{tab:p4-skill}",
310     r" \footnotesize",
311     r" \begin{tabular}{lrrrr}",
312     r" \toprule",
313     r" 发布版本 & 覆盖时段 & 平均绝对误差 & 相对均价 & 共同时段误差 \\",
314     r" \midrule",
315 ]
316 spans = {"0点版本": "0--144", "6点版本": "36--144",
317          "12点版本": "72--144", "18点版本": "108--144"}
318 best = min(keys, key=lambda k: sk[k]["共同时段平均绝对误差_元每kWh"])
319 for k in keys:
320     v = sk[k]
321     s = f"{v['共同时段平均绝对误差_元每kWh']:.4f}"
322     if k == best:
323         s = rf"\textbf{{{s}}}"
324     lines.append(f" {k} & {spans[k]} & {v['平均绝对误差_元每kWh']:.4f}"
325                 f" & {pct(v['相对均价'], 2)} & {s} \\\\")
326 flat = sk["对照_全天均价"]
327 lines += [
328     r" \midrule",
329     f" 对照：全天用同一个均价 & 全天 & {flat['平均绝对误差_元每kWh']:.4f}"
330     f" & {pct(flat['相对均价'], 2)} & --- \\\\",
331     r" \bottomrule",
332     r" \end{tabular}",
333     r" \par\vspace{2pt}",
334     r" \begin{minipage}{.92\textwidth}\footnotesize",
335     r" 注：`覆盖时段`是时段索引范围，第 $h$ 个版本只能修正它发布之后的"
336     r"时段，因此四个版本的`平均绝对误差`覆盖的时段\textbf{互不相同}"
337     r" ($0{:}00$ 版覆盖全天 $144$ 段，$18{:}00$ 版只覆盖最后 $36$ 段)，"
338     r"\textbf{直接比大小是错的}。最后一列把四个版本都限制在 $18{:}00$--$24{:}00$"
339     r"（四个版本都覆盖该段）上求平均绝对误差，才是可比的版本间排序；加粗者为"
340     r"其中的最优。最后一行的`全天用同一个均价`是没有任何日内形状信息的"
341     r"预测，它的误差远大于四个版本，说明形状信息确有价值。",
342     r" \end{minipage}",
343     r"\end{table}",
344     "",
345 ]
346 return "\n".join(lines)
347
348 # ----- 表：风险口径
349 def tab_risk(res: dict) -> str:
350     """表：价格加权分位数与普通分位数的对照，以及残差—价格的协方差检查。"""
351     rw = res["风险_加权对照"]
352     c = res["风险_价格协方差"]
353     # 「等权·下确界」与「纯加权相对抬升」是分位数定义修正后新增的两列。旧的结果
354     # 文件里没有，硬取会 KeyError；这里给出可执行的提示而不是让生成器崩掉。
355     if rw and "等权下确界均值_kWh" not in rw[0]:
356         return ("% 风险余量对照表需要分位数定义修正后的结果文件，"
357                 "请先运行：python3 03_代码/p4_microgrid.py --report-only\n")
358     lines = [
359         r"\begin{table}[htbp]",
360         r" \centering",
361         r" \caption{风险余量口径对照：以实际电价为权重的分位数 vs 等权分位数}"
362         r" (正式区间按每 7 天抽样求均值)}",
363         r" \label{tab:p4-risk}",
364         r" \footnotesize",
365

```

```

366     r" \begin{tabular}{lrrrrr}",
367     r" \toprule",
368     r" \alpha$ & 价格加权/(kWh) & 等权·线性/(kWh) & 等权·下确界/(kWh)"
369     r" & 比线性插值 & 比下确界 \\",
370     r" \midrule",
371 ]
372 for r in rw:
373     # JSON 里  $\alpha=0.50$  的相对抬升是 NaN (分母为负, 比值无意义)。读到 NaN 时
374     # 必须写成 ``----``——直接格式化会印出 ``+nan%``, 那不是"无意义"的写法,
375     # 而像是一个算错的数。
376     v = r["相对抬升"]
377     rel = "----" if v is None or not math.isfinite(v) else signed_pct(v)
378     q = r.get("纯加权相对抬升")
379     relq = "----" if q is None or not math.isfinite(q) else signed_pct(q)
380     lines.append(f" {r[' $\alpha$ ']:.2f} & {r['价格加权均值_kWh']:.3f}"
381                 f" & {r['普通分位数均值_kWh']:.3f}"
382                 f" & {r['等权下确界均值_kWh']:.3f}"
383                 f" & {rel} & {relq} \\\\"
384 lines += [
385     r" \bottomrule",
386     r" \end{tabular}",
387     r" \par\vspace{2pt}",
388     r" \begin{minipage}{.92\textwidth}\footnotesize",
389     r" 注: ``等权·线性``沿用问题二的做法, 取 \texttt{numpy} 默认的线性插值"
390     r"分位数; ``等权·下确界``与``价格加权``同取下确界定义, 故未列是"
391     r"\textbf{纯粹的加权效应}, 而倒数第二列还混入了取值约定之差。"
392     r"$\alpha=0.50$ 处的余量为负, 因为净负荷预测误差的中位数本身略小于零"
393     r"(日间预测略偏高), 此时``相对抬升``无意义, 故以 ``----`` 表示。",
394     r" \end{minipage}",
395     r" \end{table}",
396     "",
397     r" \begin{table}[htbp]",
398     r" \centering",
399     r" \caption{净负荷预测误差  $\varepsilon$  与实际电价的协方差检查"
400     r"(框架 3.2 节)}",
401     r" \label{tab:p4-cov}",
402     r" \footnotesize",
403     r" \begin{tabular}{lr}",
404     r" \toprule",
405     r" 项目 & 数值 \\",
406     r" \midrule",
407     f" 样本数 & {int(c['样本数']):,} \\\\",
408     f" 合并口径相关系数 & {c['合并相关系数']:+.4f}$ \\\\",
409     f" 逐时段相关系数均值 & {c['逐时段相关系数均值']:+.3f}$ \\\\",
410     f" 逐时段相关系数范围 & "
411     f"${c['逐时段相关系数最小']:+.3f}$~${c['逐时段相关系数最大']:+.3f}$ \\\\",
412     f" 逐时段相关为正的的比例 & {pct(c['逐时段相关为正的的比例'], 1)} \\\\",
413     r" \midrule",
414     f" 电价最高 20\% 时段的  $\varepsilon$  均值"
415     f" & {c['高价20%_误差均值_kW']:+.1f}$~kW \\\\",
416     f" 电价最低 20\% 时段的  $\varepsilon$  均值"
417     f" & {c['低价20%_误差均值_kW']:+.1f}$~kW \\\\",
418     f" 两者之差 & {c['两端误差均值差_kW']:+.1f}$~kW \\\\",
419     r" \bottomrule",
420     r" \end{tabular}",
421     r" \par\vspace{2pt}",
422     r" \begin{minipage}{.92\textwidth}\footnotesize",
423     r" 注: 合并口径把全部 (日期, 时段) 样本混在一起, 被``时段之间``的电价"
424     r"差异 (傍晚贵、凌晨便宜) 主导; 把时段固定住再看, 相关系数由 "
425     f"${c['合并相关系数']:+.3f}$ 升到 ${c['逐时段相关系数均值']:+.3f}$, "
426     rf"且 {pct(c['逐时段相关为正的的比例'], 0)} 的时段为正。这说明"
427     r"$\mathrm{Cov}(P,R)>0$ 是\textbf{同一时段内}的性质, "
428     r"故风险余量必须逐时段计算, 不能对全年做一次汇总——这正是框架 3.4 节"
429     r"按  $\$t\$$  单独求分位数的理由。",

```

```

430         r" \end{minipage}",
431         r"\end{table}",
432         "",
433     ]
434     return "\n".join(lines)
435
436 # ----- 表：两分支汇总
437 def tab_branches(res: dict) -> str:
438     """表：result4-2 与 result4-3 的费用汇总与相互比较。"""
439     a, b = res["4-2 主策略"], res["4-3 主策略"]
440     # (行名, 4-2 的键, 4-3 的键, 小数位): 两个分支的键不一定同名
441     rows = [
442         ("计划量 ($0{:}00$ 定下的量) /kWh", "计划量_kWh", "初始计划量_kWh", 0),
443         ("最终生效常规量/kWh", "计划量_kWh", "最终常规量_kWh", 0),
444         ("上调量/kWh", None, "上调量_kWh", 0),
445         ("下调量/kWh", None, "下调量_kWh", 0),
446         ("紧急购电量/kWh", "紧急购电量_kWh", "紧急购电量_kWh", 0),
447         ("弃电量/kWh", "弃电量_kWh", "弃电量_kWh", 0),
448         ("计划费/元", "计划费_元", "计划费_元", 0),
449         ("调整费/元", None, "调整费_元", 0),
450         ("紧急费/元", "紧急费_元", "紧急费_元", 0),
451         ("合计费用/元", "合计费用_元", "合计费用_元", 0),
452         ("期初储电量/kWh", "期初储电量_kWh", "期初储电量_kWh", 0),
453         ("期末储电量/kWh", "期末储电量_kWh", "期末储电量_kWh", 0),
454     ]
455     lines = [
456         r"\begin{table}[htbp]",
457         r" \centering",
458         r" \caption{两个交付文件在 2025 年 2 月 1 日--12 月 31 日 (334 天) 的"}
459         r"实际运行结果",
460         r" \label{tab:p4-branch}",
461         r" \footnotesize",
462         r" \begin{tabular}{lrr}",
463         r" \toprule",
464         r" 指标 & result4-2 (计划不得修改) & result4-3 (6/12/18 可调整) \\",
465         r" \midrule",
466     ]
467     for name, k42, k43, nd in rows:
468         va = None if k42 is None else a.get(k42)
469         vb = None if k43 is None else b.get(k43)
470         sa = "----" if va is None else fmt(va, nd)
471         sb = "----" if vb is None else fmt(vb, nd)
472         if k42 == "合计费用_元":
473             sa, sb = rf"\textbf{{{sa}}}", rf"\textbf{{{sb}}}"
474         lines.append(f" {name} & {sa} & {sb} \\\\")
475         if k43 == "紧急购电量_kWh":
476             lines.append(r" \midrule")
477     lines += [
478         r" \bottomrule",
479         r" \end{tabular}",
480         r" \par\vspace{2pt}",
481         r" \begin{minipage}{.92\textwidth}\footnotesize",
482         r" 注：两个分支都在\textbf{同一条附件 4 实际价格路径}上独立回放并独立"
483         r"标定，费用一栏不可相加。两者均按\textbf{对应交付时段的实际电价}结算"
484         r" (含初始计划费)。主计费口径取退款：被取消的部分按 $50\%$ 的违约电价"
485         r"计，故 result4-3 全年实际发生下调；result4-2 全天冻结，无任何调整量。",
486         r" \end{minipage}",
487         r"\end{table}",
488         "",
489     ]
490     return "\n".join(lines)
491
492
493

```

```

494 # ----- 表：策略对照
495 def tab_strategies(res: dict) -> str:
496     """表：三种价格信息条件下的策略对照（框架 6.1 节）。"""
497     rows = res.get("策略对照")
498     if not rows:
499         return "% 策略对照尚未计算（需以 --strategies 运行 p4_microgrid.py）\n"
500     lines = [
501         r"\begin{table}[htbp]",
502         r" \centering",
503         r" \caption{三种价格信息条件下的策略在同一实际价格路径上的独立回放",
504         r" (框架 6.1 节) }",
505         r" \label{tab:p4-strategy}",
506         r" \footnotesize",
507         r" \begin{tabular}{llrrrrr}",
508         r" \toprule",
509         r" 分支 & 策略 & 计划费/元 & 调整费/元 & 紧急费/元 & 合计费用/元 \\",
510         r" \midrule",
511     ]
512     for r in rows:
513         if r.get("模式") == "":
514             # ΔJ 行
515             lines.append(f" \\\midrule\n {r['分支']} & \\\textit{{{r['策略']}}}"
516                 f" & --- & --- & ---"
517                 f" & \\\textbf{{{fmt(r['合计费用_元'], 0)}}}"
518                 f" ({signed(float(r['相对降幅']) * 100)}) \\\\"")
519             continue
520             lines.append(
521                 f" {r['分支']} & {r['策略']} & {fmt(r.get('计划费_元', 0), 0)}"
522                 f" & {fmt(r.get('调整费_元', 0), 0)}"
523                 f" & {fmt(r.get('紧急费_元', 0), 0)}"
524                 f" & \\\textbf{{{fmt(r['合计费用_元'], 0)}}} \\\\"")
525     lines += [
526         r" \bottomrule",
527         r" \end{tabular}",
528         r" \par\vspace{2pt}",
529         r" \begin{minipage}{.92\textwidth}\footnotesize",
530         r" 注：三种策略看到的 price 信息不同，但\textbf{都在同一条实际价格路径上",
531         r" 结算}，故费用可直接相减。$\Delta J$ 取`固定电价参考`减`波动电价预测`，",
532         r" 即适应波动电价值多少钱。`未来价格已知参考`假设当天价格在 0:00 已全部",
533         r" 公布，作为\textbf{信息增强上界}单独报告（题目未提供该信息条件，故不作",
534         r" 主策略、不计入 $\Delta J$）。三者各自独立标定，用各自信息集下的历史。",
535         r" \end{minipage}",
536         r"\end{table}",
537         "",
538     ]
539     return "\n".join(lines)
540
541 # ----- 表：组合比较
542 def tab_combos(res: dict) -> str:
543     """表：波动电价下 4-3 的八种预报组合（框架 6.2 节）。"""
544     rows = res.get("预报组合")
545     if not rows:
546         return "% 预报组合尚未计算（需以 --sweep 运行 p4_microgrid.py）\n"
547     base = next((r["合计费用_元"] for r in rows if r["使用预报"] == "0"), None)
548     lines = [
549         r"\begin{table}[htbp]",
550         r" \centering",
551         r" \caption{波动电价下 4-3 的八种预报使用组合（各组合单独标定参数，",
552         r" 评价区间与计费口径完全一致) }",
553         r" \label{tab:p4-combo}",
554         r" \footnotesize",
555         r" \begin{tabular}{lrrrrrrr}",
556         r" \toprule",
557

```

```

558     r"      使用的预报时刻 & 初始计划费/元 & 调整费/元 & 紧急费/元 "
559     r"& 合计费用/元 & 相对 $\varnothing$ & 期末储电量/kWh \\",
560     r"      \midrule",
561 ]
562 for r in sorted(rows, key=lambda x: x["合计费用_元"]):
563     tot = r["合计费用_元"]
564     rel = "----" if base is None or r["使用预报"] == "0" else \
565         signed((tot - base) / base * 100)
566     name = "仅 0:00" if r["使用预报"] == "0" else r["使用预报"]
567     lines.append(
568         f"      {name} & {fmt(r['计划费_元'], 0)}"
569         f" & {fmt(r['调整费_元'], 0)} & {fmt(r['紧急费_元'], 0)}"
570         f" & \\\textbf{{{fmt(tot, 0)}}} & {rel}"
571         f" & {fmt(r['期末储电量_kWh'], 1)} \\\\"
572     lines += [
573         r"      \bottomrule",
574         r"      \end{tabular}",
575         r"      \par\vspace{2pt}",
576         r"      \begin{minipage}{.92\textwidth}\footnotesize",
577         r"      注：与问题三的一张表对照可以看出价格波动改变了什么——在固定电价下"
578         r"      用不用新预报'主要体现在调整费与紧急费的此消彼长；在波动电价下，"
579         r"      更新预报同时改善了\textbf{电量}与\textbf{价格}两项判断（6:00 起既能看到"
580         r"      当天更准的负载与光伏，也能用当天已实现的价格修正剩余时段的电价预测），"
581         r"      因此增量收益的来源更宽。所有方案都使用 0:00 发布的预报，组合列出的时刻"
582         r"      是额外使用的版本。",
583         r"      \end{minipage}",
584         r"\end{table}",
585         ""
586     ]
587     return "\n".join(lines)
588
589
590 def tab_combo_detail(res: dict) -> str:
591     """表：4-3 八种预报组合在 12 条覆盖边上的边际价值。
592
593     原先这张表是手抄进 moved_body.tex 的，重算后数字对不上（正是它导致了
594     PDF 里的一批过时数字），故改为与其余各表同源生成。
595     """
596     rows = res.get("预报组合")
597     if not rows:
598         return "% 预报组合尚未计算（需以 --sweep 运行 p4_microgrid.py）\n"
599     cost = {}
600     for r in rows:
601         key = frozenset() if r["使用预报"] == "0" else \
602             frozenset(int(t) for t in
603                 r["使用预报"].strip("{}").split(",") if t.strip())
604         cost[key] = r["合计费用_元"]
605     # 12 条覆盖边：从 0 逐层向上，每个集合再补一个尚未加入的时刻。
606     edges = []
607     for s in sorted(cost, key=lambda k: (len(k), sorted(k))):
608         for h in (6, 12, 18):
609             if h in s or (s | {h}) not in cost:
610                 continue
611             edges.append((s, h, cost[s] - cost[s | {h}]))
612     # 按起点集合分组（同一起点的几条边相邻），与原表的阅读顺序一致。
613     edges.sort(key=lambda e: (len(e[0]), sorted(e[0]), e[1]))
614     lo = min(range(len(edges)), key=lambda i: edges[i][2])
615
616     def name(s) -> str:
617         r"""集合列。空集用 \varnothing，其余写成 $\{6,18\}$ 的紧凑形式：
618         CJK 标点进数学模式在 xelatex 下会排版异常，故用半角逗号。"""
619         return r"$\varnothing$" if not s else \
620             r"${{" + ",".join(str(h) for h in sorted(s)) + "}}$"
621

```

```

622 lines = [
623     r"\begin{table}[htbp]",
624     r" \centering",
625     r" \caption{4-3 八种预报组合的边际价值（波动电价，各组合独立标定；"
626     r"单位：元）}",
627     r" \label{tab:p4-combo-detail}",
628     r" \footnotesize",
629     r" \begin{tabular}{llrr}",
630     r" \toprule",
631     r" 已有预报 $$ & 再加一版 $$ & 增量 $\Delta_h(S)$ & 折合 \\",
632     r" \midrule",
633 ]
634 prev = None
635 for i, (s, h, d) in enumerate(edges):
636     if prev is not None and len(s) != len(prev):
637         lines.append(r" \midrule")
638     prev = s
639     if i == lo:
640         tail_cells = (r"\textbf{%s} & \textbf{%s 万元}"
641                      % (fmt(d, 0), fmt(d / 1e4, 2)))
642     else:
643         tail_cells = f"{fmt(d, 0)} & {fmt(d / 1e4, 2)} 万元"
644     lines.append(f" {name(s)} & ${h}{:}{00}$ & {tail_cells} \\\\")
645 lines += [
646     r" \bottomrule",
647     r" \end{tabular}",
648     r" \par\vspace{2pt}",
649     r" \begin{minipage}{.92\textwidth}\footnotesize",
650     r" 注：$\Delta_h(S)=J(S)-J(S\cup\{h\})$ 为在已有集合 $$ 之上再订一版"
651     r" $$ 所省下的费用。所有组合都在同一条附件 4 实际价格路径上结算，且各自"
652     r"重新标定 $\alpha, \rho, \lambda$，因此增量可以横向比较。加粗行是"
653     r"\textbf{边际价值最低}的一条（已有 $\{6{:}00, 18{:}00\}$ 时再加"
654     r"$12{:}00$）；主计费口径下全部 $12$ 条边的增量\textbf{均为正}，"
655     r"不存在\textbf{多一版预报反而有害}的情形——该辨析及逐边的费用构成"
656     r"规律见附录该节说明。",
657     r" \end{minipage}",
658     r"\end{table}",
659     "",
660 ]
661 return "\n".join(lines)
662
663 # ----- 表：校验
664 def tab_check(res: dict) -> str:
665     """表：问题四各项强制检查的实测最坏值与未通过天数。"""
666     keys42 = ["信息边界", "计划冻结", "物理可行性", "状态累计", "费用一致性"]
667     keys43 = ["信息边界", "调整权限", "物理可行性", "状态累计", "费用一致性"]
668     f42, f43 = res["4-2 校验失败天数"], res["4-3 校验失败天数"]
669     v42, v43 = res["4-2 校验数值口径"], res["4-3 校验数值口径"]
670     lines = [
671         r"\begin{table}[htbp]",
672         r" \centering",
673         r" \caption{两个分支在 334 个交付日上的检查结果（未通过天数均为零）}",
674         r" \label{tab:p4-check}",
675         r" \footnotesize",
676         r" \begin{tabular}{lrr}",
677         r" \toprule",
678         r" 检查项 & result4-2 未通过天数 & result4-3 未通过天数 \\",
679         r" \midrule",
680     ]
681     for k in keys42:
682         lines.append(f" {k} & {f42[k]} & {f43.get(k, 0)} \\\\")
683     for k in keys43:
684         if k not in keys42:
685             lines.append(f" {k} & {f42.get(k, 0)} & {f43[k]} \\\\")

```

```

686         lines.append(f"      {k} & {f42.get(k, 0)} & {f43[k]} \\\\")
687     lines += [
688         r"      \midrule",
689         r"      \multicolumn{3}{l}{\textit{数值口径的最坏值}} \\",
690         r"      \midrule",
691         f"      逐段电量平衡最大残差/kWh & ${sci(v42['逐段平衡最大残差_kWh'])}$"
692         f" & ${sci(v43['逐段平衡最大残差_kWh'])}$ \\\\",
693         f"      计划费重算最大偏差/元 & ${sci(v42['计划费重算最大偏差_元'])}$"
694         f" & ${sci(v43['计划费重算最大偏差_元'])}$ \\\\",
695         f"      调整费重算最大偏差/元 & --- "
696         f"& ${sci(v43['调整费重算最大偏差_元'])}$ \\\\",
697         f"      紧急费重算最大偏差/元 & ${sci(v42['紧急费重算最大偏差_元'])}$"
698         f" & ${sci(v43['紧急费重算最大偏差_元'])}$ \\\\",
699         f"      跨日衔接最大偏差/kWh & ${sci(v42['跨日衔接最大偏差_kWh'])}$"
700         f" & ${sci(v43['跨日衔接最大偏差_kWh'])}$ \\\\",
701         f"      全局储电量范围/kWh & "
702         f"{fmt(v42['全局最小储电量_kWh'], 1)}~{fmt(v42['全局最大储电量_kWh'], 1)}"
703         f" & {fmt(v43['全局最小储电量_kWh'], 1)}~"
704         f"{fmt(v43['全局最大储电量_kWh'], 1)} \\\\",
705         r"      \midrule",
706         f"      储电下界余量/kWh & {fmt(v42['储电下界余量_kWh'], 3)}"
707         f" & {fmt(v43['储电下界余量_kWh'], 3)} \\\\",
708         f"      储电上界余量/kWh & {fmt(v42['储电上界余量_kWh'], 3)}"
709         f" & {fmt(v43['储电上界余量_kWh'], 3)} \\\\",
710         r"      \bottomrule",
711         r"      \end{tabular}",
712         r"      \par\vspace{2pt}",
713         r"      \begin{minipage}{.92\textwidth}\footnotesize",
714         r"      注：残差均为浮点误差量级 ( $<10^{-12}$  kWh)，说明约束是被精确满足"
715         r"而非近似满足。储电量上下界余量恰为零，说明日末储备与容量约束在个别"
716         r"日期是\textbf{紧的}——这正是风险余量与储备线起作用的地方，不是缺陷。"
717         r"``计划冻结''在 result4-2 下的可核验内容是逐段电量平衡：执行层唯一的外部"
718         r"常规购电来源是 0:00 定下的计划量，日内不存在第二次常规购电。",
719         r"      \end{minipage}",
720         r"    \end{table}",
721         "",
722     ]
723     return "\n".join(lines)
724
725 # ----- 表：不退款口径敏感性 (4-3, 附录)
726 def tab_refund(res: dict) -> str:
727     """表：4-3 在主计费口径（退款）与不退款口径下的对照。
728
729     主口径取题面的字面读法（被取消部分按 50% 违约电价计，调减每度 0.5p）；
730     "不退款、只追加"是另一种更保守的读法，作为独立敏感性报告。两种口径下模型
731     与执行规则完全相同，只有费用函数不同，因此这张表回答的是"主结论对这条题面
732     歧义有多敏感"，而不是给出第二个答案。
733     """
734     main, no_ref = res.get("4-3 主策略"), res.get("4-3 不退款口径")
735     if not main or not no_ref:
736         return ("% 不退款口径未计算（需先由 --refund-main 换口径）\n")
737     rows = [
738         ("合计费用", "合计费用_元", 2),
739         ("其中 计划费", "计划费_元", 2),
740         ("其中 调整费", "调整费_元", 2),
741         ("其中 紧急费", "紧急费_元", 2),
742         ("计划量（初始）", "初始计划量_kWh", 2),
743         ("最终常规量", "最终常规量_kWh", 2),
744         ("上调量", "上调量_kWh", 2),
745         ("下调量", "下调量_kWh", 2),
746         ("紧急购电量", "紧急购电量_kWh", 2),
747         ("弃电量", "弃电量_kWh", 2),
748         ("期末储电量", "期末储电量_kWh", 2),
749     ]

```

```

750     ("调整提交次数", "调整提交次数", 0),
751 ]
752 lines = [
753     r"\begin{table}[htbp]",
754     r" \centering",
755     r" \caption{4-3 在主计费口径（退款）与不退款口径下的对照（两口径各自独立"
756     r"标定并回放，模型与执行规则完全相同，只换费用函数与可行域）}",
757     r" \label{tab:p4-refund}",
758     r" \footnotesize",
759     r" \begin{tabular}{lrrr}",
760     r" \toprule",
761     r" 指标 & 主口径（退款） & 不退款口径 & 差异（不退款  $\$-\$$  主口径） \\",
762     r" \midrule",
763 ]
764 for name, key, nd in rows:
765     a, b = main.get(key), no_ref.get(key)
766     if a is None or b is None:
767         continue
768     d = b - a
769     dstr = "----" if abs(d) < 5e-7 * max(1.0, abs(a)) else \
770         f"{d:+,.{nd}f}"
771     lines.append(f" {name} & {fmt(a, nd)} & {fmt(b, nd)} & {dstr} \\\\"")
772 lines += [
773     r" \bottomrule",
774     r" \end{tabular}",
775     r" \par\vspace{2pt}",
776     r" \begin{minipage}{.92\textwidth}\footnotesize",
777     r" 注：两种口径的差别不只是一个系数。主口径下调减每度只花  $\$0.5p\$$ ，",
778     r" 是一条真实可用的通道，全年下调量非零；不退款口径把计划费理解为",
779     r" 无条件全额、 $\$50\%$  只作\textbf{额外}违约费，此时调减每度净付",
780     r"  $\$1.5p\$$ 、最优解不会真正调减， $\$x_t \geq g_t^0\$$  成为可以外加的简化约束，",
781     r" \textbf{可行域本身变窄}。因此两条臂的\textbf{标定结果与最终计划都}",
782     r" 不同，表中的差异同时包含参数差异与规则差异，不能读作同一个解的两种",
783     r" 计价。这里关心的是\textbf{ $\$4\$-\$3\$$  相对  $\$4\$-\$2\$$  的方向是否依然成立}：",
784     r" 两种口径下  $\$4\$-\$3\$$  都低于  $\$4\$-\$2\$$ ，主结论不依赖这条题面歧义。",
785     r" \end{minipage}",
786     r"\end{table}",
787     "",
788 ]
789 return "\n".join(lines)
790
791
792 HEADER = [
793     "% 本文件由 03_代码/p4_tables.py 自动生成，请勿手工修改。",
794     "% 数据源：06_支撑材料/p4_results.json",
795     "",
796 ]
797
798
799 def main() -> None:
800     with open(SUP / "p4_results.json", encoding="utf-8") as fh:
801         res = json.load(fh)
802         # 正文只留两张最必需的交付表：
803         # tab_branches 两个交付文件的全年结果——题目明确要求的两份文件，必须在正文；
804         # tab_strategy 三种价格信息条件的策略对照—— $\Delta J$  的全部数字，正文逐条讨论。
805         # 其余全部入附录。正文只有 30 页额度，把策略对照表放附录也有内容上的理由：
806         # 它与同名的费用分解图（附录图）成对阅读更自然，正文该处已给出全部关键数字。
807         body = HEADER + [
808             tab_branches(res),
809         ]
810         tail = HEADER + [
811             tab_price_struct(res),
812             tab_strategies(res),
813             # 选型表紧挨候选复核表：前者回答"参数怎么定的"，后者回答"定下来之后

```

```

814     # 在评价区间上表现如何", 两张表连读才看得出选型与评价已经分开。
815     tab_select(res),
816     tab_forecast(res),
817     tab_beta(res),
818     tab_skill4(res),
819     tab_risk(res),
820     tab_combos(res),
821     tab_check(res),
822 ]
823 if "--split" in sys.argv:
824     OUT_TEX.write_text("\n".join(body), encoding="utf-8")
825     OUT_TEX_TAIL.write_text("\n".join(tail), encoding="utf-8")
826     OUT_TEX_REFUND.write_text(
827         HEADER[0] + "\n" + HEADER[1] + "\n\n" + tab_refund(res),
828         encoding="utf-8")
829     OUT_TEX_COMBO.write_text(
830         HEADER[0] + "\n" + HEADER[1] + "\n\n" + tab_combo_detail(res),
831         encoding="utf-8")
832     print(f"写入 {OUT_TEX} (正文: 两分支汇总 + 策略对照)")
833     print(f"写入 {OUT_TEX_TAIL} (附录: 价格结构 + 预测选型 + 风险口径"
834           f" + 组合比较 + 校验)")
835     print(f"写入 {OUT_TEX_REFUND} (附录: 不退款口径敏感性)")
836     print(f"写入 {OUT_TEX_COMBO} (附录: 八种组合的边际价值明细表)")
837     return
838     OUT_TEX.write_text("\n".join(body + tail), encoding="utf-8")
839     print(f"写入 {OUT_TEX}")
840
841
842 if __name__ == "__main__":
843     main()

```

(3) p4_figures.py: 插图生成

```

1  # -*- coding: utf-8 -*-
2  r"""
3  2026 CUMCM C 题 问题四 —— 论文插图生成
4  输出到 04_图/, 300 dpi, 供 LaTeX 直接 \includegraphics 引用。
5
6  六张图对应框架第八·九节:
7  图 1 附件 4 电价的结构分解  $P =$  附件 1 曲线 + 星期偏移
8  图 2 价格预测: 0:00 与 6:00 两个版本对同一天的实际电价
9  图 3 三种价格信息条件下的策略费用分解与  $\Delta J$ 
10 图 4 4-2 的冻结计划与 4-3 的初始/最终购电曲线
11图 5 两个分支的实际储电量与参考轨迹
12图 6 八种预报组合的费用分解
13图 7 逐日费用与累计差额
14
15数据来源 (先运行 03_代码/p4_microgrid.py 生成):
16 06_支撑材料/p4_results.json
17 06_支撑材料/p4_detail_42.csv
18 06_支撑材料/p4_detail_43.csv
19 运行: python3 03_代码/p4_figures.py
20 """
21
22 from __future__ import annotations
23
24 import datetime
25 import json
26 import sys
27 from pathlib import Path
28
29 import numpy as np

```

```

30 import pandas as pd
31 import matplotlib.pyplot as plt
32
33 sys.path.insert(0, str(Path(__file__).resolve().parent))
34 from cumcm_plot import savefig, setup_plot # noqa: E402
35 from p1_microgrid import N, E_MIN, E_MAX, TAU # noqa: E402
36 from p4_microgrid import ( # noqa: E402
37     PriceForecaster, load_attach4, load_attach1,
38     select_forecast_params, REPORT_START,
39 )
40
41 setup_plot()
42
43 ROOT = Path(__file__).resolve().parents[1]
44 SUP = ROOT / "06_支撑材料"
45 OUT = ROOT / "04_图"
46
47 C_FIXED = "#7f8c8d"
48 C_PRICE = "#c0392b"
49 C_Fcost = "#2980b9"
50 C_FCST6 = "#16a085"
51 C_ACT = "#c0392b"
52 C_G0 = "#2980b9"
53 C_X = "#c0392b"
54 C_EMG = "#e67e22"
55 C_SOC42 = "#8e44ad"
56 C_SOC43 = "#16a085"
57 C_REF = "#7f8c8d"
58 C_PLANFEE = "#34495e"
59 C_ADJFEE = "#e67e22"
60 C_EMGFEE = "#c0392b"
61
62 SPECIAL = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
63 SPECIAL_TITLE = {"2025-03-20": "2025-03-20 (春分)",
64                  "2025-06-21": "2025-06-21 (夏至)",
65                  "2025-09-23": "2025-09-23 (秋分)",
66                  "2025-12-21": "2025-12-21 (冬至)"}
67 WD = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
68
69
70 def hours() -> np.ndarray:
71     return np.arange(N) * 10 / 60
72
73
74 def load_inputs():
75     res = json.loads((SUP / "p4_results.json").read_text(encoding="utf-8"))
76     det42 = pd.read_csv(SUP / "p4_detail_42.csv", encoding="utf-8-sig")
77     det43 = pd.read_csv(SUP / "p4_detail_43.csv", encoding="utf-8-sig")
78     return res, det42, det43
79
80
81 def panels(ncol: int = 2):
82     fig, axes = plt.subplots(2, ncol, figsize=(11.2, 4.8))
83     return fig, np.asarray(axes).ravel()
84
85
86 def _day(det: pd.DataFrame, d: str) -> pd.DataFrame:
87     return det[det["日期"] == d]
88
89
90 # ----- 图 1
91 def fig1_structure() -> None:
92     """图 1: 左——一周的实际电价与附件 1 曲线; 右——残差的星期均值。"""
93     PRICE = load_attach4()

```

```

94 FIXED = load_attach1()["电价"].to_numpy(float)
95 R = PRICE - FIXED
96 d0 = datetime.date(2025, 1, 1) # 0=周一 ... 6=周日
97 wd = np.array([(d0 + datetime.timedelta(days=n)).weekday()
98                 for n in range(PRICE.shape[0])])
99
100 t = hours()
101 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.2, 3.3),
102                                gridspec_kw={"width_ratios": [2.0, 1.0]})
103
104 n0 = 60 # 2025-03-02 起一周
105 for k in range(7):
106     n = n0 + k
107     lo = wd[n] in (4, 5)
108     ax1.plot(t + 24 * k, PRICE[n], color=C_PRICE if lo else C_Fcost,
109             lw=1.1, alpha=0.9 if lo else 0.55, zorder=4 if lo else 3)
110 for k in range(7):
111     ax1.plot(t + 24 * k, FIXED, color="k", lw=1.0, ls="--", alpha=0.75,
112             zorder=5)
113 ax1.plot([], [], color=C_PRICE, lw=1.4, label="附件 4 实际电价 (周五/周六)")
114 ax1.plot([], [], color=C_Fcost, lw=1.4, label="附件 4 实际电价 (其余曜日)")
115 ax1.plot([], [], color="k", lw=1.0, ls="--", label="附件 1 固定电价曲线 $F$")
116 ax1.set_xlim(0, 24 * 7)
117 ax1.set_xticks(np.arange(0, 24 * 7 + 1, 24))
118 ax1.set_xticklabels(["周一", "周二", "周三", "周四", "周五", "周六",
119                     "周日", "次周一"], fontsize=8)
120 ax1.set_xlabel("日期 (2025-03-03 起一周)")
121 ax1.set_ylabel("电价 / (元/kWh)")
122 ax1.set_title("实际电价 = 固定曲线 + 星期偏移", fontsize=11)
123 ax1.legend(fontsize=8, loc="upper left")
124
125 means = [float(R[wd == w].mean()) for w in range(7)]
126 cols = [C_PRICE if w in (4, 5) else C_Fcost for w in range(7)]
127 ax2.bar(np.arange(7), means, 0.62, color=cols)
128 for i, v in enumerate(means):
129     ax2.annotate(f"{v:+.3f}", (i, v), ha="center",
130                 va="bottom" if v > 0 else "top", fontsize=8,
131                 xytext=(0, 3 if v > 0 else -3),
132                 textcoords="offset points")
133 ax2.axhline(0, color="k", lw=0.8)
134 hi = np.mean([means[k] for k in (0, 1, 2, 3, 6)])
135 lo_ = (means[4] + means[5]) / 2
136 ax2.axhline(hi, color=C_Fcost, lw=1.0, ls=":")
137 ax2.axhline(lo_, color=C_PRICE, lw=1.0, ls=":")
138 ax2.annotate(f"差 {hi - lo_:.3f} 元/kWh", (5.5, (hi + lo_) / 2),
139             fontsize=8, ha="right", color="k",
140             bbox=dict(fc="white", ec="gray", lw=0.5, alpha=0.85))
141 ax2.set_xticks(np.arange(7))
142 ax2.set_xticklabels(WD, fontsize=8)
143 ax2.set_ylabel("残差 $R$ 的星期均值 / (元/kWh)")
144 ax2.set_title("残差的星期结构", fontsize=11)
145 fig.suptitle("附件 4 实时电价并非另一条曲线, 而是附件 1 曲线叠加星期偏移",
146             fontsize=12, y=1.02)
147 fig.tight_layout()
148 savefig("p4_fig1_price_structure.png")

```

----- 图 2

```
def fig2_forecast() -> None:
```

```
"""图 2: 两个发布版本的价格预测与当天实际电价 (选四个特征日)。
```

```
预测器参数与正式计算完全一致: 只在 1 月预热期 (评价区间之外) 选一次、
冻结后全程复用, 见 p4_microgrid.select_forecast_params。
```

```
"""
```

```

158 PRICE = load_attach4()
159 pp, _ = select_forecast_params(PRICE, 1, REPORT_START)
160 pf = PriceForecaster(PRICE, pp)
161 idx = {str(np.datetime64("2025-01-01") + np.timedelta64(n, "D")): n
162         for n in range(PRICE.shape[0])}
163 t = hours()
164
165 fig, axes = panels()
166 for ax, d in zip(axes, SPECIAL):
167     n = idx[d]
168     act = PRICE[n]
169     q0 = pf.forecast0(n)
170     # hi 是发布序号 (0/1/2/3 ↔ 0:00/6:00/12:00/18:00), 不是小时数:
171     # 传 6 会让 t0 = 36*6 = 216 (即 15:00), 由图从第 36 时段画起就自相矛盾。
172     q6 = pf.forecast_at(n, 1)
173     ax.plot(t, act, color=C_ACT, lw=2.0, label="实际电价", zorder=6)
174     ax.plot(t, q0, color=C_FCOST, lw=1.2, ls="--", label="0:00 预测")
175     ax.plot(t[36:], q6[36:], color=C_FCST6, lw=1.4, ls="-.",
176            label="6:00 修正后预测")
177     ax.fill_between(t[36:], q0[36:], q6[36:], where=q0[36:] > q6[36:],
178                    color=C_FCST6, alpha=0.18, step="mid")
179     ax.set_xlim(0, 24)
180     ax.set_xticks(range(0, 25, 6))
181     ax.set_xlabel("时刻 / h")
182     ax.set_ylabel("电价 / (元/kWh)")
183     ax.set_title(SPECIAL_TITLE[d], fontsize=11)
184 h, lab = axes[0].get_legend_handles_labels()
185 fig.legend(h, lab, fontsize=9, ncol=3, loc="lower center",
186           bbox_to_anchor=(0.5, -0.035), frameon=False)
187 fig.suptitle("价格预测与实际电价 (6:00 起用当日已实现价格修正剩余时段)",
188            fontsize=12, y=1.0)
189 fig.tight_layout(rect=(0, 0.045, 1, 1))
190 savefig("p4_fig2_forecast.png")
191
192 # ----- 图 3
193 def fig3_strategy(res: dict) -> None:
194     """图 3: 三种价格信息条件下的策略费用分解 (两分支并列) 与 ΔJ。"""
195     # 用 .get: 只跑了 --report-only 或没跑 --strategies 时这个键不存在,
196     # 此时应当安静跳过, 而不是让整张图的生成中断
197     rows = [r for r in res.get("策略对照", []) if r.get("模式")]
198     if not rows:
199         print(" 跳过图 3: 策略对照尚未计算")
200         return
201     order = ["forecast", "fixed", "oracle"]
202     label = {r["模式"]: r["策略"] for r in rows}
203
204     fig, ax = plt.subplots(figsize=(9.6, 3.35))
205     xs, plan, adj, emg, names = [], [], [], [], []
206     pos = 0.0
207     for br in ("4-2", "4-3"):
208         for m in order:
209             r = next((q for q in rows if q["分支"] == br and q["模式"] == m), None)
210             if r is None:
211                 continue
212             xs.append(pos)
213             plan.append(r["计划费_元"] / 1e4)
214             adj.append(r["调整费_元"] / 1e4)
215             emg.append(r["紧急费_元"] / 1e4)
216             names.append(f"{br}\n{label[m]}")
217             pos += 1
218         pos += 0.6
219     plan, adj, emg = np.array(plan), np.array(adj), np.array(emg)
220     tot = plan + adj + emg
221

```

```

222 ax.bar(xs, plan, 0.6, color=C_PLANFEE, label="计划购电费")
223 ax.bar(xs, adj, 0.6, bottom=plan, color=C_ADJFEE, label="调整费")
224 ax.bar(xs, emg, 0.6, bottom=plan + adj, color=C_EMGFEE, label="紧急购电费")
225 for x, v in zip(xs, tot):
226     ax.annotate(f"{v:,.1f}", (x, v), ha="center", va="bottom", fontsize=8,
227                xytext=(0, 2), textcoords="offset points")
228 ax.set_xticks(xs)
229 ax.set_xticklabels(names, fontsize=8)
230 ax.set_ylabel("费用 / 万元")
231 ax.axvline(2.8, color="gray", lw=0.8, ls="--")
232 ax.set_title("三种价格信息条件下的策略：同一实际价格路径，各自独立回放",
233             fontsize=11)
234 ax.legend(fontsize=9)
235 fig.tight_layout()
236 savefig("p4_fig3_strategy.png")
237
238 out = pd.DataFrame({
239     "分支": [b for b in ("4-2", "4-3") for _ in order],
240     "策略": [label[m] for _ in ("4-2", "4-3") for m in order],
241     "计划费_万元": plan, "调整费_万元": adj, "紧急费_万元": emg,
242     "合计_万元": tot,
243 })
244 out.to_csv(SUP / "p4_strategy_fees.csv", index=False, encoding="utf-8-sig")
245 print(f" 另存 {SUP / 'p4_strategy_fees.csv'}")
246
247
248 # ----- 图 4
249 def fig4_purchase(det42: pd.DataFrame, det43: pd.DataFrame) -> None:
250     """图 4: 4-2 的冻结计划与 4-3 的初始/最终购电，叠加紧急补购。"""
251     t = hours()
252     fig, axes = panels()
253     for ax, d in zip(axes, SPECIAL):
254         a = _day(det42, d)
255         b = _day(det43, d)
256         g42 = a["计划购电_kWh"].to_numpy()
257         e42 = a["紧急购电_kWh"].to_numpy()
258         g0 = b["初始计划购电_kWh"].to_numpy()
259         x = b["最终购电_kWh"].to_numpy()
260         e43 = b["紧急购电_kWh"].to_numpy()
261         ax.plot(t, g42, color=C_FIXED, lw=1.5, ls=":", label="4-2 计划 $g_t$")
262         ax.plot(t, g0, color=C_G0, lw=1.3, ls="--", label="4-3 初始 $g^0_t$")
263         ax.plot(t, x, color=C_X, lw=1.5, label="4-3 最终 $x_t$")
264         up = x > g0 + 1e-9
265         if up.any():
266             ax.fill_between(t, g0, x, where=up, color=C_X, alpha=0.22,
267                             step="mid")
268         if (e43 > 1e-6).any():
269             ax.fill_between(t, 0, e43, where=e43 > 1e-6, color=C_EMG,
270                             alpha=0.5, step="mid", label="4-3 紧急购电 $r_t$")
271         if (e42 > 1e-6).any():
272             ax.fill_between(t, 0, -e42, where=e42 > 1e-6, color=C_FIXED,
273                             alpha=0.5, step="mid", label="4-2 紧急购电 $r_t$")
274         ax.axhline(0, color="k", lw=0.7)
275         ax.set_xlim(0, 24)
276         ax.set_xticks(range(0, 25, 6))
277         ax.set_xlabel("时刻 / h")
278         ax.set_ylabel("十分钟电量 / kWh")
279         ax.set_title(SPECIAL_TITLE[d], fontsize=11)
280     axes[0].legend(fontsize=7.5, loc="upper right", ncol=1)
281     fig.suptitle("4-2 的冻结计划与 4-3 的初始/最终购电 (6/12/18 时允许调整)",
282                 fontsize=12, y=1.0)
283     fig.tight_layout()
284     savefig("p4_fig4_purchase.png")
285

```

```

286
287 # ----- 图 5
288 def fig5_storage(det42: pd.DataFrame, det43: pd.DataFrame) -> None:
289     """图 5: 两个分支的实际储电量与各自的参考轨迹。"""
290     t = hours()
291     fig, axes = panels()
292     for ax, d in zip(axes, SPECIAL):
293         a = _day(det42, d)
294         b = _day(det43, d)
295         E42 = a["储电量_kWh"].to_numpy()
296         E43 = b["储电量_kWh"].to_numpy()
297         ref = b["参考储电量_kWh"].to_numpy()
298         ax.plot(t, ref, color=C_REF, lw=1.0, ls="--", label="参考轨迹 $\bar{E}_t$")
299         ax.plot(t, E42, color=C_SOC42, lw=1.5, label="4-2 储电量 $E_t$")
300         ax.plot(t, E43, color=C_SOC43, lw=1.5, label="4-3 储电量 $E_t$")
301         ax.axhline(E_MIN, color="gray", lw=0.8, ls=":")
302         ax.axhline(E_MAX, color="gray", lw=0.8, ls=":")
303         for h in (0, 6, 12, 18):
304             ax.axvline(h, color="k", lw=0.6, ls=":", alpha=0.45)
305         ax.set_xlim(0, 24)
306         ax.set_ylim(E_MIN - 250, E_MAX + 250)
307         ax.set_xticks(range(0, 25, 6))
308         ax.set_xlabel("时刻 / h")
309         ax.set_ylabel("储电量 / kWh")
310         ax.set_title(SPECIAL_TITLE[d], fontsize=11)
311     axes[0].legend(fontsize=8, loc="lower left")
312     fig.suptitle("两个分支的实际储电量 (竖虚线为 0/6/12/18 时决策点)",
313                 fontsize=12, y=1.0)
314     fig.tight_layout()
315     savefig("p4_fig5_storage.png")
316
317
318 # ----- 图 6
319 def fig6_combos(res: dict) -> None:
320     """图 6: 八种预报组合的费用分解与总额。"""
321     rows = res.get("预报组合")
322     if not rows:
323         print(" 跳过图 6: 预报组合尚未计算")
324         return
325     rows = sorted(rows, key=lambda r: r["合计费用_元"])
326     labels = ["仅 0:00" if r["使用预报"] == "0" else r["使用预报"] for r in rows]
327     plan = np.array([r["计划费_元"] for r in rows]) / 1e4
328     adj = np.array([r["调整费_元"] for r in rows]) / 1e4
329     emg = np.array([r["紧急费_元"] for r in rows]) / 1e4
330     tot = plan + adj + emg
331
332     fig, ax = plt.subplots(figsize=(10.0, 3.4))
333     xp = np.arange(len(labels))
334     ax.bar(xp, plan, 0.62, color=C_PLANFEE, label="计划购电费")
335     ax.bar(xp, adj, 0.62, bottom=plan, color=C_ADJFEE, label="调整费")
336     ax.bar(xp, emg, 0.62, bottom=plan + adj, color=C_EMGFEE, label="紧急购电费")
337     for i, v in enumerate(tot):
338         ax.annotate(f"{v:,.1f}", (i, v), ha="center", va="bottom", fontsize=8,
339                    xytext=(0, 2), textcoords="offset points")
340     ax.set_xticks(xp)
341     ax.set_xticklabels(labels, rotation=20, ha="right")
342     ax.set_ylabel("费用 / 万元")
343     ax.set_title("波动电价下 4-3 的八种预报组合 (各组合单独标定)", fontsize=11)
344     ax.legend(fontsize=9)
345     fig.tight_layout()
346     savefig("p4_fig6_combos.png")
347
348
349 # ----- 图 7

```

```

350 def fig7_daily(d42: pd.DataFrame, d43: pd.DataFrame) -> None:
351     """图 7: 逐日费用与累计差额, 看收益的月份分布。"""
352     x = pd.to_datetime(d42["日期"])
353     c42 = d42["合计费用_元"].to_numpy(float)
354     c43 = d43["合计费用_元"].to_numpy(float)
355     cum = np.cumsum(c42 - c43)
356
357     fig, axes = plt.subplots(3, 1, figsize=(11.0, 5.4), sharex=True)
358     axes[0].plot(x, c42 / 1e4, color=C_S0C42, lw=0.9, label="4-2 冻结计划")
359     axes[0].plot(x, c43 / 1e4, color=C_S0C43, lw=0.9, label="4-3 可调整")
360     axes[0].set_ylabel("日费用 / 万元")
361     axes[0].legend(fontsize=8, ncol=2)
362     axes[1].bar(x, (c42 - c43) / 1e4, color=C_FCOST, width=1.0)
363     axes[1].axhline(0, color="k", lw=0.8)
364     axes[1].set_ylabel("日差额 / 万元")
365     axes[2].plot(x, cum / 1e4, color=C_PRICE, lw=1.1)
366     axes[2].axhline(0, color="k", lw=0.8)
367     axes[2].set_ylabel("累计差额 / 万元")
368     axes[2].set_xlabel("日期")
369     fig.suptitle("4-3 相对 4-2 (冻结计划) 省下的费用及其累计 (正数表示 4-3 更省)",
370                 fontsize=12, y=0.995)
371     fig.tight_layout()
372     savefig("p4_fig7_daily.png")
373
374
375 def main() -> None:
376     res, det42, det43 = load_inputs()
377     fig1_structure()
378     print(" 图 1 价格结构分解")
379     fig2_forecast()
380     print(" 图 2 价格预测与实际")
381     fig3_strategy(res)
382     print(" 图 3 三策略费用分解")
383     fig4_purchase(det42, det43)
384     print(" 图 4 冻结计划与初始/最终购电")
385     fig5_storage(det42, det43)
386     print(" 图 5 实际储电量")
387     fig6_combos(res)
388     print(" 图 6 八种预报组合")
389     d42 = pd.read_csv(SUP / "p4_daily_42.csv", encoding="utf-8-sig")
390     d43 = pd.read_csv(SUP / "p4_daily_43.csv", encoding="utf-8-sig")
391     fig7_daily(d42, d43)
392     print(" 图 7 逐日费用与累计差额")
393     print(f"输出目录: {OUT}")
394
395
396 if __name__ == "__main__":
397     main()

```

J 问题四结果文件说明

结果按题目要求的 result4-2.xlsx、result4-3.xlsx 与支撑材料中的内部文件两套输出。

(1) result4-2.xlsx (问题四分支 1: 全天计划固定)

- 工作表 Sheet1: 逐日汇总, 共 334 行 (2025-02-01 至 2025-12-31), 每行含日期、0:00 计划购电量、实际收到总电量、紧急购电量与三项费用;
- 工作表 Sheet2-Sheet5: 按附件 5 划分口径给出的指定时间段细化记录;
- 每行的储电量记录为当天 24:00 的值, 与次日 0:00 的初值一致;
- 4-2 全天计划保持不变, 故表中不设置调整提交列, 最终常规购电量恒等于计划购电量。

(2) result4-3.xlsx (问题四分支 2: 滚动调整)

- 工作表 Sheet1: 逐日汇总, 除计划量外另含最终常规购电量、上调量、下调量与调整提交次数;
- 工作表 Sheet2-Sheet5: 0:00、6:00、12:00、18:00 四个发布时刻的细化记录, 逐日给出该时刻是否提交调整、提交了哪些时段、调整前后的购电量与当时所用的价格预测版本;
- 记录口径与 result3.xlsx 一致, 便于两份交付文件直接对照。

(3) 内部结果文件 (支撑材料 06_ 支撑材料/)

- p4_detail_42.csv、p4_detail_43.csv: 逐时段明细 (各 $334 \times 144 = 48096$ 行), 同时输出预测电价、实际电价、预估计划费与实际计划费四列, 用于逐时段分析预测费用与实际费用的差异;
- p4_daily_42.csv、p4_daily_43.csv: 逐日汇总 (各 334 行), 含全天电量、三项费用、充放电与紧急购电的加权均价、储电量与当日生效参数;
- p4_strategies.csv: 固定价、波动价预测、波动价事后完全信息三种策略在两个分支上的独立时序仿真结果, 即表 25 的原始记录;
- p4_combinations.csv: 8 种预报组合的全年汇总, 即表 32 的原始记录;
- p4_price_forecast_candidates.csv、p4_price_beta.csv: 价格预测候选与日内修正系数扫描的原始记录, 即表 27、表 28 的评价区间情形 (选型不在评价区间上做, 选型期的对表见表 26, 随 p4_results.json 的“预测模型选型”字段一并保存);
- p4_risk_weight.csv: 价格加权与等权两种风险余量的逐 α 对照, 即表 30 的原始记录;
- p4_results.json: 正文全部汇总数字、校验结果、标定记录、价格结构四条断言与预测模型选型复核的机器可读版本。附表 24-33 由 p4_tables.py 直接读取该文件与明细 CSV 自动生成 (p4_tables.tex、p4_tables_appendix.tex)。

表 35 四个分支的求解规模与耗时（本机单次完整运行）

任务	内容	说明
4-2 主策略	365 天滚动，含 9 次标定	1 月联合标定 $5 \times 6 \times 6 = 180$ 组，此后每 42 天在 $5 \times 6 = 30$ 组上重选 α, ρ
4-3 主策略	365 天滚动，含 9 次标定	同上；两分支的参数网格与时序仿真窗口完全一致
策略对照	2 分支 $\times$ 3 价格模式，各自独立标定	共 6 次完整时序仿真
组合比较	8 个预报组合，各自独立标定	共 8 次完整时序仿真
不退款口径	4-3 在不退款口径下重新标定与时序仿真 ($x_t \geq g_t^0$)	独立敏感性分析

K 支撑材料文件列表

按规范第五条，随论文提交的全部支撑材料为一个压缩包，其内容如下表所列。除题目要求的两类结果文件外，其余均为程序的中间与最终输出，均由源码自动生成；问题三所用的六个脚本已在“完整源程序”一节全文列出，其余脚本的提交形式与之相同。

(1) 题目要求的结果文件

- **result1.xlsx**: 问题一要求的结果，含 144 行逐时段计划购电量与充放电量；
- **result2.xlsx**: 问题二要求的结果，按附件 5 的划分口径自动切分为表 1（计划购电量）、表 2（充放电量）、表 3（紧急购电量）三张工作表，并额外附一张全天汇总表（模板三表一列未改，见正文问题二一节的说明）；
- **result3.xlsx**: 问题三要求的结果，含计划购电量、调整购电量、充放电量、紧急购电量四张工作表（与附件 5 模板同一套表名与列口径）；
- **result4-2.xlsx**、**result4-3.xlsx**: 问题四要求的两份结果，分别对应“计划全天冻结”与“滚动调整”两个分支，口径与 **result2.xlsx**/**result3.xlsx** 保持一致。

(2) 源程序（源码/，共 21 个脚本）

- **p1_microgrid.py**、**p2_microgrid.py**、**p3_microgrid.py**、**p4_microgrid.py**: 四个问题的主求解器；
- **p3_backtest.py**、**p3_export.py**: 问题三的全年因果滚动时序仿真与结果导出（含九项校验）；
- **p3_analysis.py**、**p3_calibration.py**: 问题三的量化分析（八种预报使用方式、两处口径敏感性、单参数扫描、预报精度）与 1 月标定及样本外检验；
- **p3_rolling_microgrid.py**: 问题三早期版本的建模库。它不再产生论文中的任何数字，但问题四的求解器复用了其中的预报面板、误差表与调整求解器（**p4_microgrid.py** 直接 import 本模块），故一并提交；
- **solve2_final.py**: 问题二的独立参考实现。它与 **p2_microgrid.py** 由不同成员各自独立编写（标定网格、重选周期与时序仿真窗口均独立设定），二者对 **result2.xlsx** 的全年费用相互印证（数值对照见正文问题二一节）；
- **p1_figures.py**、**p2_figures.py**、**p3_figures.py**、**p4_figures.py**: 正文插图生成；
- **p2_tables.py**、**p3_tables.py**、**p4_tables.py**: 论文表格生成；
- **p2_alphasweep.py**: 问题二的 α 灵敏度扫描；
- **p3_alignment_scan.py**: 附件 3 时间口径判定；
- **p4_price_scan.py**: 附件 4 价格结构的独立复核；
- **cumcm_plot.py**: 绘图样式与中文字体配置。

以上脚本在 Python 3.11 下只需 **numpy/scipy/pulp/pandas/openpyxl/matplotlib** 即可按依赖顺序直接运行（混合整数模型用 **scipy.optimize.milp**，问题三的两个

线性规划用 `pulp` 自带的 CBC 求解器), 不依赖任何外部数据文件 (输入全部取自题目附件), 也不需要手工输入任何中间结果。

(3) 结果数据

- `p1_detail.csv`、`p1_results.json`: 问题一的逐时段明细与汇总指标;
- `p2_detail.csv`、`p2_daily.csv`、`p2_calibration.csv`、`p2_results.json`、`p2_alpha_sensitivity.json`: 问题二的逐时段明细、逐日汇总、滚动标定记录、汇总指标与灵敏度扫描结果;
- `p3_spec.json`: 问题三四个指定日期的逐时段明细与当日汇总, 即表 7–10 的原始记录;
- `p3_analysis.json`、`p3_main_daily.csv`、`p3_main_arrays.npz`: 问题三的八种预报使用方式汇总、逐日汇总与主策略逐时段数组, 即表 18 与图 4、14a–15 的原始记录;
- `p3_forecast_skill.csv`、`p3_calibration.json`: 四个发布版本的预报精度与 1 月标定面 (含样本外检验), 即表 19 与表 22 的原始记录;
- `p3_alignment_scan.json`: 附件 3 三种时间口径的扫描结果, 对应正文 7.2 节的判定;
- `p4_detail_42.csv`、`p4_detail_43.csv`、`p4_daily_42.csv`、`p4_daily_43.csv`: 问题四两个分支的逐时段明细与逐日汇总;
- `p4_results.json`: 问题四的全部汇总指标与逐项校验结果;
- `p4_combinations.csv`: 问题四 8 种预报组合的实际总费用与费用构成, 即表 32 的原始记录;
- `p4_strategies.csv`、`p4_strategy_fees.csv`: 三种价格信息条件下的策略对照, 即表 25 与图 13 的原始记录;
- `p4_price_scan.json`: 附件 4 价格结构分解、星期效应与候选预测模型误差的独立复核结果 (由 `p4_price_scan.py` 直接读取附件重算, 不引用任何求解器中间变量);
- `p4_price_forecast_candidates.csv`、`p4_price_beta.csv`、`p4_risk_weight.csv`: 价格预测候选方案、日内修正系数 β 的扫描与风险权重口径的对照记录。

L 补充分析与讨论

本附录给出基准对照、灵敏度检验、成因分解及关键模型设定的补充结果。

12.1 问题一：基准对照与灵敏度分析

12.1.1 基准对照：储能与光伏的价值

为量化储能设备与光伏发电的经济价值，构造“光伏供电、缺口直接向外网购买、电池保持不动”的基准方案： $g_t^{\text{base}} = \max\{\ell_t - v_t, 0\}$, $c_t = d_t = 0$, $E_t \equiv 6000$, 多余光伏弃用。该方案满足同样的日循环约束，因而是原问题的一个可行解，可用于检验优化结果并衡量储能的经济贡献。

表 36 不同方案的经济性对照

方案	全天购电量/kWh	全天购电费/元	弃光电量/kWh
无光伏无储能	111024.81	88319.36	—
无储能	61789.94	48052.05	6247.96
本模型（含储能）	59482.70	35126.95	0.00

由表 36 可见，配置储能设备后，全天购电费由 48052.05 元降至 35126.95 元，节省 12925.10 元，降幅为 26.90%；较“无光伏无储能”方案节省 53192.41 元（60.23%）。含储能方案的购电量由 61789.94 kWh 降至 59482.70 kWh，这是因为储能将无储能方案中的 6247.96 kWh 光伏盈余转移到无光照时段使用，从而提高光伏本地消纳率并降低电网购电量。优化费用 35126.95 元低于基准费用 48052.05 元，与上述可行性论证一致。

12.1.2 灵敏度分析

题目只给出一个“充放电效率为 90%”的参数，其在充电与放电两侧的分配方式存在解释空间：可以是单程效率（ $\eta_c = \eta_d = 0.9$ ，往返 0.81），也可以是往返效率（对称折算为 $\eta_c = \eta_d = \sqrt{0.9} \approx 0.9487$ ，往返 0.90）。此外，储能的日循环水平、充放电决策的时间粒度也均可能影响结论。为此设计六组对照口径：

表 37 不同建模口径下的求解结果对照

口径	全天购电量/kWh	全天购电费/元	弃光/kWh
主模型（往返 0.81, $E_0 = 6000$, 10 min）	59482.70	35126.95	0.00
放电无损（ $\eta_c = 0.90$, $\eta_d = 1.00$, 往返 0.90）	57580.98	33416.53	0.00
对称折算（ $\eta_c = \eta_d = \sqrt{0.90}$, 往返 0.90）	57526.24	33801.50	0.00
E_0 自由（仅要求 $E_0 = E_{144}$, 收敛至 8550）	59482.70	35118.60	0.00
粗化为 1 小时粒度	59924.95	36771.29	768.44
粗化为 4 小时粒度	61302.10	43648.06	2293.67

注：“往返”指 $\eta_c \eta_d$ ；除注明外均为 $\eta_c = \eta_d = 0.90$ 、 $E_0 = 6000$ kWh、10 分钟决策粒度。

对表 37 的分析可得以下结论：

(1) 效率参数的影响。若将 90% 的效率解释为往返效率并对称折算（ $\eta_c = \eta_d = \sqrt{0.9}$ ），购电费为 33801.50 元，较主模型降低 3.77%；若极端地假设放电无损（ $\eta_c = 0.90$, $\eta_d = 1.00$ ），购电费为 33416.53 元，降低 4.87%。两者构成单程/往返两种解释下的费用区间 [33416.53, 35126.95] 元，跨度不足 5%。原因是放电效率越低，储能放电时需额外多消耗的储存电量越多，越直接地削弱峰谷套利收益。**两种口径下“谷充峰放”的策略结构与零弃光结论均不变**，说明主模型对效率参数的分配方式不敏感。

(2) 初始储电量的影响。若解除 $E_0 = 6000$ kWh 的固定约束、仅要求日循环 $E_0 = E_{144}$ ，模型自动收敛到 $E_0 = 8550$ kWh，购电费仅下降 8.35 元（0.024%）。这表明在当前数据和约束下，初始储电量对最优费用的影响较小，采用题目给定的 $E_0 = 6000$ kWh 不改变主要结果。

(3) 决策粒度的影响。当充放电决策由 10 分钟粗化为 1 小时时，购电费上升 4.68%

并产生 768.44 kWh 弃光；粗化为 4 小时时，购电费上升 24.26%（达 43648.06 元），弃光量进一步增至 2293.67 kWh。原因是粗粒度下储能功率在各时段内保持恒定，对电价与光伏出力变化的响应能力下降，峰谷调节和光伏消纳受到影响。因此，10 分钟决策粒度能更充分地利用时段信息。

12.2 问题二：弃用电量的成因

全年弃用电量达 2463544.98 kWh，约占同期光伏发电量 19068890.0 kWh 的 12.9%。题目未对弃电设置惩罚或收益，因此弃电不产生额外结算费用，但可以反映日前计划与实际运行之间的偏差。

按逐时段口径分解全年弃电，可以看到它来自**两个性质不同的来源**。71.8% 的弃电（1769824.2 kWh）发生在实际光伏大于实际负载的时段，属于光伏富余超出储能吸纳能力——储能受 5000 kW 功率上限与 10800 kWh 容量上限双重约束，在光伏最集中的时段无法全额吸收；风险修正又使日前计划量系统性偏高，与高于预期的实际光伏叠加后进一步加剧了这一部分。另 28.2% 的弃电（693720.7 kWh）发生在**光伏并无富余**的时段，其中 137052.5 kWh 甚至出现在光伏出力为零的深夜与凌晨——这部分电量不可能来自光伏，只能来自**已按计划电价付费、但因实际负载低于预期而用不掉的计划购电量**。按各时段电价核算，这部分被弃掉的已购电量当初共付出 466267 元，约占全年计划购电费的 3.54%。因此，这部分弃电对应为降低紧急购电风险而增加的计划购电支出。

计划购电量同时影响弃电和紧急购电。本文在固定 $\rho = 1$ 、 $\lambda = 0$ 下重跑了 $\alpha \in \{0.50, 0.70, 0.80, 0.90\}$ 四点： α 由 0.80 降至 0.50 时，弃电量从 2628.6 MWh 降至 1493.7 MWh，但紧急购电量由 177.9 MWh 增至 636.1 MWh，总费用从 1403.8 万元升至 1508.8 万元；反向升至 0.90 时，紧急购电量降至 86.2 MWh，弃电量却升至 3369.1 MWh，总费用回升至 1421.6 万元； $\alpha = 0.70$ 为 1410.8 万元。四点中 $\alpha = 0.80$ 取到最小值且两侧点均高于它，表明风险余量需要在弃电与紧急购电之间权衡。（这四点均固定 $\rho = 1$ ；主策略因滚动重标定在 8 个窗口选到不同参数，全年 1393.97 万元还略低于该网格最优解。）逐点记录见支撑材料。

图 8 给出紧急购电量的季节分布与逐日费用构成。紧急购电费用在时间上较为集中。334 天中有 143 天发生紧急购电，但其中位日仅 317.1 kWh；而最严重的 6 月 1 日单日紧急购电 10545.11 kWh、费用 56753.15 元，一天就占去全年紧急购电费的 7.5%，其后紧接的 6 月 2 日（7244.83 kWh、42449.26 元）与 12 月 2 日、7 月 6 日同样显著高出常态。这说明全年紧急购电费主要由少数连续高负荷时段构成。6 月初的连日高负荷与光伏出力波动同时出现，使储能在数日内持续被压至下限，缺口只能在 5 倍电价下补足。

这一分布表明，额外储能容量或预测资源可优先用于高缺口日期。

12.3 问题二：模型设定与运行含义

问题二以连续储能状态连接相邻运行日，以过去 35 天的实际总费用滚动标定风险余量与储备线。该设定把跨日储能价值通过连续状态和后续决策纳入全年仿真；在给定的 5000 kW 功率上限下，策略较无储能基准节省 24.73% 的费用。

12.4 问题三：运行设定与结算规则

每个六小时调整区间仅在区间起点提交一次调整，因此每个时段只相对初始计划 g_t^0 结算一次。每次求解只使用当时已发布的信息，故 14,977,809 元的对照费用与 13,418,626 元的主策略费用均对应各自可得的信息集。参数以 2025 年 1 月实际总费用标定，并在评价期固定；主口径下调减部分退回原价并收取 50% 违约金，不退款口径的独立结果列于表 21。

12.5 问题四：预测与结算设定

附件 4 与附件 1 的年平均相同，逐时段均值最大偏差仅 5.15×10^{-5} 元/kWh；两者差值的星期结构解释 41.7% 的方差。模型据此将日内形状、星期特征和价格加权误差分位数结合为价格预测与风险修正。第 8.4 节的正协方差结果表明，价格加权分位数能够将高价时段的缺口风险直接纳入购电计划。主结果按退款口径结算：取消部分退回原价，并按交易时刻价格的 50% 支付违约金；不退款口径的独立时序仿真见附录第 4.1 节。